

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



ĐỒ ÁN LIÊN NGÀNH

**ĐỀ TÀI: NÂNG CAO HIỆU SUẤT PHÁT TRIỂN PHẦN MỀM TRONG
PHƯƠNG THỨC LINH HOẠT (AGILE) SỬ DỤNG CƠ SỞ DỮ LIỆU ĐỒ
THỊ/ĐỒ HỌA (GRAPH DATABASE)**

Giảng viên hướng dẫn: TS. Nguyễn Lệ Thu

Sinh viên thực hiện: Hoàng Lê Đức Huy (23010298)

Hoàng Minh Quân (23010315)

Khoá: K17 – 2023 – 2027

Lớp tín chỉ: CSE703014-1-3-25(N01)

Hà Nội, tháng 6 năm 2026

LỜI CẢM ƠN

Chúng tôi xin trân trọng cảm ơn **TS. Nguyễn Lệ Thu** đã tận tình hướng dẫn, đóng góp những ý kiến quý báu và tạo điều kiện thuận lợi để chúng tôi có thể hoàn thành đồ án cơ sở này.

Trong suốt quá trình thực hiện đề tài, cô đã luôn theo sát, định hướng và hỗ trợ nhóm chúng tôi kịp thời trong việc xác định hướng nghiên cứu, xây dựng nội dung và hoàn thiện sản phẩm. Những góp ý chuyên môn của cô không chỉ giúp chúng tôi nâng cao chất lượng đồ án mà còn giúp chúng tôi tích lũy thêm nhiều kiến thức và kinh nghiệm thực tế.

Chúng tôi cũng xin gửi lời cảm ơn chân thành tới quý thầy cô trong khoa đã truyền đạt những kiến thức nền tảng và tạo môi trường học tập thuận lợi để chúng tôi có thể hoàn thành đồ án này.

Mặc dù đã có nhiều cố gắng, nhưng do thời gian và kinh nghiệm còn hạn chế, đồ án không tránh khỏi những thiếu sót. Nhóm chúng tôi rất mong nhận được sự đóng góp ý kiến của thầy cô để đồ án được hoàn thiện hơn.

Chúng tôi xin chân thành cảm ơn!

LỜI CAM ĐOAN

Chúng tôi cam đoan Đồ án liên ngành là sản phẩm trí tuệ của tập thể chúng tôi. Mọi thông tin, dữ liệu, hình ảnh, etc... được sử dụng từ các nguồn khác đều được trích dẫn đầy đủ và có thể tìm thấy các tài liệu liên quan thông qua mục tài liệu tham khảo.

Chúng tôi xin chịu trách nhiệm hoàn toàn về nội dung của Đồ án liên ngành mà chúng tôi đã nộp.

Hà Nội, ngày ... tháng ... năm

Hoàng Lê Đức Huy, Hoàng Minh Quân (ký tên)

MỤC LỤC

MỤC LỤC	4
DANH MỤC HÌNH ẢNH	9
DANH MỤC BẢNG	12
Chương 1: Tổng quan đề tài	14
1.1. Lý do chọn đề tài	14
1.2. Mục tiêu và phạm vi nghiên cứu	14
1.3. Đối tượng và phương pháp nghiên cứu	15
1.3.1. Đối tượng nghiên cứu	15
1.3.2. Phương pháp nghiên cứu	15
1.4. Ý nghĩa khoa học và thực tiễn	15
Chương 2: Cơ sở lý thuyết	17
2.1. Tổng quan lĩnh vực	17
2.2. Các khái niệm và mô hình liên quan	18
2.2.1. Domain-Driven Design (DDD)	18
2.2.1.1. Khái niệm	18
2.2.1.2. Kiến trúc phân tầng trong Domain-Driven Design	18
2.2.1.3. DDD kết hợp với kiến trúc 3 lớp.	18
2.2.2. Model-View-ViewModel (MVVM)	19
2.2.1.1. Khái niệm	19
2.2.1.2. Các thành phần trong kiến trúc MVVM	20
2.2.1.3. MVVM kết hợp với Domain-Driven Design	20
2.2.1.4. Lợi ích của việc kết hợp DDD và MVVM	20
2.2.3. UML (Unified Modeling Language)	21
2.2.4. Java Spring Boot	21
2.2.5. Flutter	21
2.2.6. Cơ sở dữ liệu quan hệ	22
2.2.7. KeyCloak	25
2.2.8. JobRunr	26
2.2.9. Docker	27
2.2.10. WebSocket	27
2.2.11. Simple Mail Transfer Protocol (SMTP)	28
2.2.11.1. Khái niệm	28
2.2.11.2. Vai trò của SMTP	28
2.2.11.3. SMTP kết hợp với xử lý bất đồng bộ	28
2.2.12. GraphQL	28
2.2.12.1. Khái niệm	28
2.2.12.2. Vai trò của GraphQL trong hệ thống	29
2.2.12.3. GraphQL kết hợp với kiến trúc hệ thống	29
2.2.12.4. Ưu điểm của GraphQL	29

2.2.13. NLP & Dependency Parsing for User Story	29
2.2.14. Semantic Normalization và Canonical Mapping	30
2.2.15. Association Rule Mining (Apriori)	30
2.2.16. Graph Centrality for Story Prioritization	31
2.2.17. Weakly-supervised Redundancy Classification	31
2.2.18. Hybrid Analytical Pipeline (MySQL -> NLP/ML -> Neo4j)	32
2.3. Các công trình, nghiên cứu liên quan	32
2.4. Công nghệ, ngôn ngữ và công cụ sử dụng	33
Chương 3: Phân tích hệ thống	35
3.1. Khảo sát hiện trạng	35
3.1.1. Quy trình nghiệp vụ hiện tại	35
3.1.2. Các vấn đề, hạn chế	35
3.2. Phân tích yêu cầu	36
3.2.1 Yêu cầu nghiệp vụ	36
3.2.2. Yêu cầu chức năng	36
3.2.3. Yêu cầu phi chức năng	37
3.3. Mô hình ca sử dụng (Use Case Model)	38
3.3.1. Danh sách Use Case	38
3.3.2. Biểu đồ Use Case tổng quát	42
3.4. Các biểu đồ phân tích	48
3.4.1. Sơ đồ tuần tự (Sequence Diagram)	48
3.4.1.1. Sơ đồ tuần tự đăng ký tài khoản (Sequence Diagram Register)	48
3.4.1.2. Sơ đồ tuần tự xem hồ sơ (Sequence Diagram Get My Profile)	50
3.4.1.3. Sơ đồ tuần tự lấy tất cả hồ sơ (Sequence Diagram Get All Profile)	52
3.4.1.4. Sơ đồ tuần tự tạo không gian làm việc (Sequence Diagram Create Workspace)	54
3.4.1.5. Sơ đồ tuần tự cập nhật không gian làm việc (Sequence Diagram Update Workspace)	55
3.4.1.6. Sơ đồ tuần tự lấy tất cả không gian làm việc (Sequence Diagram Get All Workspace)	58
3.4.1.7. Sơ đồ tuần tự lấy thành viên (Sequence Diagram Get Members)	60
3.4.1.8. Sơ đồ tuần tự xóa không gian làm việc (Sequence Diagram Delete Workspace)	62
3.4.1.9. Sơ đồ tuần tự mời thành viên (Sequence Diagram Invite Member)	64
3.4.1.10. Sơ đồ tuần tự đồng ý lời mời (Sequence Diagram Accept)	66
3.4.1.11. Sơ đồ tuần tự từ chối lời mời (Sequence Diagram Deny)	68
3.4.1.12. Sơ đồ tuần tự lấy lời mời đang chờ xử lý (Sequence Diagram Get My Pending Invitation)	70
3.4.1.13. Sơ đồ tuần tự lấy thông báo chưa đọc (Sequence Diagram Get Unread)	72
3.4.1.14. Sơ đồ tuần tự đánh dấu thông báo là đã đọc (Sequence Diagram Mark As Read)	74
3.4.1.15. Sơ đồ tuần tự đánh dấu tất cả thông báo đã đọc (Sequence Diagram Mark	

all as Read Notification)	76
3.4.1.16: Sơ đồ tuần tự tạo Sprint (Sequence Diagram Create Sprint)	78
3.4.1.17: Sơ đồ tuần tự lấy danh sách tất cả sprints (Sequence Diagram Get All Sprints)	80
3.4.1.18: Sơ đồ tuần tự thêm User Story vào Sprint (Sequence Diagram Add User Story To Sprint)	82
3.4.1.19: Sơ đồ tuần tự xóa User Story ra khỏi Sprint (Sequence Diagram Remove User Story From Sprint)	84
3.4.1.20: Sơ đồ tuần tự lấy tất cả User Stories trong Sprint (Sequence Diagram Get User Stories Of Sprint)	86
3.4.1.21: Sơ đồ tuần tự bắt đầu một Sprint (Sequence Diagram Start Sprint)	87
3.4.1.22: Sơ đồ tuần tự hoàn thành Sprint (Sequence Diagram Complete Sprint)	89
3.4.1.23: Sơ đồ tuần tự tạo User Story (Sequence Diagram Create User Story)	91
3.4.1.24: Sơ đồ tuần tự lấy danh sách User Stories trong Product Backlog (Sequence Diagram Get User Stories In Product Backlog)	93
3.4.1.25: Sơ đồ tuần tự cập nhật trạng thái của User Story (Sequence Diagram Update User Story Status)	95
3.4.1.26: Sơ đồ tuần tự xóa câu chuyện người dùng (Sequence Diagram Delete User Story)	97
3.4.1.27: Sơ đồ tuần tự cho đồ thị các câu chuyện người dùng (Sequence Diagram Get Workspace Graph)	99
3.4.1.28: Sơ đồ tuần tự phân loại câu chuyện người dùng trùng lặp (redundancy classification)	102
3.4.2 Biểu đồ Lớp (Class Diagram)	106
3.5. Kết luận chương	106
Chương 4: Thiết kế hệ thống	107
4.1. Thiết kế cơ sở dữ liệu	107
4.1.1. ERD	107
4.1.2. Lược đồ quan hệ logic	108
4.1.3. Cấu trúc các Collection	110
4.1.3.1. Collection Profile (Identity / User Service — MySQL)	110
4.1.3.2. Cụm Workspace & Agile	110
4.1.3.3. Cụm Cộng tác & Thông báo (MySQL)	113
4.1.3.4. Cụm Phân tích User Story (Analyze Service — MySQL)	114
4.1.3.5. Cụm Đồ thị Neo4j (Graph DB)	117
4.2. Thiết kế thành phần phần mềm	119
4.2.1. Biểu đồ lớp (Class Diagram) chi tiết	119
4.2.2. Biểu đồ đóng gói (Package Diagram) chi tiết	120
4.2.3. Sơ đồ Thực thể Quan hệ Logic (Logical ERD Diagram)	121
4.2.3.1. Sơ đồ Thực thể Quan hệ Logic - Backend	121
4.2.3.2. Sơ đồ Thực thể Quan hệ Logic - analyze_user_stories	122
4.3. Công thức và thuật toán tính toán trong hệ thống	128
4.3.1. Phân tích ngữ nghĩa và chuẩn hóa từ vựng (Semantic Normalization)	128

4.3.1.1. Mục đích	128
4.3.1.2. Độ tương đồng phi tuyến (Non-linear Fusion Similarity)	128
4.3.1.3. Chỉ số Jaccard cho ngữ cảnh (Context Similarity)	128
4.3.1.4. Thuật toán Union-Find — gom nhóm từ vựng	129
4.3.2. Khai phá luật kết hợp (Association Rules)	129
4.3.2.1. Mục đích	129
4.3.2.2. Thuật toán Apriori	129
4.3.2.3. Các chỉ số sức mạnh của luật	129
4.3.3. Chấm điểm mức độ ưu tiên (Priority Scoring)	129
4.3.3.1. Mục đích	129
4.3.3.2. Độ trung tâm đồ thị (Graph Centrality)	129
4.3.3.3. Điểm tổng hợp cấu trúc (Struct Score)	129
4.3.3.4. Chuẩn hóa (Scaling) và hàm Sigmoid	130
4.3.3.5. Công thức tinh chỉnh cuối cùng	130
4.3.4. Phân loại và phát hiện dư thừa (Redundancy Classification)	130
4.3.4.1. Mục đích	130
4.3.4.2. Mô hình học máy	130
4.3.4.3. Công thức dự phòng (Rule-based Fallback)	131
4.3.4.4. Gom nhóm	131
4.3.5. Công thức trên đồ thị	131
4.3.5.1. Phân bố tọa độ S–V–O (Column Layout)	131
4.3.5.2. Kéo thả (Drag Node và Matrix Transformation)	131
4.3.5.3. Max-Pooling Priority	131
4.3.5.4. Lọc ngưỡng (Threshold Culling)	131
4.3.5.5. Active Neighbor Search	132
4.3.5.6. Quadratic Bezier Curve	132
4.3.5.7. Glow Pulse Animation	132
4.3.5.8. Dashed Circle Zoning	132
4.3.5.9. Lasso Polygon Tracing	132
4.3.6. Tổng kết	132
4.4. Thiết kế giao diện người dùng (Giao diện Web)	133
4.4.1. Giao diện trang đích (Landing page)	133
4.4.2. Giao diện đăng ký	134
4.4.3. Giao diện đăng nhập	136
4.4.4. Giao diện trang chủ	136
4.4.5. Giao diện các mẫu không gian làm việc (Space templates)	138
4.4.6. Giao diện khởi tạo Workspace	139
4.4.7. Giao diện mời thành viên	140
4.4.8. Giao diện Backlog	140
4.4.9. Giao diện Graph Backlog	141
4.4.10. Giao diện Sprint	148

4.4.11. Giao diện Summary	149
4.4.12. Giao diện Graph Sprint	150
4.4.13. Giao diện thông báo (Notifications)	152
4.4.14. Giao diện mời (Invitations)	156
4.4.15. Giao diện hồ sơ (Profile)	157
4.5. Thiết kế giao diện người dùng (Giao diện mobile)	158
4.5.1. Tối ưu hóa không gian và Điều hướng (Navigation)	158
4.5.2. Tái cấu trúc hiển thị dữ liệu (Data Layout)	160
4.5.3. Tương tác vuốt chạm (Gesture-based Interaction)	161
4.6. Kết luận chương	162
Chương 5: Cài đặt và kết quả thử nghiệm	163
5.1. Môi trường triển khai	163
5.1.1. Hệ điều hành và phần cứng	163
5.1.2. Phần mềm và công cụ hỗ trợ	163
5.1.3. Kết quả mong đợi	163
5.2. Đánh giá hệ thống (Kết luận và hướng phát triển)	164
5.2.1. Kết quả đạt được	164
5.2.2. Hạn chế	165
5.2.3. Hướng phát triển tiếp theo	167
5.2.3.1. Triển khai hệ thống trong môi trường thực tế (Production Environment)	167
5.2.3.2. Hoàn thiện quy trình CI/CD tự động	168
5.2.3.3. Nâng cấp pipeline phân tích ngôn ngữ và đồ thị tri thức	168
5.2.3.4. Tối ưu hiệu năng, bảo mật và trải nghiệm người dùng	170
Thuật ngữ chuyên môn	171
Tài Liệu Tham Khảo:	172

DANH MỤC HÌNH ẢNH

Hình 1: Sơ đồ kiến trúc Domain-Driven Design (DDD)	19
Hình 2: Sơ đồ KeyCloak	25
Hình 3: Biểu đồ use case tổng quát	42
Hình 4: Biểu đồ use case register	43
Hình 5: Biểu đồ use case login	43
Hình 6: Biểu đồ use case Workspace	44
Hình 7: Biểu đồ use case Sprint	45
Hình 8: Biểu đồ use case profile	46
Hình 9: Biểu đồ use case notification	46
Hình 10: Biểu đồ use case notification system	46
Hình 11: Biểu đồ use case User Story	47
Hình 12: Biểu đồ use case graph	47
Hình 13: Sơ đồ tuần tự đăng ký tài khoản (Sequence Diagram Register)	48
Hình 14: Sơ đồ tuần tự xem hồ sơ (Sequence Diagram Get My Profile)	50
Hình 15: Sơ đồ tuần tự lấy tất cả hồ sơ (Sequence Diagram Get All Profile)	52
Hình 16: Sơ đồ tuần tự tạo không gian làm việc (Sequence Diagram Create Workspace)	54
Hình 17: Sơ đồ tuần tự cập nhật không gian làm việc	55
Hình 18: Sơ đồ tuần tự lấy tất cả không gian làm việc (Sequence Diagram Get All Workspace)	58
Hình 19: Sơ đồ tuần tự lấy thành viên (Sequence Diagram Get Members)	60
Hình 20: Sơ đồ tuần tự xóa không gian làm việc (Sequence Diagram Delete Workspace)	62
Hình 21: Sơ đồ tuần tự mời thành viên (Sequence Diagram Invite Member)	64
Hình 22: Sơ đồ tuần tự đồng ý lời mời (Sequence Diagram Accept)	66
Hình 23: Sơ đồ tuần tự từ chối lời mời (Sequence Diagram Deny)	68
Hình 24: Sơ đồ tuần tự lấy lời mời đang chờ xử lý (Sequence Diagram Get My Pending Invitation)	70
Hình 25: Sơ đồ tuần tự lấy thông báo chưa đọc (Sequence Diagram Get Unread)	72
Hình 26: Sơ đồ tuần tự đánh dấu thông báo là đã đọc (Sequence Diagram Mark As Read)	74
Hình 27: Sơ đồ tuần tự đánh dấu tất cả thông báo đã đọc (Sequence Diagram Mark all as Read Notification)	76
Hình 28: Sơ đồ tuần tự tạo Sprint (Sequence Diagram Create Sprint)	78
Hình 29: Sơ đồ tuần tự lấy danh sách tất cả sprints (Sequence Diagram Get All Sprints)	80
Hình 30: Sơ đồ tuần tự thêm User Story vào Sprint (Sequence Diagram Add User Story To Sprint)	82
Hình 31: Sơ đồ tuần tự xóa User Story ra khỏi Sprint (Sequence Diagram Remove User Story From Sprint)	84
Hình 32: Sơ đồ tuần tự lấy tất cả User Stories trong Sprint (Sequence Diagram Get User Stories Of Sprint)	86
Hình 33: Sơ đồ tuần tự bắt đầu một Sprint (Sequence Diagram Start Sprint)	88
Hình 34: Sơ đồ tuần tự hoàn thành Sprint (Sequence Diagram Complete Sprint)	90

Hình 35: Sơ đồ tuần tự tạo User Story (Sequence Diagram Create User Story)	91
Hình 36: Sơ đồ tuần tự lấy danh sách User Stories trong Product Backlog (Sequence Diagram Get User Stories In Product Backlog)	93
Hình 37: Sơ đồ tuần tự cập nhật trạng thái của câu chuyện người dùng (Sequence Diagram Update User Story Status)	95
Hình 38: Sơ đồ tuần tự xóa câu chuyện người dùng (Sequence Diagram Delete User Story)	97
Hình 39: Sơ đồ tuần tự cho đồ thị toàn bộ câu chuyện người dùng của sản phẩm (Sequence Diagram Get Workspace Graph)	99
Hình 40: Sơ đồ tuần tự phân loại câu chuyện người dùng trùng lặp (redundancy classification)	104
Hình 41: Biểu đồ lớp của Ứng dụng (Class diagram)	106
Hình 42: Biểu đồ ERD vật lý	108
Hình 43: Biểu đồ ERD vật lý cho analyze_story	108
Hình 44: Biểu đồ lớp (Class Diagram)	119
Hình 45: Biểu đồ đóng gói (Package Diagram)	120
Hình 46: Sơ đồ Thực thể Quan hệ Logic (Logical ERD Diagram)	121
Hình 47: Sơ đồ Thực thể Quan hệ Logic - analyze_user_stories	123
Hình 48: Đồ thị tri thức & ML	125
Hình 49: Luồng logic	127
Hình 50: Giao diện trang đích	133
Hình 51: Giao diện đăng ký	134
Hình 52: Giao diện đăng ký	135
Hình 53: Giao diện đăng ký	136
Hình 54: Giao diện đăng ký	136
Hình 55: Giao diện đăng nhập	137
Hình 56: Giao diện trang chủ	137
Hình 57: Giao diện trang chủ cơ chế điều hướng (Auth_gate)	138
Hình 58: Giao diện trang chủ lựa chọn chế độ	139
Hình 59: Giao diện trang chủ chế độ đêm (Night mode)	139
Hình 60: Giao diện Space Templates	139
Hình 61: Giao diện tạo Workspace	140
Hình 62: Giao diện mời	141
Hình 63: Giao diện Backlog	142
Hình 64: Giao diện Backlog	142
Hình 65: Giao diện Backlog	143
Hình 66: Giao diện Backlog	143
Hình 67: Giao diện Backlog	144
Hình 68: Giao diện Backlog	144
Hình 69: Giao diện Backlog	145
Hình 70: Giao diện Backlog	145
Hình 71: Giao diện Backlog chức năng vẽ	146
Hình 72: Giao diện Backlog chức năng vẽ	146

Hình 73: Giao diện Backlog chức năng chọn	147
Hình 74: Giao diện Backlog chức năng thu	147
Hình 75: Giao diện Backlog chức năng mở	148
Hình 76: Giao diện Backlog chức năng lọc	148
Hình 77: Giao diện Backlog chức năng lọc	149
Hình 78 Giao diện Sprint	149
Hình 79: Giao diện Sprint	150
Hình 80: Giao diện Summary	151
Hình 81: Giao diện Graph Sprint	151
Hình 82: Giao diện Graph Sprint	152
Hình 83: Giao diện Graph Sprint	152
Hình 84 Giao diện Graph Sprint	153
Hình 85: Giao diện thông báo	154
Hình 86: Giao diện thông báo	155
Hình 87: Giao diện thông báo	156
Hình 88: Giao diện lời mời	157
Hình 89: Giao diện lời mời	158
Hình 90: Giao diện hồ sơ	159
Hình 91: Giao diện bị ẩn	160
Hình 92: Giao diện backlog	161
Hình 93: Giao diện backlog	162
Hình 94: Giao diện backlog	163

DANH MỤC BẢNG

Bảng 1: Các công trình, nghiên cứu liên quan	32
Bảng 2: Công nghệ, ngôn ngữ và công cụ sử dụng	33
Bảng 3: Danh sách Use Case	38
Bảng 4: Đặc tả chức năng đăng ký tài khoản (Register)	48
Bảng 5: Đặc tả chức năng xem hồ sơ (Get My Profile)	50
Bảng 6: Đặc tả chức năng lấy tất cả hồ sơ (Get All Profile)	52
Bảng 7: Đặc tả chức năng tạo không gian làm việc (Create Workspace)	54
Bảng 8: Đặc tả chức năng cập nhật không gian làm việc (Update Workspace)	56
Bảng 9: Đặc tả chức năng lấy tất cả không gian làm việc (Get All Workspace)	58
Bảng 10: Đặc tả chức năng lấy thành viên (Get Members)	60
Bảng 11: Đặc tả chức năng xóa không gian làm việc (Delete Workspace)	62
Bảng 12: Đặc tả chức năng mời thành viên (Invite Member)	64
Bảng 13: Đặc tả chức năng đồng ý lời mời (Accept)	66
Bảng 14: Đặc tả chức năng từ chối lời mời (Deny)	68
Bảng 15: Đặc tả chức năng lấy lời mời đang chờ xử lý (Get My Pending Invitation)	70
Bảng 16: Đặc tả chức năng lấy thông báo chưa đọc (Get Unread)	72
Bảng 17: Đặc tả chức năng đánh dấu thông báo là đã đọc (Mark As Read)	74
Bảng 18: Đặc tả chức năng đánh dấu tất cả thông báo đã đọc	76
Bảng 19: Đặc tả chức năng tạo Sprint (Create Sprint)	78
Bảng 20: Đặc tả chức năng lấy danh sách tất cả sprints (Get All Sprints)	80
Bảng 21: Đặc tả chức năng thêm User Story vào Sprint (Add User Story To Sprint)	82
Bảng 22: Đặc tả chức năng xóa User Story ra khỏi Sprint	84
Bảng 23: Đặc tả chức năng lấy tất cả User Stories trong Sprint (Get User Stories Of Sprint)	86
Bảng 24: Đặc tả chức năng bắt đầu một Sprint (Start Sprint)	88
Bảng 25: Đặc tả chức năng hoàn thành Sprint (Complete Sprint)	90
Bảng 26: Đặc tả chức năng tạo User Story (Create User Story)	91
Bảng 27: Đặc tả chức năng lấy danh sách User Stories trong Product Backlog (Get User Stories In Product Backlog)	93
Bảng 28: Đặc tả chức năng cập nhật trạng thái của câu chuyện người dùng (Update User Story Status)	95
Bảng 29: Đặc tả chức năng xóa câu chuyện người dùng (Delete User Story)	97
Bảng 30: Đặc tả chức năng cho đồ thị toàn bộ câu chuyện người dùng của sản phẩm (Get Workspace Graph)	99
Bảng 31: Đặc tả chức năng phân loại câu chuyện người dùng trùng lặp (redundancy classification)	104
Bảng 32: Bảng Collection Profile	110
Bảng 33: Bảng Collection Workspace	110
Bảng 34: Bảng Collection Backlog	111
Bảng 35: Bảng Collection Sprint	111
Bảng 36: Bảng Collection UserStory	112

Bảng 37: Bảng Collection WorkspaceRole	112
Bảng 38: Bảng Collection WorkspaceMember	113
Bảng 39: Bảng Collection WorkspaceInvitation	113
Bảng 40: Bảng Collection Notification	113
Bảng 41: Bảng Collection ApiKey	114
Bảng 42: Bảng Collection AnalyzeStoryResult	114
Bảng 43: Bảng Collection KnowledgeTerm	115
Bảng 44: Bảng Collection KnowledgeRelation	116
Bảng 45: Bảng Collection AnalyzeStatistic	117
Bảng 46: Thuộc tính node Term	117
Bảng 47: Thuộc tính node UserStory	117
Bảng 48: Các loại relationship Neo4j	118
Bảng 49: Thuộc tính ClassificationMetrics	118
Bảng 49: Bảng mô tả sơ đồ Thực thể Quan hệ Logic (Logical ERD Diagram)	121
Bảng 50: Bảng mô tả sơ đồ Thực thể Quan hệ Logic - analyze_user_stories	123
Bảng 51: Bảng mô tả Đồ thị tri thức & ML	125
Bảng 52: Bảng mô tả Luồng logic	127

PHÂN CÔNG NHIỆM VỤ ĐỒ ÁN

TT	Mã sinh viên	Họ và tên	Nhiệm vụ phân công	Nội dung đã thực hiện
1	23010298	Hoàng Lê Đức Huy	Xây dựng Backend, vẽ diagram, viết báo cáo	Thiết kế API, vẽ diagram, thiết kế cơ sở dữ liệu quan hệ (MySQL master-slave), xây dựng cơ sở dữ liệu đồ thị Neo4j, xây dựng phân tích analyze_user_stories (FastAPI, RabbitMQ, spaCy, WordNet/Word2Vec, Apriori, priority, redundancy), viết báo cáo
2	23010315	Hoàng Minh Quân	Xây dựng Frontend, viết báo cáo	Thiết kế giao diện, xây dựng ý tưởng để hiển thị đồ thị, tương tác, xây dựng giao diện mobile, Tài liệu hóa pipeline phân tích và tìm hiểu về các thuật toán & công thức, viết báo cáo

Chương 1: Tổng quan đề tài

1.1. Lý do chọn đề tài

Trong môi trường phát triển phần mềm, các phương pháp phát triển phần mềm linh hoạt (Agile) [1] ngày càng được áp dụng rộng rãi nhằm đáp ứng yêu cầu thay đổi nhanh của khách hàng và thị trường. Agile giúp các nhóm phát triển tăng tính thích ứng, rút ngắn chu kỳ phát triển theo từng vòng lặp (iteration) [1] và nâng cao chất lượng sản phẩm với các khái niệm cốt lõi như Sprint, User Story, Backlog và Team Member. Tuy nhiên, trong thực tế triển khai Agile, đặc biệt là với các dự án có số lượng lớn user story, task, Sprint và mối quan hệ phụ thuộc phức tạp, việc quản lý, theo dõi và phân tích dữ liệu dự án vẫn còn gặp nhiều khó khăn.

Các hệ thống quản lý Agile truyền thống thường sử dụng cơ sở dữ liệu quan hệ (Relational Database) [21]. Mặc dù phù hợp với dữ liệu có cấu trúc, mô hình này bộc lộ hạn chế khi cần biểu diễn và truy vấn các mối quan hệ phức tạp như: quan hệ phụ thuộc giữa các user story, liên kết giữa Sprint – task – thành viên, hoặc phân tích mức độ ảnh hưởng khi thay đổi yêu cầu. Các truy vấn dạng liên kết sâu (JOIN nhiều bảng) [21] thường gây ảnh hưởng đến hiệu năng và khó mở rộng.

Trong khi đó, cơ sở dữ liệu đồ thị (Graph Database) [3] cho phép mô hình hóa dữ liệu dưới dạng các đỉnh (node) và cạnh (relationship) [3], rất phù hợp để biểu diễn các mối quan hệ trong hệ thống Agile, việc duyệt quan hệ nhiều bước có thể hiệu quả hơn nhờ cơ chế index-free adjacency [3] trên graph engine gốc. Việc áp dụng Graph Database giúp truy vấn mối quan hệ nhanh hơn, trực quan hơn và hỗ trợ tốt cho việc phân tích, trực quan hóa dữ liệu dự án.

Xuất phát từ những vấn đề trên, nhóm lựa chọn đề tài **“Nâng cao hiệu suất phát triển phần mềm trong phương thức linh hoạt (Agile) sử dụng cơ sở dữ liệu đồ thị (Graph Database)”** nhằm nghiên cứu và xây dựng một hệ thống hỗ trợ quản lý Agile hiệu quả hơn, kết hợp giữa kiến trúc backend hiện đại với Java Spring Boot [12] và ứng dụng frontend đa nền tảng bằng Flutter [13].

1.2. Mục tiêu và phạm vi nghiên cứu

● Mục tiêu:

- Xây dựng một hệ thống hỗ trợ quản lý quy trình phát triển phần mềm theo phương thức Agile, tập trung vào các thành phần như Workspace, Sprint, User Story và Member.
- Ứng dụng cơ sở dữ liệu đồ thị (Graph Database) để mô hình hóa giữa các thực thể trong Agile, phân tích các user story ra thành các nodes dưới dạng đồ thị để dễ quản lý.
- Phát triển backend bằng Java Spring Boot, áp dụng kiến trúc Domain-Driven Design (DDD) nhằm đảm bảo tính hiệu quả và dễ bảo trì [2].
- Xây dựng giao diện người dùng bằng Flutter, hỗ trợ đa nền tảng [8], giúp người dùng dễ tương tác và theo dõi tiến độ dự án

- Cung cấp chức năng cơ bản của một hệ thống Agile như: tạo và quản lý Sprint, user story, cập nhật trạng thái công việc.
- **Phạm vi:**
 - Hệ thống được phát triển phục vụ cho các nhóm phát triển phần mềm áp dụng phương pháp Agile, bao gồm Project Manager, Developer và các thành viên liên quan.
 - Phạm vi nghiên cứu:
 - Quản lý dữ liệu Agile (Workspace, Sprint, User Story, Member).
 - Phân tích User Story và biểu diễn thành mối quan hệ giữa các nodes bằng Graph Database.
 - Hiển thị thông tin và tiến độ dự án thông qua giao diện Flutter.

1.3. Đối tượng và phương pháp nghiên cứu

1.3.1. Đối tượng nghiên cứu

- Quy trình phát triển phần mềm theo phương thức Agile.
- Các thực thể và mối quan hệ trong hệ thống Agile như Sprint, User Story, Member.
- Việc ứng dụng cơ sở dữ liệu đồ thị trong quản lý và phân tích dữ liệu dự án phần mềm.

1.3.2. Phương pháp nghiên cứu

- Khảo sát và nghiên cứu: Tìm hiểu về Agile, Graph Database, Spring Boot và Flutter
- Phân tích hệ thống: Xây dựng mô hình nghiệp vụ và biểu đồ UML (Use Case Diagram, Activity Diagram, Sequence Diagram, ...)
- Thiết kế và lập trình: Triển khai hệ thống backend bằng Java Spring Boot và frontend bằng Flutter

1.4. Ý nghĩa khoa học và thực tiễn

- **Về mặt khoa học:**
Đề tài tập trung vào khả năng ứng dụng cơ sở dữ liệu đồ thị trong quản lý quy trình phát triển phần mềm Agile nhằm làm rõ cấu trúc và mối quan hệ giữa các thành phần trong hệ thống. Thông qua việc mô hình hóa Agile dưới dạng đồ thị, bài toán quản lý được biểu diễn một cách trực quan và dễ quan sát, giúp hạn chế trùng lặp dữ liệu và phản ánh chính xác các mối quan hệ phức tạp. Cách tiếp cận này cho phép phân tích hiệu quả các liên kết trong hệ thống, từ đó nâng cao chất lượng truy vấn và hỗ trợ quá trình phân tích, đánh giá trong phát triển phần mềm Agile.
- **Về mặt thực tiễn:**
Về mặt thực tiễn, hệ thống được xây dựng có thể đóng vai trò như một mô hình tham khảo cho các nhóm phát triển phần mềm áp dụng Agile. Việc kết hợp Graph Database với Java Spring Boot và Flutter giúp hệ thống có khả năng mở rộng, dễ bảo trì và phù hợp với nhiều nền tảng.

Ngoài ra, đề tài còn giúp sinh viên nâng cao kỹ năng phân tích hệ thống, thiết kế kiến trúc phần mềm và ứng dụng công nghệ mới vào thực tế phát triển phần mềm.

Chương 2: Cơ sở lý thuyết

2.1. Tổng quan lĩnh vực

Trong những năm gần đây, phát triển phần mềm đã có nhiều thay đổi nhằm đáp ứng yêu cầu ngày càng cao về tốc độ, chất lượng và khả năng thích ứng với thay đổi. Các phương pháp phát triển truyền thống như thác nước (Waterfall) [22] bộc lộ nhiều hạn chế khi yêu cầu hệ thống thay đổi thường xuyên, chu kỳ phát triển kéo dài và khó phản hồi kịp thời với nhu cầu của khách hàng.

Để khắc phục những vấn đề này, phương thức phát triển phần mềm linh hoạt (Agile) [1] đã ra đời và nhanh chóng được áp dụng rộng rãi trong các dự án phần mềm hiện đại. Agile tập trung vào việc chia nhỏ công việc, phát triển theo từng vòng lặp ngắn (iteration), thường xuyên phản hồi và cải tiến liên tục. Các khái niệm cốt lõi trong Agile bao gồm Sprint, User Story, Backlog [23] và Team Member, tạo nên một hệ sinh thái quản lý công việc có mối quan hệ chặt chẽ với nhau.

Tuy nhiên, trong các dự án Agile có quy mô trung bình đến lớn, số lượng user story, task và mối quan hệ phụ thuộc giữa chúng ngày càng gia tăng. Việc lưu trữ và truy vấn dữ liệu Agile bằng các hệ quản trị cơ sở dữ liệu quan hệ truyền thống gặp khó khăn khi phải xử lý nhiều quan hệ phức tạp, dẫn đến truy vấn chậm và khó trực quan hóa dữ liệu.

Trong bối cảnh đó, cơ sở dữ liệu đồ thị (Graph Database) nổi lên như một giải pháp phù hợp để mô hình hóa và quản lý các mối quan hệ trong hệ thống Agile. Graph Database cho phép biểu diễn dữ liệu dưới dạng các đỉnh (node) và cạnh (relationship) [3], giúp việc truy vấn mối quan hệ, phân tích sự phụ thuộc và trực quan hóa tiến độ dự án trở nên hiệu quả hơn.

Dự án này hướng đến việc xây dựng một hệ thống hỗ trợ quản lý phát triển phần mềm theo phương thức Agile, trong đó:

- Quản lý các thực thể chính như Workspace, Sprint, User Story, và Member.
- Biểu diễn mối quan hệ giữa các thực thể bằng cơ sở dữ liệu đồ thị.
- Hỗ trợ theo dõi tiến độ, trạng thái công việc và mối quan hệ phụ thuộc trong dự án.
- Cung cấp giao diện trực quan giúp người dùng dễ dàng nắm bắt tình trạng phát triển phần mềm.

Hệ thống được phát triển với backend sử dụng Java Spring Boot nhằm đảm bảo tính ổn định, khả năng mở rộng và bảo trì, kết hợp với frontend Flutter để xây dựng giao diện đa nền tảng (web và mobile). Việc áp dụng Graph Database trong kiến trúc hệ thống giúp nâng cao hiệu suất truy vấn, hỗ trợ phân tích dữ liệu Agile và cải thiện hiệu quả quản lý dự án phần mềm.

2.2. Các khái niệm và mô hình liên quan

2.2.1. Domain-Driven Design (DDD)

2.2.1.1. Khái niệm

Domain-Driven Design (DDD) là phương pháp thiết kế và phát triển phần mềm tập trung vào nghiệp vụ cốt lõi (domain) [2]. Thay vì chú trọng vào công nghệ hay cấu trúc dữ liệu, DDD nhấn mạnh việc mô hình hóa chính xác các khái niệm nghiệp vụ, giúp hệ thống phản ánh đúng các hoạt động của bài toán thực tế. DDD giúp hệ thống dễ thích nghi với sự thay đổi của yêu cầu, giảm độ phức tạp khi mở rộng tính năng, tăng khả năng bảo trì và tái sử dụng mã nguồn.

2.2.1.2. Kiến trúc phân tầng trong Domain-Driven Design

DDD thường được triển khai theo mô hình kiến trúc phân tầng nhằm tách biệt rõ ràng trách nhiệm của từng thành phần [2].

Kiến trúc này gồm 4 phần chính:

a) Presentation Layer

Tầng trình bày chịu trách nhiệm:

- Nhận yêu cầu từ phía người dùng
- Gọi các dịch vụ ứng dụng để xử lý nghiệp vụ
- Trả kết quả về cho client

b) Application Layer

Tầng ứng dụng điều phối các use case của hệ thống:

- Thực hiện các nghiệp vụ ở mức ứng dụng
- Gọi các Domain Model để xử lý logic
- Quản lý transaction và luồng xử lý

c) Domain Layer

Domain Layer là trung tâm của hệ thống, chứa các quy tắc nghiệp vụ quan trọng nhất.

Thành phần gồm:

- Entity: Biểu diễn các đối tượng (Ví dụ: Sprint, User Story)
- Domain Service: Xử lý các nghiệp vụ không thuộc một entity nào

d) Infrastructure Layer

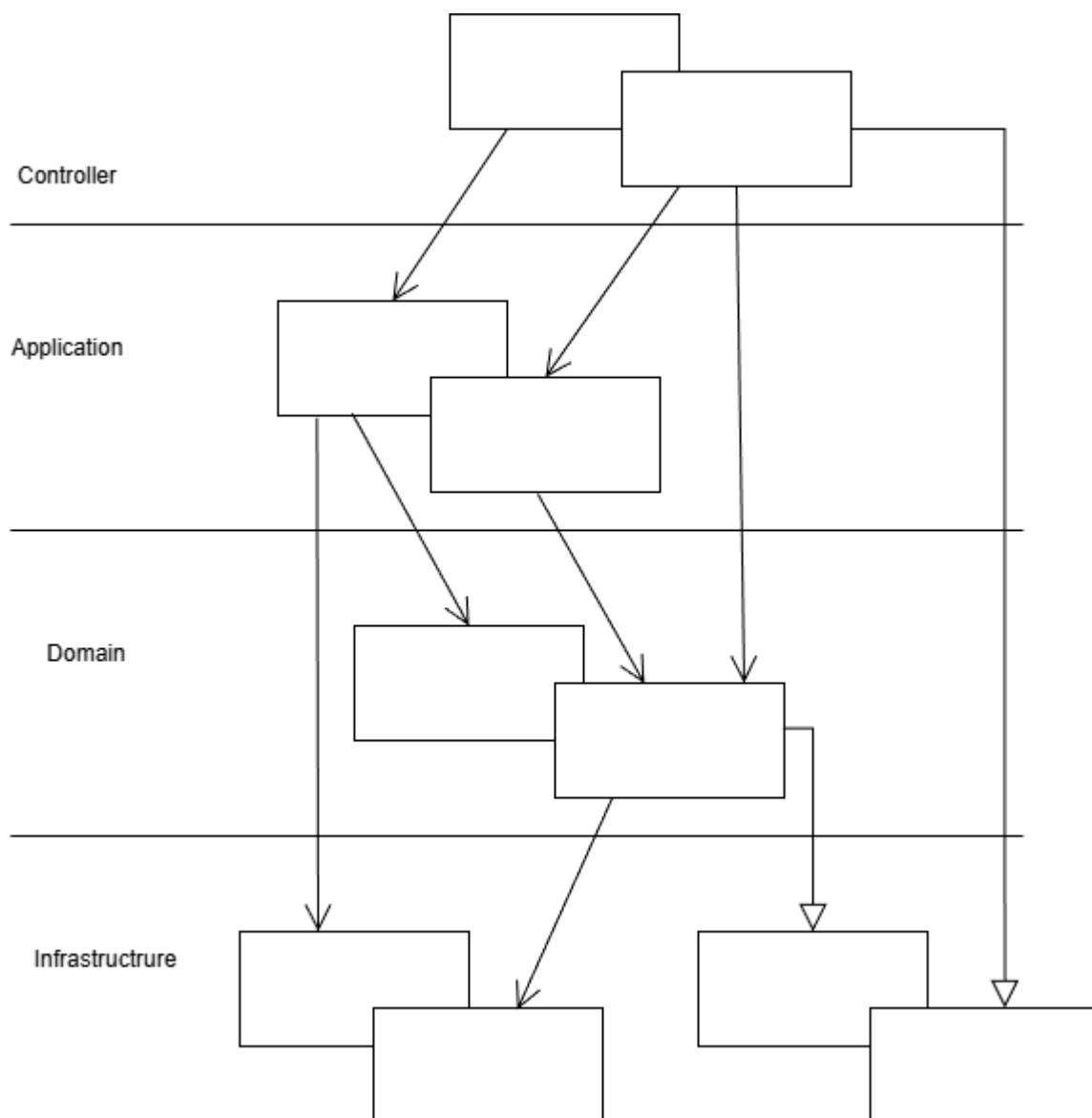
Tầng hạ tầng chịu trách nhiệm:

- Truy cập cơ sở dữ liệu
- Triển khai các Repository

2.2.1.3. DDD kết hợp với kiến trúc 3 lớp.

Kiến trúc 3 lớp	Domain-Driven Design
Presentation Layer	Presentation Layer

	(Controller)
Business Logic Layer	Application + Domain Layer
Data Access Layer	Infrastructure Layer



Hình 1: Sơ đồ kiến trúc Domain-Driven Design (DDD)

2.2.2. Model-View-ViewModel (MVVM)

2.2.1.1. Khái niệm

Model–View–ViewModel (MVVM) [24] là mô hình kiến trúc phổ biến trong phát triển giao diện người dùng, đặc biệt phù hợp với các ứng dụng hiện đại có giao diện tương tác cao.

MVVM giúp tách biệt rõ ràng giữa giao diện hiển thị, logic xử lý giao diện, và dữ liệu, từ đó nâng cao khả năng bảo trì, kiểm thử và mở rộng hệ thống frontend.

2.2.1.2. Các thành phần trong kiến trúc MVVM

Kiến trúc MVVM gồm ba thành phần chính:

- View: là tầng giao diện người dùng, chịu trách nhiệm:
 - Hiển thị dữ liệu lên UI
 - Nhận tương tác người dùng
 - Liên kết với ViewModel

View sẽ phản ánh trạng thái do ViewModel cung cấp.

- ViewModel: ViewModel đóng vai trò trung gian giữa View và Model
 - Quản lý trạng thái của giao diện
 - Xử lý logic trình bày (presentation logic)
 - Gọi API từ Backend để lấy hoặc cập nhật dữ liệu
 - Chuyển đổi dữ liệu từ dạng backend sang dạng phù hợp với UI
- Model

Model trong frontend đại diện cho:

- Dữ liệu nhận từ backend (DTO, Response Model)
- Cấu trúc dữ liệu phục vụ hiển thị

2.2.1.3. MVVM kết hợp với Domain-Driven Design

Trong hệ thống, kiến trúc MVVM ở frontend được thiết kế phù hợp và tương thích với Domain-Driven Design ở backend, nhằm đảm bảo sự phân tách rõ ràng về trách nhiệm giữa các tầng.

Backend (DDD)	Frontend (MVVM)
Presentation Layer (Controller)	ViewModel (gọi API)
Application Layer	ViewModel (điều phối usecase UI)
Domain Layer	Không hiển thị trực tiếp
Infrastructure Layer	Không hiển thị trực tiếp
Application Layer (DTO/Response)	Model

Frontend không truy cập trực tiếp Domain Model, chỉ giao tiếp với backend thông qua API, đảm bảo:

- Domain Logic được đảm bảo ở backend.
- Frontend tập trung vào trải nghiệm người dùng

2.2.1.4. Lợi ích của việc kết hợp DDD và MVVM

Việc kết hợp Domain-Driven Design ở backend và MVVM ở frontend mang lại các lợi ích sau:

- Phân tách rõ ràng các trách nhiệm giữa frontend và backend
- Dễ dàng mở rộng hệ thống theo từng phần độc lập
- Thuận lợi cho kiểm thử (đơn vị kiểm thử với ViewModel, domain test backend)
- Tăng khả năng bảo trì và tái sử dụng mã nguồn

- Phù hợp với các hệ thống lớn, có nghiệp vụ phức tạp

2.2.3. UML (Unified Modeling Language)

UML [25] là ngôn ngữ mô hình hóa thống nhất được sử dụng để mô tả, trực quan hóa, và thiết kế hệ thống phần mềm.

Trong dự án, UML được sử dụng để mô tả:

- Use Case Diagram: mô tả các chức năng của hệ thống và vai trò của người dùng.
- Class Diagram: mô tả cấu trúc dữ liệu, các lớp và mối quan hệ giữa chúng.
- Sequence Diagram: mô tả trình tự tương tác giữa các thành phần.

Việc sử dụng UML giúp nhóm phát triển hiểu rõ kiến trúc tổng thể, giảm sai sót trong thiết kế, và dễ bảo trì khi mở rộng hệ thống.

2.2.4. Java Spring Boot

Java Spring Boot [12] là một framework phát triển ứng dụng backend phổ biến, được xây dựng trên nền tảng Spring Framework, nhằm đơn giản hóa quá trình cấu hình và triển khai ứng dụng Java. Spring Boot cho phép lập trình viên nhanh chóng xây dựng các hệ thống backend có hiệu năng cao, dễ mở rộng và dễ bảo trì.

Trong dự án này, Java Spring Boot được sử dụng để xây dựng backend cho hệ thống quản lý phát triển phần mềm theo phương thức sản xuất phần mềm linh hoạt (Agile). Spring Boot đảm nhiệm vai trò xử lý các yêu cầu từ phía client, triển khai các nghiệp vụ cốt lõi, quản lý dữ liệu và tích hợp với các công nghệ khác như MySQL, Neo4j, Keycloak và JobRunr.

Ưu điểm của Java Spring Boot

- Cấu hình đơn giản: cơ chế cài đặt tự động (auto-configuration) [12] giúp giảm thiểu cấu hình thủ công.
- Hiệu năng và ổn định cao: phù hợp với các hệ thống backend quy mô vừa và lớn.
- Hỗ trợ lập trình bất đồng bộ: giúp xử lý nhiều request đồng thời hiệu quả.
- Hệ sinh thái phong phú: dễ dàng tích hợp với các công nghệ như MySQL, Neo4j, Keycloak, và JobRunr.
- Bảo mật cao: hỗ trợ tốt cho các cơ chế xác thực và phân quyền khi kết hợp với Keycloak.
- Dễ triển khai và mở rộng: phù hợp cho các hệ thống phát triển theo Agile.

2.2.5. Flutter

Flutter là một framework mã nguồn mở do Google phát triển [13], được sử dụng để xây dựng giao diện người dùng (UI) đa nền tảng từ một mã nguồn duy nhất. Flutter cho phép phát

triển ứng dụng chạy trên Android, iOS và Web, phù hợp với các hệ thống hiện đại yêu cầu tính linh hoạt và khả năng mở rộng.

Trong dự án này, Flutter được sử dụng để xây dựng frontend cho hệ thống quản lý phát triển phần mềm theo phương thức Agile. Ứng dụng Flutter đảm nhiệm vai trò hiển thị dữ liệu, tương tác với người dùng và giao tiếp với backend thông qua các API được cung cấp bởi hệ thống Java Spring Boot.

Flutter hỗ trợ kiến trúc widget-based, cho phép xây dựng giao diện theo các thành phần độc lập, giúp mã nguồn rõ ràng, dễ bảo trì và tái sử dụng. Nhờ cơ chế cập nhật giao diện linh hoạt, ứng dụng có thể hiển thị động các thông tin như Sprint, user story, tiến độ công việc và mối quan hệ giữa các thành phần trong dự án Agile.

Ưu điểm của Flutter:

- Đa nền tảng: phát triển một lần, triển khai trên nhiều nền tảng (Android, iOS, Web).
- Hiệu năng cao: sử dụng cơ chế render riêng, không phụ thuộc vào WebView.
- Giao diện linh hoạt: dễ dàng xây dựng UI hiện đại và trực quan.
- Tái sử dụng mã nguồn: giảm thời gian phát triển và bảo trì.
- Tích hợp tốt với backend: dễ dàng kết nối với các API REST của Spring Boot.
- Cộng đồng và hệ sinh thái phát triển mạnh: nhiều thư viện hỗ trợ cho việc xây dựng ứng dụng.

2.2.6. Cơ sở dữ liệu quan hệ

Cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) [21] là mô hình lưu trữ phổ biến, trong đó dữ liệu được tổ chức thành các bảng (table) gồm hàng (record) và cột (field). Các bảng liên kết với nhau qua khóa chính (Primary Key) [21] và khóa ngoại (Foreign Key) [21], giúp duy trì mối quan hệ và tính nhất quán dữ liệu.

Trong hệ thống quản lý và phân tích User Story theo phương pháp Agile của đề tài, MySQL được dùng làm hệ quản trị CSDL quan hệ chính. Tuy nhiên, MySQL không được triển khai theo một mô hình duy nhất cho toàn hệ thống, mà được tổ chức theo hai vai trò tương ứng hai tầng dịch vụ:

(1) MySQL nghiệp vụ — Backend Java (master-slave replication)

Tầng backend (be/nckh, Spring Boot) lưu dữ liệu nghiệp vụ có cấu trúc ổn định, quan hệ rõ ràng, phục vụ các thao tác CRUD đồng bộ. Các thực thể chính gồm:

Thực thể	Vai trò
----------	---------

Profile	Hồ sơ người dùng cục bộ (email, username, tham chiếu Keycloak)
Workspace	Không gian làm việc của dự án
Backlog	Product backlog, quan hệ 1–1 với Workspace
Sprint	Vòng đời iteration (tên, trạng thái ToDo / InProgress / Done)
User Story	Mô tả yêu cầu, trạng thái, gắn Workspace / Sprint / Backlog
WorkspaceRole, WorkspaceMember	Vai trò và thành viên trong workspace
WorkspaceInvitation, Notification, ApiKey	Mời thành viên, thông báo, bảo mật API

Để đảm bảo hiệu năng và khả năng mở rộng đọc, backend triển khai MySQL replication master–slave [26]:

- Master: nhận mọi thao tác ghi (tạo/cập nhật/xóa Workspace, Sprint, User Story, ...). Spring Boot định tuyến qua RoutingDataSource và DataSourceContextHolder.
- Slave (read replica): phục vụ các truy vấn đọc, giảm tải cho master khi số lượng truy vấn tăng.

Cấu hình master/slave được khai báo trong application.yml (spring.datasource.master, spring.datasource.slave); môi trường local có thể dựng bằng Docker Compose (mysql-master-daln, mysql-slave-daln) với binlog replication.

Kiến trúc này phù hợp nghiệp vụ Agile: dữ liệu có schema cố định, cần ACID [21] khi tạo story, chuyển sprint, cập nhật trạng thái; đồng thời tách luồng đọc để hệ thống scale theo hướng đọc nhiều hơn ghi.

(2) MySQL phân tích — Service Python analyze_user_stories (instance đơn):

Service phân tích (analyze_user_stories, FastAPI) sử dụng MySQL thông thường (một instance, kết nối qua DATABASE_URL), không triển khai master–slave riêng. Trên instance này lưu các bảng phát sinh từ pipeline NLP/ML:

- AnalyzeStoryResult — kết quả trích xuất S-V-O và bản canonical
- KnowledgeTerm — catalog thuật ngữ chuẩn hóa theo workspace
- KnowledgeRelation — quan hệ giữa các term (luật kết hợp)
- AnalyzeStatistic — tần suất term theo sprint

Service Python đọc dữ liệu nghiệp vụ user_story từ MySQL slave của backend, không ghi ngược lên schema nghiệp vụ. Như vậy:

- Backend vẫn là nguồn sự thật (source of truth) cho User Story.
- Service phân tích ghi kết quả phân tích vào MySQL riêng và đồng bộ đồ thị sang Neo4j.
- Luồng đọc story qua replica tránh cạnh tranh tải ghi với master khi consumer RabbitMQ [20] xử lý hàng loạt sự kiện USER_STORY_CREATED.

(3) Vai trò giữa MySQL và Neo4j:

Khi người dùng tạo User Story, di chuyển story (start/complete sprint) hoặc kích hoạt rebuild graph, hệ thống cần phân tích mối quan hệ phụ thuộc, tương đồng ngữ nghĩa, trùng lặp và ưu tiên — dữ liệu có quan hệ phức tạp, đa chiều, phù hợp mô hình đồ thị hơn mô hình bảng nhiều JOIN.

Do đó, đề tài áp dụng kiến trúc hybrid store:

Store	Vai trò	Triển khai trong đề tài
MySQL (BE, master–slave)	Dữ liệu nền và nghiệp vụ Agile	Ghi trên master; đọc có thể qua slave
MySQL (analyze, đơn)	Kết quả phân tích, kho tri thức	Ghi/đọc trên một instance
Neo4j	Đồ thị tri thức sau phân tích	Node Term, UserStory; cạnh PERFORM, TARGET, SIMILAR, ASSOCIATED, REDUNDANT_WITH

Sự kiện nghiệp vụ trên MySQL (sau commit) được phát qua RabbitMQ; consumer Python parse, lưu MySQL phân tích và cập nhật Neo4j bất đồng bộ, không chặn API đồng bộ của người dùng.

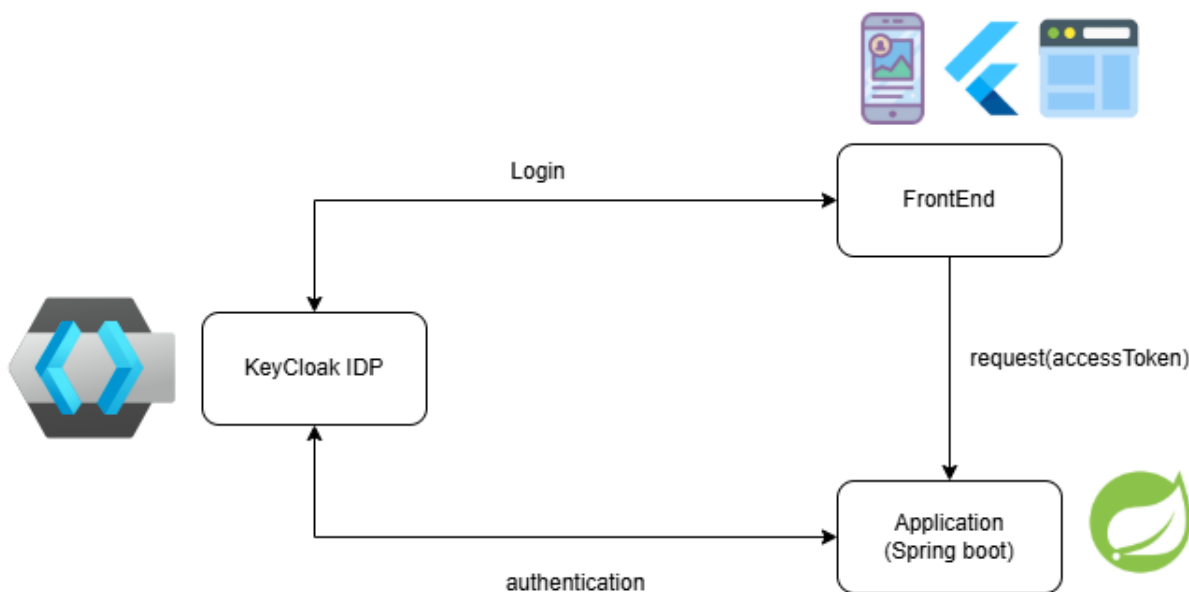
Ưu điểm của cơ sở dữ liệu quan hệ (MySQL):

- ACID: đảm bảo toàn vẹn khi tạo workspace, gán sprint, cập nhật trạng thái story.
- SQL: truy vấn CRUD, báo cáo, aggregation quen thuộc; tích hợp tốt với Spring Boot + JPA/Hibernate.
- Cấu trúc ổn định: phù hợp Profile, Workspace, Sprint, User Story và ràng buộc FK nghiệp vụ.
- Replication master–slave (tầng BE): tăng throughput đọc, hỗ trợ service phân tích đọc replica mà không ảnh hưởng ghi nghiệp vụ.

Ưu điểm của cơ sở dữ liệu đồ thị (Neo4j):

- Biểu diễn quan hệ tự nhiên: node và relationship phản ánh trực quan chuỗi Subject → Action → Object, tương đồng term, luật kết hợp, cặp story trùng lặp — thay vì nhiều JOIN sâu trên SQL.
- Hiệu suất duyệt quan hệ: index-free adjacency [3] hỗ trợ traversal nhiều bước (multi-hop) ổn định hơn khi số JOIN SQL tăng.
- Schema linh hoạt: bổ sung loại cạnh hoặc thuộc tính (ví dụ source, story_id, workspace_id trên cạnh) mà không refactor toàn schema quan hệ.
- Cypher [3] trực quan: phù hợp truy vấn đồ thị workspace/sprint/backlog; backend expose qua GraphQL [17].
- Graph algorithms: Neo4j GDS [4] hỗ trợ degree [11], betweenness [11], ... phục vụ tính ưu tiên và phân tích cấu trúc (đã dùng trong pipeline batch).

2.2.7. KeyCloak



Hình 2: Sơ đồ KeyCloak

Keycloak [14] là một nền tảng Identity and Access Management (IAM) mã nguồn mở, cung cấp các chức năng xác thực (Authentication) và phân quyền (Authorization) cho ứng dụng. Keycloak hỗ trợ các chuẩn bảo mật phổ biến như OAuth 2.0, OpenID Connect (OIDC) [15] và JSON Web Token (JWT) [16], giúp việc quản lý người dùng và bảo mật hệ thống trở nên tập trung, linh hoạt và an toàn.

Trong dự án này, Keycloak được sử dụng để quản lý xác thực và phân quyền người dùng cho hệ thống quản lý phát triển phần mềm theo phương thức Agile. Thay vì tự triển khai cơ chế đăng nhập và quản lý phiên, hệ thống ủy quyền toàn bộ quá trình xác thực cho Keycloak, từ đó giảm độ phức tạp và tăng tính bảo mật.

Keycloak được áp dụng cho các chức năng sau:

- Authentication: Quản lý đăng ký, đăng nhập và xác thực người dùng thông qua cơ chế cấp phát JWT Access Token.
- Authorization: Quản lý vai trò (role) và quyền truy cập của người dùng trong hệ thống.
- Session Management: Quản lý phiên đăng nhập tập trung, hỗ trợ đăng xuất, hết hạn phiên và đăng nhập một lần (Single Sign-On – SSO).
- Token Validation: Backend sử dụng Java Spring Boot để kiểm tra và xác thực token do Keycloak cấp, đảm bảo chỉ người dùng hợp lệ mới có quyền truy cập các API.

Việc tích hợp Keycloak giúp hệ thống tách biệt rõ ràng giữa xử lý nghiệp vụ và xử lý bảo mật, đồng thời tăng khả năng mở rộng khi số lượng người dùng tăng cao hoặc khi tích hợp thêm các ứng dụng khác trong tương lai.

Ưu điểm của Keycloak

- Hỗ trợ đầy đủ các chuẩn xác thực và phân quyền hiện đại (OAuth 2.0, OIDC, JWT).
- Quản lý người dùng, vai trò và quyền truy cập một cách tập trung.
- Dễ dàng tích hợp với các ứng dụng backend như Spring Boot.
- Hỗ trợ Single Sign-On (SSO), nâng cao trải nghiệm người dùng.
- Giảm thiểu rủi ro bảo mật so với việc tự xây dựng hệ thống xác thực thủ công.

Việc sử dụng Keycloak trong dự án góp phần nâng cao tính an toàn, nhất quán và khả năng mở rộng của hệ thống, đồng thời đảm bảo quá trình phát triển phần mềm Agile được vận hành trên một nền tảng bảo mật vững chắc.

2.2.8. JobRunr

JobRunr [27] là một thư viện mã nguồn mở hỗ trợ xử lý tác vụ nền (background job processing) dành cho các ứng dụng Java, đặc biệt phù hợp với các hệ thống sử dụng Spring Boot. JobRunr cho phép thực thi các công việc bất đồng bộ ngoài luồng xử lý chính của ứng dụng, giúp cải thiện hiệu năng và khả năng phản hồi của hệ thống.

Trong dự án này, JobRunr được sử dụng để xử lý các tác vụ nền liên quan đến hệ thống quản lý phát triển phần mềm theo phương thức Agile, cụ thể bao gồm:

- Gửi email dưới nền: gửi email thông báo khi người dùng được mời vào Workspace, khi Sprint được bắt đầu hoặc khi user story thay đổi trạng thái.
- Xử lý tác vụ bất đồng bộ: thực hiện các công việc không yêu cầu phản hồi ngay lập tức, tránh làm chậm luồng xử lý chính của hệ thống.
- Lập lịch công việc: hỗ trợ thực hiện các tác vụ theo lịch (scheduled jobs) hoặc trì hoãn (delayed jobs) như gửi nhắc nhở tiến độ Sprint.

Ưu điểm của JobRunr

- Tích hợp tốt với Java Spring Boot.
- Hỗ trợ thực thi tác vụ nền, lập lịch công việc và xử lý bất đồng bộ hiệu quả.

- Cho phép theo dõi, quản lý và retry các job khi xảy ra lỗi.
- Giảm tải cho luồng xử lý chính, nâng cao hiệu suất và khả năng phản hồi của hệ thống.
- Phù hợp với các hệ thống có nhu cầu gửi email, xử lý thông báo hoặc các tác vụ nền khác.

2.2.9. Docker

Docker [28] là một nền tảng mã nguồn mở cho phép đóng gói và triển khai ứng dụng cùng với toàn bộ môi trường và các phụ thuộc của nó dưới dạng Container [28]. Việc sử dụng Docker giúp đảm bảo ứng dụng hoạt động nhất quán trong các môi trường khác nhau như máy cá nhân, môi trường phát triển và môi trường triển khai.

Trong dự án này, Docker chưa được sử dụng để đóng gói toàn bộ hệ thống, mà được áp dụng ở mức hỗ trợ hạ tầng, cụ thể cho các dịch vụ bên thứ ba, bao gồm:

- Keycloak: triển khai dịch vụ xác thực và phân quyền người dùng dưới dạng container.
- Neo4j: triển khai cơ sở dữ liệu đồ thị phục vụ phân tích mối quan hệ trong hệ thống Agile.

2.2.10. WebSocket

WebSocket [29] là một giao thức truyền thông cho phép thiết lập kết nối hai chiều (bidirectional) [29] và duy trì liên tục giữa client và server. Không giống mô hình HTTP truyền thống (request–response), WebSocket cho phép server chủ động gửi dữ liệu đến client ngay khi có sự kiện xảy ra, từ đó đáp ứng tốt các yêu cầu thời gian thực (real-time).

Trong dự án này, Socket.IO [30] được sử dụng như một thư viện hỗ trợ xây dựng cơ chế giao tiếp real-time dựa trên WebSocket. Socket.IO cung cấp các cơ chế quản lý kết nối, xử lý sự kiện và fallback, giúp việc triển khai giao tiếp thời gian thực trở nên đơn giản và ổn định hơn.

Cụ thể, Socket.IO được sử dụng để:

- Gửi sự kiện thời gian thực từ client đến backend khi xảy ra các hành động quan trọng như mời thành viên vào Workspace hoặc thay đổi trạng thái Sprint.
- Kích hoạt quy trình gửi email: khi backend nhận được sự kiện qua Socket.IO, hệ thống sẽ tạo background job thông qua JobRunr để gửi email thông báo dưới nền.
- Đảm bảo phản hồi nhanh cho người dùng mà không làm gián đoạn luồng xử lý chính của hệ thống.

Cơ chế này giúp tách biệt rõ ràng giữa:

- Xử lý sự kiện thời gian thực (WebSocket / Socket.IO)
- Xử lý tác vụ nặng và chậm (gửi email) thông qua JobRunr

Nhờ đó, hệ thống vừa đảm bảo tính thời gian thực (real-time), vừa duy trì được hiệu năng và độ ổn định khi số lượng người dùng tăng lên.

Ưu điểm của WebSocket và Socket.IO

- Hỗ trợ giao tiếp hai chiều, thời gian thực giữa client và server
- Giảm số lượng request HTTP(s), tiết kiệm tài nguyên hệ thống.
- Dễ dàng triển khai cơ chế event-driven trong ứng dụng.
- Phù hợp cho các chức năng thông báo, cập nhật trạng thái và kích hoạt xử lý nền.
- Kết hợp hiệu quả với JobRunr để xử lý các tác vụ bất đồng bộ như gửi email.

2.2.11. Simple Mail Transfer Protocol (SMTP)

2.2.11.1. Khái niệm

SMTP [31] là giao thức tiêu chuẩn được sử dụng để gửi thư qua mạng. SMTP hoạt động theo mô hình client-server, trong đó mail client hoặc application server sẽ gửi yêu cầu đến mail server để chuyển tiếp email đến người nhận

2.2.11.2 Vai trò của SMTP

Trong ứng dụng, SMTP được sử dụng để gửi các email thông báo đến người dùng khi xảy ra các sự kiện trong hệ thống, ví dụ:

- Thông báo khi người dùng được mời tham gia Workspace
- Thông báo khi thay đổi trạng thái Sprint
- Thông báo khi có cập nhật liên quan đến User Story hoặc Product Backlog

2.2.11.3. SMTP kết hợp với xử lý bất đồng bộ

Để tránh ảnh hưởng đến hiệu năng của hệ thống, việc gửi email sẽ không được xử lý trực tiếp trong luồng request chính. Thay vào đó, hệ thống áp dụng cơ chế xử lý bất đồng bộ thông qua JobRunr.

Quy trình xử lý:

1. Client gửi sự kiện thông qua WebSocket hoặc API.
2. Backend tiếp nhận sự kiện và tạo background job.
3. JobRunr thực hiện gửi email thông qua SMTP.
4. Email được gửi đến người dùng mà không gián đoạn luồng xử lý chính.

Cơ chế này giúp hệ thống:

- Tăng tốc phản hồi cho người dùng
- Giảm tải cho server khi gửi email số lượng lớn
- Đảm bảo khả năng mở rộng hệ thống

2.2.12. GraphQL

2.2.12.1. Khái niệm

GraphQL [17] là một ngôn ngữ truy vấn dữ liệu và runtime cho API do Facebook phát triển. GraphQL cho phép client chủ động xác định chính xác dữ liệu cần lấy từ server, thay vì server trả về dữ liệu cố định như trong Rest API.

GraphQL sử dụng một endpoint duy nhất để xử lý nhiều loại truy vấn khác nhau, giúp tối ưu hóa việc truyền tải dữ liệu và giảm số lượng yêu cầu (Request).

2.2.12.2. Vai trò của GraphQL trong hệ thống

Trong dự án, GraphQL được sử dụng để cung cấp API truy vấn dữ liệu liên quan đến user story, Backlog, Sprint và các thông tin liên quan đến graph dữ liệu.

GraphQL cho phép:

- Truy vấn dữ liệu linh hoạt theo nhu cầu hiển thị của frontend
- Giảm tình trạng lấy dữ liệu dư thừa (over-fetching)
- Giảm tình trạng thiếu dữ liệu cần thiết (under-fetching)
- Hỗ trợ truy vấn dữ liệu có cấu trúc phức tạp như dữ liệu dạng đồ thị

2.2.12.3. GraphQL kết hợp với kiến trúc hệ thống

GraphQL được triển khai ở backend như một lớp API giao tiếp giữa frontend và domain logic của hệ thống.

Quy trình hoạt động:

1. Frontend gửi truy vấn GraphQL đến server.
2. GraphQL Resolver tiếp nhận truy vấn và gọi Application Layer.
3. Application Layer xử lý nghiệp vụ thông qua Domain Layer.
4. Dữ liệu được trả về theo đúng cấu trúc mà client yêu cầu.

Cơ chế này giúp:

- Tách biệt rõ ràng giữa API và logic nghiệp vụ
- Tăng khả năng mở rộng hệ thống
- Tối ưu hiệu suất truyền tải dữ liệu

2.2.12.4. Ưu điểm của GraphQL

- Cho phép client kiểm soát dữ liệu trả về
- Giảm số lượng endpoint cần xây dựng
- Phù hợp với hệ thống có dữ liệu phức tạp
- Hỗ trợ tốt cho hệ thống đồ thị
- Tăng hiệu quả khi kết hợp với frontend

2.2.13. NLP & Dependency Parsing for User Story

Trong các hệ thống Agile, User Story thường được viết dưới dạng ngôn ngữ tự nhiên, phổ biến theo mẫu:

“As a <role>, I want to <action> <object>, so that <benefit>.” [23]

Tuy nhiên, dữ liệu thực tế có thể đa dạng về cấu trúc câu, từ đồng nghĩa, hoặc chứa nhiều mệnh đề phụ. Vì vậy, cần áp dụng xử lý ngôn ngữ tự nhiên (NLP) [32] để chuẩn hóa và trích xuất thông tin có cấu trúc.

Dependency Parsing [18], [32] là kỹ thuật phân tích cú pháp phụ thuộc giữa các từ trong câu, cho phép xác định quan hệ ngữ pháp như chủ ngữ (subject), động từ chính (verb), tân ngữ (object), mệnh đề phụ (clause), v.v.

Trong bài toán này, Dependency Parsing được dùng để trích xuất bộ ba S–V–O (Subject–Verb–Object) [18] từ User Story.

Quy trình tổng quát:

1. Tiền xử lý văn bản đầu vào (chuẩn hóa khoảng trắng, chữ hoa/thường).
2. Phân tích dependency để xác định mệnh đề chính.
3. Tách subject, action, object từ mệnh đề chính.

4. Áp dụng cơ chế fallback cho mệnh đề phụ (ví dụ advcl) khi object không nằm ở mệnh đề chính.
5. Trả kết quả theo cấu trúc chuẩn phục vụ các bước phân tích tiếp theo.

Việc chuyển User Story từ dạng tự do sang dạng SVO giúp giảm nhiễu ngôn ngữ, tăng tính nhất quán và là nền tảng cho các bước semantic normalization, rule mining và đánh giá mức độ ưu tiên.

2.2.14. *Semantic Normalization và Canonical Mapping*

Sau khi trích xuất SVO, các từ hành động/đối tượng thường tồn tại nhiều biến thể gần nghĩa (ví dụ: create, register, sign up). Nếu không chuẩn hóa, cùng một ý nghĩa nghiệp vụ sẽ bị tách thành nhiều nút khác nhau trong đồ thị, làm giảm chất lượng phân tích.

Semantic Normalization nhằm gom các thuật ngữ gần nghĩa về một đại diện chuẩn (canonical term), gồm các bước:

- Tính độ tương tự ngữ nghĩa giữa các cặp từ.
- Đánh giá thêm mức tương đồng ngữ cảnh (context overlap), ví dụ mức giao nhau của tập object đi kèm action.
 - Phân loại cặp từ thành:
 - Auto-merge: đủ tin cậy để tự động gộp.
- Ambiguous: cần xem xét thêm.
- Dùng cấu trúc Union-Find (Disjoint Set Union) [9] để hợp nhất các từ thuộc cùng nhóm và tạo ánh xạ canonical.

Kết quả cuối cùng là canonical map: mỗi từ gốc được ánh xạ về một đại diện chuẩn.

Lợi ích chính:

1. Giảm trùng lặp thuật ngữ.
2. Tăng độ ổn định khi tạo transaction và luật kết hợp.
3. Cải thiện chất lượng graph analytics và redundancy classification.

2.2.15. *Association Rule Mining (Apriori)*

Để phát hiện các mẫu đồng xuất hiện giữa các thành phần nghiệp vụ trong User Story, hệ thống sử dụng Association Rule Mining với thuật toán Apriori [5].

Mỗi User Story được biểu diễn thành một transaction gồm các token đã chuẩn hóa (subject, action, object). Từ tập transaction này, Apriori tìm các frequent itemsets theo ngưỡng hỗ trợ tối thiểu.

Sau đó sinh các luật dạng:

$$X \Rightarrow Y$$

với ba chỉ số chính [5]:

- Support: tần suất $X \cup Y$ xuất hiện trong toàn bộ transaction.
- Confidence: xác suất xuất hiện Y khi đã có X.
- Lift: mức độ phụ thuộc giữa X và Y, với:
 - lift > 1: tương quan dương (đồng xuất hiện mạnh hơn ngẫu nhiên),
 - lift = 1: độc lập,

- $lift < 1$: tương quan âm.

Trong đề tài, luật kết hợp được dùng như một tín hiệu tri thức nghiệp vụ bổ sung cho mô hình phân loại redundant và tinh chỉnh mức ưu tiên backlog.

2.2.16. Graph Centrality for Story Prioritization

Cơ sở dữ liệu đồ thị cho phép mô hình hóa các thực thể và quan hệ của User Story dưới dạng node/relationship. Trên đồ thị này, mức độ “quan trọng” của một story có thể được suy ra từ vị trí cấu trúc của các node liên quan.

Hai độ đo centrality được sử dụng:

1. Degree Centrality [4]: số lượng liên kết trực tiếp của một node.
2. Betweenness Centrality [4]: mức độ node nằm trên các đường đi ngắn nhất giữa các node khác (vai trò cầu nối).

Với mỗi story, hệ thống tổng hợp điểm cấu trúc từ các object liên quan, ví dụ:

$$x_s = 0.5 \cdot \overline{degree}_s + 0.5 \cdot \overline{betweenness}_s$$

Sau đó áp dụng chuẩn hóa phi tuyến qua hàm sigmoid [33] để đưa về miền (0, 1):

$$priority(s) = \frac{1}{1 + e^{-(\alpha x_s + \beta)}}$$

Cách tiếp cận này cho phép ưu tiên các story có liên kết mạnh hoặc giữ vai trò “trung tâm” trong miền nghiệp vụ, thay vì chỉ dựa vào đánh giá cảm tính.

2.2.17. Weakly-supervised Redundancy Classification

Gán nhãn trùng lặp thủ công cho mọi cặp User Story là tốn kém. Vì vậy, đề tài áp dụng weak supervision [10]: tạo nhãn yếu bằng heuristic rồi dùng để huấn luyện mô hình phân loại.

Quy trình:

1. Sinh tập cặp story theo cách pairwise.
2. Trích xuất đặc trưng cho từng cặp, gồm:
 - mức giống nhau của subject/action/object,
 - semantic similarity,
 - rule confidence/lift,
 - chênh lệch priority.
3. Gán weak labels:
 - Nhãn dương khi tương đồng cao + có tín hiệu luật mạnh.
 - Nhãn âm khi tương đồng thấp + tín hiệu luật yếu.
 - Các trường hợp chưa rõ để unlabeled.
4. Huấn luyện các baseline classifier (ví dụ Logistic Regression, Random Forest, HistGradientBoosting) thuộc thư viện scikit-learn [19].
5. Dự đoán xác suất redundant và so ngưỡng để ra quyết định.

Ngoài ra, các cặp redundant có thể được gom cụm bằng **Union-Find [9]** để tạo nhóm story trùng lặp theo thành phần liên thông.

Cách làm này giúp hệ thống mở rộng tốt trong bối cảnh thiếu dữ liệu gán nhãn chuẩn.

2.2.18. Hybrid Analytical Pipeline (MySQL -> NLP/ML -> Neo4j)

Đề tài sử dụng mô hình pipeline lai, kết hợp dữ liệu quan hệ và phân tích đồ thị:

1. MySQL lưu dữ liệu nghiệp vụ gốc (workspace, sprint, user story, trạng thái, thông tin người dùng).
2. NLP layer trích xuất SVO và chuẩn hóa ngữ nghĩa từ User Story.
3. Mining/ML layer:
 - khai phá luật kết hợp (Apriori) [5],
 - tính điểm ưu tiên cấu trúc,
 - phân loại redundant theo weak supervision.
4. **Neo4j** lưu tri thức phân tích dưới dạng graph (term, quan hệ tương đồng, luật, redundancy links), phục vụ truy vấn liên kết sâu và trực quan hóa.
5. Kết quả phân tích được phản hồi về backlog để hỗ trợ:
 - sắp xếp ưu tiên,
 - phát hiện trùng lặp,
 - tăng chất lượng lập kế hoạch sprint.

Pipeline này tận dụng thế mạnh của từng thành phần:

- **MySQL** cho dữ liệu giao dịch ổn định
- **NLP/ML** cho suy luận từ văn bản
- **Neo4j** cho phân tích quan hệ phức tạp và giải thích trực quan.

2.3. Các công trình, nghiên cứu liên quan

Bảng 1: Các công trình, nghiên cứu liên quan

Hệ thống / Nghiên cứu	Mô tả	Ưu điểm	Nhược điểm
Jira	Jira Software là một công cụ quản lý dự án Agile phổ biến, hỗ trợ Scrum và Kanban, cho phép quản lý Sprint, Backlog, user story và task.	Hỗ trợ đầy đủ các quy trình Agile và Scrum. Giao diện trực quan, nhiều báo cáo theo dõi tiến độ. Khả năng mở rộng cao và tích hợp với nhiều công cụ khác.	Kiến trúc phức tạp, khó tùy chỉnh sâu. Chi phí sử dụng cao đối với các nhóm lớn

Azure DevOps Boards	Azure DevOps Boards là hệ thống quản lý công việc Agile do Microsoft phát triển, hỗ trợ quản lý Backlog, Sprint và theo dõi tiến độ phát triển phần mềm.	Tích hợp tốt với hệ sinh thái DevOps. Hỗ trợ Agile ở quy mô doanh nghiệp. Giao diện rõ ràng, dễ theo dõi tiến độ	Phụ thuộc nhiều vào hệ sinh thái Microsoft. Dữ liệu được lưu trữ theo mô hình quan hệ, chưa tối ưu cho phân tích quan hệ phức tạp.
Dự án hiện tại	Dự án của nhóm tập trung xây dựng một hệ thống hỗ trợ quản lý phát triển phần mềm theo phương thức Agile, kết hợp Java Spring Boot, Flutter và Graph Database (Neo4j) để nâng cao hiệu suất xử lý và phân tích dữ liệu.	Kết hợp cơ sở dữ liệu quan hệ (MySQL) và cơ sở dữ liệu đồ thị (Neo4j) theo đúng vai trò. Mô hình hóa các mối quan hệ giữa Sprint, User Story và Member.	Quy mô nhỏ, chủ yếu ở mức Prototype. Chưa triển khai các chức năng nâng cao như dự đoán tiến độ hoặc AI hỗ trợ.

2.4. Công nghệ, ngôn ngữ và công cụ sử dụng

Bảng 2: Công nghệ, ngôn ngữ và công cụ sử dụng

Loại	Công nghệ / Công cụ	Vai trò
Ngôn ngữ lập trình	Java & Dart	Ngôn ngữ chính cho backend & frontend.
Ngôn ngữ lập trình	Python	Xây dựng dịch vụ phân tích User Story (NLP/ML)
Framework Backend	Java Spring Boot	Xây dựng API của dự án
Frontend	Flutter	Xây dựng giao diện người dùng
Framework API	FastAPI, Uvicorn	Cung cấp API phân tích và chạy service Python
Cơ sở dữ liệu quan hệ và cơ sở dữ liệu đồ thị	MySQL & Neo4J	Lưu dữ liệu.

ORM / DB Access	SQLAlchemy, PyMySQL	Kết nối và truy xuất dữ liệu MySQL
NLP	spaCy, en_core_web_sm	Dependency parsing, trích xuất Subject-Verb-Object
Semantic Similarity	Word2Vec	Tính tương đồng ngữ nghĩa và hỗ trợ canonical mapping
Data Mining	mlxtend (Apriori, association_rules)	Khai phá luật kết hợp support/confidence/lift
Machine Learning	scikit-learn	Phân loại trùng lặp User Story (redundancy classification)
Phân tích dữ liệu	pandas, numpy, scipy	Tiền xử lý, feature engineering, tính toán số liệu
Graph Integration	Neo4j Python Driver, Neo4j GDS	Lưu đồ thị tri thức và chạy thuật toán centrality
Thao tác chạy nền (Background Job)	JobRunr	Là 1 job queue hỗ trợ cho những thao tác chạy dưới nền bất đồng bộ
Message Queue	RabbitMQ, pika	Đồng bộ tác vụ phân tích theo cơ chế hàng đợi
Quản lý mã nguồn	Git, GitHub	Quản lý phiên bản mã nguồn và làm việc nhóm
Môi trường phát triển	IntelliJ IDEA & Visual Studio Code	IDE chính cho viết mã
Công cụ UML	Draw.io và Visual Paradigm	Vẽ biểu đồ UML và biểu đồ kiến trúc hệ thống

Chương 3: Phân tích hệ thống

3.1. Khảo sát hiện trạng

3.1.1. Quy trình nghiệp vụ hiện tại

Hiện nay, trong các phần mềm phát triển linh hoạt (Agile), các công cụ quản lý như Jira, Trello, Azure DevOps được sử dụng phổ biến để hỗ trợ việc quản lý **Workspace, Sprint, Product Backlog, và User Story**. Các hệ thống này cho phép người dùng tạo và quản lý user story, phân công công việc, theo dõi trạng thái và đánh giá tiến độ phát triển phần mềm theo từng Sprint. Tuy nhiên, phần lớn các hệ thống hiện tại chủ yếu dựa vào cơ sở dữ liệu quan hệ, trong đó dữ liệu được tổ chức dưới dạng bảng. Ngoài ra, các tác vụ như phân tích user story hay tổng hợp dữ liệu khi bắt đầu một Sprint thường được xử lý theo cách tuần tự hoặc thủ công. Do đó, thực trạng hiện nay đặt ra nhu cầu về hệ thống có khả năng hỗ trợ phân tích mối quan hệ giữa các user story.

3.1.2. Các vấn đề, hạn chế

Mô hình nguyên khối (Monolithic) truyền thống gặp một số hạn chế:

- Khi một module gặp sự cố, có thể ảnh hưởng đến toàn bộ hệ thống.
- Việc bảo trì và cập nhật tính năng mới mất nhiều thời gian, vì phải triển khai lại toàn bộ ứng dụng.
- Tốc độ phản hồi giảm khi lưu lượng truy cập tăng đột biến
- Thiếu khả năng phân tán công việc và tái sử dụng, gây lãng phí tài nguyên máy chủ.

Giải pháp của đề tài là phát triển hệ thống theo mô hình monolithic tối ưu, kết hợp Domain-Driven Design (DDD), trong đó:

- Áp dụng DDD để tách biệt rõ ràng các tầng Presentation (Trình diễn), Application (Ứng dụng), Domain và Infrastructure (Miền và cơ sở hạ tầng), giúp hệ thống tập trung vào nghiệp vụ cốt lõi, dễ bảo trì và mở rộng.
- Sử dụng cơ sở dữ liệu quan hệ (MySQL) để lưu trữ các dữ liệu nghiệp vụ có cấu trúc rõ ràng như Profile, Workspace, Sprint và User Story.
- Kết hợp cơ sở dữ liệu đồ thị (Neo4j) để lưu trữ và phân tích các dữ liệu phát sinh sau khi bắt đầu Sprint, đặc biệt là các mối quan hệ và sự tương đồng giữa các user story, nhằm hỗ trợ phân tích nghiệp vụ hiệu quả hơn.
- Ứng dụng JobRunr để xử lý các tác vụ nền (background jobs) như phân tích user story hoặc gửi email thông báo, giúp giảm tải cho luồng xử lý chính và cải thiện tốc độ phản hồi của hệ thống.
- Tích hợp Keycloak để quản lý xác thực và phân quyền tập trung, nâng cao tính bảo mật và khả năng mở rộng trong tương lai.

3.2. Phân tích yêu cầu

3.2.1 Yêu cầu nghiệp vụ

Hệ thống được xây dựng nhằm nâng cao hiệu suất phát triển phần mềm Agile thông qua hai nhóm nghiệp vụ chính: **quản lý quy trình và phân tích hỗ trợ quyết định**.

Nhóm nghiệp vụ quản lý quy trình

- Quản lý tài khoản và xác thực người dùng.
- Quản lý Workspace và thành viên theo vai trò.
- Quản lý Sprint và vòng đời Sprint.
- Quản lý User Story trong Product Backlog và Sprint Backlog.

Nhóm nghiệp vụ phân tích hỗ trợ quyết định

- Khi Sprint được bắt đầu, hệ thống tự động phân tích tập User Story thuộc phạm vi workspace/sprint.
- Trích xuất tri thức từ User Story để xây dựng đồ thị quan hệ.
- Tính điểm ưu tiên cho từng User Story nhằm hỗ trợ sắp xếp backlog.
- Phát hiện các cặp User Story có khả năng trùng lặp/đồng nghĩa để giảm dư thừa và cải thiện chất lượng backlog.
- Cung cấp kết quả phân tích dưới dạng dữ liệu truy vấn và trực quan hóa đồ thị để Product Owner/Scrum Master ra quyết định nhanh hơn.

3.2.2. Yêu cầu chức năng

- **Authentication & User Management**
 - Đăng ký/đăng nhập thông qua Keycloak.
 - Backend xác minh access token cho các API cần bảo vệ.
 - Quản lý hồ sơ người dùng (thông tin hiển thị, email, vai trò).
 - Theo dõi thời gian tạo/cập nhật dữ liệu.
- **Workspace Management**
 - Tạo, cập nhật, xóa workspace.
 - Quản lý thành viên workspace (mời, thêm/xóa, phân vai trò).
 - Truy vấn danh sách workspace theo người dùng.
- **Sprint Management**
 - Tạo sprint, cập nhật thông tin sprint.
 - Bắt đầu/kết thúc sprint.
 - Thêm/xóa user story vào/ra sprint.
- **User Story Management**
 - Tạo, cập nhật, xóa user story.
 - Cập nhật trạng thái user story (To Do, In Progress, Done).
 - Truy vấn danh sách backlog theo workspace/sprint.
- **User Story Analysis Pipeline**

- Kích hoạt phân tích khi người dùng gửi danh sách user stories hoặc bắt đầu sprint hoặc theo yêu cầu.
- Tiền xử lý và trích xuất thành phần nghiệp vụ từ user story.
- Chuẩn hóa ngữ nghĩa/canonical mapping để giảm trùng thuật ngữ.
- Xây dựng luật kết hợp từ dữ liệu story.
- Tính điểm ưu tiên cho story dựa trên phân tích đồ thị.
- Phân loại trùng lặp cặp story và nhóm story redundant.
- Lưu kết quả phân tích vào Neo4j để phục vụ truy vấn đồ thị.
- **Result Query & Visualization Support**
 - Truy vấn điểm ưu tiên story (priority ban đầu, priority tinh chỉnh, priority cuối).
 - Truy vấn các cặp redundant có xác suất cao (top-K).
 - Truy vấn group redundant theo story.
 - Truy vấn thống kê mô hình phân loại (model name, metrics).
- **Background Job & Notification**
 - Chạy các tác vụ nặng dưới dạng background job (phân tích, gửi thông báo).
 - Theo dõi trạng thái job; hỗ trợ retry khi lỗi tạm thời.
 - Gửi thông báo tiến trình/kết quả phân tích theo sự kiện hệ thống.
- **Data Management**
 - MySQL lưu dữ liệu nghiệp vụ có cấu trúc ổn định.
 - Neo4j lưu dữ liệu tri thức phân tích và quan hệ đồ thị.
 - Đồng bộ khóa định danh giữa hai hệ dữ liệu để đảm bảo truy vết.

3.2.3. Yêu cầu phi chức năng

- **Hiệu năng:**
 - API quản lý nghiệp vụ phản hồi nhanh trong tải bình thường.
 - Tác vụ phân tích chạy bất đồng bộ, không chặn request chính.
 - Truy vấn đồ thị phải tối ưu cho các câu hỏi liên kết sâu.
- **Tính sẵn sàng:**
 - Hệ thống hỗ trợ vận hành liên tục cho chức năng quản lý chính.
 - Cơ chế retry cho background jobs để hạn chế mất tác vụ.
- **Bảo mật:**
 - Xác thực tập trung qua Keycloak; API bảo vệ bằng token.
 - Phân quyền theo vai trò ở mức workspace/chức năng.
 - Không lộ thông tin nhạy cảm trong log và phản hồi lỗi.
 - Kênh truyền thông cần sử dụng giao thức an toàn trong môi trường triển khai.
- **Khả năng mở rộng (Scalability):**
 - Có thể tăng năng lực xử lý tác vụ phân tích khi số lượng story/sprint tăng.
 - Mô hình tách dữ liệu nghiệp vụ (MySQL) và dữ liệu phân tích (Neo4j) để mở rộng độc lập.

- Dễ thêm thuật toán phân tích mới mà không ảnh hưởng luồng nghiệp vụ chính.
- **Khả năng phục hồi:**
 - Job được lưu trạng thái và có thể tiếp tục xử lý sau sự cố tạm thời.
 - Lỗi phân tích không làm gián đoạn chức năng CRUD cốt lõi của hệ thống.
 - Có cơ chế backup/restore cho dữ liệu quan trọng.
- **Tính nhất quán dữ liệu:**
 - Dữ liệu phân tích phải gắn đúng workspace_id, story_id.
 - Chuẩn hóa key cặp term/story để tránh trùng lệch do thứ tự hoặc khác biệt chữ hoa/thường.
- **Khả năng quan sát (Observability):**
 - Ghi log theo từng bước pipeline phân tích.
 - Có chỉ số theo dõi: số story xử lý, số cặp redundant, thời gian job, tỉ lệ lỗi/retry.

3.3. Mô hình ca sử dụng (Use Case Model)

3.3.1. Danh sách Use Case

Bảng 3: Danh sách Use Case

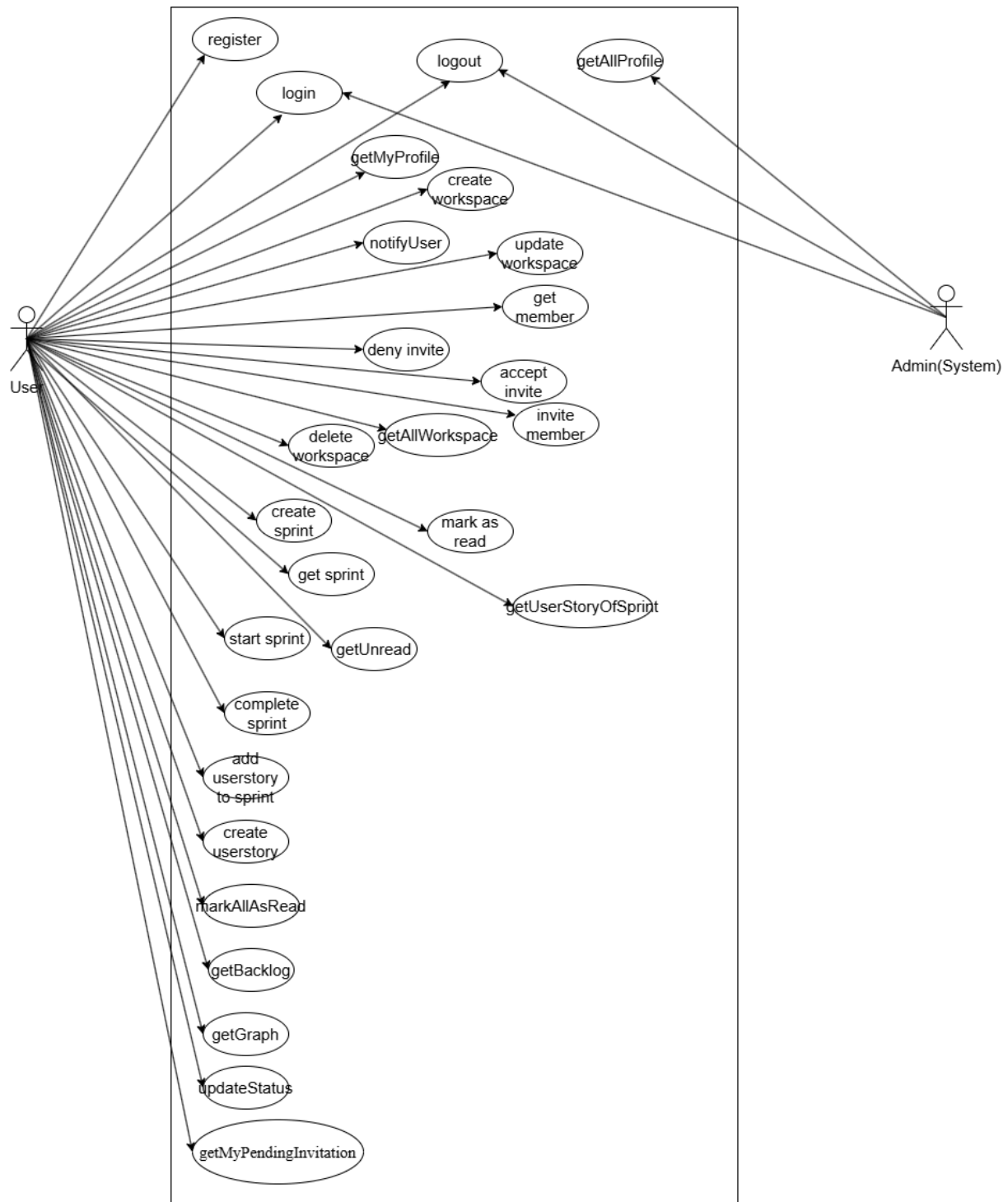
STT	Tên use case	Mô tả
1	Register	Người dùng đăng ký bằng email. Sau khi xác thực, người dùng nhập username, password, verified password.
2	Login	Người dùng nhập thông tin username hoặc email, password
3	Logout	Người dùng có thể đăng xuất khỏi ứng dụng
4	getMyProfile	Người dùng có thể xem thông tin cá nhân của mình.
5	GetAllProfiles	Admin có thể quản lý toàn bộ profile của hệ thống

6	createWorkspace	Cho phép người dùng tạo một Workspace mới để quản lý Backlog, user story và các tài nguyên liên quan.
7	inviteMemberToWorkspace	Cho phép Owner/Admin mời người dùng khác tham gia vào Workspace với một vai trò cụ thể (ví dụ: Member, Viewer).
8	updateWorkspace	Cho phép người dùng có quyền chỉnh sửa thông tin của một Workspace đã tồn tại.
9	deleteWorkspace	Cho phép Owner/Admin có thể xóa Workspace
10	getMember	Cho phép Owner/Admin có thể xem danh sách thành viên có trong Workspace
11	getAllWorkspace	Hiển thị toàn bộ danh sách Workspace của người dùng.
12	accept	Người dùng có thể chấp nhận lời mời tham gia vào Workspace
13	deny	Người dùng có thể từ chối lời mời tham gia vào Workspace
14	getMyPendingInvitation	Cho phép người dùng xem danh sách các lời mời tham gia Workspace đang ở trạng thái chờ (Pending).
15	notifyUser	Gửi thông báo đến người dùng khi họ được mời tham gia vào Workspace,

		hoặc nhận thông báo đồng ý hoặc từ chối tham gia Workspace
16	getUnread	Cho phép người dùng xem danh sách các thông báo chưa được đánh dấu là đã đọc.
17	markAsRead	Đánh dấu thông báo đã đọc
18	markAllAsRead	Đánh dấu tất cả thông báo đã đọc
19	createSprint	Cho phép người dùng có quyền tạo một Sprint mới trong Workspace để quản lý và thực hiện các user story trong một khoảng thời gian xác định.
20	getSprints	Cho phép người dùng xem danh sách các Sprint thuộc một Workspace.
21	startSprint	Cho phép người dùng có quyền bắt đầu một Sprint đã được tạo trước đó, chuyển trạng thái của Sprint sang ACTIVE để bắt đầu thực hiện các user story.
22	completeSprint	Cho phép người dùng có quyền kết thúc một Sprint đang hoạt động, chuyển trạng thái của Sprint sang COMPLETED và tổng kết các user story trong Sprint.
23	addUserStoryToSprint	Cho phép người dùng có quyền thêm một user story từ Backlog vào một Sprint cụ thể để thực hiện trong Sprint đó.

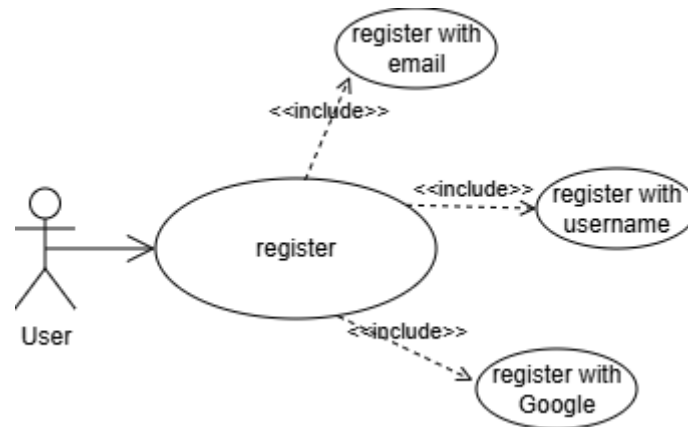
24	removeUserStoryFromSprint	Cho phép người dùng có quyền loại bỏ một user story khỏi Sprint và đưa story đó trở lại Backlog.
25	getUserStoriesOfSprint	Cho phép người dùng xem danh sách các user story thuộc một Sprint cụ thể trong Workspace.
26	createUserStory	Cho phép người dùng tạo một user story mới trong Backlog của Workspace để mô tả một yêu cầu hoặc chức năng cần phát triển.
27	getBacklog	Cho phép người dùng xem danh sách các user story thuộc Backlog của một Workspace
28	updateStatus	Cho phép người dùng cập nhật trạng thái của một user story trong quá trình thực hiện Sprint.
29	deleteUserStory	Cho phép người dùng xóa User Story
30	getGraph	Cho phép người dùng truy vấn và hiển thị dữ liệu đồ thị biểu diễn mối quan hệ giữa các user story và các thành phần được phân tích (actor, action, object) trong một Workspace.

3.3.2. Biểu đồ Use Case tổng quát



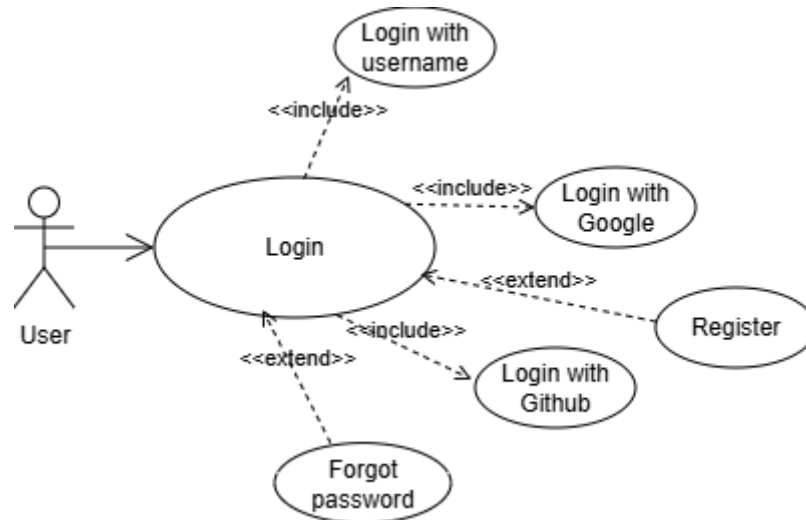
Hình 3: Biểu đồ use case tổng quát

- Register: Người dùng đăng ký bằng email. Sau khi xác thực, người dùng nhập username, password, firstname, lastname, date-of-birth (dob). (có thể đăng ký bằng Google, phone, Facebook).



Hình 4: Biểu đồ use case register

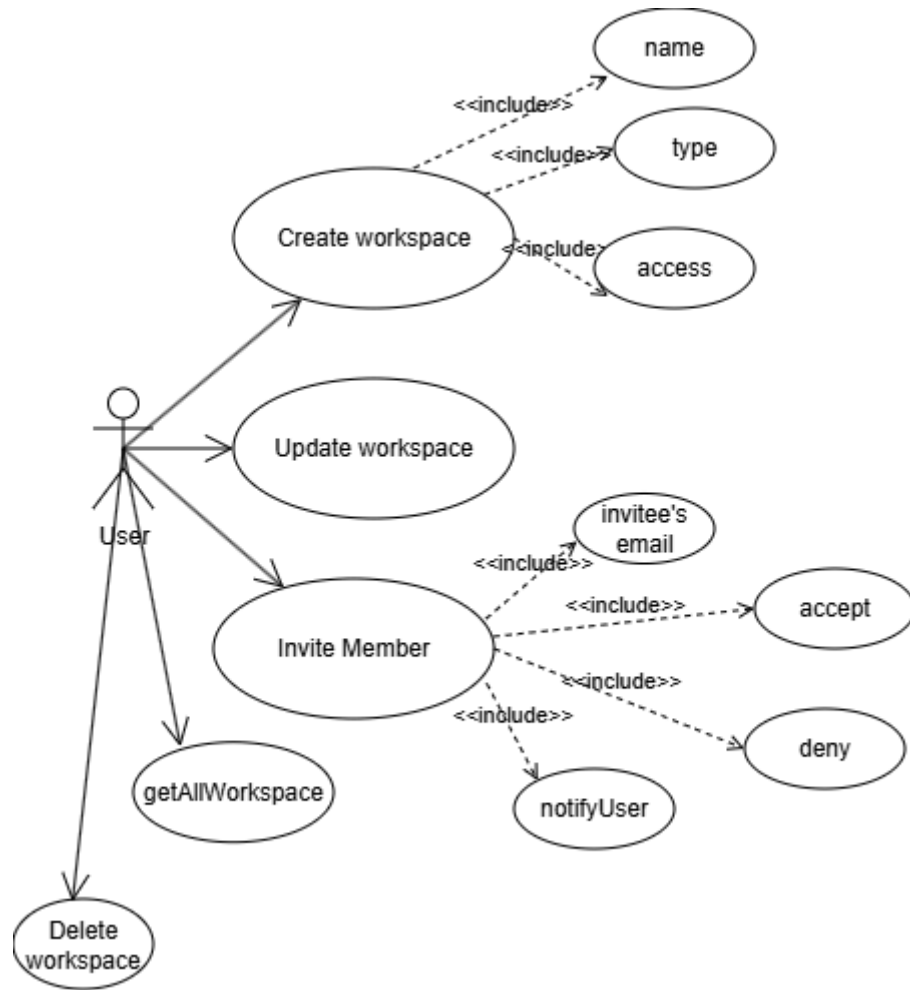
- Login: Người dùng nhập thông tin email, password (có thể đăng nhập bằng Google, username, Github), bên cạnh đó người dùng có thể đăng ký tài khoản nếu không có tài khoản. Khi quên mật khẩu thì có thể bấm “*Forgot password*” để đổi mật khẩu.



Hình 5: Biểu đồ use case login

- Create Workspace: Người dùng có thể tạo Workspace, thêm tên, loại (Team Manage ,...), access (Open, Private, ...). Khi Workspace được tạo, Product Backlog sẽ được tự động tạo.
- Update Workspace: Người dùng có thể sửa đổi Workspace nếu người dùng đó có vai trò Admin trong Workspace đó, họ có thể đổi tên, kiểu Workspace, và sự truy cập.
- Invite Member: Admin của một Workspace có thể mời người khác vào Workspace thông qua email. Khi được mời, người được mời sẽ được nhận thông báo thông qua gmail, nếu người được mời online thì sẽ có thông báo tới họ. Sau khi được mời, người dùng có thể đồng ý hoặc từ chối lời mời tham gia Workspace đó.

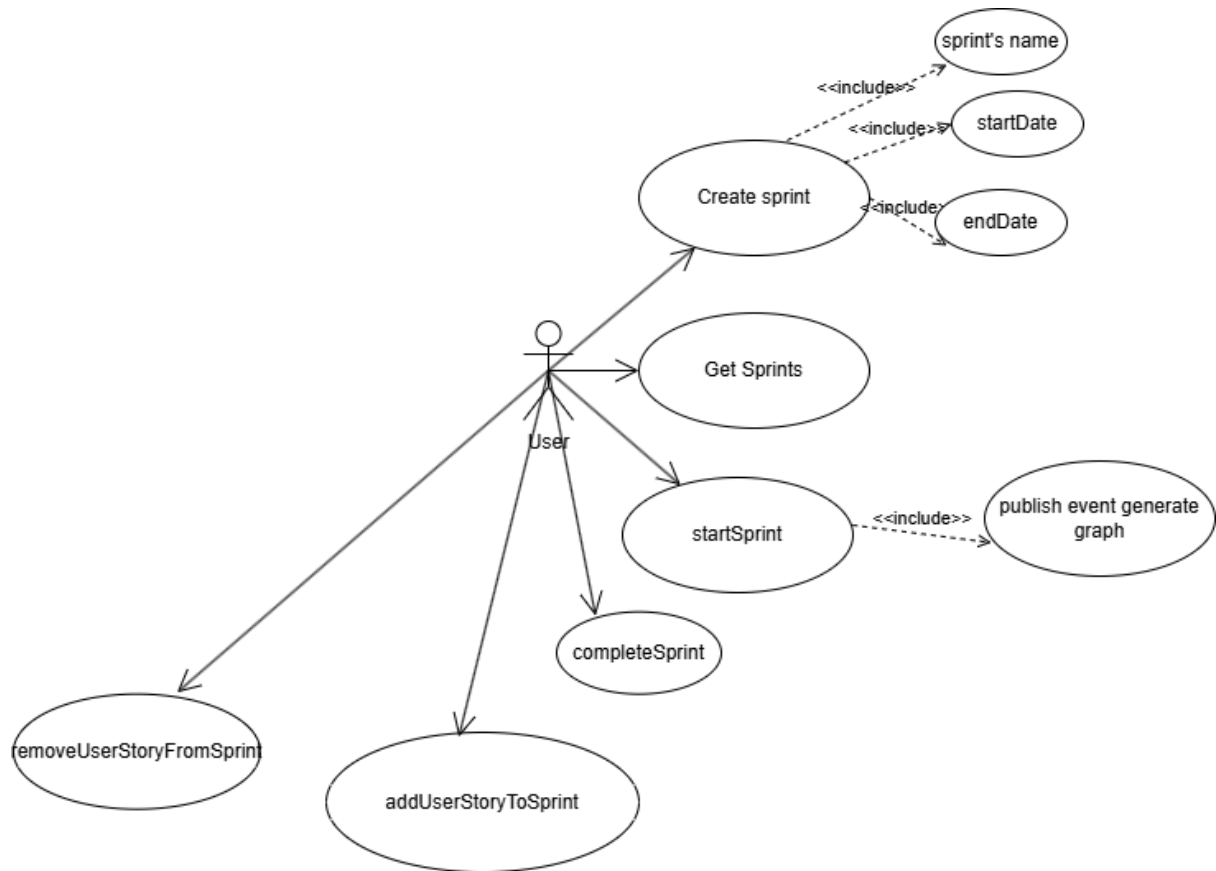
- Get All Workspace: Người dùng có thể thấy được hết Workspace mà họ tham gia hoặc đã tạo
- Delete Workspace: Người dùng có vai trò Admin có thể xóa Workspace



Hình 6: Biểu đồ use case Workspace

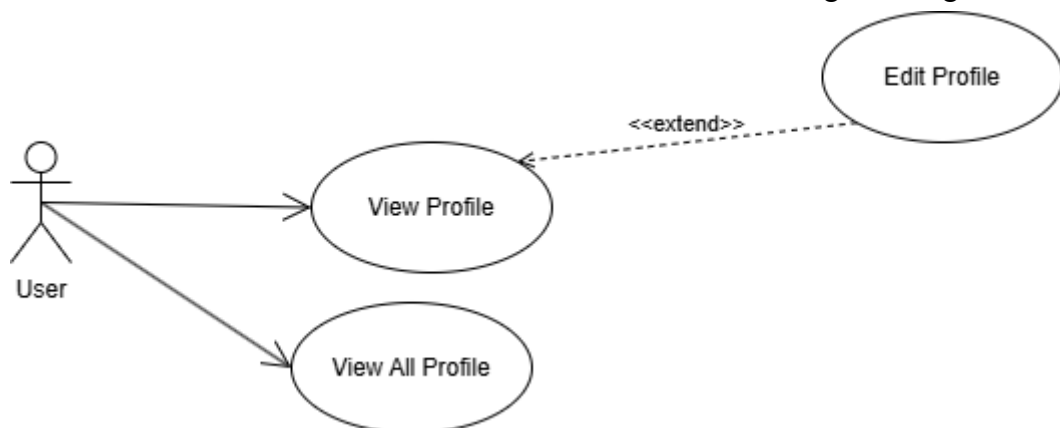
- Sprint:
 - Create Sprint: Người dùng có thể tạo Sprint (name, startDate, endDate). Khi Sprint được tạo, Sprint đó sẽ được để ở trạng thái “ToDo”
 - Get Sprints: Người dùng có thể xem được tất cả Sprints có trong Workspace đó
 - Add User Story To Sprint: Người dùng có thể thêm user story ở Product Backlog vào Sprint.
 - Remove User Story From Sprint: Người dùng có thể xóa user story ra khỏi Sprint, user story đó sẽ được chuyển về Product Backlog.
 - Start Sprint: Khi start Sprint, lúc đó một event sẽ được chạy để tạo ra được đồ thị sau khi đã phân tích user story. Lúc này Sprint sẽ trong trạng thái In_Progress và chỉ được start một Sprint.

- Complete Sprint: Sau khi complete Sprint, Sprint sẽ được chuyển về trạng thái Done, những user story nào có trạng thái khác trạng thái Done thì sẽ được chuyển lại về Product Backlog.



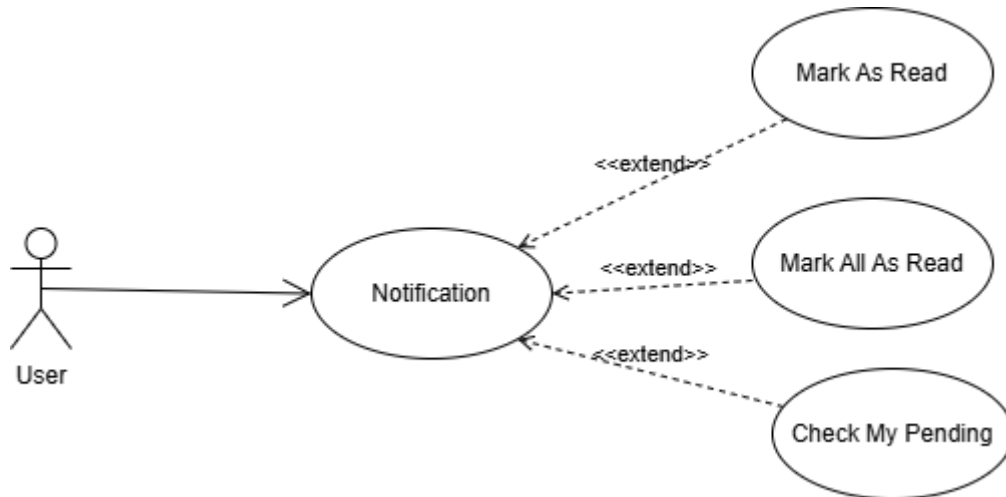
Hình 7: Biểu đồ use case Sprint

- Profile:
 - View Profile: Người dùng có thể xem profile của mình. Bên cạnh đó, người dùng có thể chỉnh sửa profile.
 - View All Profile: Admin có thể xem tất cả Profile trong hệ thống



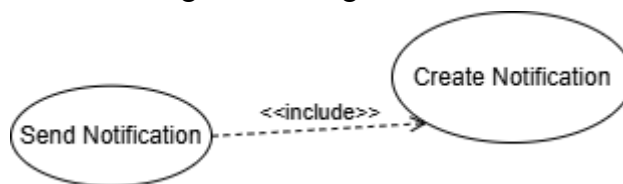
Hình 8: Biểu đồ use case profile

- Notification:
 - Mark As Read: Người dùng có thể đánh dấu thông báo đã đọc
 - Mark All As Read: Người dùng có thể đánh dấu tất cả thông báo đã đọc
 - Check My Pending: Người dùng có thể kiểm tra được những lời mời đã gửi đến họ.



Hình 9: Biểu đồ use case notification

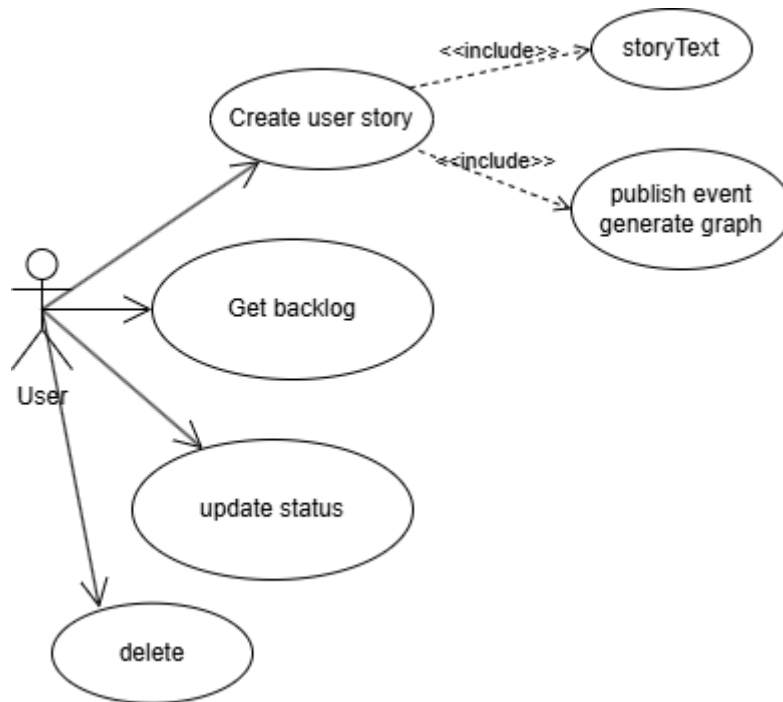
- Create Notification: Hệ thống tạo ra thông báo khi có sự kiện mới.



Hình 10: Biểu đồ use case notification system

- User Story:

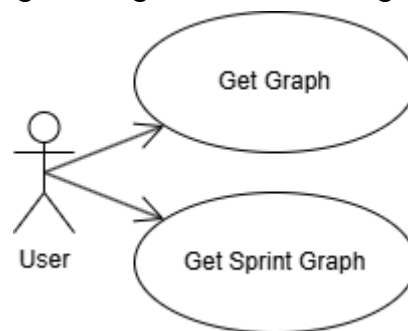
- Create User Story: Người dùng nhập user story, lúc này user story sẽ được thêm vào Product Backlog, sau đó sẽ công khai sự kiện (publish event) để phân tích user story đó ra thành từng node rồi tạo ra graph ở Product Backlog.
- Get Backlog: Người dùng có thể thấy toàn bộ user story có trong Product Backlog.
- Update Status: User Story có thể được cập nhật sang các trạng thái như “ToDo”, “InProgress”, “Done”.
- Delete: Người dùng có thể xóa user story.



Hình 11: Biểu đồ use case User Story

- Graph:

- Get Graph: Người dùng có thể xem được graph đã được phân tích ở Product Backlog
- Get Sprint Graph: Người dùng có thể xem được graph sau khi Sprint start.

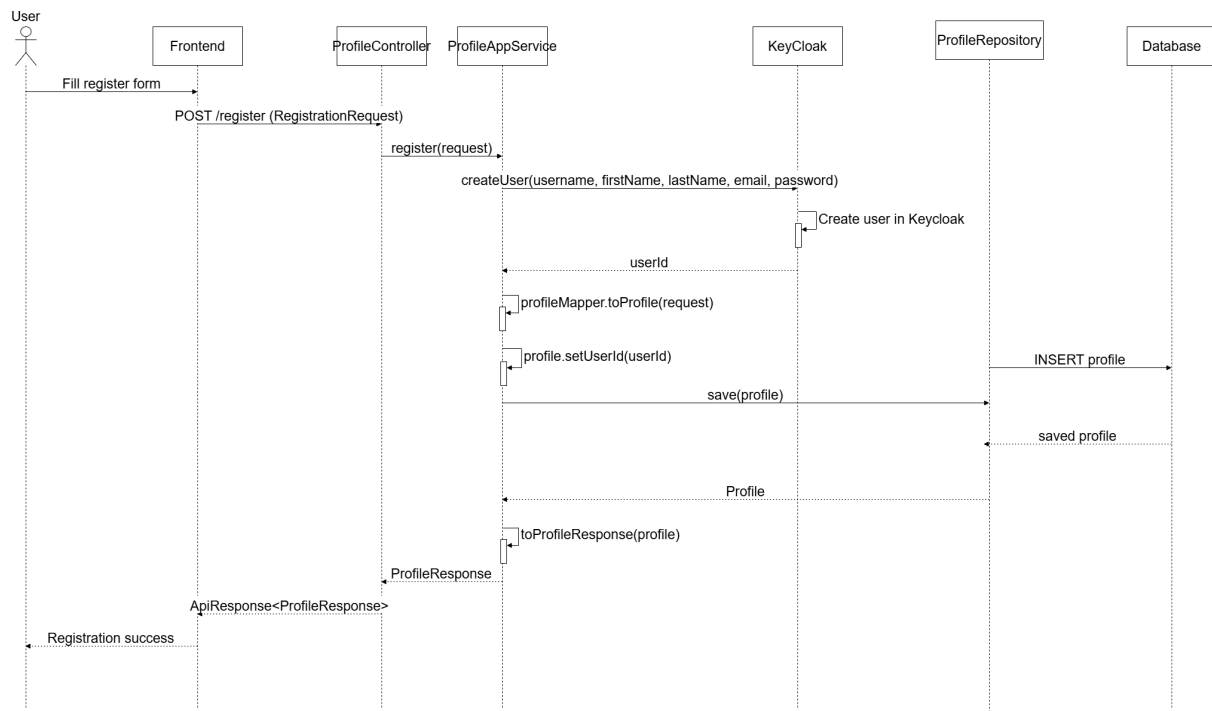


Hình 12: Biểu đồ use case graph

3.4. Các biểu đồ phân tích

3.4.1. Sơ đồ tuần tự (Sequence Diagram)

3.4.1.1. Sơ đồ tuần tự đăng ký tài khoản (Sequence Diagram Register)



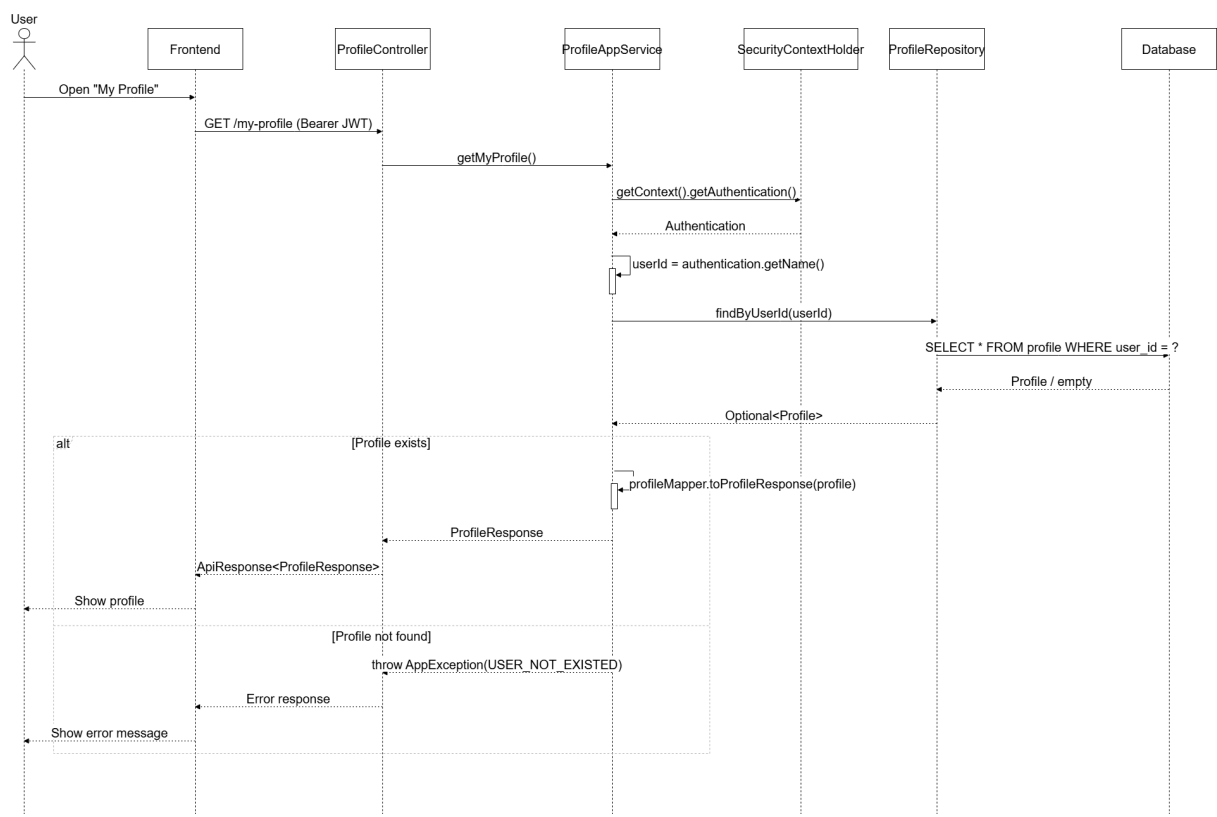
Hình 13: Sơ đồ tuần tự đăng ký tài khoản (Sequence Diagram Register)

Bảng 4: Đặc tả chức năng đăng ký tài khoản (Register)

Thành phần	Nội dung chi tiết
Tên usecase	Đăng ký tài khoản
Tác nhân	Người dùng
Mô tả	Quá trình người dùng đăng ký tài khoản mới. Hệ thống sẽ tạo tài khoản định danh trên KeyCloak và lưu thông tin hồ sơ vào cơ sở dữ liệu
Sự kiện kích hoạt	Người dùng nhấn nút xác nhận sau khi điền đầy đủ thông tin vào form đăng ký trên Frontend.
Điều kiện trước	- Người dùng đã truy cập vào trang đăng ký của ứng dụng.

	- Kết nối giữa Backend với Keycloak và Database đang hoạt động ổn định.
Điều kiện sau	- Tài khoản được tạo thành công trên KeyCloak - Hồ sơ người dùng được lưu trữ trong Database
Luồng sự kiện chính	1. Người dùng điền thông tin vào form đăng ký và gửi đi 2. Gửi yêu cầu đến POST /register đến ProfileController 3. Gọi phương thức register(request) của ProfileAppService 4. ProfileAppService sẽ gửi yêu cầu createUser (bao gồm username, email, password ...) đến KeyCloak 5. KeyCloak sẽ khởi tạo tài khoản và trả về userId cho ProfileAppService 6. ProfileAppService sẽ thực hiện ánh xạ dữ liệu (ProfileMapper) và gán userId nhận được vào Profile 7. ProfileAppService gọi save(profile) thông qua ProfileRepository 8. ProfileRepository sẽ INSERT profile vào Database 9. Database xác nhận lưu dữ liệu thành công và trả về thông tin saved profile 10. ProfileAppService sẽ chuyển đổi dữ liệu sang định dạng phản hồi 11. ProfileController trả về ApiResponse<ProfileResponse> cho Frontend 12. Frontend sẽ trả về thông báo “Registration success” cho User

3.4.1.2. Sơ đồ tuần tự xem hồ sơ (Sequence Diagram Get My Profile)



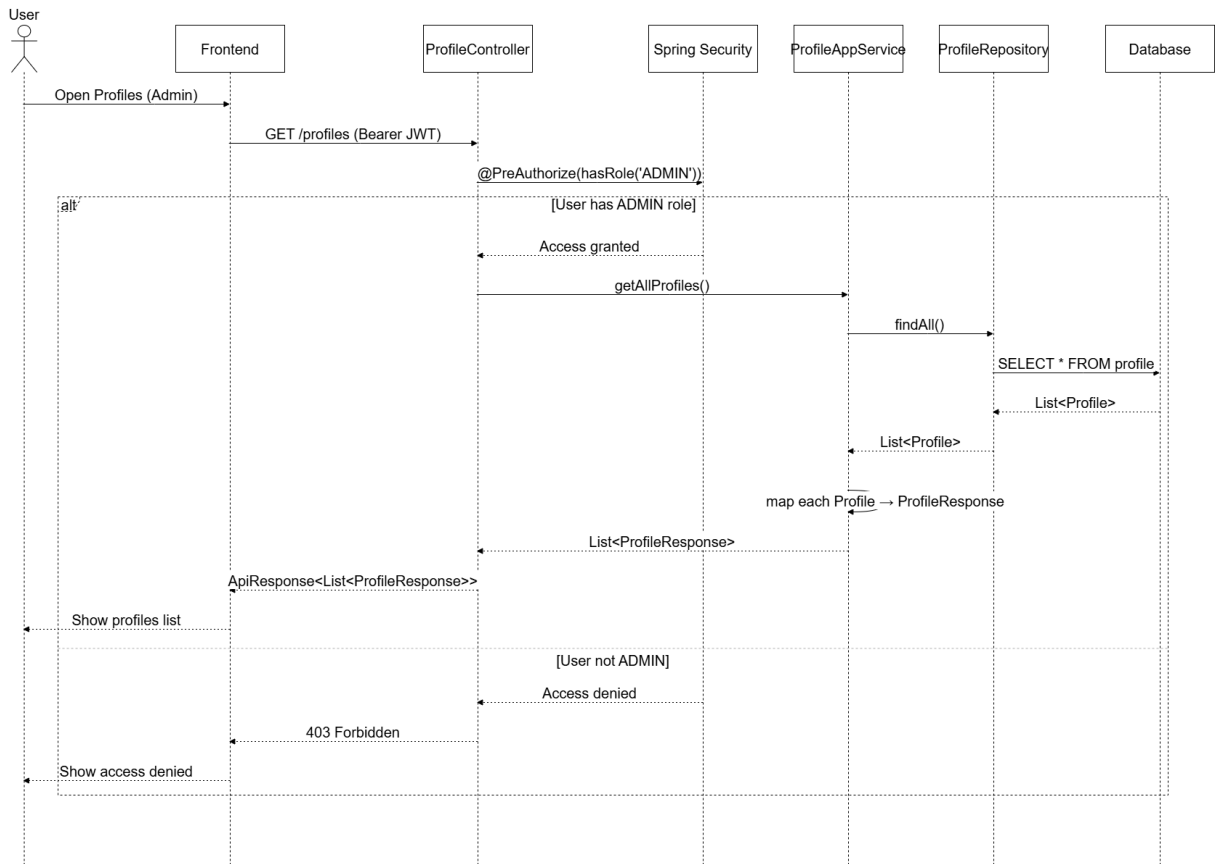
Hình 14: Sơ đồ tuần tự xem hồ sơ (Sequence Diagram Get My Profile)

Bảng 5: Đặc tả chức năng xem hồ sơ (Get My Profile)

Thành phần	Nội dung chi tiết
Tên usecase	Xem hồ sơ của tôi
Tác nhân	Người dùng
Mô tả	Người dùng yêu cầu xem thông tin chi tiết về hồ sơ cá nhân của mình. Hệ thống sẽ trích xuất ID người dùng từ mã thông báo bảo mật, truy vấn cơ sở dữ liệu và hiển thị thông tin hoặc báo lỗi nếu không tìm thấy.
Sự kiện kích hoạt	Người dùng nhấn vào mục "My Profile" trên giao diện ứng dụng.
Điều kiện trước	- Người dùng đã đăng nhập thành công. - Yêu cầu gửi đi phải kèm theo mã thông báo hợp lệ

Điều kiện sau	- Thông tin hồ sơ được hiển thị trên giao diện người dùng (nếu tồn tại). - Hoặc một thông báo lỗi được hiển thị (nếu hồ sơ không tồn tại).
Luồng sự kiện chính	1. Người dùng mở mục "My Profile". 2. Gửi yêu cầu GET /my-profile đến ProfileController. 3. Gọi phương thức getMyProfile() từ ProfileAppService. 4. ProfileAppService sẽ gửi yêu cầu getContext().getAuthentication() đến SecurityContextHolder. 5. SecurityContextHolder sẽ trả về thông tin Authentication. 6. ProfileAppService sẽ trích xuất userId từ đối tượng Authentication. 7. ProfileAppService gọi findByUserId(userId) từ ProfileRepository. 8. ProfileRepository sẽ thực thi câu lệnh SQL: SELECT * FROM profile WHERE user_id = ? xuống Database. 9. Database trả về dữ liệu Profile. 10. ProfileRepository sẽ trả về kết quả dưới dạng Optional<Profile> cho ProfileAppService. 11. ProfileAppService (Trường hợp Profile tồn tại) Thực hiện profileMapper.toProfileResponse(profile). 12. ProfileAppService sẽ trả về ProfileResponse cho ProfileController. 13. ProfileController sẽ trả về ApiResponse<ProfileResponse> cho Frontend. 14. Frontend hiển thị thông tin hồ sơ
Luồng sự kiện thay thế	Hồ sơ không tồn tại • Tại bước 9, nếu Database không tìm thấy bản ghi nào tương ứng, nó sẽ trả về kết quả empty • Tại bước 11, nếu không tìm thấy Profile, ProfileAppService sẽ kiểm tra và trả về một ngoại lệ: throw AppException(USER_NOT_EXISTED) • ProfileController tiếp nhận ngoại lệ và gửi một Error response cho Frontend. • Frontend hiển thị thông báo cho người dùng

3.4.1.3. Sơ đồ tuần tự lấy tất cả hồ sơ (Sequence Diagram Get All Profile)



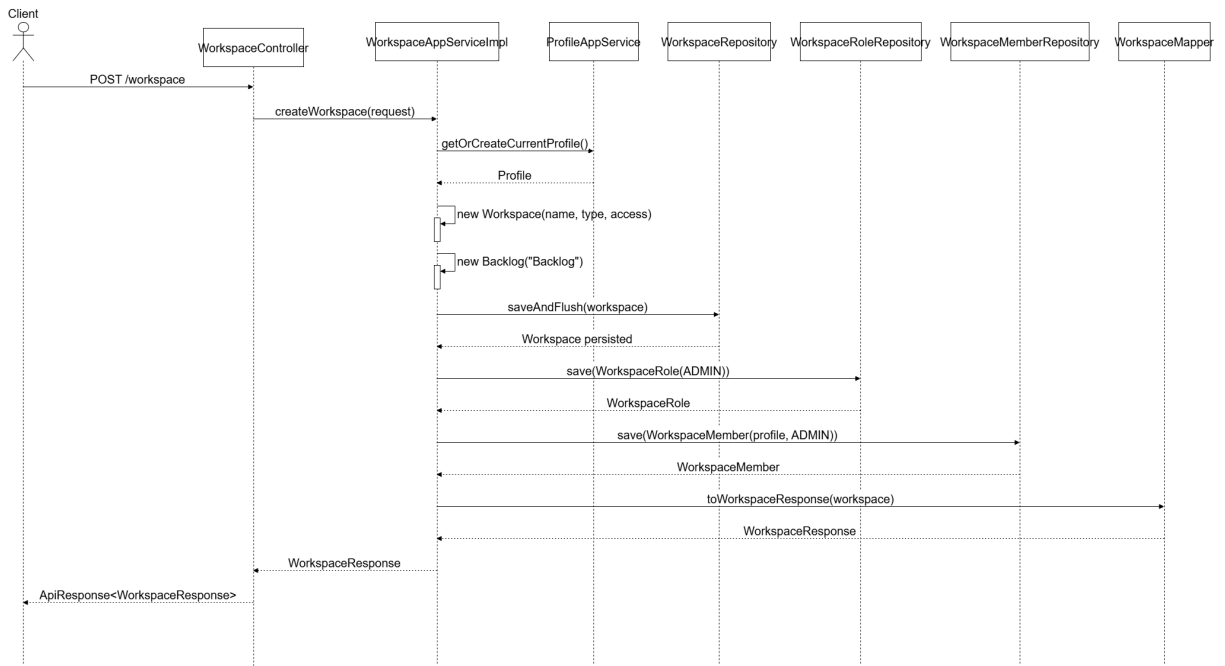
Hình 15: Sơ đồ tuần tự lấy tất cả hồ sơ (Sequence Diagram Get All Profile)

Bảng 6: Đặc tả chức năng lấy tất cả hồ sơ (Get All Profile)

Thành phần	Nội dung chi tiết
Tên usecase	Lấy tất cả hồ sơ
Tác nhân	Admin
Mô tả	Quản trị viên yêu cầu xem toàn bộ danh sách hồ sơ người dùng. Hệ thống kiểm tra quyền hạn trước khi truy xuất dữ liệu.
Sự kiện kích hoạt	Quản trị viên nhấn chọn mục "User"
Điều kiện trước	- Người dùng đã đăng nhập.
Điều kiện sau	- Danh sách toàn bộ hồ sơ được hiển thị hoặc truy cập bị từ chối nếu không có quyền Admin.
Luồng sự kiện chính	1. User thực hiện hành động "Open Profiles (Admin)". 2. Frontend gửi yêu cầu GET /profiles kèm Bearer JWT tới

	ProfileController. 3. Spring Security thực hiện kiểm tra quyền bằng annotation <code>@PreAuthorize(hasRole('ADMIN'))</code>. 4. Trường hợp [User has ADMIN role]: • Spring Security thông báo "Access granted" cho ProfileController. • ProfileController gọi hàm <code>getAllProfiles()</code> trong ProfileAppService. • ProfileAppService gọi <code>findAll()</code> từ ProfileRepository. • ProfileRepository thực hiện truy vấn <code>SELECT * FROM profile</code> trong Database. • Database trả về danh sách <code>List<Profile></code>. • ProfileAppService thực hiện chuyển đổi (map) từng Profile sang ProfileResponse. • ProfileAppService trả về <code>List<ProfileResponse></code> cho ProfileController. • ProfileController trả về <code>ApiResponse<List<ProfileResponse>></code> cho Frontend. • Frontend hiển thị danh sách hồ sơ cho User.
Luồng sự kiện thay thế	[User not ADMIN] • Nếu người dùng không có quyền quản trị, Spring Security phản hồi "Access denied". ProfileController gửi mã lỗi 403 Forbidden về Frontend, và Frontend hiển thị thông báo từ chối truy cập.

3.4.1.4. Sơ đồ tuần tự tạo không gian làm việc (Sequence Diagram Create Workspace)



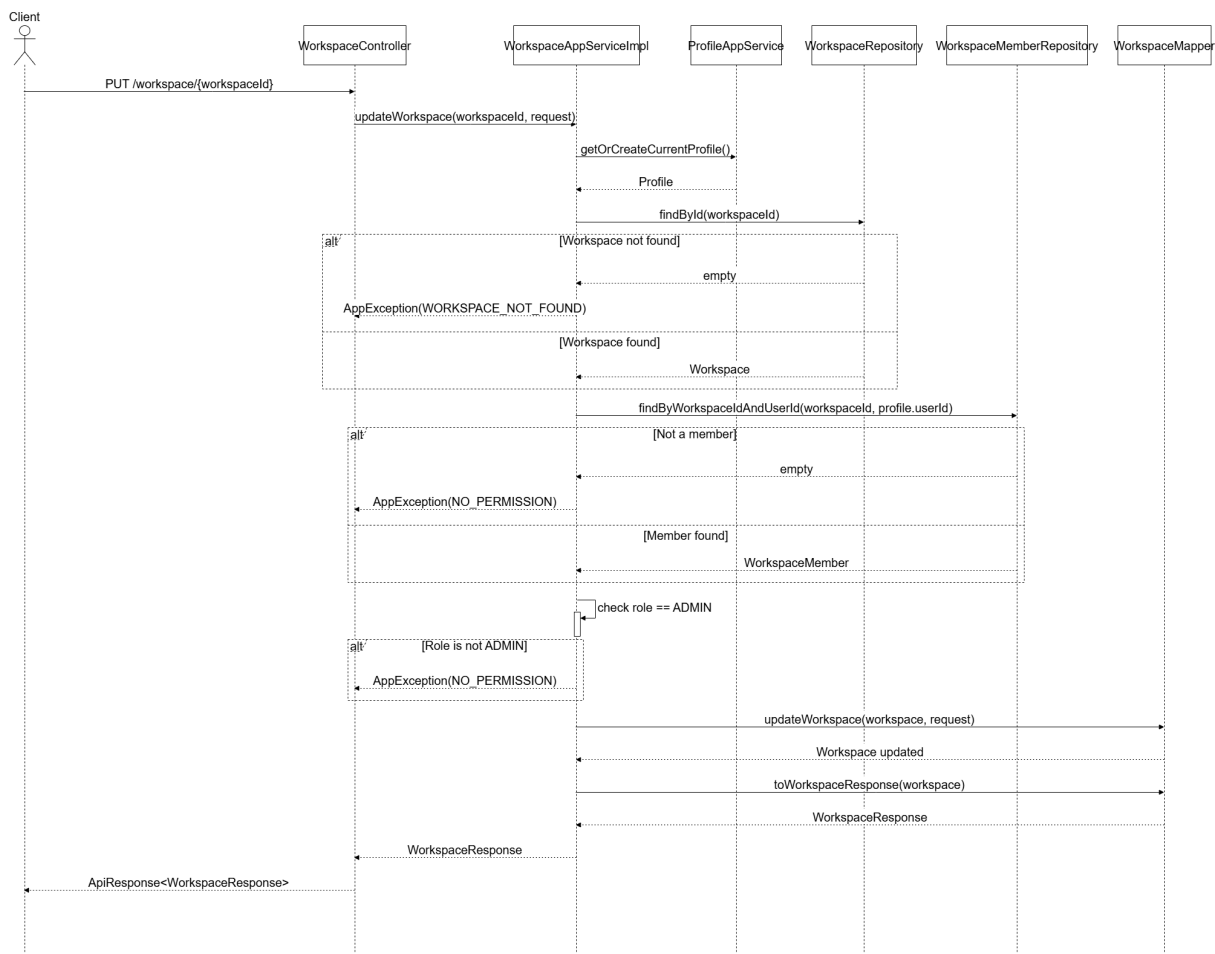
Hình 16: Sơ đồ tuần tự tạo không gian làm việc (Sequence Diagram Create Workspace)

Bảng 7: Đặc tả chức năng tạo không gian làm việc (Create Workspace)

Thành phần	Nội dung chi tiết
Tên usecase	Tạo không gian làm việc
Tác nhân	Người dùng
Mô tả	Người dùng tạo một Workspace mới. Hệ thống sẽ tạo đồng thời Backlog, thiết lập vai trò Admin cho người tạo và lưu thông tin thành viên.
Sự kiện kích hoạt	Người dùng nhấn tạo workspace trên giao diện ứng dụng.
Điều kiện trước	- Người dùng đã đăng nhập thành công.
Điều kiện sau	- Workspace mới được tạo thành công cùng với các thực thể liên quan (Role, Member, Backlog).
Luồng sự kiện chính	1. Client gửi yêu cầu tạo Workspace tới WorkspaceController. 2. WorkspaceController gọi createWorkspace(request) tới WorkspaceAppServiceImpl 3. WorkspaceAppServiceImpl gọi ProfileAppService để lấy thông tin hồ sơ người dùng hiện tại

	(getOrCreateCurrentProfile). 4. WorkspaceAppServiceImpl khởi tạo đối tượng Workspace mới (tên, loại, quyền truy cập) và một Backlog mặc định. 5. WorkspaceAppServiceImpl gọi saveAndFlush(workspace) tới WorkspaceRepository để lưu vùng làm việc. 6. Hệ thống thực hiện lưu vai trò quản trị thông qua WorkspaceRoleRepository (save(WorkspaceRole(ADMIN))). 7. Hệ thống thiết lập người tạo là thành viên quản trị qua WorkspaceMemberRepository (save(WorkspaceMember(profile, ADMIN))). 8. WorkspaceMapper chuyển đổi kết quả sang WorkspaceResponse. 9. Client nhận phản hồi thành công ApiResponse<WorkspaceResponse>.
--	---

3.4.1.5. Sơ đồ tuần tự cập nhật không gian làm việc (Sequence Diagram Update Workspace)



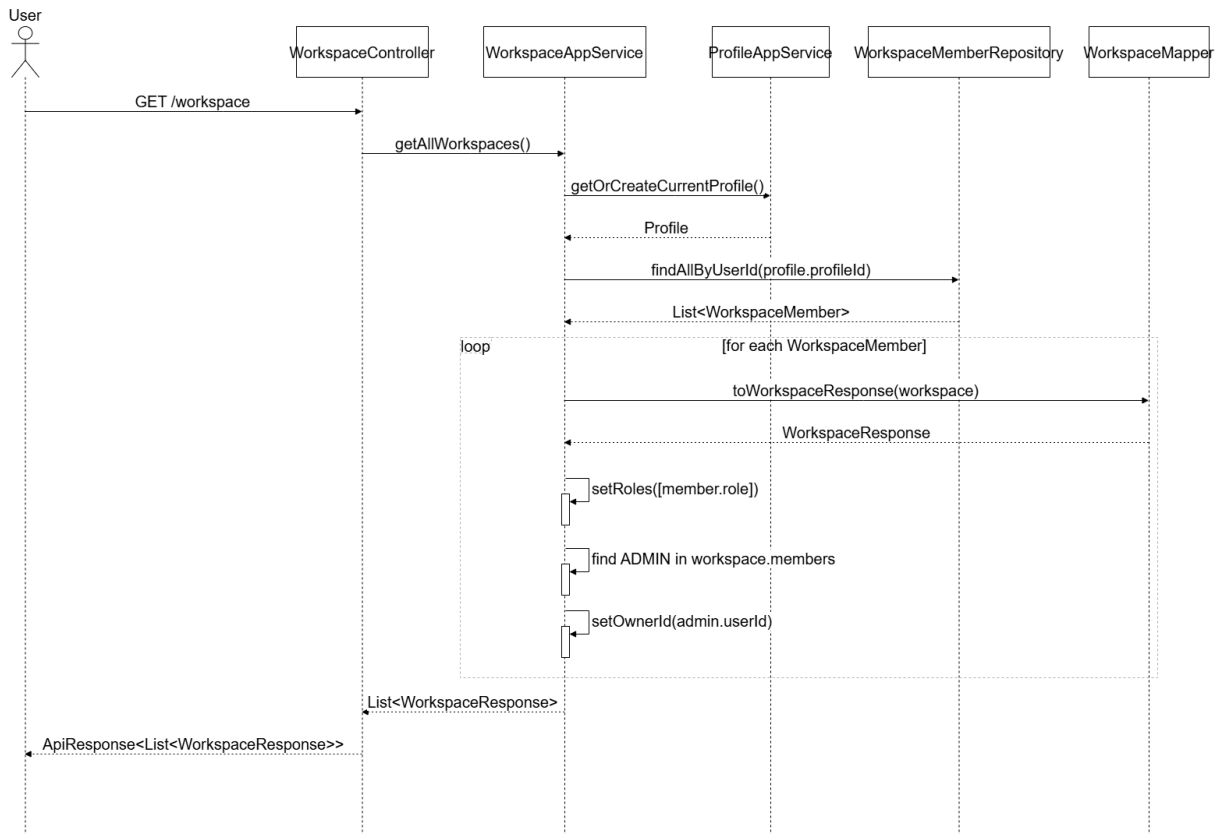
Hình 17: Sơ đồ tuần tự cập nhật không gian làm việc

Bảng 8: Đặc tả chức năng cập nhật không gian làm việc (Update Workspace)

Thành phần	Nội dung chi tiết
Tên usecase	Cập nhật Không gian làm việc.
Tác nhân	Người dùng: Phải là Admin của workspace đó
Mô tả	Cho phép người dùng chỉnh sửa các thông tin cơ bản của một không gian làm việc hiện có như tên, mô tả, hoặc cấu hình quyền truy cập.
Sự kiện kích hoạt	Người dùng nhấn nút "Lưu thay đổi" (Save Changes) sau khi chỉnh sửa thông tin trong phần cài đặt Workspace.
Điều kiện trước	- Người dùng đã đăng nhập và được xác thực. - Không gian làm việc (Workspace) cần cập nhật phải tồn tại trong hệ thống. - Người dùng phải có vai trò ADMIN trong Workspace đó.
Điều kiện sau	- Thông tin mới của Workspace được cập nhật vào cơ sở dữ liệu. - Hệ thống ghi lại dấu vết (nếu có hệ thống log) và hiển thị thông báo thành công.
Luồng sự kiện chính	1. Client gửi yêu cầu PUT /workspace/{workspaceId} kèm theo dữ liệu cần cập nhật 2. WorkspaceController tiếp nhận yêu cầu và gọi phương thức updateWorkspace(id, request) của WorkspaceAppService. 3. WorkspaceAppService gọi getOrCreateCurrentProfile() từ ProfileAppService để xác định người thực hiện. 4. WorkspaceAppService gọi findById(id) từ WorkspaceRepository để lấy thông tin Workspace hiện tại. 5. WorkspaceAppService kiểm tra quyền hạn: Gọi WorkspaceMemberRepository để xác nhận người dùng có quyền ADMIN trong Workspace này. 6. WorkspaceAppService sử dụng WorkspaceMapper để cập nhật các thuộc tính mới từ request vào đối tượng Workspace hiện tại. 7. WorkspaceAppService gọi save(workspace) thông qua WorkspaceRepository. 8. WorkspaceRepository thực hiện lệnh UPDATE dữ liệu trong Database. 9. Database phản hồi trạng thái cập nhật thành công. 10. WorkspaceAppService chuyển đổi đối tượng đã cập nhật

	sang WorkspaceResponse. 11. WorkspaceController trả về ApiResponse<WorkspaceResponse> cho Client. 12. Client hiển thị thông báo "Cập nhật thành công" và cập nhật lại giao diện.
Luồng sự kiện thay thế	Không tìm thấy Workspace: • Tại bước 4, nếu không tìm thấy ID, hệ thống ném ngoại lệ WORKSPACE_NOT_FOUND. • Trả về mã lỗi 404 cho Client. Không có quyền chỉnh sửa (Forbidden): • Tại bước 5, nếu người dùng không phải là ADMIN của Workspace, hệ thống ném ngoại lệ ACCESS_DENIED. • Trả về mã lỗi 403 cho Client. Tên Workspace đã tồn tại: • Tại bước 6, nếu kiểm tra tên mới bị trùng, hệ thống báo lỗi WORKSPACE_NAME_ALREADY_EXISTS. • Trả về mã lỗi 400 cho Client.

3.4.1.6. Sơ đồ tuần tự lấy tất cả không gian làm việc (Sequence Diagram Get All Workspace)



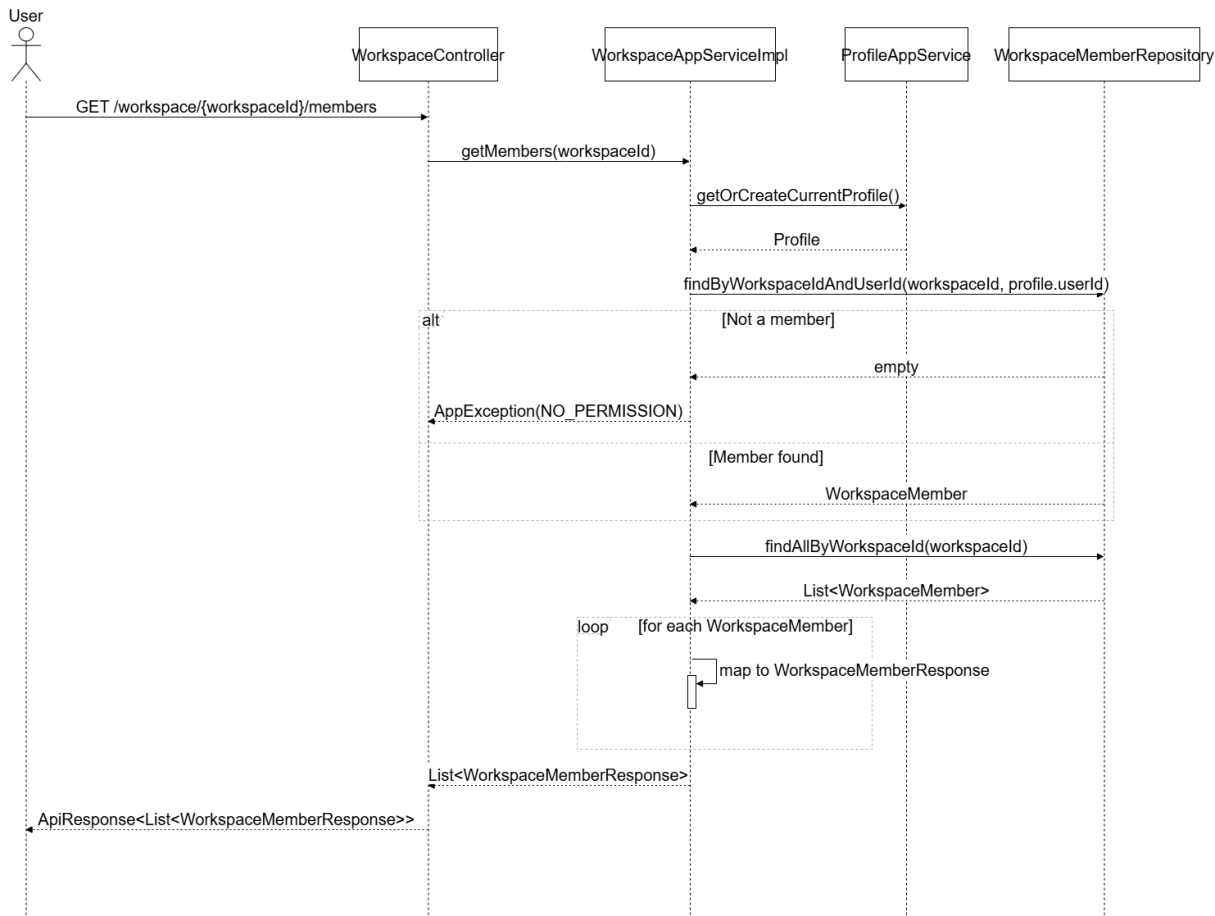
Hình 18: Sơ đồ tuần tự lấy tất cả không gian làm việc (Sequence Diagram Get All Workspace)

Bảng 9: Đặc tả chức năng lấy tất cả không gian làm việc (Get All Workspace)

Thành phần	Nội dung chi tiết
Tên usecase	Lấy danh sách không gian làm việc.
Tác nhân	Người dùng
Mô tả	Hệ thống truy xuất tất cả các không gian làm việc mà người dùng hiện tại tham gia, xác định vai trò của người dùng và thông tin quản trị viên trong từng không gian đó để hiển thị.
Sự kiện kích hoạt	Người dùng truy cập vào trang danh sách không gian làm việc hoặc ứng dụng tự động tải danh sách khi khởi động.
Điều kiện trước	- Người dùng đã xác thực thành công và gửi yêu cầu kèm mã định danh hợp lệ.
Điều kiện sau	- Danh sách các không gian làm việc được trả về cho người dùng với đầy đủ thông tin về vai trò và chủ sở hữu.

Luồng sự kiện chính	1. User gửi yêu cầu GET /workspace. 2. WorkspaceController tiếp nhận yêu cầu và gọi getAllWorkspaces() từ WorkspaceAppService. 3. WorkspaceAppService gọi getOrCreateCurrentProfile() từ ProfileAppService để lấy thông tin hồ sơ người dùng hiện tại. 4. WorkspaceAppService gọi findAllByUserId(profile.userId) từ WorkspaceMemberRepository. 5. WorkspaceMemberRepository trả về danh sách các bản ghi thành viên List<WorkspaceMember>. 6. WorkspaceAppService bắt đầu vòng lặp (Loop) cho mỗi WorkspaceMember trong danh sách: • Gọi WorkspaceMapper để chuyển đổi thông tin workspace sang WorkspaceResponse. • Thực hiện setRoles([member.role]) để gán vai trò của người dùng hiện tại trong workspace đó. • Tìm kiếm quản trị viên (find ADMIN) trong danh sách thành viên của workspace. • Thực hiện setOwnerId(admin.userId) để xác định chủ sở hữu không gian làm việc. 7. WorkspaceAppService trả về danh sách List<WorkspaceResponse> hoàn chỉnh cho WorkspaceController. 8. WorkspaceController trả về ApiResponse<List<WorkspaceResponse>> cho người dùng.
Luồng sự kiện thay thế	Trường hợp người dùng không tham gia Workspace nào: • Tại bước 5, WorkspaceMemberRepository trả về danh sách rỗng. • Vòng lặp không thực hiện. • Hệ thống trả về ApiResponse với danh sách trắng (empty list).

3.4.1.7. Sơ đồ tuần tự lấy thành viên (Sequence Diagram Get Members)



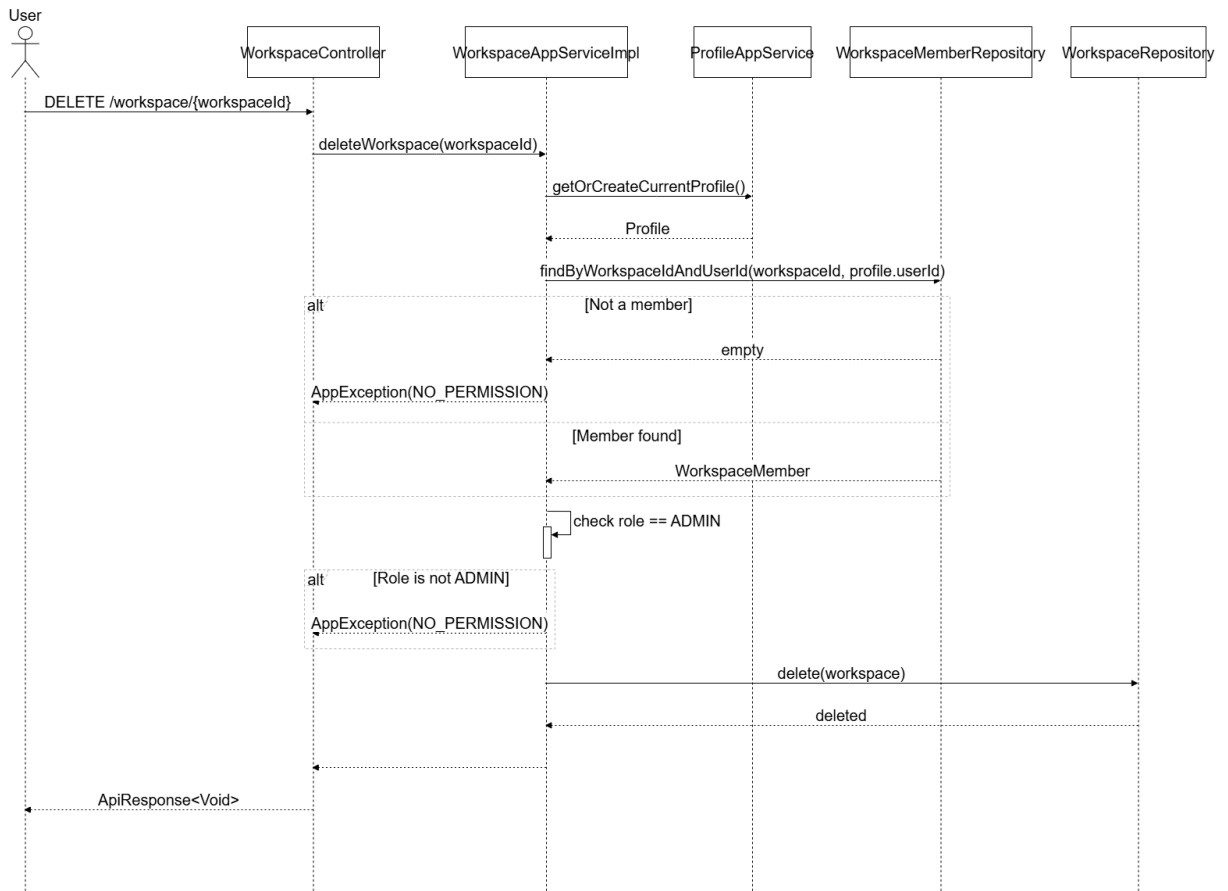
Hình 19: Sơ đồ tuần tự lấy thành viên (Sequence Diagram Get Members)

Bảng 10: Đặc tả chức năng lấy thành viên (Get Members)

Thành phần	Nội dung chi tiết
Tên usecase	Lấy danh sách thành viên trong Workspace.
Tác nhân	Người dùng
Mô tả	Người dùng yêu cầu xem danh sách tất cả các thành viên tham gia vào một Workspace. Hệ thống sẽ kiểm tra xem người dùng yêu cầu có thuộc Workspace đó hay không trước khi hiển thị.
Sự kiện kích hoạt	User gửi yêu cầu GET /workspace/{workspaceId}/members.
Điều kiện trước	- WorkspaceId hợp lệ và người dùng đã xác thực.
Điều kiện sau	- Trả về danh sách thành viên đã được map sang định dạng Response.

Luồng sự kiện chính	1. User gửi yêu cầu tới WorkspaceController. 2. WorkspaceController gọi getMembers(workspaceId) từ WorkspaceAppServiceImpl. 3. WorkspaceAppServiceImpl xác định hồ sơ người dùng hiện tại qua ProfileAppService. 4. Hệ thống gọi WorkspaceMemberRepository để kiểm tra quyền truy cập của người dùng đối với Workspace này. 5. Trường hợp [Member found]: • Hệ thống gọi findAllByWorkspaceId(workspaceId) từ WorkspaceMemberRepository để lấy toàn bộ danh sách thành viên. • Thực hiện vòng lặp [for each WorkspaceMember]: Ánh xạ (map) từng thành viên sang WorkspaceMemberResponse. • WorkspaceController trả về danh sách cuối cùng qua ApiResponse<List<WorkspaceMemberResponse>>.
Luồng sự kiện thay thế	[Not a member]: • Nếu người dùng không thuộc Workspace, hệ thống ném ngoại lệ AppException(NO_PERMISSION) và hiển thị lỗi trên giao diện.

3.4.1.8. Sơ đồ tuần tự xóa không gian làm việc (Sequence Diagram Delete Workspace)



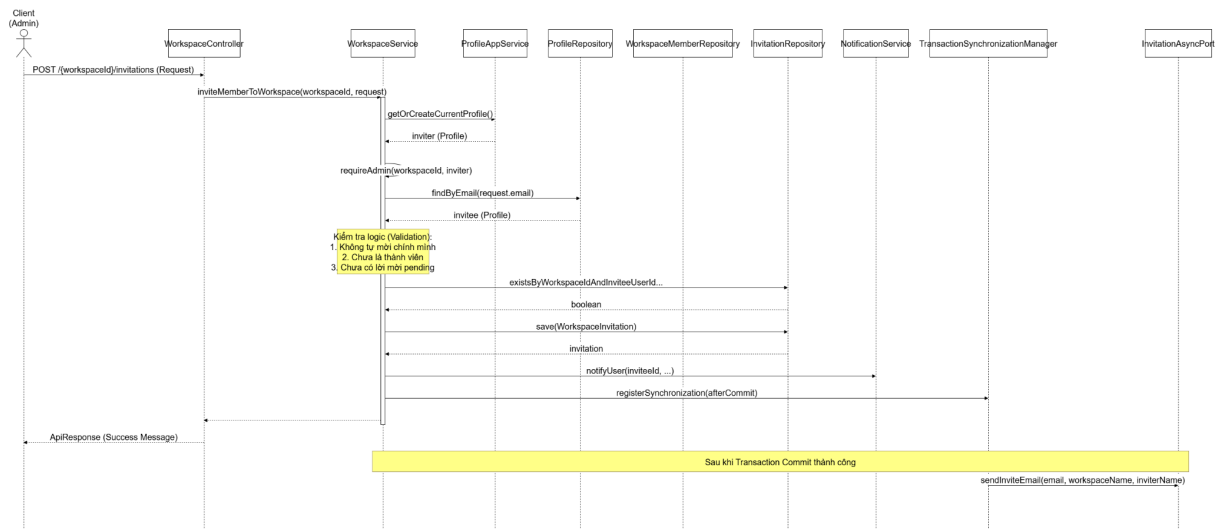
Hình 20: Sơ đồ tuần tự xóa không gian làm việc (Sequence Diagram Delete Workspace)

Bảng 11: Đặc tả chức năng xóa không gian làm việc (Delete Workspace)

Thành phần	Nội dung chi tiết
Tên usecase	Xóa không gian làm việc.
Tác nhân	Người dùng: Phải là Admin của workspace đó
Mô tả	Cho phép người quản trị xóa hoàn toàn một không gian làm việc khỏi hệ thống.
Sự kiện kích hoạt	Người dùng nhấn nút "Xóa Workspace" trong phần cài đặt quản trị.
Điều kiện trước	- Người dùng đã đăng nhập thành công. - WorkspaceId tồn tại trong hệ thống.
Điều kiện sau	- Không gian làm việc bị xóa vĩnh viễn khỏi cơ sở dữ liệu.

	- Tất cả dữ liệu liên quan (thành viên, quyền...) bị loại bỏ (theo cơ chế cascade hoặc logic nghiệp vụ).
Luồng sự kiện chính	1. User gửi yêu cầu DELETE /workspace/{workspaceId}. 2. WorkspaceController tiếp nhận yêu cầu và gọi deleteWorkspace(workspaceId) của WorkspaceAppServiceImpl. 3. WorkspaceAppServiceImpl gọi getOrCreateCurrentProfile() từ ProfileAppService để xác định hồ sơ người thực hiện. 4. WorkspaceAppServiceImpl kiểm tra quyền thành viên bằng cách gọi findByWorkspaceIdAndUserId từ WorkspaceMemberRepository. 5. Hệ thống xác nhận người dùng là thành viên (Member found). 6. WorkspaceAppServiceImpl kiểm tra vai trò của thành viên (check role == ADMIN). 7. Hệ thống xác nhận vai trò là quản trị viên. 8. WorkspaceAppServiceImpl gọi lệnh delete(workspace) thông qua WorkspaceRepository. 9. WorkspaceRepository thực hiện xóa dữ liệu trong Database và phản hồi trạng thái "deleted" 10. WorkspaceAppServiceImpl phản hồi kết quả thành công về cho WorkspaceController. 11. WorkspaceController trả về ApiResponse<Void> (Thành công không kèm dữ liệu) cho người dùng.
Luồng sự kiện thay thế	Người dùng không phải thành viên (Not a member): • Tại bước 4, nếu kết quả trả về là empty. • Hệ thống ném ngoại lệ AppException(NO_PERMISSION). • Trả về thông báo lỗi cho người dùng. Thành viên không có quyền ADMIN (Role is not ADMIN): • Tại bước 6, nếu vai trò không phải là quản trị viên. • Hệ thống ném ngoại lệ AppException(NO_PERMISSION). • Trả về thông báo lỗi cho người dùng.

3.4.1.9. Sơ đồ tuần tự mời thành viên (Sequence Diagram Invite Member)



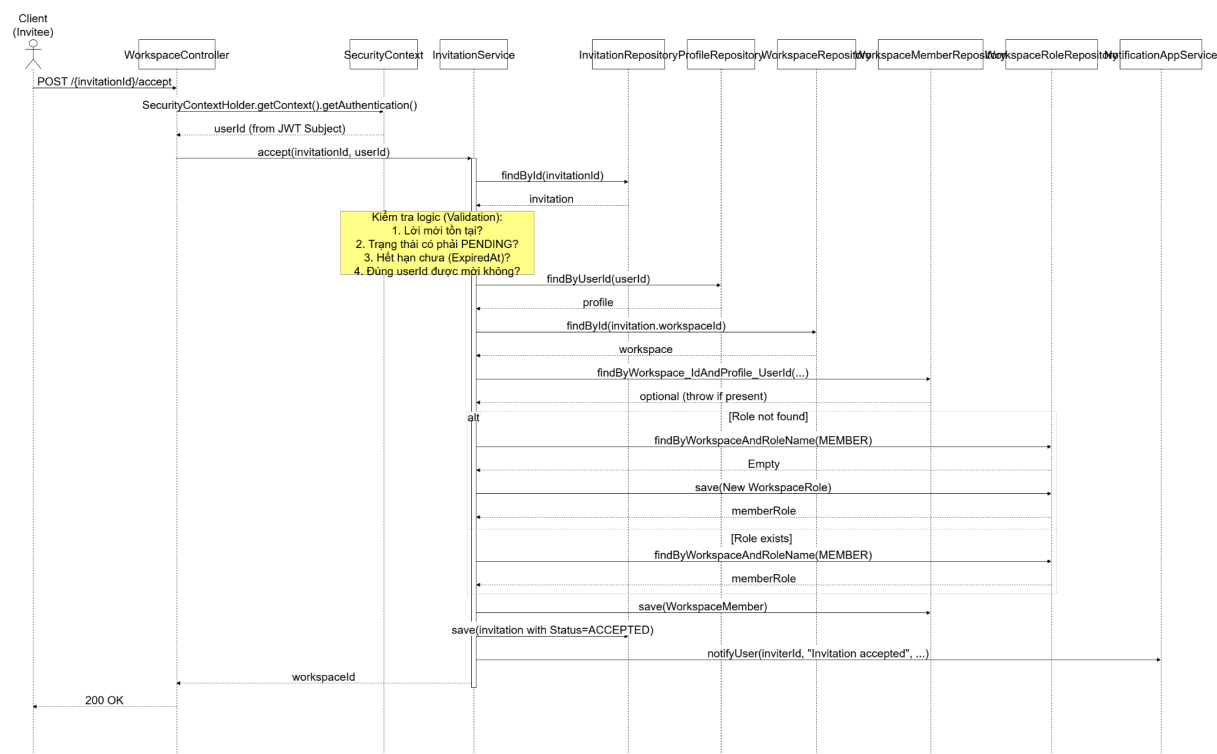
Hình 21: Sơ đồ tuần tự mời thành viên (Sequence Diagram Invite Member)

Bảng 12: Đặc tả chức năng mời thành viên (Invite Member)

Thành phần	Nội dung chi tiết
Tên usecase	Mời thành viên vào không gian làm việc.
Tác nhân	Người gửi lời mời, phải có quyền quản trị trong Workspace.
Mô tả	Cho phép quản trị viên của một Workspace gửi lời mời tham gia cho người dùng khác thông qua email. Hệ thống sẽ kiểm tra các điều kiện hợp lệ, lưu lời mời và gửi thông báo/email sau khi giao dịch (transaction) thành công.
Sự kiện kích hoạt	Admin nhập email và gửi yêu cầu mời thành viên từ giao diện quản lý Workspace.
Điều kiện trước	- Người mời đã đăng nhập và có quyền ADMIN trong Workspace tương ứng. - Người được mời phải đã có tài khoản (Profile) trên hệ thống (dựa trên email).
Điều kiện sau	- Một bản ghi lời mời mới được lưu vào InvitationRepository. - Một thông báo hệ thống được gửi đến người được mời. - Một email mời được gửi đi sau khi quá trình lưu dữ liệu hoàn tất.
Luồng sự kiện chính	1. Client (Admin) gửi yêu cầu POST /{workspaceId}/invitations kèm theo thông tin Request.

	2. WorkspaceController gọi inviteMemberToWorkspace(workspaceId, request) của WorkspaceService. 3. WorkspaceService gọi getOrCreateCurrentProfile() từ ProfileAppService để lấy thông tin người mời (inviter). 4. WorkspaceService kiểm tra quyền quản trị bằng hàm nội bộ requireAdmin(workspaceId, inviter). 5. WorkspaceService gọi findByEmail(request.email) từ ProfileRepository để lấy thông tin người được mời (invitee). 6. WorkspaceService kiểm tra logic nghiệp vụ (Validation): • Không được tự mời chính mình. • Người được mời chưa phải là thành viên Workspace. • Chưa có lời mời nào đang chờ xử lý (pending). 7. WorkspaceService kiểm tra sự tồn tại của lời mời qua existsByWorkspaceIdAndInviteeUserId tại InvitationRepository. 8. WorkspaceService gọi save(WorkspaceInvitation) để lưu lời mời mới vào InvitationRepository. 9. WorkspaceService gọi notifyUser(inviteeId, ...) thông qua NotificationService để gửi thông báo trong ứng dụng. 10. WorkspaceService đăng ký sự kiện afterCommit với TransactionSynchronizationManager để xử lý tác vụ sau giao dịch. 11. WorkspaceController trả về ApiResponse (Success Message) cho Client. 12. Hệ thống sau khi Transaction Commit thành công: Gọi sendInviteEmail(...) qua InvitationAsyncPort để gửi email thực tế.
Luồng sự kiện thay thế	Người được mời không tồn tại: • Tại bước 5, nếu không tìm thấy Profile theo email, hệ thống ném ngoại lệ USER_NOT_FOUND. Vi phạm logic Validation: • Tại bước 6, nếu vi phạm bất kỳ điều kiện nào (ví dụ: đã là thành viên), hệ thống trả về lỗi nghiệp vụ tương ứng (ví dụ: ALREADY_MEMBER). Lỗi gửi Email: • Nếu quá trình gửi email ở bước 12 thất bại, lời mời vẫn được bảo toàn trong database (vì transaction đã commit), nhưng hệ thống sẽ ghi log lỗi gửi mail để xử lý sau.

3.4.1.10. Sơ đồ tuần tự đồng ý lời mời (Sequence Diagram Accept)



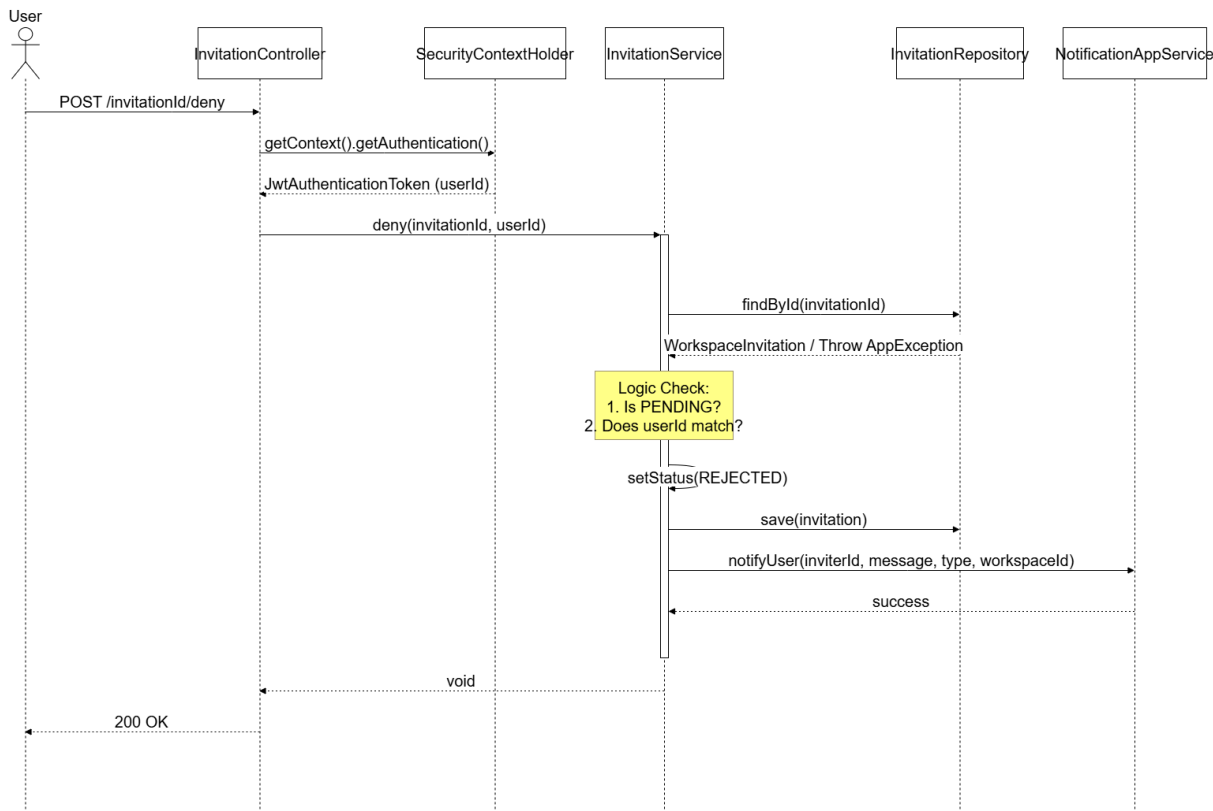
Hình 22: Sơ đồ tuần tự đồng ý lời mời (Sequence Diagram Accept)

Bảng 13: Đặc tả chức năng đồng ý lời mời (Accept)

Thành phần	Nội dung chi tiết
Tên usecase	Chấp nhận lời mời vào không gian làm việc.
Tác nhân	Người nhận được lời mời tham gia Workspace.
Mô tả	Người dùng đồng ý tham gia vào một không gian làm việc dựa trên lời mời đã nhận. Hệ thống sẽ kiểm tra tính hợp lệ của lời mời, thiết lập vai trò thành viên và cập nhật trạng thái lời mời.
Sự kiện kích hoạt	Người dùng nhấn nút "Chấp nhận" (Accept) từ email hoặc thông báo trong ứng dụng.
Điều kiện trước	- Người dùng đã đăng nhập và có mã định danh (userId) hợp lệ từ JWT. - Lời mời (invitationId) phải tồn tại trong hệ thống.
Điều kiện sau	- Người dùng trở thành thành viên chính thức của Workspace với vai trò MEMBER. - Trạng thái lời mời được cập nhật thành ACCEPTED.

	- Quản trị viên (người mời) nhận được thông báo về việc lời mời đã được chấp nhận.
Luồng sự kiện chính	1. Client (Invitee) gửi yêu cầu POST /{invitationId}/accept. 2. WorkspaceController lấy userId từ SecurityContext và gọi accept(invitationId, userId) tới InvitationService. 3. InvitationService tìm kiếm lời mời thông qua InvitationRepository. 4. InvitationService kiểm tra logic (Validation): • Lời mời có tồn tại không? • Trạng thái có phải là PENDING không? • Lời mời còn hạn không (ExpiredAt)? • Có đúng userId được mời không? 5. InvitationService lấy thông tin hồ sơ (Profile) và không gian làm việc (Workspace) liên quan. 6. InvitationService kiểm tra quyền thành viên hiện tại. Nếu chưa có, tiến hành lấy hoặc tạo vai trò MEMBER cho Workspace đó. 7. InvitationService gọi save(WorkspaceMember) để thêm người dùng vào danh sách thành viên của Workspace. 8. InvitationService cập nhật trạng thái lời mời: save(invitation with Status=ACCEPTED). 9. InvitationService gọi notifyUser(...) qua NotificationAppService để báo cho người mời rằng lời mời đã được chấp nhận. 10. WorkspaceController trả về mã phản hồi 200 OK và workspaceId cho người dùng.
Luồng sự kiện thay thế	Lời mời không hợp lệ: • Tại bước 4, nếu bất kỳ điều kiện kiểm tra nào thất bại (ví dụ: lời mời đã hết hạn hoặc đã được chấp nhận trước đó), hệ thống ném ngoại lệ và trả về lỗi cho người dùng. Người dùng đã là thành viên: • Nếu người dùng đã có tên trong danh sách thành viên của Workspace đó, hệ thống có thể bỏ qua bước tạo thành viên mới và chỉ cập nhật trạng thái lời mời.

3.4.1.11. Sơ đồ tuần tự từ chối lời mời (Sequence Diagram Deny)



Hình 23: Sơ đồ tuần tự từ chối lời mời (Sequence Diagram Deny)

Bảng 14: Đặc tả chức năng từ chối lời mời (Deny)

Thành phần	Nội dung chi tiết
Tên usecase	Từ chối lời mời tham gia không gian làm việc.
Tác nhân	User (Người dùng): Người nhận được lời mời và không muốn tham gia Workspace.
Mô tả	Cho phép người dùng từ chối một lời mời tham gia Workspace. Hệ thống sẽ cập nhật trạng thái lời mời và thông báo cho người gửi biết yêu cầu đã bị từ chối.
Sự kiện kích hoạt	Người dùng nhấn nút "Từ chối" (Deny) trong danh sách lời mời hoặc từ email thông báo.
Điều kiện trước	- Người dùng đã đăng nhập (Hệ thống có thể trích xuất userId từ JwtAuthenticationToken). - Lời mời tồn tại trong cơ sở dữ liệu và đang ở trạng thái chờ xử lý (PENDING).

Điều kiện sau	- Trạng thái lời mời được cập nhật thành REJECTED. - Người mời (Admin) nhận được thông báo về việc lời mời bị từ chối. - Không có bản ghi thành viên nào được tạo ra trong Workspace.
Luồng sự kiện chính	1. User gửi yêu cầu POST /invitationId/deny. 2. InvitationController lấy thông tin xác thực từ SecurityContextHolder để xác định userId. 3. InvitationController gọi phương thức deny(invitationId, userId) tới InvitationService. 4. InvitationService tìm kiếm lời mời bằng findById(invitationId) thông qua InvitationRepository. 5. InvitationService kiểm tra logic (Logic Check): • Trạng thái lời mời có phải là PENDING không? • userId hiện tại có khớp với người được mời không? 6. InvitationService thực hiện setStatus(REJECTED) cho đối tượng lời mời. 7. InvitationService gọi save(invitation) để lưu thay đổi vào InvitationRepository. 8. InvitationService gọi notifyUser(...) qua NotificationAppService để gửi thông báo cho người mời (inviterId). 9. InvitationService phản hồi trạng thái thành công (success) về cho Controller. 10. InvitationController trả về mã phản hồi 200 OK cho người dùng.
Luồng sự kiện thay thế	Lời mời không tồn tại hoặc sai thông tin: • Tại bước 5, nếu không tìm thấy lời mời hoặc thông tin userId không khớp, hệ thống ném ra ApplicationException. Lời mời đã được xử lý: • Nếu trạng thái không phải là PENDING (đã chấp nhận hoặc đã từ chối trước đó), hệ thống sẽ báo lỗi logic và không thực hiện cập nhật.

3.4.1.12. Sơ đồ tuần tự lấy lời mời đang chờ xử lý (Sequence Diagram Get My Pending Invitation)



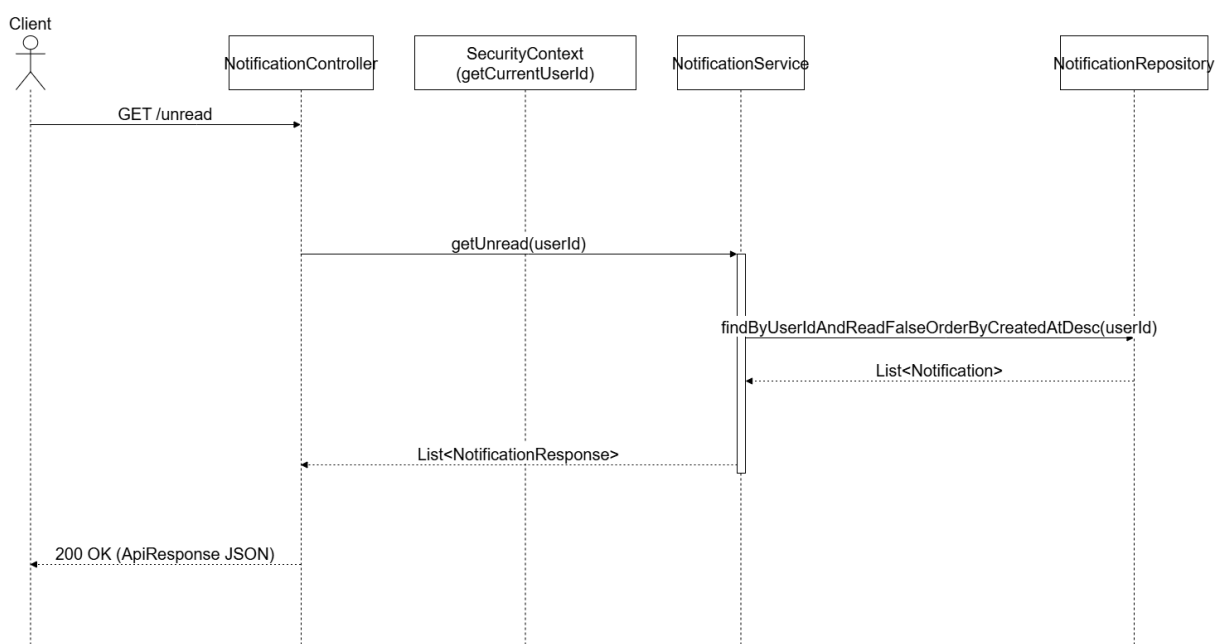
Hình 24: Sơ đồ tuần tự lấy lời mời đang chờ xử lý (Sequence Diagram Get My Pending Invitation)

Bảng 15: Đặc tả chức năng lấy lời mời đang chờ xử lý (Get My Pending Invitation)

Thành phần	Nội dung chi tiết
Tên usecase	Xem danh sách lời mời đang chờ xử lý.
Tác nhân	Người dùng muốn kiểm tra các lời mời tham gia không gian làm việc mà họ chưa phản hồi.
Mô tả	Cho phép người dùng xem tất cả các lời mời vào Workspace hiện đang ở trạng thái chờ (PENDING) dành riêng cho tài khoản của họ.
Sự kiện kích hoạt	Người dùng truy cập vào mục "Lời mời" hoặc trung tâm thông báo trên giao diện ứng dụng.
Điều kiện trước	- Người dùng đã đăng nhập thành công vào hệ thống. - Yêu cầu gửi đi phải có mã thông báo định danh (JWT) hợp lệ để hệ thống trích xuất userId.
Điều kiện sau	- Danh sách các lời mời (nếu có) được hiển thị đầy đủ thông tin cho người dùng.

Luồng sự kiện chính	1. Client/User gửi yêu cầu GET /pending tới hệ thống. 2. InvitationController gọi getContext().getAuthentication() từ SecurityContextHolder để xác thực phiên làm việc. 3. InvitationController trích xuất userId từ JwtAuthenticationToken. 4. InvitationController gọi phương thức getMyPendingInvitations(userId) của InvitationService. 5. InvitationService thực hiện truy vấn findByInviteeUserIdAndStatus(userId, PENDING) thông qua InvitationRepository. 6. InvitationRepository trình lục lại cơ sở dữ liệu và trả về List<WorkspaceInvitation>. 7. InvitationService thực hiện ánh xạ (map) danh sách thực thể sang danh sách phản hồi List<InvitationResponse>. 8. InvitationController nhận kết quả từ Service và trả về mã 200 OK kèm theo danh sách dữ liệu dưới dạng JSON. 9. Client/User hiển thị danh sách lời mời lên màn hình.
Luồng sự kiện thay thế	Trường hợp không có lời mời nào: • Tại bước 6, InvitationRepository trả về một danh sách rỗng (Empty List). • Hệ thống vẫn trả về mã 200 OK nhưng với mảng dữ liệu trống []. • Giao diện người dùng hiển thị thông báo: "Bạn không có lời mời nào đang chờ xử lý".

3.4.1.13. Sơ đồ tuần tự lấy thông báo chưa đọc (Sequence Diagram Get Unread)



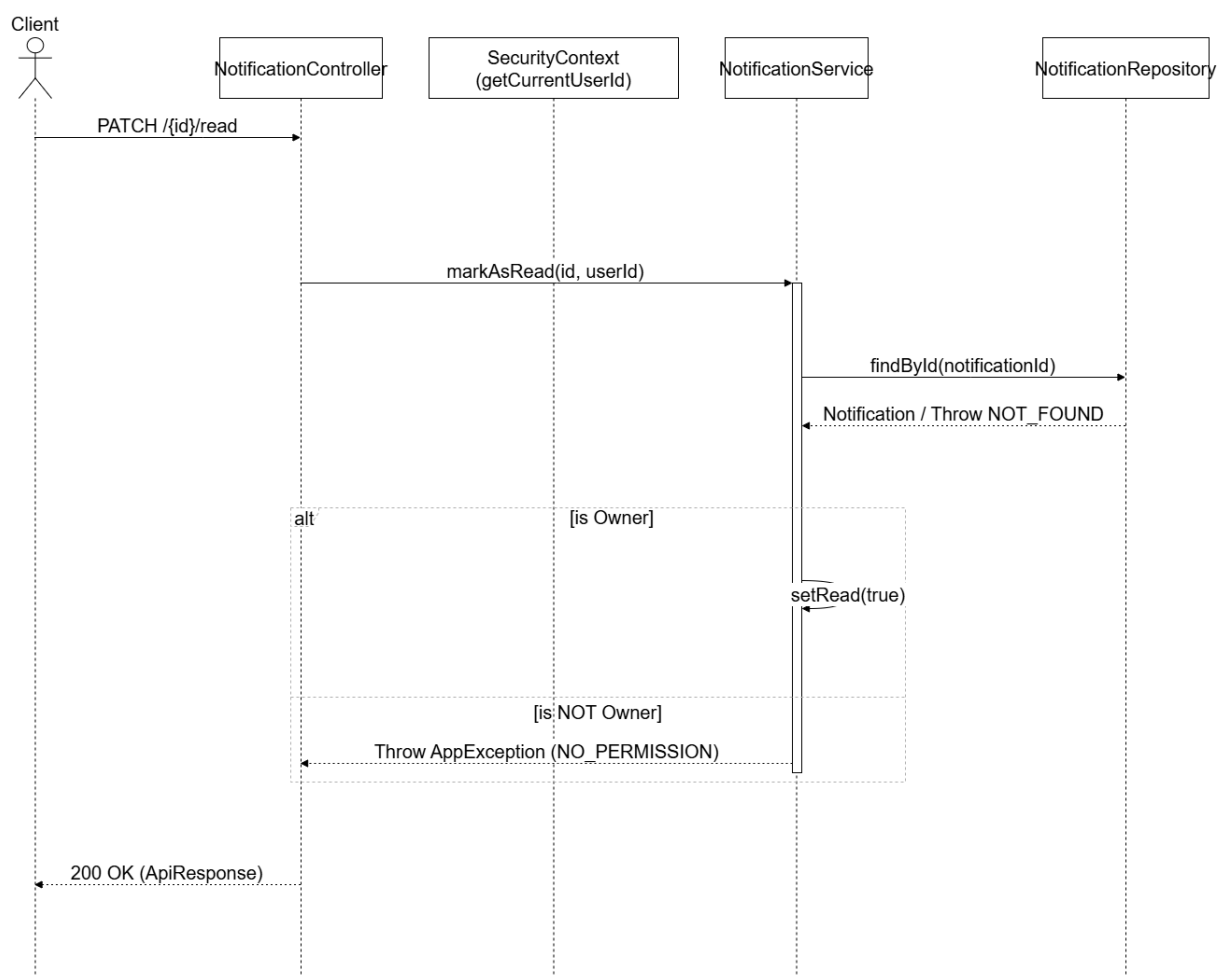
Hình 25: Sơ đồ tuần tự lấy thông báo chưa đọc (Sequence Diagram Get Unread)

Bảng 16: Đặc tả chức năng lấy thông báo chưa đọc (Get Unread)

Thành phần	Nội dung chi tiết
Tên usecase	Xem danh sách thông báo chưa đọc.
Tác nhân	Người dùng cuối muốn kiểm tra các thông báo mới.
Mô tả	Cho phép người dùng lấy toàn bộ danh sách các thông báo chưa đọc của mình. Hệ thống sẽ truy xuất dữ liệu từ cơ sở dữ liệu và sắp xếp theo thứ tự thời gian mới nhất.
Sự kiện kích hoạt	Người dùng nhấn vào biểu tượng thông báo (hình cái chuông) trên giao diện ứng dụng.
Điều kiện trước	- Người dùng đã đăng nhập thành công vào hệ thống. - Hệ thống xác định được userId hiện tại thông qua Security Context.
Điều kiện sau	- Danh sách thông báo chưa đọc được trả về và hiển thị trên màn hình của người dùng.
Luồng sự kiện chính	1. User gửi yêu cầu GET /unread tới hệ thống. 2. NotificationController gọi getCurrentUserId() từ SecurityContext để xác định danh tính người dùng.

	3. NotificationController chuyển tiếp yêu cầu xử lý bằng cách gọi getUnread(userId) tới NotificationService. 4. NotificationService thực hiện truy vấn thông qua NotificationRepository với phương thức findByUserIdAndReadFalseOrderByCreatedAtDesc(userId). 5. NotificationRepository tìm kiếm trong cơ sở dữ liệu và trả về danh sách thực thể List<Notification>. 6. NotificationService ánh xạ (Map) danh sách thực thể sang danh sách phản hồi List<NotificationResponse>. 7. NotificationController nhận kết quả từ Service và đóng gói vào ApiResponse JSON. 8. Client nhận phản hồi 200 OK và hiển thị các thông báo chưa đọc cho người dùng.
Luồng sự kiện thay thế	Trường hợp không có thông báo mới: • Tại bước 5, NotificationRepository trả về một danh sách rỗng. • Hệ thống vẫn trả về mã 200 OK với mảng dữ liệu trống. • Giao diện người dùng hiển thị thông báo: "Bạn không có thông báo mới nào".

3.4.1.14. Sơ đồ tuần tự đánh dấu thông báo là đã đọc (Sequence Diagram Mark As Read)



Hình 26: Sơ đồ tuần tự đánh dấu thông báo là đã đọc (Sequence Diagram Mark As Read)

Bảng 17: Đặc tả chức năng đánh dấu thông báo là đã đọc (Mark As Read)

Thành phần	Nội dung chi tiết
Tên usecase	Đánh dấu thông báo đã đọc.
Tác nhân	Người dùng tương tác với hệ thống.
Mô tả	Cho phép người dùng chuyển trạng thái của một thông báo cụ thể từ "Chưa đọc" sang "Đã đọc". Hệ thống sẽ kiểm tra quyền sở hữu để đảm bảo người dùng chỉ có thể đánh dấu thông báo của chính họ.
Sự kiện kích hoạt	Người dùng nhấn vào một thông báo cụ thể hoặc nhấn nút "Đánh dấu đã đọc" trên giao diện

Điều kiện trước	- Người dùng đã đăng nhập và có phiên làm việc hợp lệ. - Thông báo (id) phải tồn tại trong cơ sở dữ liệu.
Điều kiện sau	- Trạng thái read của thông báo được cập nhật thành true. - Thông báo không còn xuất hiện trong danh sách "Chưa đọc".
Luồng sự kiện chính	1. Client gửi yêu cầu PATCH /{id}/read tới hệ thống. 2. NotificationController gọi getCurrentUserId() từ SecurityContext để xác định chủ thể yêu cầu. 3. NotificationController gọi phương thức markAsRead(id, userId) của NotificationService. 4. NotificationService gọi findById(notificationId) từ NotificationRepository để lấy thông tin thông báo. 5. Hệ thống kiểm tra quyền sở hữu (Ownership Check): • Trường hợp [is Owner]: Nếu userId của thông báo khớp với userId người gửi yêu cầu. • Thực hiện setRead(true) cho đối tượng thông báo. 6. NotificationService lưu thay đổi vào cơ sở dữ liệu và phản hồi thành công về cho Controller. 7. NotificationController trả về mã phản hồi 200 OK (ApiResponse) cho Client.
Luồng sự kiện thay thế	Không tìm thấy thông báo: • Tại bước 4, nếu không tìm thấy bản ghi, hệ thống ném ngoại lệ NOT_FOUND. Không có quyền truy cập (is NOT Owner): • Tại bước 6, nếu người dùng cố gắng đánh dấu thông báo của người khác. • Hệ thống ném ngoại lệ ApplicationException(NO_PERMISSION). • Trả về mã lỗi tương ứng cho Client.

3.4.1.15. Sơ đồ tuần tự đánh dấu tất cả thông báo đã đọc (Sequence Diagram Mark all as Read Notification)



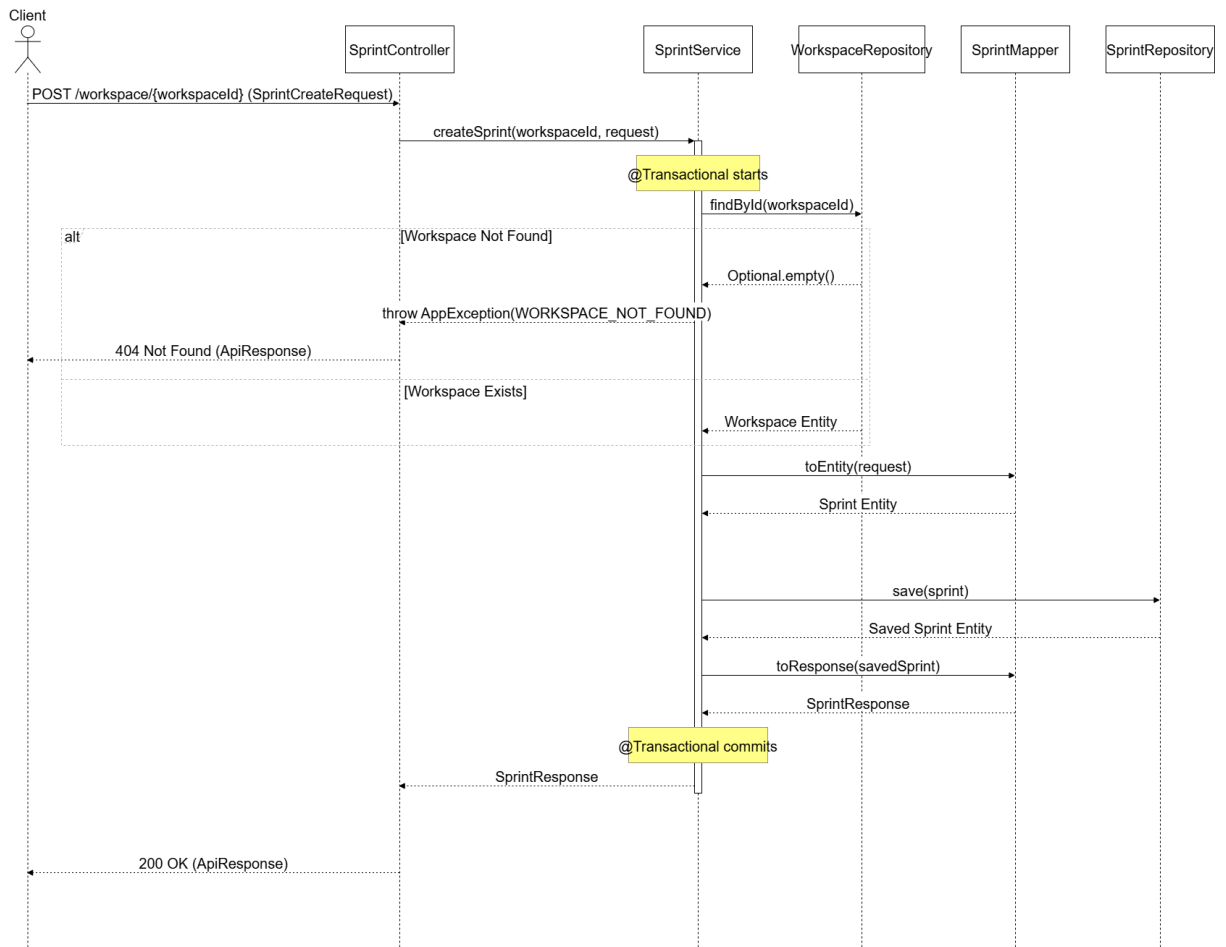
Hình 27. Sơ đồ tuần tự đánh dấu tất cả thông báo đã đọc (Sequence Diagram Mark all as Read Notification)

Bảng 18: Đặc tả chức năng đánh dấu tất cả thông báo đã đọc

Thành phần	Nội dung chi tiết
Tên usecase	Đánh dấu tất cả thông báo là đã đọc.
Tác nhân	Người muốn xóa trạng thái "chưa đọc" cho toàn bộ thông báo của mình.
Mô tả	Cho phép người dùng cập nhật trạng thái của tất cả các thông báo thuộc sở hữu của mình từ "Chưa đọc" sang "Đã đọc" chỉ bằng một thao tác duy nhất.
Sự kiện kích hoạt	Người dùng nhấn vào nút "Đánh dấu tất cả là đã đọc" (Mark all as read) trong giao diện thông báo.
Điều kiện trước	- Người dùng đã đăng nhập thành công và có phiên làm việc hợp lệ. - Hệ thống xác định được danh tính người dùng thông qua mã định danh (userId).
Điều kiện sau	- Tất cả các bản ghi thông báo có userId tương ứng trong cơ sở dữ

	liệu đều có thuộc tính read = true. - Giao diện người dùng cập nhật số lượng thông báo chưa đọc về 0.
Luồng sự kiện chính	1. Client gửi yêu cầu PATCH /read-all tới hệ thống. 2. NotificationController gọi getCurrentUserId() từ SecurityContext để lấy ID của người dùng hiện tại. 3. NotificationController chuyển tiếp yêu cầu xử lý bằng cách gọi markAllAsRead(userId) tới NotificationService. 4. NotificationService gọi phương thức markAllAsReadByUserId(userId) của NotificationRepository. 5. NotificationRepository thực thi câu lệnh SQL: UPDATE Notification n SET n.read = true WHERE n.userId = :userId. 6. Database cập nhật hàng loạt các bản ghi phù hợp và trả về kết quả thành công (void). 7. NotificationController nhận phản hồi từ Service và trả về mã trạng thái 200 OK cho Client.
Luồng sự kiện thay thế	Trường hợp người dùng không có thông báo nào: • Hệ thống vẫn thực thi câu lệnh SQL (không có bản ghi nào bị ảnh hưởng). • Trả về mã 200 OK như bình thường vì về mặt logic, mục tiêu "tất cả đã đọc" đã được đáp ứng.

3.4.1.16: Sơ đồ tuần tự tạo Sprint (Sequence Diagram Create Sprint)



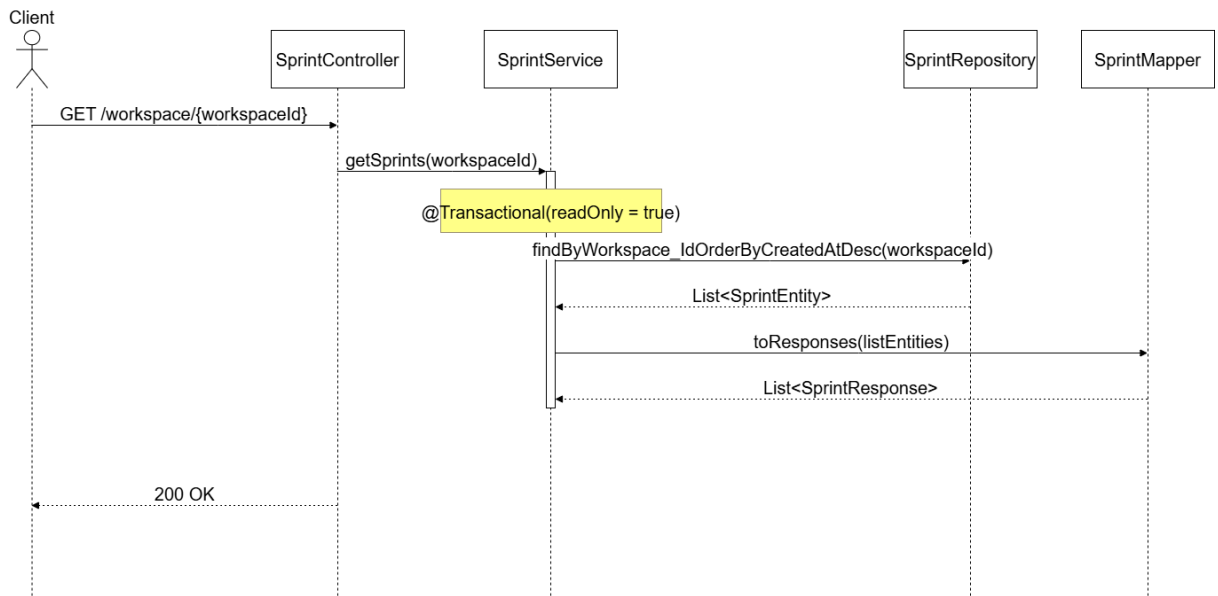
Hình 28: Sơ đồ tuần tự tạo Sprint (Sequence Diagram Create Sprint)

Bảng 19: Đặc tả chức năng tạo Sprint (Create Sprint)

Thành phần	Nội dung chi tiết
Tên usecase	Tạo Sprint mới.
Tác nhân	Người dùng (thường là quản trị viên hoặc quản lý dự án) thực hiện tạo chu kỳ làm việc.
Mô tả	Cho phép người dùng tạo một Sprint mới trong một Không gian làm việc (Workspace) cụ thể. Hệ thống sẽ kiểm tra sự tồn tại của Workspace trước khi lưu thông tin Sprint.
Sự kiện kích hoạt	Người dùng gửi yêu cầu tạo Sprint từ giao diện quản lý dự án.
Điều kiện trước	- Người dùng đã đăng nhập và có quyền tác động vào Workspace mục tiêu.

	- WorkspaceId cung cấp phải chính xác.
Điều kiện sau	- Một thực thể Sprint mới được lưu trữ thành công trong cơ sở dữ liệu. - Mọi thay đổi dữ liệu được đảm bảo tính toàn vẹn thông qua Transaction.
Luồng sự kiện chính	1. Client gửi yêu cầu POST /workspace/{workspaceId} kèm theo SprintCreateRequest. 2. SprintController tiếp nhận yêu cầu và gọi createSprint(workspaceId, request) tới SprintService. 3. SprintService bắt đầu Giao dịch (@Transactional starts). 4. SprintService gọi findById(workspaceId) từ WorkspaceRepository. 5. Hệ thống kiểm tra điều kiện [Workspace Exists]: Xác nhận Workspace có tồn tại. 6. SprintService gọi toEntity(request) qua SprintMapper để chuyển đổi dữ liệu yêu cầu sang thực thể Sprint. 7. SprintService gọi save(sprint) tới SprintRepository để lưu xuống cơ sở dữ liệu. 8. SprintService gọi toResponse(savedSprint) qua SprintMapper để tạo đối tượng phản hồi. 9. SprintService hoàn tất Giao dịch (@Transactional commits). 10. SprintController trả về SprintResponse kèm mã 200 OK (ApiResponse).
Luồng sự kiện thay thế	Không tìm thấy Không gian làm việc (Workspace Not Found): • Tại bước 5, nếu WorkspaceRepository trả về Optional.empty(). • SprintService ném ra ngoại lệ ApplicationException(WORKSPACE_NOT_FOUND). • SprintController trả về mã lỗi 404 Not Found cho Client.

3.4.1.17: Sơ đồ tuần tự lấy danh sách tất cả sprints (Sequence Diagram Get All Sprints)



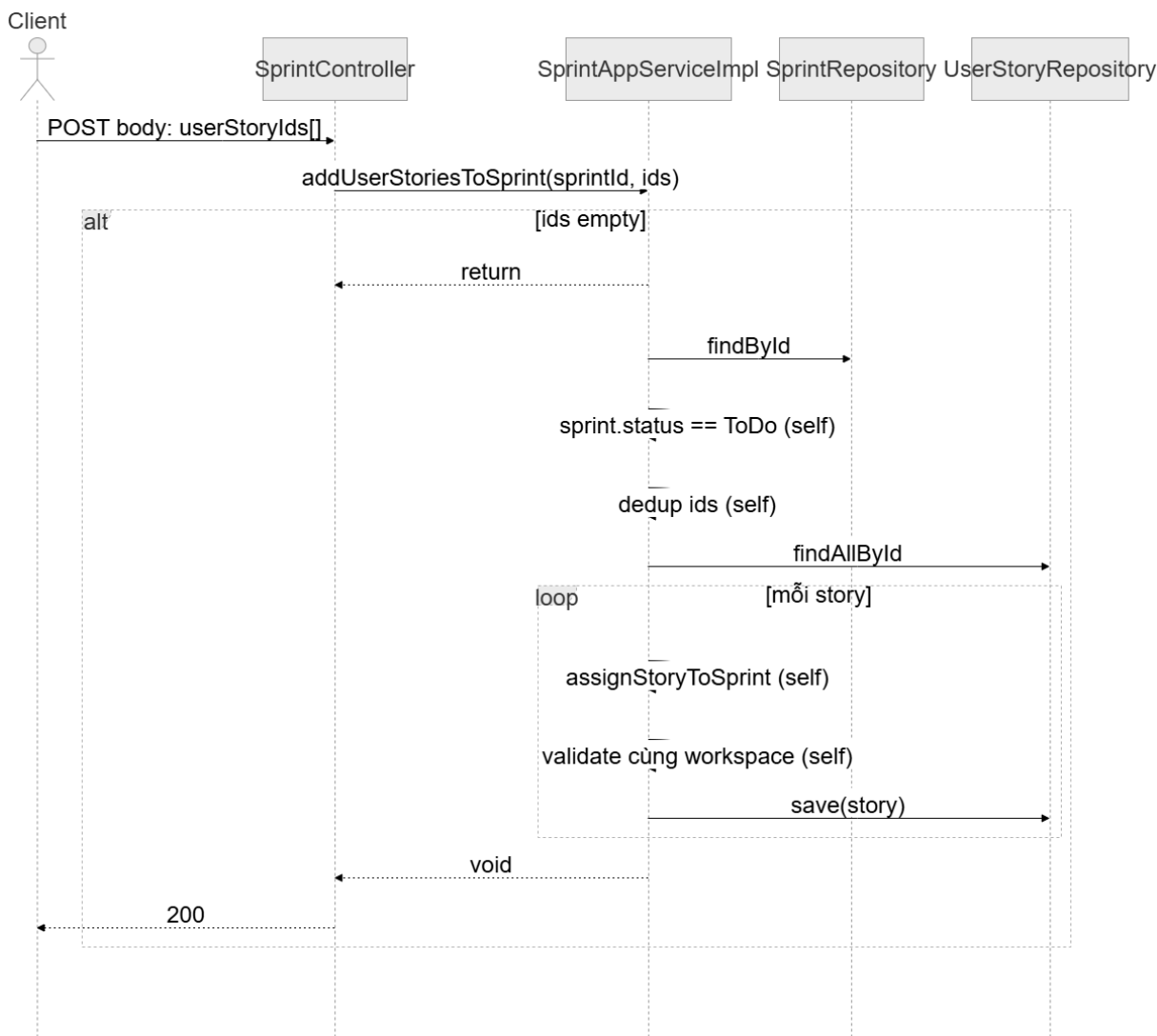
Hình 29: Sơ đồ tuần tự lấy danh sách tất cả sprints (Sequence Diagram Get All Sprints)

Bảng 20: Đặc tả chức lấy danh sách tất cả sprints (Get All Sprints)

Thành phần	Nội dung chi tiết
Tên usecase	Xem danh sách Sprint trong Workspace.
Tác nhân	Người dùng muốn xem lộ trình các chu kỳ làm việc.
Mô tả	Truy xuất toàn bộ danh sách các Sprint thuộc về một Không gian làm việc (Workspace) cụ thể, được sắp xếp theo thời gian khởi tạo giảm dần.
Sự kiện kích hoạt	Người dùng truy cập vào tab "Sprints" hoặc "Backlog" trong giao diện Workspace.
Điều kiện trước	- Người dùng đã được xác thực quyền truy cập vào Workspace (workspaceId). - workspaceId phải tồn tại trong hệ thống.
Điều kiện sau	- Danh sách Sprint được hiển thị cho người dùng theo thứ tự mới nhất lên đầu.
Luồng sự kiện chính	1. Client gửi yêu cầu GET /workspace/{workspaceId} tới hệ thống. 2. SprintController tiếp nhận yêu cầu và gọi getSprints(workspaceId) của SprintService. 3. SprintService bắt đầu giao dịch chỉ đọc

	((@Transactional(readOnly = true))). 4. SprintService gọi findByWorkspace_IdOrderByCreatedAtDesc(workspaceId) từ SprintRepository. 5. SprintRepository truy vấn cơ sở dữ liệu và trả về List<SprintEntity>. 6. SprintService gọi toResponses(listEntities) thông qua SprintMapper để chuyển đổi sang định dạng Response. 7. SprintService trả về List<SprintResponse> cho Controller. 8. SprintController phản hồi mã 200 OK kèm danh sách dữ liệu cho Client.
Luồng sự kiện thay thế	Trường hợp Workspace chưa có Sprint nào: • Tại bước 5, SprintRepository trả về một danh sách rỗng. • Hệ thống vẫn trả về 200 OK với mảng trống []. • Giao diện hiển thị: "Chưa có Sprint nào được tạo trong Workspace này".

3.4.1.18: Sơ đồ tuần tự thêm User Story vào Sprint (Sequence Diagram Add User Story To Sprint)



Hình 30: Sơ đồ tuần tự thêm User Story vào Sprint (Sequence Diagram Add User Story To Sprint)

Bảng 21: Đặc tả chức năng thêm User Story vào Sprint (Add User Story To Sprint)

Thành phần	Nội dung chi tiết
Tên usecase	Thêm User Story vào Sprint (Add User Stories to Sprint).
Tác nhân	Người dùng thực hiện điều phối các yêu cầu vào chu kỳ làm việc.
Mô tả	Ca sử dụng này cho phép người dùng gán một hoặc nhiều User Story vào một Sprint cụ thể. Hệ thống sẽ kiểm tra trạng thái của Sprint (phải ở trạng thái ToDo), lọc bỏ các ID trùng lặp, kiểm tra tính hợp lệ (thuộc cùng Workspace) và tiến hành lưu thông tin cập

	nhập cho từng User Story.
Sự kiện kích hoạt	Client gửi yêu cầu API POST (kèm body chứa mảng userStoryIds[]) tới hệ thống.
Điều kiện trước	- Sprint với ID tương ứng tồn tại trong cơ sở dữ liệu và đang ở trạng thái ToDo. - Các User Story trong danh sách truyền vào phải tồn tại và thuộc cùng Workspace với Sprint.
Điều kiện sau	- Các User Story hợp lệ được cập nhật liên kết tới Sprint mục tiêu trong cơ sở dữ liệu.
Luồng sự kiện chính	1. Client gửi yêu cầu HTTP POST kèm theo body chứa danh sách userStoryIds[] tới SprintController. 2. SprintController tiếp nhận và gọi phương thức addUserStoriesToSprint(sprintId, ids) của SprintAppServiceImpl. 3. SprintAppServiceImpl gọi hàm findById tới SprintRepository để truy vấn thông tin Sprint. 4. SprintAppServiceImpl tự kiểm tra trạng thái của Sprint (đảm bảo sprint.status == ToDo). 5. SprintAppServiceImpl thực hiện loại bỏ các ID trùng lặp trong danh sách đầu vào (dedup ids). 6. SprintAppServiceImpl gọi findAllById tới UserStoryRepository để lấy danh sách các đối tượng User Story. 7. Hệ thống bắt đầu vòng lặp (loop) để xử lý từng User Story: • Tự thực hiện gán User Story vào Sprint (assignStoryToSprint). • Tự kiểm tra tính hợp lệ để đảm bảo User Story và Sprint thuộc cùng một workspace (validate cùng workspace). • Gọi hàm save(story) tới UserStoryRepository để lưu thông tin cập nhật vào cơ sở dữ liệu. 8. Sau khi vòng lặp kết thúc, SprintAppServiceImpl trả về void cho SprintController. 9. SprintController trả về mã HTTP 200 (OK) cho Client.
Luồng sự kiện thay thế	Danh sách ID truyền vào bị trống (alt: ids empty): • Tại bước 2 của luồng chính, nếu SprintAppServiceImpl phát hiện danh sách userStoryIds[] trống. • Hàm sẽ kết thúc sớm (return) và trả về trực tiếp cho SprintController mà không truy vấn cơ sở dữ liệu hay lưu trữ gì thêm. • SprintController vẫn trả về mã HTTP 200 cho Client.

3.4.1.19: Sơ đồ tuần tự xóa User Story ra khỏi Sprint (Sequence Diagram Remove User Story From Sprint)



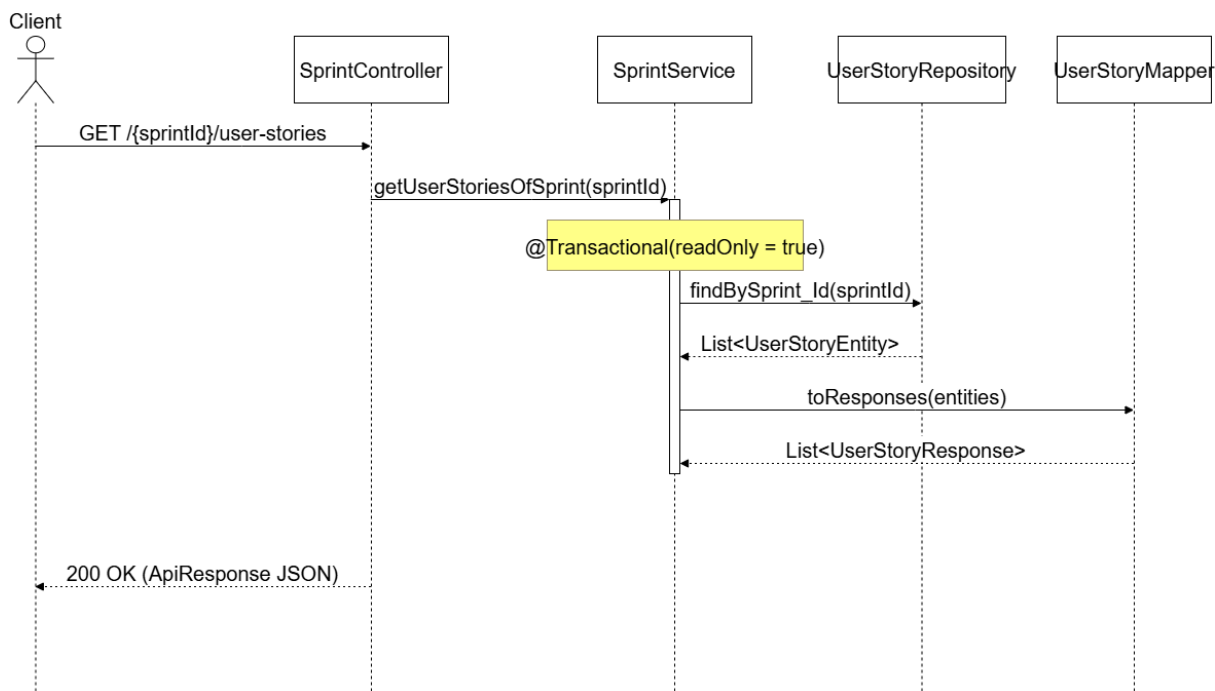
Hình 31: Sơ đồ tuần tự xóa User Story ra khỏi Sprint (Sequence Diagram Remove User Story From Sprint)

Bảng 22: Đặc tả chức năng xóa User Story ra khỏi Sprint

Thành phần	Nội dung chi tiết
Tên usecase	Xóa User Story khỏi Sprint.
Tác nhân	Người dùng (Quản lý dự án/Scrum Master) thực hiện điều chỉnh kế hoạch Sprint.
Mô tả	Cho phép người dùng gỡ bỏ một User Story ra khỏi Sprint hiện tại. User Story sau khi gỡ bỏ sẽ không còn thuộc về Sprint đó (trạng thái sprint_id trở thành null).

Sự kiện kích hoạt	Client gửi yêu cầu xóa thông qua API DELETE /user-stories/{userId}.
Điều kiện trước	- Người dùng đã đăng nhập và có quyền chỉnh sửa trong không gian làm việc. - User Story mục tiêu phải tồn tại trong hệ thống.
Điều kiện sau	- Liên kết giữa User Story và Sprint bị hủy bỏ. - Cơ sở dữ liệu được cập nhật thành công (trường sprint_id của User Story chuyển về null).
Luồng sự kiện chính	1. Client gửi yêu cầu DELETE /user-stories/{userId} tới hệ thống. 2. SprintController tiếp nhận yêu cầu và gọi phương thức removeUserStoryFromSprint(userId) của SprintService. 3. SprintService bắt đầu giao dịch (@Transactional starts). 4. SprintService gọi findById(userId) từ UserStoryRepository để kiểm tra sự tồn tại của story. 5. Hệ thống trường hợp [Story Found]: • Nhận thực thể UserStory Entity từ Repository. • Thực hiện lệnh story.setSprint(null) để gỡ bỏ liên kết với Sprint. 6. SprintService hoàn tất giao dịch (@Transactional commits): Cập nhật bản ghi xuống Database (SET sprint_id = null). 7. SprintController trả về mã phản hồi 200 OK (ApiResponse) cho Client.
Luồng sự kiện thay thế	Không tìm thấy User Story (Story Not Found): • Tại bước 4, nếu UserStoryRepository trả về Optional.empty(). • SprintService ném ra ngoại lệ ApplicationException(USER_STORY_NOT_FOUND). • SprintController phản hồi mã lỗi 404 Not Found về cho Client.

3.4.1.20: Sơ đồ tuần tự lấy tất cả User Stories trong Sprint (Sequence Diagram Get User Stories Of Sprint)



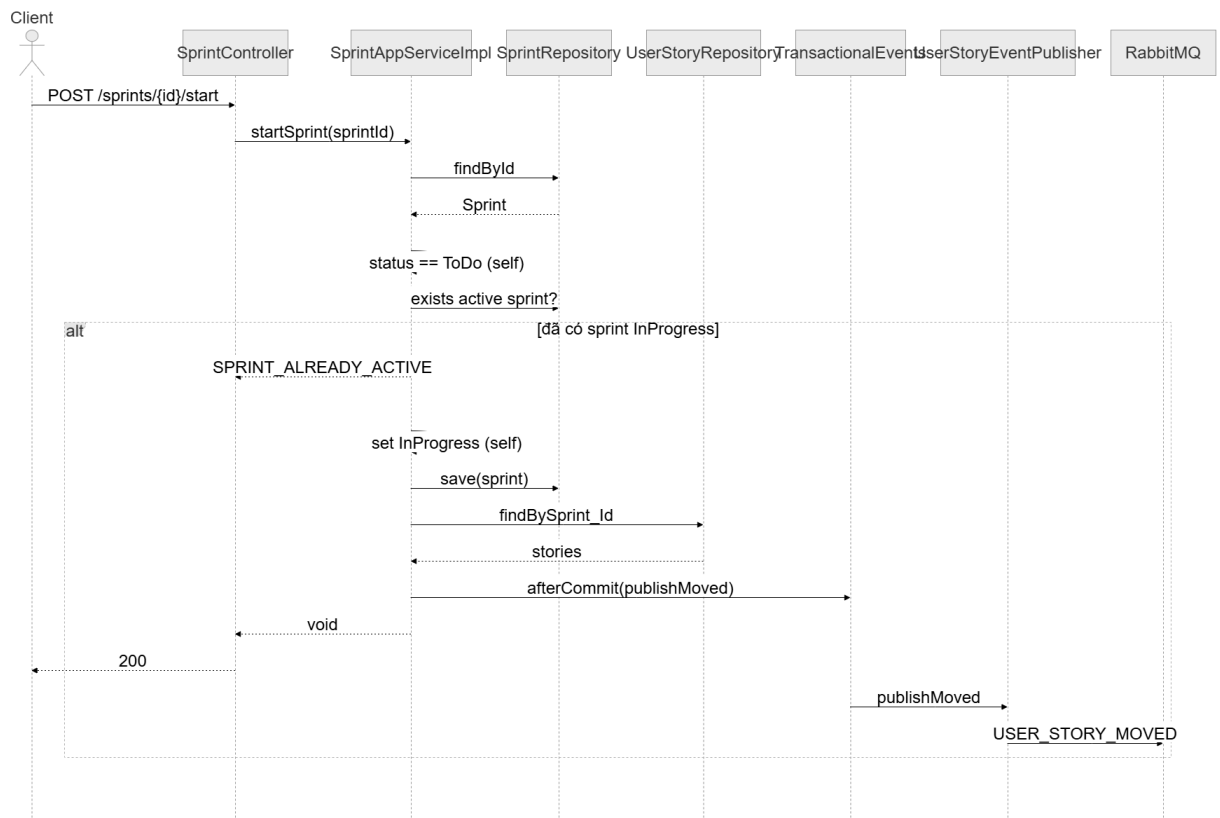
Hình 32: Sơ đồ tuần tự lấy tất cả User Stories trong Sprint (Sequence Diagram Get User Stories Of Sprint)

Bảng 23: Đặc tả chức năng lấy tất cả User Stories trong Sprint (Get User Stories Of Sprint)

Thành phần	Nội dung chi tiết
Tên usecase	Xem danh sách User Story của Sprint.
Tác nhân	Người dùng muốn kiểm tra các yêu cầu/công việc đã được lập kế hoạch cho một chu kỳ cụ thể.
Mô tả	Truy xuất toàn bộ các User Story đã được gán vào một Sprint cụ thể để hiển thị trên giao diện quản lý.
Sự kiện kích hoạt	Người dùng chọn vào một Sprint cụ thể trong danh sách.
Điều kiện trước	- Người dùng đã đăng nhập và có quyền xem thông tin trong Workspace chứa Sprint này. - sprintId cung cấp phải tồn tại trong hệ thống.
Điều kiện sau	- Danh sách các User Story thuộc Sprint được trả về đầy đủ.
Luồng sự kiện chính	1. Client gửi yêu cầu GET /{sprintId}/user-stories tới hệ thống. 2. SprintController tiếp nhận yêu cầu và gọi getUserStoriesOfSprint(sprintId) tới SprintService.

	3. SprintService bắt đầu giao dịch chỉ đọc (@Transactional(readOnly = true)). 4. SprintService gọi phương thức findBySprint_Id(sprintId) từ UserStoryRepository. 5. UserStoryRepository thực hiện truy vấn cơ sở dữ liệu và trả về List<UserStoryEntity>. 6. SprintService gọi toResponses(entities) thông qua UserStoryMapper để chuyển đổi dữ liệu thực thể sang DTO. 7. SprintService trả về danh sách List<UserStoryResponse> cho Controller. 8. SprintController phản hồi mã 200 OK (ApiResponse JSON) kèm danh sách dữ liệu cho Client.
Luồng sự kiện thay thế	Trường hợp Sprint trống (No User Stories): • Tại bước 5, UserStoryRepository trả về danh sách rỗng. • Hệ thống vẫn trả về 200 OK với dữ liệu là một mảng trống []. • Giao diện người dùng sẽ hiển thị thông báo: "Sprint này chưa có User Story nào".

3.4.1.21: Sơ đồ tuần tự bắt đầu một Sprint (Sequence Diagram Start Sprint)



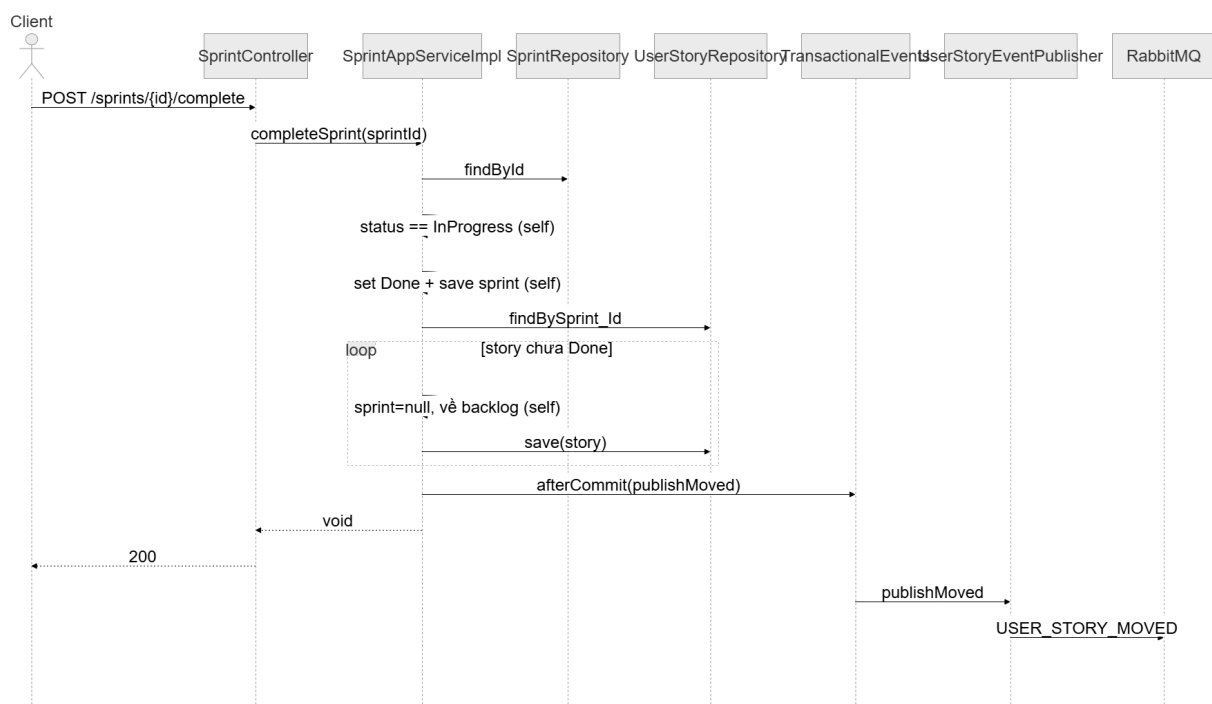
Hình 33: Sơ đồ tuần tự bắt đầu một Sprint (Sequence Diagram Start Sprint)

Bảng 24: Đặc tả chức năng bắt đầu một Sprint (Start Sprint)

Thành phần	Nội dung chi tiết
Tên usecase	Bắt đầu Sprint.
Tác nhân	Người dùng thực hiện kích hoạt chu kỳ làm việc.
Mô tả	Ca sử dụng này cho phép người dùng bắt đầu một Sprint. Hệ thống sẽ kiểm tra xem Sprint đó có hợp lệ để bắt đầu hay không (phải ở trạng thái ToDo và không có Sprint nào khác đang chạy). Nếu hợp lệ, hệ thống cập nhật trạng thái Sprint thành InProgress và phát ra sự kiện thông báo (qua RabbitMQ) về việc các User Story trong Sprint này đã được chuyển trạng thái.
Sự kiện kích hoạt	Client gửi yêu cầu kích hoạt thông qua API POST /{sprintId}/start.
Điều kiện trước	- Sprint với {id} tương ứng tồn tại trong cơ sở dữ liệu. - Sprint đang ở trạng thái ToDo. - Hệ thống hàng đợi message (RabbitMQ) đang hoạt động.
Điều kiện sau	- Trạng thái của Sprint trong cơ sở dữ liệu được cập nhật thành InProgress. - Thông điệp sự kiện USER_STORY_MOVED được gửi thành công lên RabbitMQ.
Luồng sự kiện chính	1. Client gửi yêu cầu POST /sprints/{id}/start tới SprintController. 2. SprintController tiếp nhận và gọi phương thức startSprint(sprintId) của SprintAppServiceImpl. 3. SprintAppServiceImpl gọi hàm findById tới SprintRepository để truy vấn thông tin. 4. SprintRepository trả về đối tượng Sprint. 5. SprintAppServiceImpl tự kiểm tra điều kiện trạng thái của Sprint (đảm bảo status == ToDo). 6. SprintAppServiceImpl thực hiện kiểm tra xem có Sprint nào khác đang hoạt động không (exists active sprint?).existsByWorkspace_IdAndStatus(workspaceId, InProgress). 7. Nếu không có Sprint nào đang chạy, SprintAppServiceImpl cập nhật trạng thái của Sprint hiện tại thành InProgress (set InProgress). 8. SprintAppServiceImpl gọi save(sprint) tới SprintRepository để lưu thay đổi vào cơ sở dữ liệu.

	9. SprintAppServiceImpl gọi findBySprint_Id tới UserStoryRepository và nhận về danh sách các stories thuộc Sprint này. 10. SprintAppServiceImpl đăng ký một sự kiện kích hoạt sau khi giao dịch cơ sở dữ liệu commit thành công thông qua afterCommit(publishMoved) với TransactionalEvents. 11. SprintAppServiceImpl trả về void cho SprintController. 12. SprintController trả về mã HTTP 200 (OK) cho Client, báo hiệu API đã xử lý thành công. 3. (Tiến trình chạy ngầm sau khi Commit) TransactionalEvents gọi hàm publishMoved tới UserStoryEventPublisher. 14. UserStoryEventPublisher đẩy sự kiện USER_STORY_MOVED vào hệ thống hàng đợi RabbitMQ.
Luồng sự kiện thay thế	Đã có Sprint đang hoạt động (alt: đã có sprint InProgress): • Tại bước 6 của luồng chính, nếu hệ thống phát hiện đã có một Sprint khác đang ở trạng thái InProgress. • Hệ thống sẽ hủy bỏ thao tác và ném ra lỗi SPRINT_ALREADY_ACTIVE trả về cho SprintController. • Quá trình cập nhật trạng thái và gửi sự kiện sẽ không được thực hiện.

3.4.1.22: Sơ đồ tuần tự hoàn thành Sprint (Sequence Diagram Complete Sprint)



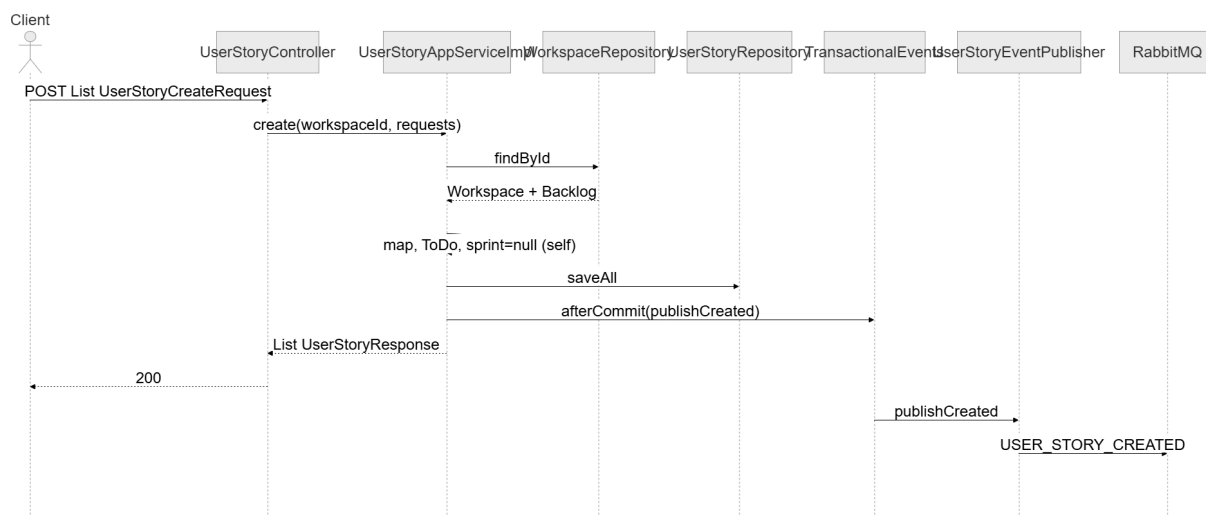
Hình 34: Sơ đồ tuần tự hoàn thành Sprint (Sequence Diagram Complete Sprint)

Bảng 25: Đặc tả chức năng hoàn thành Sprint (Complete Sprint)

Thành phần	Nội dung chi tiết
Tên usecase	Kết thúc Sprint.
Tác nhân	Người dùng thực hiện đóng chu kỳ làm việc hiện tại.
Mô tả	Ca sử dụng này cho phép người dùng kết thúc một Sprint đang diễn ra. Hệ thống sẽ cập nhật trạng thái của Sprint thành "Done". Đồng thời, hệ thống sẽ tự động quét các User Story thuộc Sprint này; những User Story chưa hoàn thành (chưa đạt trạng thái Done) sẽ được gỡ khỏi Sprint (chuyển về Backlog). Cuối cùng, hệ thống phát ra một sự kiện thông báo để cập nhật trạng thái của các User Story này qua RabbitMQ.
Sự kiện kích hoạt	Client gửi yêu cầu API POST /sprints/{id}/complete tới hệ thống.
Điều kiện trước	- Sprint với {id} tương ứng phải tồn tại trong cơ sở dữ liệu. - Sprint đang ở trạng thái InProgress (Đang thực hiện). - Hệ thống hàng đợi (RabbitMQ) đang hoạt động bình thường.
Điều kiện sau	- Sprint được cập nhật trạng thái thành Done. - Các User Story chưa hoàn thành được cập nhật trạng thái (gỡ liên kết khỏi Sprint, đưa về Product Backlog). - Sự kiện USER_STORY_MOVED được đẩy thành công lên RabbitMQ.
Luồng sự kiện chính	1. Client gửi yêu cầu POST /sprints/{id}/complete tới SprintController. 2. SprintController tiếp nhận và gọi phương thức completeSprint(sprintId) của SprintAppServiceImpl. 3. SprintAppServiceImpl gọi hàm findById tới SprintRepository để lấy thông tin chi tiết của Sprint. 4. SprintAppServiceImpl tự kiểm tra điều kiện trạng thái của Sprint (đảm bảo status == InProgress). 5. SprintAppServiceImpl cập nhật trạng thái Sprint thành Done và tiến hành lưu lại (set Done + save sprint). 6. SprintAppServiceImpl gọi findBySprint_Id tới UserStoryRepository để lấy toàn bộ danh sách các User Story thuộc Sprint này. 7. Hệ thống lặp (loop) qua các danh sách User Story chưa hoàn thành ([story chưa Done]):

	• Tự thực hiện gỡ liên kết User Story khỏi Sprint bằng cách gán sprint=null (để đưa về backlog). • Gọi save(story) tới UserStoryRepository để lưu thay đổi của từng User Story vào cơ sở dữ liệu. 8. Sau khi vòng lặp kết thúc, SprintAppServiceImpl đăng ký phát sự kiện sau khi giao dịch commit thành công thông qua afterCommit(publishMoved) với TransactionalEvents. 9. SprintAppServiceImpl trả về void cho SprintController. 10. SprintController trả về mã HTTP 200 (OK) cho Client, báo hiệu API đã xử lý xong. 11. (Tiền trình ngầm) Sau khi giao dịch database thành công, TransactionalEvents gọi hàm publishMoved tới UserStoryEventPublisher. 12. UserStoryEventPublisher gửi thông điệp sự kiện USER_STORY_MOVED vào hàng đợi RabbitMQ.
Luồng sự kiện thay thế	Nếu Sprint không tồn tại hoặc không ở trạng thái InProgress (bước 4), quá trình sẽ bị gián đoạn và trả về lỗi cho Client, các bước lưu trữ và phát sự kiện phía sau sẽ không được thực hiện.

3.4.1.23: Sơ đồ tuần tự tạo User Story (Sequence Diagram Create User Story)



Hình 35: Sơ đồ tuần tự tạo User Story (Sequence Diagram Create User Story)

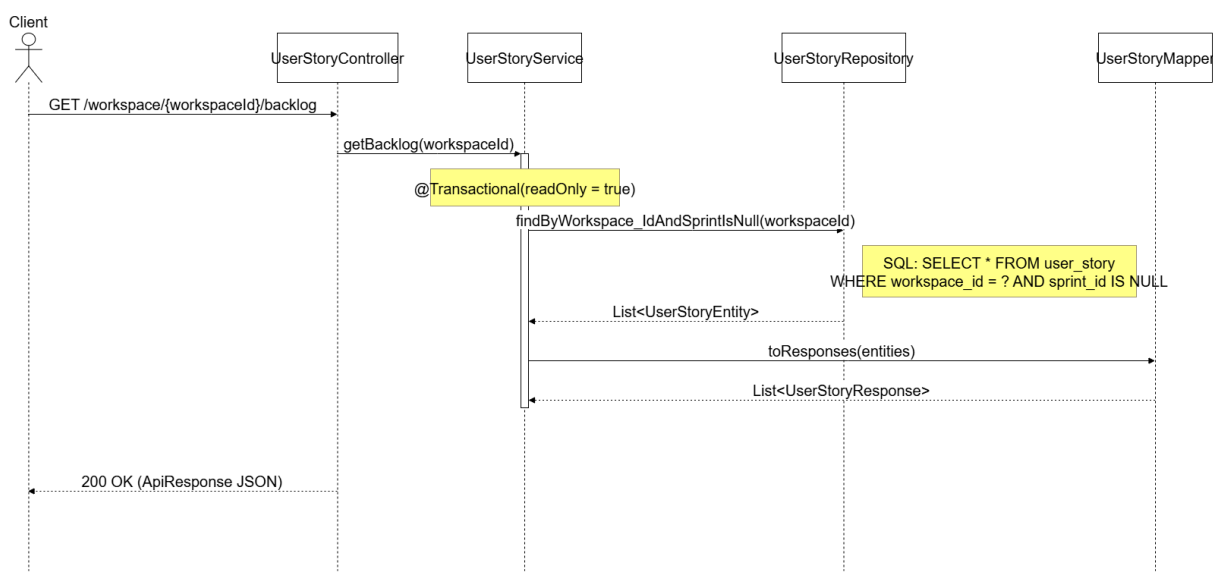
Bảng 26: Đặc tả chức năng tạo User Story (Create User Story)

Thành phần	Nội dung chi tiết
Tên usecase	Tạo mới danh sách User Story (Create User Stories)..

Tác nhân	Người dùng nhập yêu cầu mới vào hệ thống.
Mô tả	Ca sử dụng này cho phép người dùng tạo mới một hoặc nhiều User Story cùng lúc trong một Workspace cụ thể. Hệ thống sẽ tự động gán các User Story này vào Backlog (trạng thái ToDo, chưa thuộc Sprint nào) và lưu vào cơ sở dữ liệu. Sau khi lưu thành công, hệ thống sẽ phát ra một sự kiện thông báo qua RabbitMQ.
Sự kiện kích hoạt	Client gửi yêu cầu API POST kèm theo dữ liệu List UserStoryCreateRequest tới hệ thống.
Điều kiện trước	- Workspace mục tiêu phải tồn tại trong hệ thống. - Dữ liệu đầu vào (request body) hợp lệ. - Hệ thống hàng đợi (RabbitMQ) đang hoạt động bình thường.
Điều kiện sau	- Các User Story mới được tạo và lưu thành công vào cơ sở dữ liệu (gắn với Backlog, trạng thái ToDo, sprint=null). - Sự kiện USER_STORY_CREATED được đẩy thành công lên RabbitMQ.
Luồng sự kiện chính	1. Client gửi yêu cầu POST với payload là danh sách UserStoryCreateRequest tới UserStoryController. 2. UserStoryController tiếp nhận và gọi phương thức create(workspaceId, requests) của UserStoryAppServiceImpl. 3. UserStoryAppServiceImpl gọi hàm findById tới WorkspaceRepository để lấy thông tin Workspace. 4. WorkspaceRepository truy xuất và trả về dữ liệu Workspace + Backlog tương ứng. 5. UserStoryAppServiceImpl tự thực hiện ánh xạ (map) dữ liệu từ request sang thực thể (entity), thiết lập mặc định trạng thái là ToDo và sprint=null (tức là đưa vào Backlog). 6. UserStoryAppServiceImpl gọi saveAll tới UserStoryRepository để lưu toàn bộ danh sách User Story mới vào cơ sở dữ liệu. 7. UserStoryAppServiceImpl đăng ký phát sự kiện sau khi giao dịch cơ sở dữ liệu commit thành công thông qua afterCommit(publishCreated) với TransactionalEvents. 8. UserStoryAppServiceImpl trả về đối tượng List UserStoryResponse cho UserStoryController. 9. UserStoryController trả về mã HTTP 200 (OK) kèm theo danh sách kết quả cho Client. 10. (Tiến trình ngầm) Sau khi giao dịch database thành công, TransactionalEvents gọi hàm publishCreated tới UserStoryEventPublisher. 11. UserStoryEventPublisher gửi thông điệp sự kiện

	USER_STORY_CREATED vào hàng đợi RabbitMQ.
Luồng sự kiện thay thế	Nếu hàm findById ở bước 3 không tìm thấy Workspace, quá trình sẽ báo lỗi và dừng lại, các bước lưu trữ và phát sự kiện phía sau sẽ không diễn ra.

3.4.1.24: Sơ đồ tuần tự lấy danh sách User Stories trong Product Backlog (Sequence Diagram Get User Stories In Product Backlog)



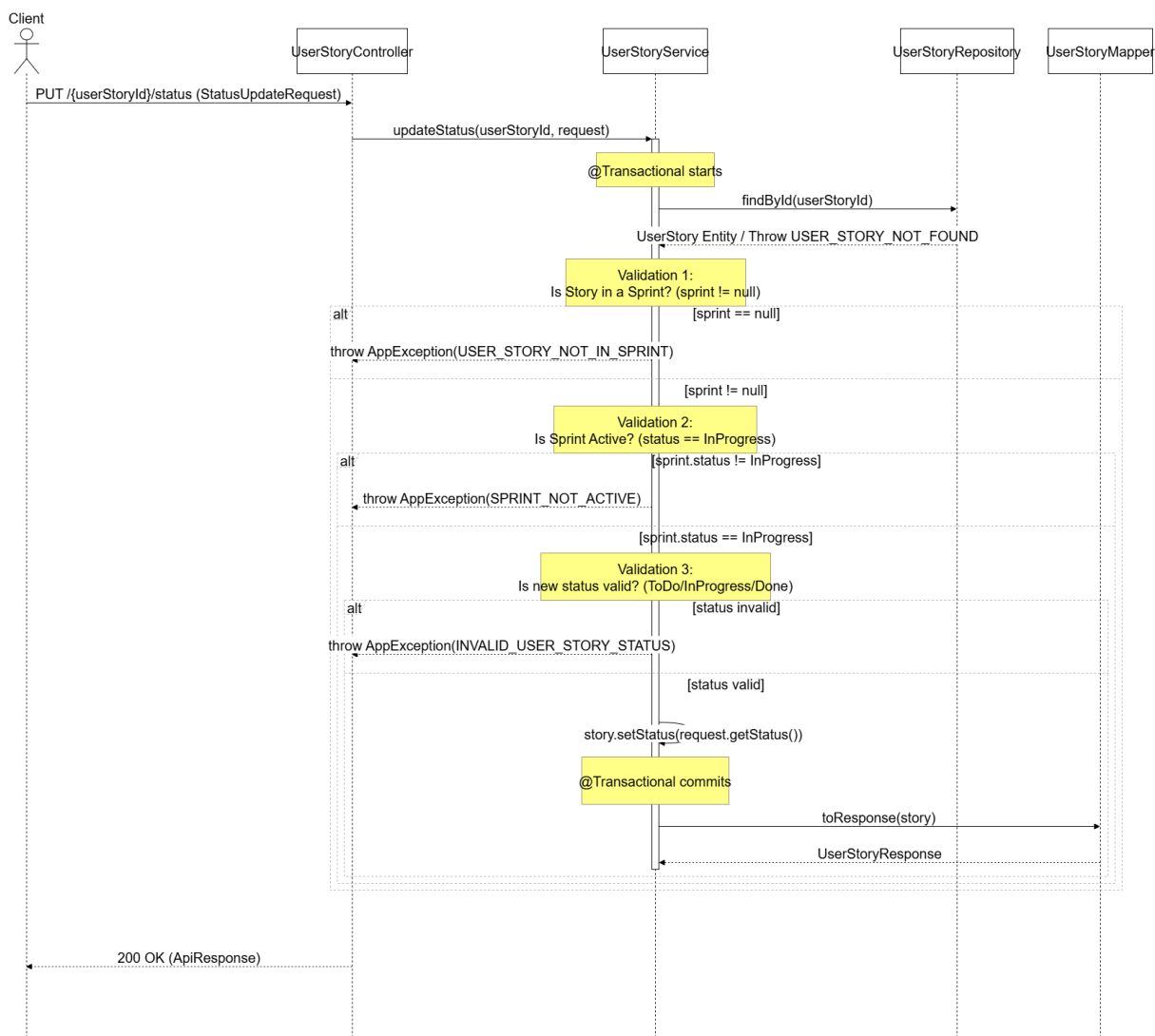
Hình 36: Sơ đồ tuần tự lấy danh sách User Stories trong Product Backlog (Sequence Diagram Get User Stories In Product Backlog)

Bảng 27: Đặc tả chức năng lấy danh sách User Stories trong Product Backlog (Get User Stories In Product Backlog)

Thành phần	Nội dung chi tiết
Tên usecase	Xem danh sách User Story trong Backlog.
Tác nhân	Người dùng muốn xem các yêu cầu chưa được lập kế hoạch vào Sprint.
Mô tả	Truy xuất tất cả các User Story thuộc về một Không gian làm việc (Workspace) cụ thể nhưng chưa được gắn vào bất kỳ Sprint nào (trạng thái sprint_id IS NULL).
Sự kiện kích hoạt	Người dùng nhấn vào tab "Backlog" trong giao diện quản lý dự án của Workspace.

Điều kiện trước	- Người dùng đã được xác thực quyền truy cập vào Workspace (workspaceId). - WorkspaceId tồn tại hợp lệ trong hệ thống.
Điều kiện sau	- Danh sách các User Story "tồn đọng" (chưa có Sprint) được hiển thị đầy đủ cho người dùng.
Luồng sự kiện chính	1. Client gửi yêu cầu GET /workspace/{workspaceId}/backlog tới hệ thống. 2. UserStoryController tiếp nhận yêu cầu và gọi phương thức getBacklog(workspaceId) của UserStoryService. 3. UserStoryService bắt đầu giao dịch chỉ đọc (@Transactional(readOnly = true)). 4. UserStoryService gọi findByWorkspace_IdAndSprintIsNull(workspaceId) từ UserStoryRepository. 5. UserStoryRepository thực thi truy vấn SQL: SELECT * FROM user_story WHERE workspace_id = ? AND sprint_id IS NULL. 6. Database trả về danh sách thực thể List<UserStoryEntity>. 7. UserStoryService gọi toResponses(entities) thông qua UserStoryMapper để chuyển đổi sang danh sách DTO. 8. UserStoryController nhận kết quả từ Service và trả về mã 200 OK (ApiResponse JSON) cho Client.
Luồng sự kiện thay thế	Trường hợp Backlog trống: • Tại bước 6, cơ sở dữ liệu không tìm thấy bản ghi nào thỏa mãn điều kiện sprint_id IS NULL. • Hệ thống vẫn trả về mã 200 OK kèm theo một mảng rỗng []. • Giao diện người dùng hiển thị thông báo: "Hiện không có yêu cầu nào trong Backlog".

3.4.1.25: Sơ đồ tuần tự cập nhật trạng thái của User Story (Sequence Diagram Update User Story Status)



Hình 37: Sơ đồ tuần tự cập nhật trạng thái của câu chuyện người dùng (Sequence Diagram Update User Story Status)

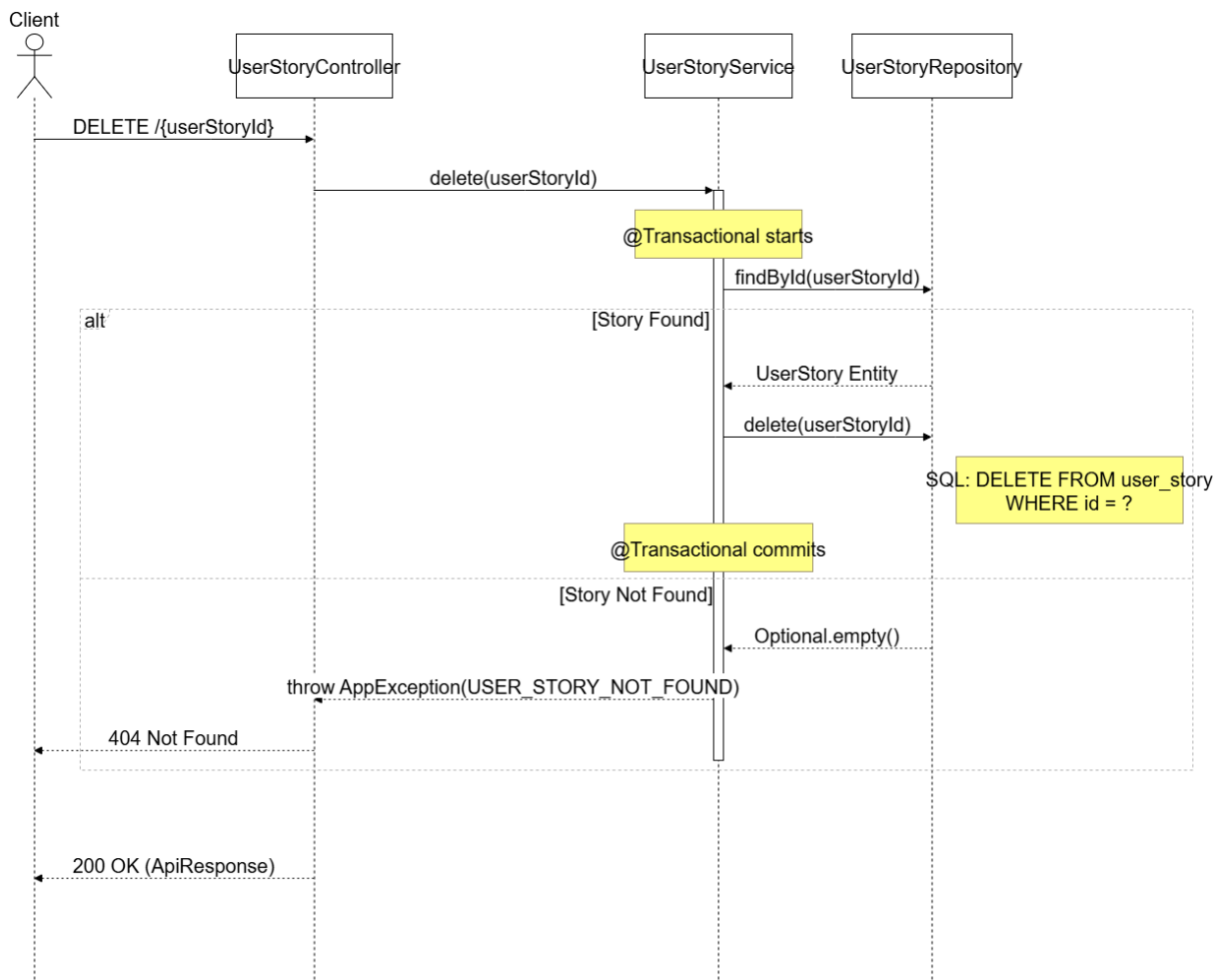
Bảng 28: Đặc tả chức năng cập nhật trạng thái của câu chuyện người dùng (Update User Story Status)

Thành phần	Nội dung chi tiết
Tên usecase	Cập nhật trạng thái User Story.
Tác nhân	Người dùng (Quản lý dự án, Scrum Master hoặc Thành viên nhóm) thực hiện cập nhật trạng thái User Story.
Mô tả	Cho phép người dùng thay đổi trạng thái tiến độ của một User Story (ví dụ: từ ToDo sang InProgress hoặc Done) khi User Story

	đó đang nằm trong một Sprint đang hoạt động.
Sự kiện kích hoạt	Client gửi yêu cầu PUT /{userId}/status kèm theo dữ liệu StatusUpdateRequest.
Điều kiện trước	- Người dùng đã đăng nhập và có quyền chỉnh sửa trong Workspace. - User Story mục tiêu phải tồn tại trong hệ thống.
Điều kiện sau	- Trạng thái của User Story được cập nhật thành công trong cơ sở dữ liệu. - Phản hồi thành công được gửi về cho Client để cập nhật giao diện
Luồng sự kiện chính	1. Client gửi yêu cầu cập nhật trạng thái tới UserStoryController. 2. UserStoryController tiếp nhận yêu cầu và gọi updateStatus(userId, request) của UserStoryService. 3. UserStoryService bắt đầu Giao dịch (@Transactional starts). 4. UserStoryService gọi findById(userId) từ UserStoryRepository để lấy thực thể hiện tại. 5. Hệ thống: Xác thực 1: Kiểm tra User Story có thuộc về một Sprint nào không (sprint != null). 6. Hệ thống: Xác thực 2: Kiểm tra Sprint liên quan có đang hoạt động không (status == InProgress). 7. Hệ thống: Xác thực 3: Kiểm tra trạng thái mới trong request có hợp lệ không (ToDo/InProgress/Done). 8. UserStoryService thực hiện story.setStatus(request.getStatus()) để cập nhật dữ liệu. 9. UserStoryService hoàn tất Giao dịch (@Transactional commits) để lưu thay đổi vào DB. 10. UserStoryService gọi toResponse(story) qua UserStoryMapper để tạo đối tượng phản hồi. 11. UserStoryController trả về UserStoryResponse kèm mã 200 OK cho Client.
Luồng sự kiện thay thế	Không tìm thấy User Story: • Tại bước 4, nếu ID không tồn tại, hệ thống ném ngoại lệ USER_STORY_NOT_FOUND. User Story chưa được đưa vào Sprint: • Tại bước 5, nếu sprint == null, hệ thống ném lỗi USER_STORY_NOT_IN_SPRINT. Sprint chưa bắt đầu hoặc đã kết thúc: • Tại bước 6, nếu trạng thái Sprint không phải là InProgress, hệ thống ném lỗi SPRINT_NOT_ACTIVE. Trạng thái mới không hợp lệ:

	Tại bước 7, nếu giá trị trạng thái gửi lên không nằm trong danh sách quy định, hệ thống ném lỗi INVALID_USER_STORY_STATUS.
--	--

3.4.1.26: Sơ đồ tuần tự xóa câu chuyện người dùng (Sequence Diagram Delete User Story)



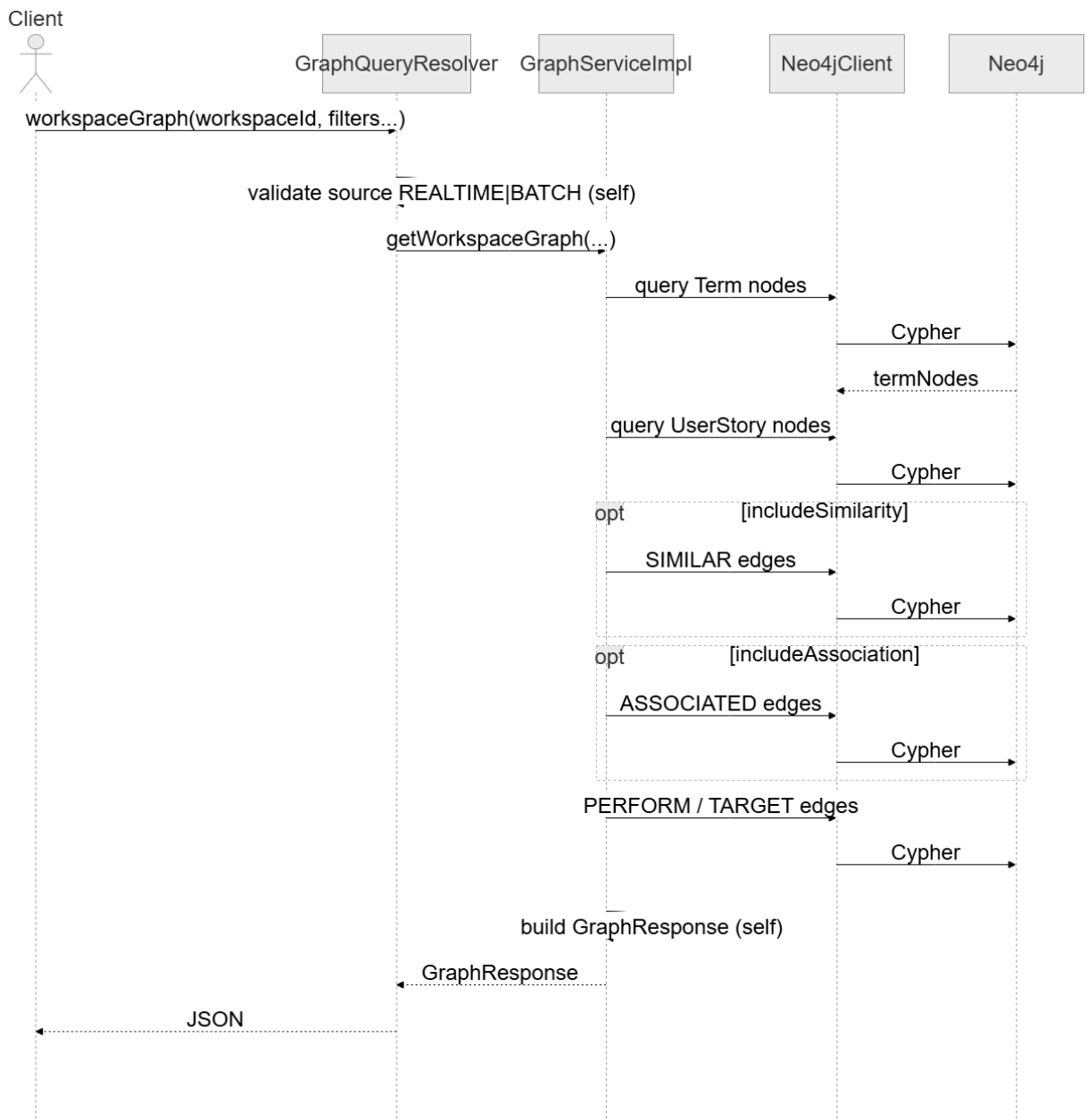
Hình 38: Sơ đồ tuần tự xóa câu chuyện người dùng (Sequence Diagram Delete User Story)

Bảng 29: Đặc tả chức năng xóa câu chuyện người dùng (Delete User Story)

Thành phần	Nội dung chi tiết
Tên usecase	Xóa User Story.
Tác nhân	Client (Người dùng).
Mô tả	Cho phép người dùng xóa hoàn toàn một User Story khỏi hệ thống. Hệ thống sẽ kiểm tra sự tồn tại của User Story trước khi thực hiện

	lệnh xóa vĩnh viễn trong cơ sở dữ liệu.
Sự kiện kích hoạt	Client gửi yêu cầu DELETE / {userStoryId} tới hệ thống.
Điều kiện trước	- Người dùng đã đăng nhập và có quyền xóa trong không gian làm việc (Workspace). - userStoryId phải tồn tại trong hệ thống.
Điều kiện sau	- User Story bị xóa vĩnh viễn khỏi bảng user_story trong cơ sở dữ liệu. - Hệ thống phản hồi trạng thái thành công cho Client.
Luồng sự kiện chính	1. Client gửi yêu cầu xóa User Story kèm theo userStoryId. 2. UserStoryController tiếp nhận yêu cầu và gọi phương thức delete(userStoryId) của UserStoryService. 3. UserStoryService bắt đầu Giao dịch (@Transactional starts). 4. UserStoryService gọi findById(userStoryId) từ UserStoryRepository để kiểm tra sự tồn tại. 5. Hệ thống: Trường hợp [Story Found]: Tìm thấy thực thể User Story. 6. UserStoryService gọi lệnh delete(userStoryId) thông qua UserStoryRepository. 7. UserStoryRepository thực thi câu lệnh SQL: DELETE FROM user_story WHERE id = ?. 8. UserStoryService hoàn tất Giao dịch (@Transactional commits). 9. UserStoryController trả về mã phản hồi 200 OK (ApiResponse) cho Client.
Luồng sự kiện thay thế	Không tìm thấy User Story (Story Not Found): • Tại bước 4, nếu UserStoryRepository trả về Optional.empty(). • UserStoryService ném ra ngoại lệ ApplicationException(USER_STORY_NOT_FOUND). • UserStoryController trả về mã lỗi 404 Not Found cho Client.

3.4.1.27: Sơ đồ tuần tự cho đồ thị các câu chuyện người dùng (Sequence Diagram Get Workspace Graph)



Hình 39: Sơ đồ tuần tự cho đồ thị toàn bộ câu chuyện người dùng của sản phẩm (Sequence Diagram Get Workspace Graph)

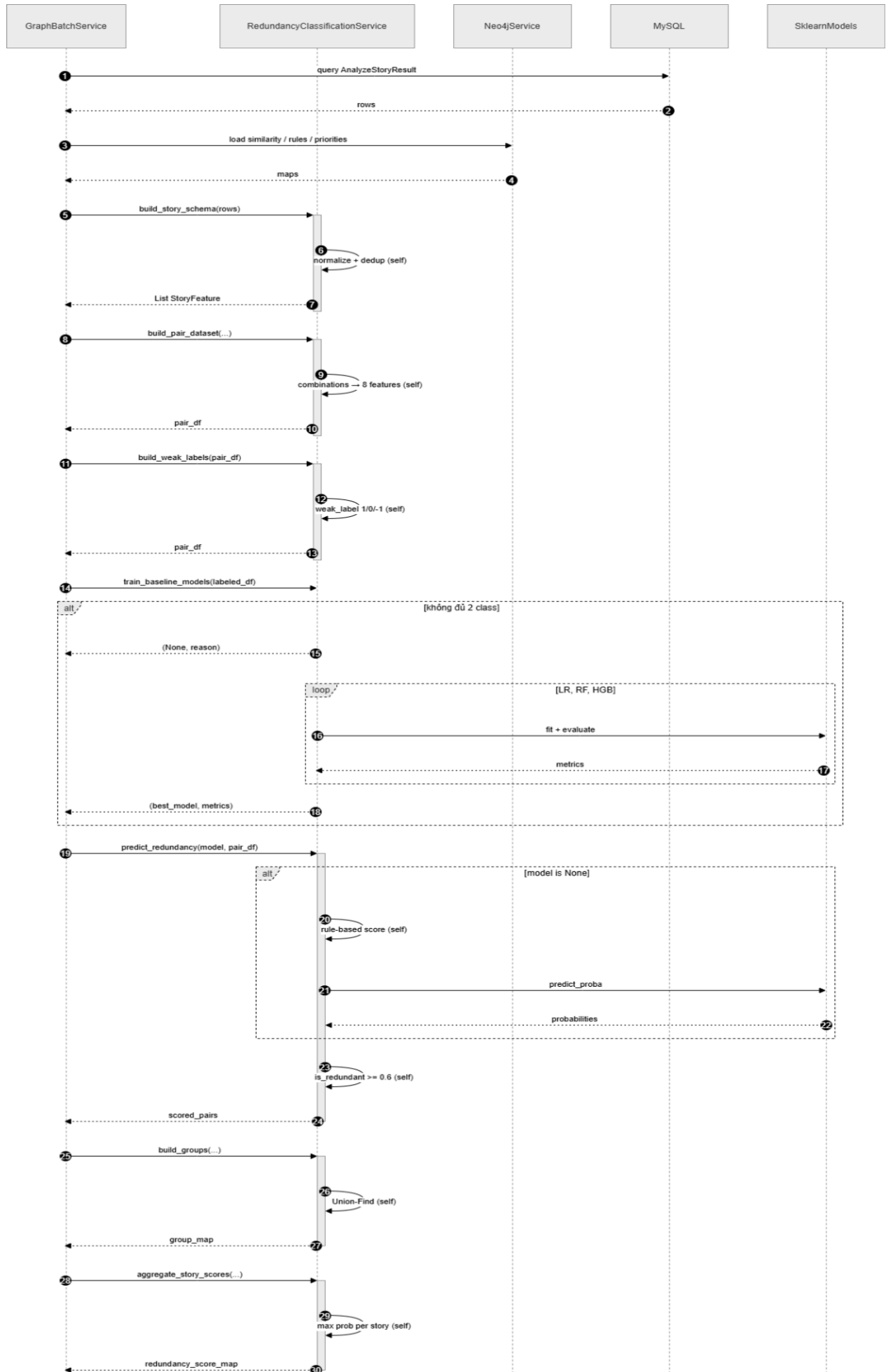
Bảng 30: Đặc tả chức năng cho đồ thị toàn bộ câu chuyện người dùng của sản phẩm (Get Workspace Graph)

Thành phần	Nội dung chi tiết
Tên usecase	Lấy dữ liệu đồ thị của Workspace (Workspace Graph Query).

Tác nhân	Người dùng muốn xem trực quan hóa mối quan hệ giữa các yêu cầu trong Workspace dưới dạng sơ đồ đồ thị.
Mô tả	Ca sử dụng này cho phép Client truy vấn toàn bộ cấu trúc đồ thị (gồm các đỉnh/nodes và cạnh/edges) của một Workspace cụ thể. Hệ thống sẽ truy vấn cơ sở dữ liệu đồ thị Neo4j để lấy các node (Term, UserStory) và các mối quan hệ giữa chúng (PERFORM, TARGET, SIMILAR, ASSOCIATED) tùy thuộc vào bộ lọc (filters) được truyền vào, sau đó trả về dữ liệu dưới dạng JSON.
Sự kiện kích hoạt	Client gửi truy vấn workspaceGraph(workspaceId, filters...) tới GraphQueryResolver.
Điều kiện trước	- Hệ thống đã kết nối thành công với cơ sở dữ liệu Neo4j. - Các tham số đầu vào (workspaceId, filters) hợp lệ.
Điều kiện sau	- Trả về cho Client một chuỗi JSON chứa đối tượng GraphResponse đại diện cho cấu trúc mạng lưới đồ thị của Workspace.
Luồng sự kiện chính	1. Client gửi truy vấn workspaceGraph(workspaceId, filters...) tới GraphQueryResolver. 2. GraphQueryResolver tự thực hiện kiểm tra và xác thực nguồn truy vấn (validate source REALTIME BATCH). 3. Tiếp theo, GraphQueryResolver gọi hàm getWorkspaceGraph(...) của GraphServiceImpl để bắt đầu lấy dữ liệu. 4. GraphServiceImpl gọi query Term nodes thông qua Neo4jClient. Neo4jClient biên dịch thành câu lệnh Cypher gửi xuống Neo4j và nhận về danh sách termNodes. 5. Tương tự, GraphServiceImpl gọi query UserStory nodes thông qua Neo4jClient. Neo4jClient chạy Cypher xuống Neo4j để lấy các đỉnh là User Story. 6. [Khởi Opt - includeSimilarity]: Nếu bộ lọc yêu cầu bao gồm độ tương đồng, GraphServiceImpl gọi lấy các cạnh SIMILAR edges thông qua Neo4jClient (chạy Cypher). 7. [Khởi Opt - includeAssociation]: Nếu bộ lọc yêu cầu bao gồm sự liên kết, GraphServiceImpl gọi lấy các cạnh ASSOCIATED edges thông qua Neo4jClient (chạy Cypher). 8. GraphServiceImpl tiếp tục gọi lấy các cạnh mối quan hệ bắt buộc là PERFORM / TARGET edges thông qua Neo4jClient (chạy Cypher). 9. Sau khi thu thập đủ dữ liệu (nodes và edges), GraphServiceImpl tự động gọi hàm build GraphResponse để tổng hợp thành đối

	tượng kết quả. 10. GraphServiceImpl trả về đối tượng GraphResponse cho GraphQueryResolver. 11. GraphQueryResolver định dạng lại thành chuỗi JSON và trả về cho Client.
Luồng sự kiện thay thế	Bộ lọc không bao gồm các tùy chọn (filters): • Tại bước 6 và 7 của luồng chính, nếu filters truyền vào không chứa cờ includeSimilarity và/hoặc includeAssociation, hệ thống sẽ bỏ qua hoàn toàn việc truy vấn các cạnh SIMILAR và ASSOCIATED xuống database để tối ưu hóa hiệu suất. Các bước sau đó vẫn tiếp tục bình thường.

3.4.1.28: Sơ đồ tuần tự phân loại câu chuyện người dùng trùng lặp (redundancy classification)



Hình 40: Sơ đồ tuần tự phân loại câu chuyện người dùng trùng lặp (redundancy classification)

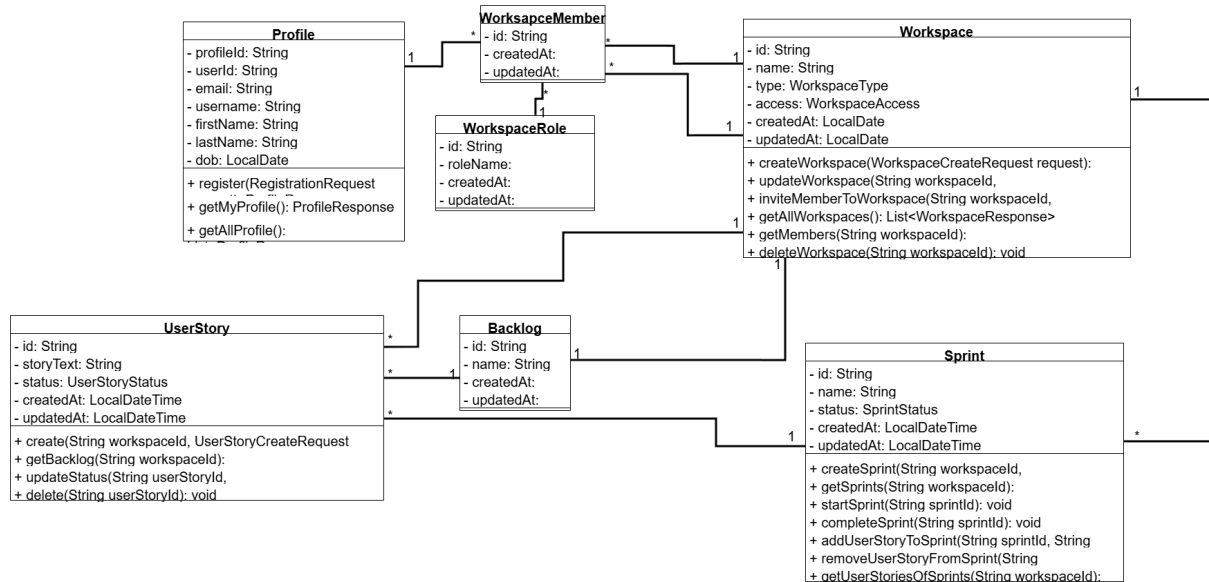
Bảng 31: Đặc tả chức năng phân loại câu chuyện người dùng trùng lặp (redundancy classification)

Thành phần	Nội dung chi tiết
Tên usecase	Phân loại và phát hiện các User Story trùng lặp.
Tác nhân	GraphBatchService (Hệ thống chạy ngầm / Batch Job).
Mô tả	Ca sử dụng này thực hiện quá trình tự động phân tích dữ liệu, trích xuất đặc trưng, huấn luyện các mô hình học máy (nếu đủ điều kiện dữ liệu) hoặc dùng luật (rule-based) để phát hiện, đánh giá xác suất và gộp nhóm các User Story trùng lặp.
Sự kiện kích hoạt	GraphBatchService bắt đầu chu kỳ xử lý phân tích và gọi đến RedundancyClassificationService.
Điều kiện trước	- Cơ sở dữ liệu MySQL và Neo4j đang hoạt động bình thường. - Đã có dữ liệu AnalyzeStoryResult trong MySQL. - Các cấu hình về độ tương tự (similarity), luật (rules) và độ ưu tiên (priorities) đã được thiết lập trong Neo4j.
Điều kiện sau	- Hệ thống tính toán và trả về một bản đồ điểm số (redundancy_score_map), trong đó chứa xác suất trùng lặp cao nhất cho từng User Story.
Luồng sự kiện chính	1. GraphBatchService truy vấn dữ liệu từ MySQL để lấy danh sách các bản ghi (rows). 2. GraphBatchService truy vấn Neo4jService để tải các cấu hình về độ tương tự, luật, độ ưu tiên và nhận về dạng maps. 3. GraphBatchService gọi build_story_schema(rows) của RedundancyClassificationService. Dịch vụ này tự chuẩn hóa và loại bỏ trùng lặp nội bộ (normalize + dedup) để trả về danh sách StoryFeature. 4. Kích hoạt build_pair_dataset(...): Dịch vụ tạo tổ hợp các cặp dữ liệu, trích xuất 8 đặc trưng (features) và trả về pair_df. 5. Kích hoạt build_weak_labels(pair_df): Dịch vụ tự gán nhãn yếu (weak_label 1/0/-1) và cập nhật lại pair_df. 6. Kích hoạt train_baseline_models(labeled_df): Dữ liệu đủ 2 class. Hệ thống lặp qua các mô hình học máy (LR, RF, HGB), gọi SklearnModels để huấn luyện (fit) và đánh giá (evaluate). Mô hình tốt nhất (best_model) và các chỉ số (metrics) được trả về. 7. Kích hoạt predict_redundancy(model, pair_df): Hệ thống gọi SklearnModels.predict_proba để lấy xác suất dự đoán. Sau đó tự

	lọc các cặp có xác suất ≥ 0.6 (<code>is_redundant ≥ 0.6</code>) và trả về danh sách <code>scored_pairs</code>. 8. Kích hoạt <code>build_groups(...)</code>: Dịch vụ áp dụng thuật toán Union-Find để gộp các cặp trùng lặp thành nhóm và trả về <code>group_map</code>. 9. Kích hoạt <code>aggregate_story_scores(...)</code>: Dịch vụ lấy xác suất trùng lặp cao nhất (<code>max prob</code>) cho mỗi story và trả về kết quả cuối cùng là <code>redundancy_score_map</code>.
Luồng sự kiện thay thế	Không đủ dữ liệu huấn luyện (Không đủ 2 class): • Tại bước 6 của luồng chính, nếu tập dữ liệu không có đủ 2 class, hệ thống sẽ bỏ qua việc gọi <code>SklearnModels</code>. • Hàm trả về (<code>None, reason</code>) thay vì <code>best_model</code>. Không có mô hình học máy (<code>model is None</code>): • Tại bước 7 của luồng chính, nếu mô hình truyền vào là <code>None</code>, hệ thống sẽ không gọi <code>SklearnModels.predict_proba</code>. • Thay vào đó, hệ thống sẽ tự tính toán điểm số dựa trên tập luật định sẵn (<code>rule-based score</code>) để đánh giá độ trùng lặp. Các bước lọc 0.6 và gộp nhóm sau đó diễn ra bình thường.

3.4.2 Biểu đồ Lớp (Class Diagram)

Từ phân tích vật lý các yêu cầu, Biểu đồ lớp của hệ thống được đưa ra và minh họa như Hình 41 gồm 7 lớp chính là Profile, User Story, WorkspaceMember, WorkspaceRole, Backlog, Workspace, và Sprint.



Hình 41: Biểu đồ lớp của Ứng dụng (Class diagram)

3.5. Kết luận chương

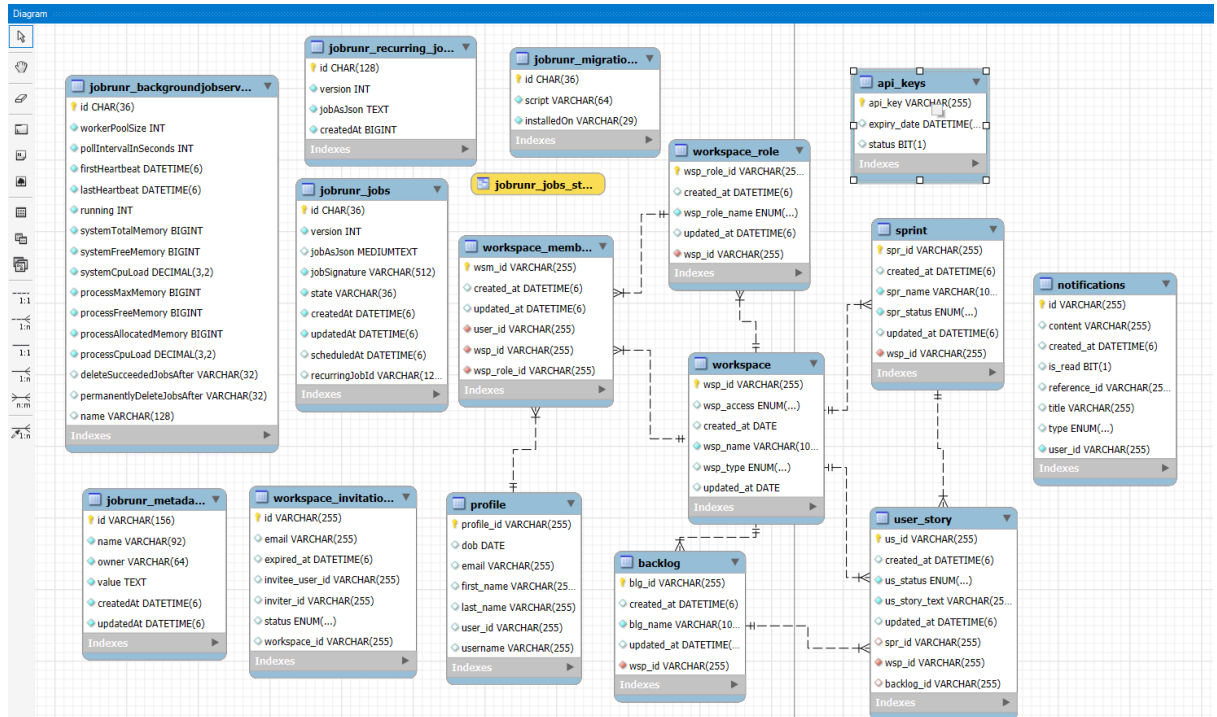
Trong chương 3, nhóm đã tiến hành phân tích hệ thống dựa trên các yêu cầu thu thập được, qua đó xác định rõ chức năng nghiệp vụ, luồng xử lý, và mối quan hệ tương tác giữa người dùng và hệ thống. Các Use Case Diagram và Sequence Diagram được xây dựng nhằm mô hình hóa chi tiết hành vi của hệ thống và cách các thành phần phối hợp với nhau trong từng chức năng cụ thể.

Kết quả phân tích giúp nhóm có cái nhìn toàn diện và logic về hệ thống, là cơ sở vững chắc để thiết kế cơ sở dữ liệu, kiến trúc, và giao diện người dùng ở chương tiếp theo.

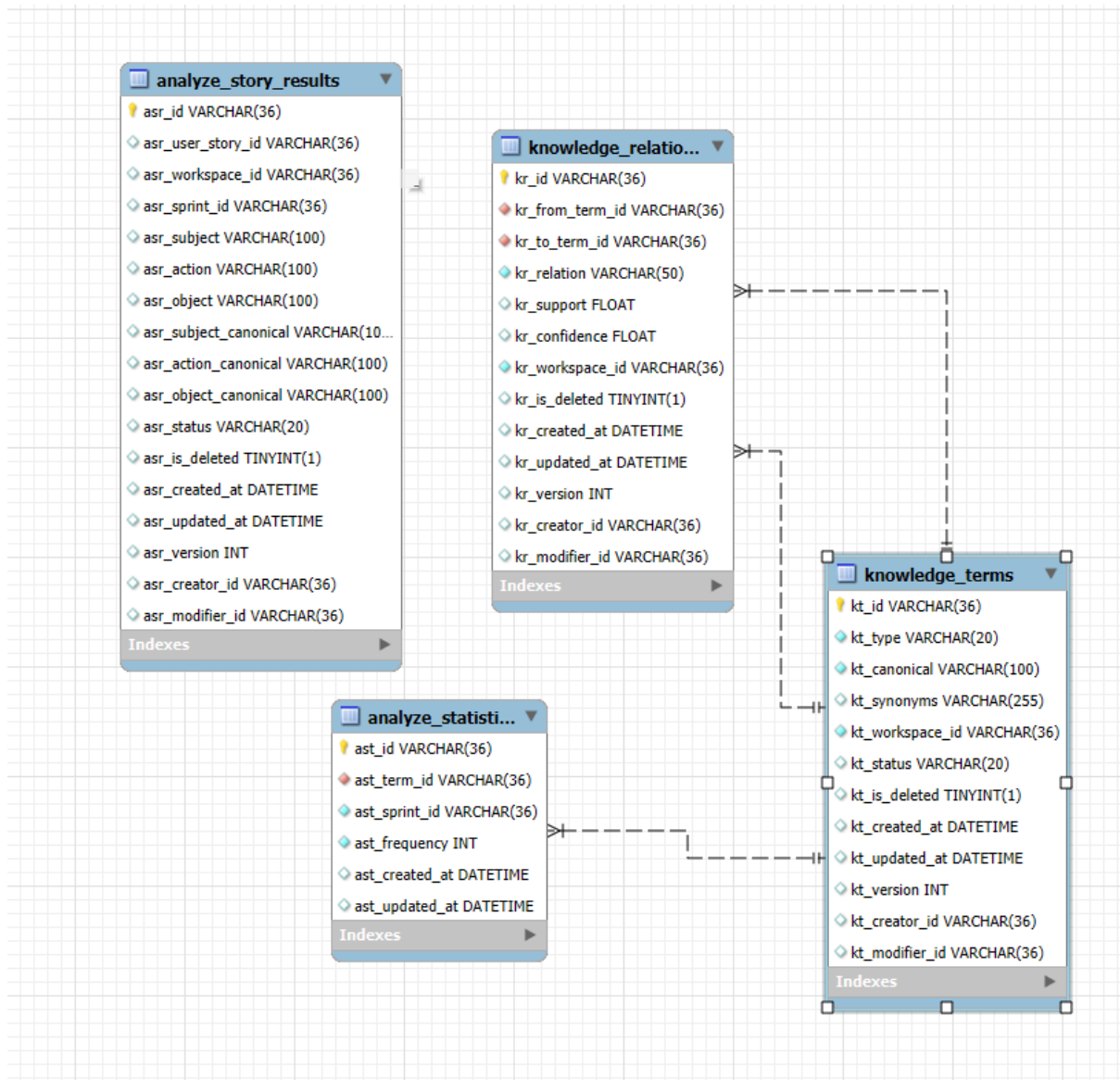
Chương 4: Thiết kế hệ thống

4.1. Thiết kế cơ sở dữ liệu

4.1.1. ERD



Hình 42: Biểu đồ ERD vật lý



Hình 43: Biểu đồ ERD vật lý cho analyze_story

4.1.2. Lược đồ quan hệ logic

Do đặc thù kiến trúc hybrid của hệ thống [3] (MySQL cho nghiệp vụ Agile và kho phân tích; Neo4j cho tri thức SVO, tương đồng, luật kết hợp và trùng lặp user story), dữ liệu được tổ chức theo các cụm store thay vì một schema đơn. Quan hệ giữa thực thể nghiệp vụ và dữ liệu phân tích được duy trì qua ID tham chiếu logic (`wsp_id`, `spr_id`, `us_id`, `asr_user_story_id`, `workspace_id` trên đồ thị), không có khóa ngoại vật lý xuyên Neo4j [3].

Tính nhất quán MySQL ↔ Neo4j ↔ dịch vụ phân tích Python được đảm bảo bằng event-driven qua RabbitMQ [20] (exchange `userstory.exchange`: `USER_STORY_CREATED`,

USER_STORY_MOVED, REBUILD_GRAPH) kết hợp TransactionalEvents.afterCommit — chỉ publish sau khi giao dịch MySQL commit.

Hình 4.1.1 thể hiện nhóm cốt lõi: Profile, Workspace, Backlog, WorkspaceRole, WorkspaceMember, Sprint, UserStory. Dưới đây mở rộng theo hình và triển khai thực tế.

Cụm Nghiệp vụ Agile (MySQL)

- Workspace: không gian làm việc (wsp_id, tên, loại, quyền truy cập). Mỗi workspace có một backlog và nhiều sprint, user story.
- Backlog: product backlog 1–1 với workspace (blg_id, wsp_id unique). Chứa story khi spr_id null.
- Sprint: vòng đời sprint (spr_id, spr_status: ToDo / InProgress / Done). Mỗi workspace tối đa một sprint InProgress.
- UserStory: yêu cầu Agile (us_id, us_story_text, us_status). Tham chiếu wsp_id, spr_id (nullable = backlog), backlog_id. Khi complete sprint, story chưa Done về backlog (spr_id = null, gán lại backlog).
- Profile: hồ sơ cục bộ (profile_id, userId Keycloak, email, username) phục vụ membership.
- WorkspaceRole: vai trò theo workspace (Owner / Admin / Member / Viewer).
- WorkspaceMember: liên kết Profile–Workspace–Role (wsm_id, user_id, wsp_id, wsp_role_id).

Cụm Cộng tác & Thông báo (MySQL)

- WorkspaceInvitation: lời mời (workspaceId, inviterId, inviteeUserId, trạng thái PENDING / ACCEPTED / REJECTED, expiredAt). Accept → tạo WorkspaceMember.
- Notification: thông báo in-app (userId, type, referenceId, is_read) phục vụ badge.
- Cụm Phân tích User Story (MySQL — service Python)
- AnalyzeStoryResult: SVO đã parse (asr_user_story_id, asr_workspace_id, asr_sprint_id, subject/action/object + canonical, asr_is_deleted). Nguồn cho batch rebuild graph.
- KnowledgeTerm: thuật ngữ chuẩn (SUBJECT | ACTION | OBJECT, kt_canonical, kt_workspace_id).
- KnowledgeRelation: quan hệ term (kr_from_term_id, kr_to_term_id, support, confidence).
- AnalyzeStatistic: tần suất term theo sprint.

Cụm Đồ thị tri thức (Neo4j)

- Term: node subject/action/object (name, workspace_id).
- UserStory (node): id, priority, redundancy, group — tham chiếu us_id MySQL.

- **PERFORM / TARGET:** SVO có story_id, sprint_id, backlog_id, source (REALTIME | BATCH) — dùng khi getWorkspaceGraph.
- **SIMILAR / ASSOCIATED:** tương đồng và luật kết hợp.
- **REDUNDANT_WITH:** cặp story trùng lặp sau ML/rule (score, group_id, model).
- **ClassificationMetrics:** metadata mô hình batch.

4.1.3. Cấu trúc các Collection

4.1.3.1. Collection Profile (Identity / User Service — MySQL)

Xác thực đăng nhập do Keycloak đảm nhiệm; bảng profile chỉ lưu hồ sơ hiển thị và liên kết membership, không lưu mật khẩu.

Bảng 32: Bảng Collection Profile

Trường	Kiểu dữ liệu	Mô tả
profile_id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
userId	VARCHAR	ID người dùng từ Keycloak (Unique logic)
email	VARCHAR	Email hiển thị / tra cứu
username	VARCHAR	Tên định danh
firstName	VARCHAR	Họ
lastName	VARCHAR	Tên
dob	DATE	Ngày sinh

4.1.3.2. Cụm Workspace & Agile

Bảng 33: Bảng Collection Workspace

Trường	Kiểu dữ liệu	Mô tả
wsp_id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
wsp_name	VARCHAR(100)	Tên không gian làm việc (NOT NULL)
wsp_type	ENUM / VARCHAR	Loại workspace: TEAM_MANAGED, COMPANY_MANAGED

wsp_access	ENUM / VARCHAR	Quyền truy cập: PRIVATE, LIMITED, OPEN
createdAt	DATE	Thời gian tạo (CreationTimestamp)
updatedAt	DATE	Thời gian cập nhật (UpdateTimestamp)

Bảng 34: Bảng Collection Backlog

Trường	Kiểu dữ liệu	Mô tả
blg_id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
blg_name	VARCHAR(100)	Tên product backlog (NOT NULL)
wsp_id	VARCHAR(36)	FK → workspace.wsp_id (Unique — 1 workspace : 1 backlog)
createdAt	DATE	Thời gian tạo (CreationTimestamp)
updatedAt	DATE	Thời gian cập nhật (UpdateTimestamp)

Bảng 35: Bảng Collection Sprint

Trường	Kiểu dữ liệu	Mô tả
spr_id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
spr_name	VARCHAR(100)	Tên sprint (NOT NULL)
spr_status	ENUM / VARCHAR	Trạng thái: ToDo, InProgress, Done
wsp_id	VARCHAR(36)	FK → workspace.wsp_id (NOT NULL)
createdAt	DATE	Thời gian tạo (CreationTimestamp)

updatedAt	DATE	Thời gian cập nhật (UpdateTimestamp)
-----------	------	---

Bảng 36: Bảng Collection UserStory

Trường	Kiểu dữ liệu	Mô tả
us_id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
us_story_text	VARCHAR(255)	Nội dung user story (NOT NULL)
us_status	ENUM / VARCHAR	ToDo, InProgress, Done
spr_id	VARCHAR(36)?	FK → sprint.spr_id; NULL = story đang ở backlog
wsp_id	VARCHAR(36)	FK → workspace.wsp_id (NOT NULL)
backlog_id	VARCHAR(36)?	FK → backlog.blg_id; gán khi story thuộc/về backlog
createdAt	DATE	Thời gian tạo (CreationTimestamp)
updatedAt	DATE	Thời gian cập nhật (UpdateTimestamp)

Bảng 37: Bảng Collection WorkspaceRole

Trường	Kiểu dữ liệu	Mô tả
wsp_role_id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
wsp_id	VARCHAR(36)	FK → workspace.wsp_id
wsp_role_name	ENUM / VARCHAR	Vai trò: ADMIN, MEMBER, VIEWER
createdAt	DATE	Thời gian tạo (CreationTimestamp)
updatedAt	DATE	Thời gian cập nhật (UpdateTimestamp)

Bảng 38: Bảng Collection WorkspaceMember

Trường	Kiểu dữ liệu	Mô tả
wsm_id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
user_id	VARCHAR(36)	FK → profile.profile_id
wsp_id	VARCHAR(36)	FK → workspace.wsp_id
wsp_role_id	VARCHAR(36)	FK → workspace_role.wsp_role_id
createdAt	DATETIME	Thời gian tạo (CreationTimestamp)
updatedAt	DATETIME	Thời gian cập nhật (UpdateTimestamp)

4.1.3.3. Cụm Cộng tác & Thông báo (MySQL)**Bảng 39: Bảng Collection WorkspaceInvitation**

Trường	Kiểu dữ liệu	Mô tả
id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
email	VARCHAR	Email người được mời
workspaceId	VARCHAR(36)	Ref → workspace.wsp_id
inviterId	VARCHAR	Ref → profile.userId (người mời)
inviteeUserId	VARCHAR?	Ref → profile.userId (sau khi đăng ký)
status	ENUM / VARCHAR	PENDING, ACCEPTED, REJECTED
expiredAt	TIMESTAMP	Hết hạn lời mời

Bảng 40: Bảng Collection Notification

Trường	Kiểu dữ liệu	Mô tả
--------	--------------	-------

id	VARCHAR(36) / UUID	Khóa chính, sinh tự động
userId	VARCHAR	Ref người nhận (Keycloak / profile)
title	VARCHAR?	Tiêu đề thông báo
content	VARCHAR	Nội dung
type	ENUM / VARCHAR	INVITATION_ACCEPTED, INVITATION_DENIED, WORKSPACE_INVITE
referenceId	VARCHAR?	ID tham chiếu (ví dụ invitationId)
is_read	BOOLEAN	Đã đọc hay chưa (mặc định false)
createdAt	TIMESTAMP	Thời gian tạo

Bảng 41: Bảng Collection ApiKey

Trường	Kiểu dữ liệu	Mô tả
api_key	VARCHAR(36) / UUID	Khóa chính / giá trị API key
status	BOOLEAN	Trạng thái kích hoạt (mặc định true)
expiryDate	TIMESTAMP?	Ngày hết hạn

4.1.3.4. Cụm Phân tích User Story (Analyze Service — MySQL)

Bảng 42: Bảng Collection AnalyzeStoryResult

Trường	Kiểu dữ liệu	Mô tả
asr_id	VARCHAR(36)	Khóa chính (UUID)
asr_user_story_id	VARCHAR(36)	Ref → user_story.us_id
asr_workspace_id	VARCHAR(36)	Ref → workspace.wsp_id
asr_sprint_id	VARCHAR(36)?	Ref sprint hiện tại (cập nhật khi MOVED)

asr_subject	VARCHAR(100)?	Chủ thể SVO (raw)
asr_action	VARCHAR(100)?	Hành động SVO (raw)
asr_object	VARCHAR(100)?	Đối tượng SVO (raw)
asr_subject_canonical	VARCHAR(100)?	Chủ thể sau chuẩn hóa semantic
asr_action_canonical	VARCHAR(100)?	Hành động canonical
asr_object_canonical	VARCHAR(100)?	Đối tượng canonical
asr_status	VARCHAR(20)?	VALID / ERROR (kết quả parse)
asr_is_deleted	BOOLEAN	Cờ xóa mềm (mặc định false)
asr_created_at	DATETIME	Thời gian tạo bản ghi phân tích
asr_updated_at	DATETIME	Thời gian cập nhật
asr_version	INT	Phiên bản bản ghi (mặc định 1)
asr_creator_id	VARCHAR(36)?	Người tạo (audit)
asr_modifier_id	VARCHAR(36)?	Người sửa (audit)

Bảng 43: Bảng Collection KnowledgeTerm

Trường	Kiểu dữ liệu	Mô tả
kt_id	VARCHAR(36)	Khóa chính (UUID)
kt_type	VARCHAR(20)	SUBJECT ACTION OBJECT (NOT NULL)
kt_canonical	VARCHAR(100)	Tên thuật ngữ chuẩn (NOT NULL)
kt_synonyms	VARCHAR(255)?	Từ đồng nghĩa / alias
kt_workspace_id	VARCHAR(36)	Ref workspace (NOT NULL)

kt_status	VARCHAR(20)	Trạng thái term (mặc định ACTIVE)
kt_is_deleted	BOOLEAN	Xóa mềm
kt_created_at	DATETIME	Thời gian tạo
kt_updated_at	DATETIME	Thời gian cập nhật
kt_version	INT	Phiên bản
kt_creator_id	VARCHAR(36)?	Audit
kt_modifier_id	VARCHAR(36)?	Audit

Bảng 44: Bảng Collection KnowledgeRelation

Trường	Kiểu dữ liệu	Mô tả
kr_id	VARCHAR(36)	Khóa chính (UUID)
kr_from_term_id	VARCHAR(36)	FK → knowledge_terms.kt_id
kr_to_term_id	VARCHAR(36)	FK → knowledge_terms.kt_id
kr_relation	VARCHAR(50)	Loại quan hệ (NOT NULL)
kr_support	FLOAT?	Support (association rule)
kr_confidence	FLOAT?	Confidence
kr_workspace_id	VARCHAR(36)	Ref workspace
kr_is_deleted	BOOLEAN	Xóa mềm
kr_created_at	DATETIME	Thời gian tạo
kr_updated_at	DATETIME	Thời gian cập nhật
kr_version	INT	Phiên bản
kr_creator_id	VARCHAR(36)?	Audit
kr_modifier_id	VARCHAR(36)?	Audit

Bảng 45: Bảng Collection AnalyzeStatistic

Trường	Kiểu dữ liệu	Mô tả
ast_id	VARCHAR(36)	Khóa chính (UUID)
ast_term_id	VARCHAR(36)	FK → knowledge_terms.kt_id (NOT NULL)
ast_sprint_id	VARCHAR(36)	Ref sprint (NOT NULL)
ast_frequency	INT	Tần suất xuất hiện term trong sprint
ast_created_at	DATETIME	Thời gian tạo
ast_updated_at	DATETIME	Thời gian cập nhật

4.1.3.5. Cụm Đồ thị Neo4j (Graph DB)**Bảng 46: Thuộc tính node Term**

Thuộc tính	Kiểu	Mô tả
name	String	Tên term (canonical))
workspace_id	String	Phạm vi workspace
label	String	Subject / Action / Object / Term

Bảng 47: Thuộc tính node UserStory

Thuộc tính	Kiểu	Mô tả
id	String	Ref user_story.us_id
priority	Float?	Độ ưu tiên ban đầu
priority_refined	Float?	Sau tính toán batch
priority_final	Float?	Sau điều chỉnh redundancy
redundancy_prob	Float?	Xác suất trùng lặp
redundancy_group_id	String?	Nhóm Union-Find

Bảng 48: Các loại relationship Neo4j

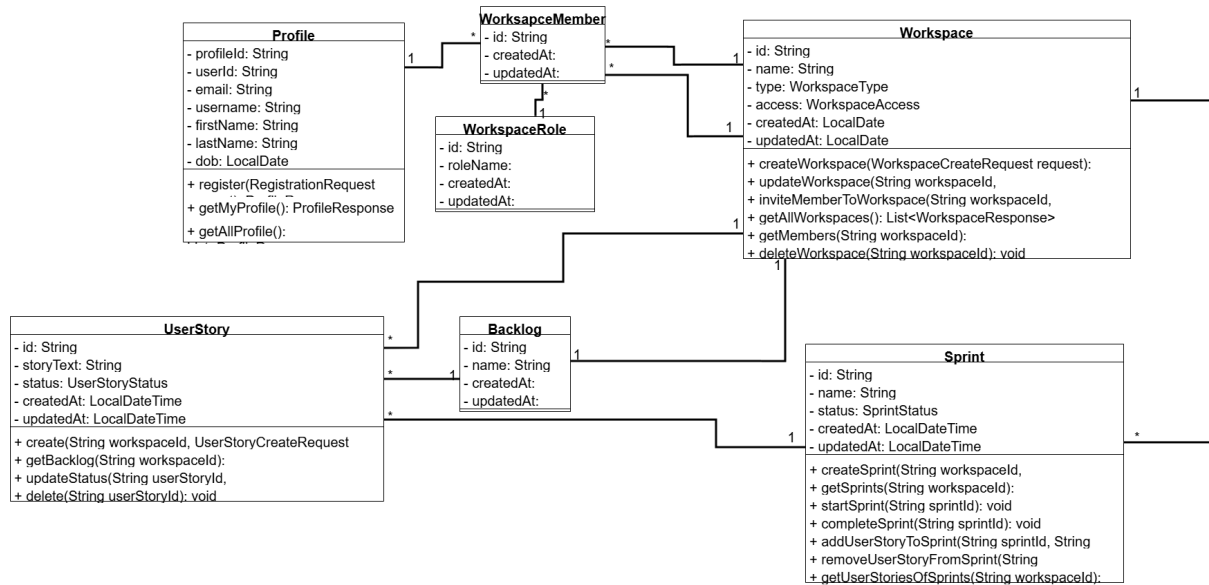
Loại	Thuộc tính chính	Mô tả
PERFORM	source, story_id, sprint_id, backlog_id	Subject → Action
TARGET	source, story_id, sprint_id, backlog_id	Action → Object
SIMILAR	score	Tương đồng giữa 2 Term
ASSOCIATED	confidence, lift	Luật kết hợp
REDUNDANT_WITH	score, group_id, model, is_redundant, workspace_id	Trùng lặp giữa 2 UserStory

Bảng 49: Thuộc tính ClassificationMetrics

Loại	Thuộc tính chính	Mô tả
workspace_id	String	Khóa workspace
model_name	String	Tên model ML batch (LR/RF/HGB)
metrics_json	String	JSON metric (F1, PR-AUC, ...)

4.2. Thiết kế thành phần phần mềm

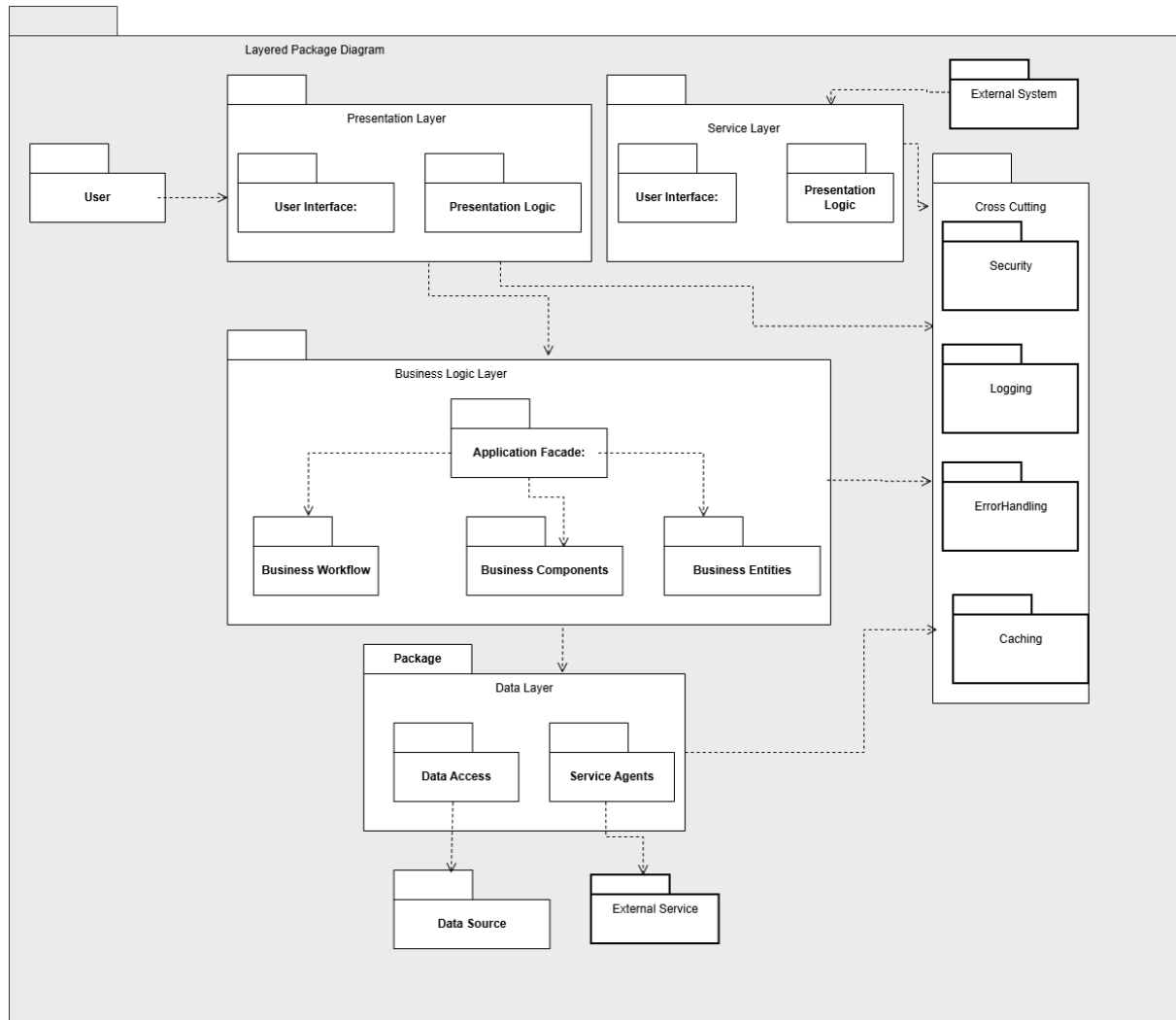
4.2.1. Biểu đồ lớp (Class Diagram) chi tiết



Hình 44: Biểu đồ lớp (Class Diagram)

Biểu đồ lớp của hệ thống được đưa ra và minh họa như Hình 44 gồm 7 lớp chính là Profile, User Story, WorkspaceMember, WorkspaceRole, Backlog, Workspace, và Sprint.

4.2.2. Biểu đồ đóng gói (Package Diagram) chi tiết



Hình 45: Biểu đồ đóng gói (Package Diagram)

Sơ đồ gói minh họa cách các thành phần của hệ thống ứng dụng được tổ chức logic thành các nhóm chức năng, được miêu tả trong Hình 45. Sơ đồ này xác định ranh giới và mối quan hệ phụ thuộc giữa các lớp logic chính:

1. **Presentation Layer:** Được xây dựng bằng Flutter, lớp này chứa tất cả các thành phần giao diện người dùng (User Interface) như Workspace, Login, Sign up, Graph... Lớp này xử lý logic hiển thị (Presentation Logic) và gửi các yêu cầu (HTTP Request) đến lớp nghiệp vụ khi người dùng tương tác.
2. **Business Logic Layer:** Đây là lớp trung tâm chứa logic nghiệp vụ cốt lõi của hệ thống. Lớp này tiếp nhận yêu cầu từ Presentation Layer và thực thi các quy trình nghiệp vụ (Business Workflow) thông qua các Business Components (thành phần xử lý) cụ thể như Handling user Authentication, Create Workspace, User Story, Generate Graph, ...

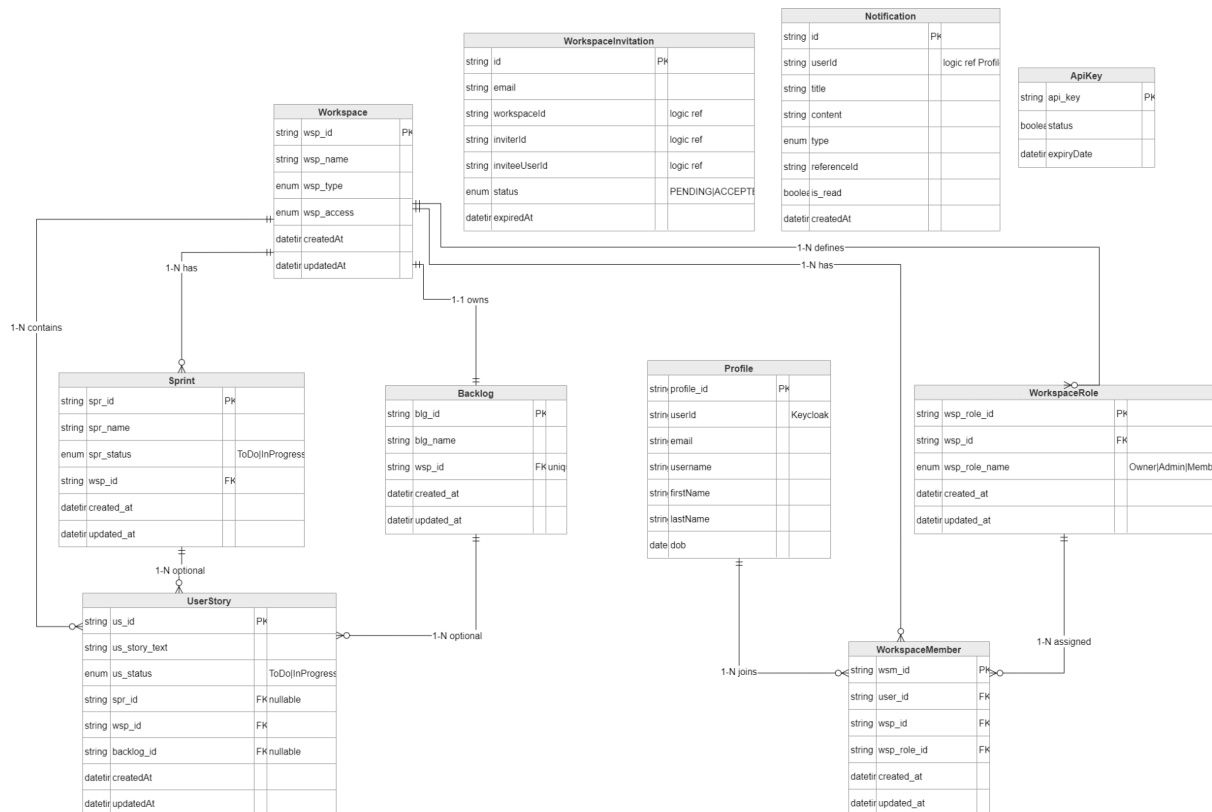
3. Data Layer: Lớp này chịu trách nhiệm hoàn toàn cho việc truy cập dữ liệu. Nó bao gồm các thành phần Data Access đóng vai trò như các "repositories" để trừu tượng hóa các truy vấn đến cơ sở dữ liệu. Lớp này tương tác với Data Source duy nhất của hệ thống là MySQL.
4. Cross-Cutting: Mặc dù không được thể hiện trên sơ đồ gốc, đây là một phần quan trọng của thiết kế, bao gồm các gói tiện ích được sử dụng bởi tất cả các lớp khác. Các gói này bao gồm Security, Logging (ghi log hệ thống), và Error Handling (quản lý ngoại lệ).

4.2.3. Sơ đồ Thực thể Quan hệ Logic (Logical ERD Diagram)

4.2.3.1. Sơ đồ Thực thể Quan hệ Logic - Backend

Store: MySQL

Phạm vi: Nghiệp vụ Agile + Cộng tác + Thông báo + Bảo mật API



Hình 46: Sơ đồ Thực thể Quan hệ Logic (Logical ERD Diagram)

Bảng 49: Bảng mô tả sơ đồ Thực thể Quan hệ Logic (Logical ERD Diagram)

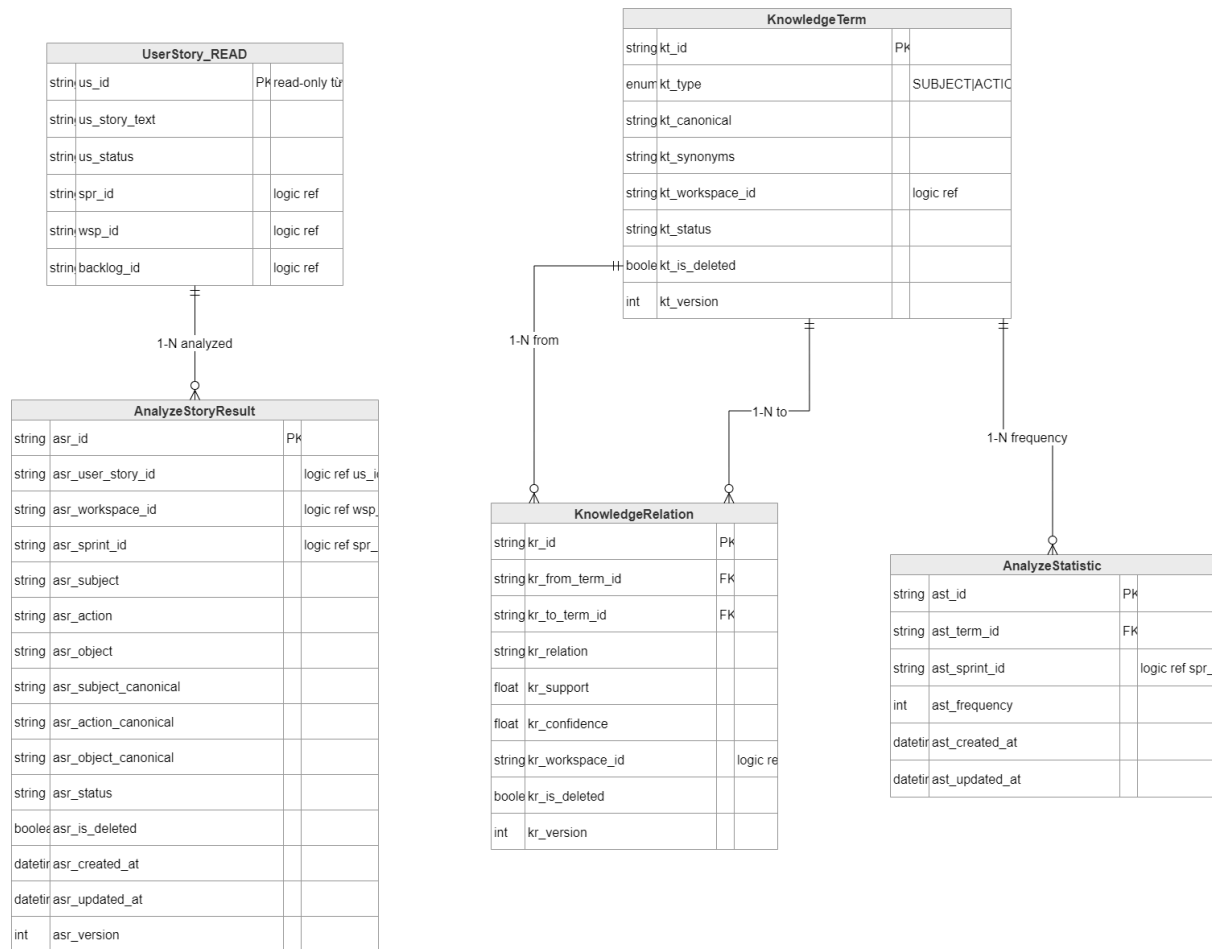
Quan hệ	Bội số	Ghi chú nghiệp vụ
Workspace ↔ Backlog	1–1	Mỗi workspace có đúng một backlog (wsp_id unique trên Backlog)

Workspace → Sprint	1–N	Nhiều sprint/workspace; tối đa 1 sprint InProgress
Workspace → UserStory	1–N	Story luôn thuộc một workspace
Sprint → UserStory	1–N	spr_id = null → story đang ở backlog
Backlog → UserStory	1–N	Gán lại khi complete sprint (story chưa Done)
Profile ↔ Workspace	N–M	Qua WorkspaceMember + vai trò WorkspaceRole
WorkspaceInvitation	Độc lập	Tham chiếu logic workspaceId, inviterId, inviteeUserId — không FK vật lý
Notification	Độc lập	userId tham chiếu logic tới Profile/Keycloak
ApiKey	Độc lập	Bảo vệ endpoint nội bộ

4.2.3.2. Sơ đồ Thực thể Quan hệ Logic - *analyze_user_stories*

Store: MySQL (bảng phân tích + đọc user_story từ database) + Neo4j (đồ thị tri thức)

Đồng bộ với BE: khóa logic us_id, wsp_id, spr_id, backlog_id (không FK xuyên service)

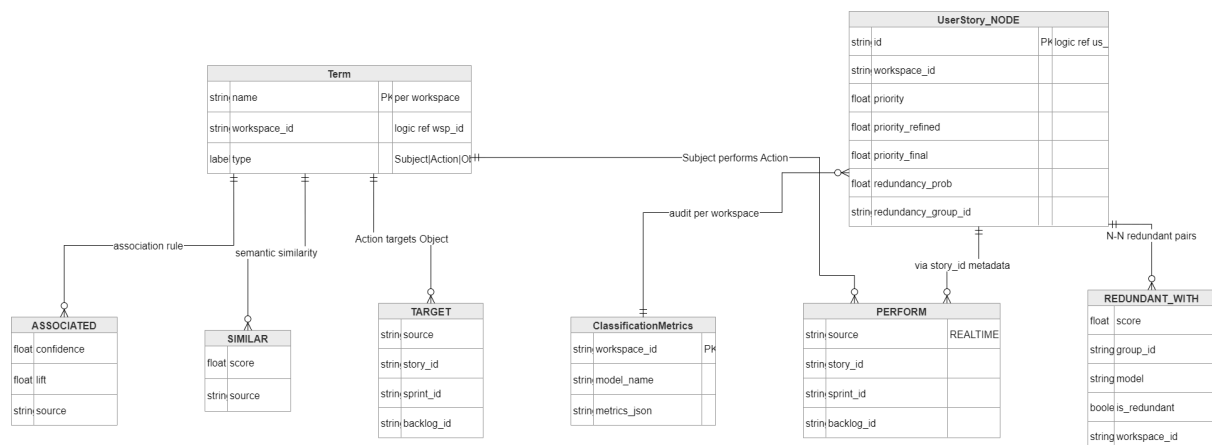


Hình 47: Sơ đồ Thực thể Quan hệ Logic - analyze_user_stories

Bảng 50: Bảng mô tả sơ đồ Thực thể Quan hệ Logic - analyze_user_stories

Quan hệ	Bộ số	Ghi chú nghiệp vụ
UserStory (read) → AnalyzeStoryResult	1–N	Một story có thể được phân tích nhiều lần (realtime/batch); consumer kiểm tra idempotent qua <code>asr_user_story_id</code> + <code>asr_is_deleted</code>
AnalyzeStoryResult → UserStory (BE)	N–1 logic	<code>asr_user_story_id</code> tham chiếu <code>us_id</code> trên BE; không có FK vật lý
AnalyzeStoryResult → Workspace / Sprint	N–1 logic	<code>asr_workspace_id</code> , <code>asr_sprint_id</code> lưu ngữ cảnh

		phân tích; cập nhật khi nhận event USER_STORY_MOVED
KnowledgeTerm → KnowledgeRelation (from)	1–N	Một term có thể là antecedent của nhiều quan hệ luật kết hợp
KnowledgeTerm → KnowledgeRelation (to)	1–N	Một term có thể là consequent của nhiều quan hệ
KnowledgeTerm → AnalyzeStatistic	1–N	Mỗi term có thể có nhiều bản ghi tần suất theo từng sprint
AnalyzeStatistic → Sprint (BE)	N–1 logic	ast_sprint_id tham chiếu sprint trên BE; rebuild khi batch theo sprint
KnowledgeTerm ↔ KnowledgeTerm	N–M	Qua bảng KnowledgeRelation (kr_from_term_id, kr_to_term_id); sinh từ Apriori (support, confidence) [5]
UserStory (read)	Mirror logic	Bảng user_story chỉ đọc từ MySQL slave của BE; Python không ghi ngược

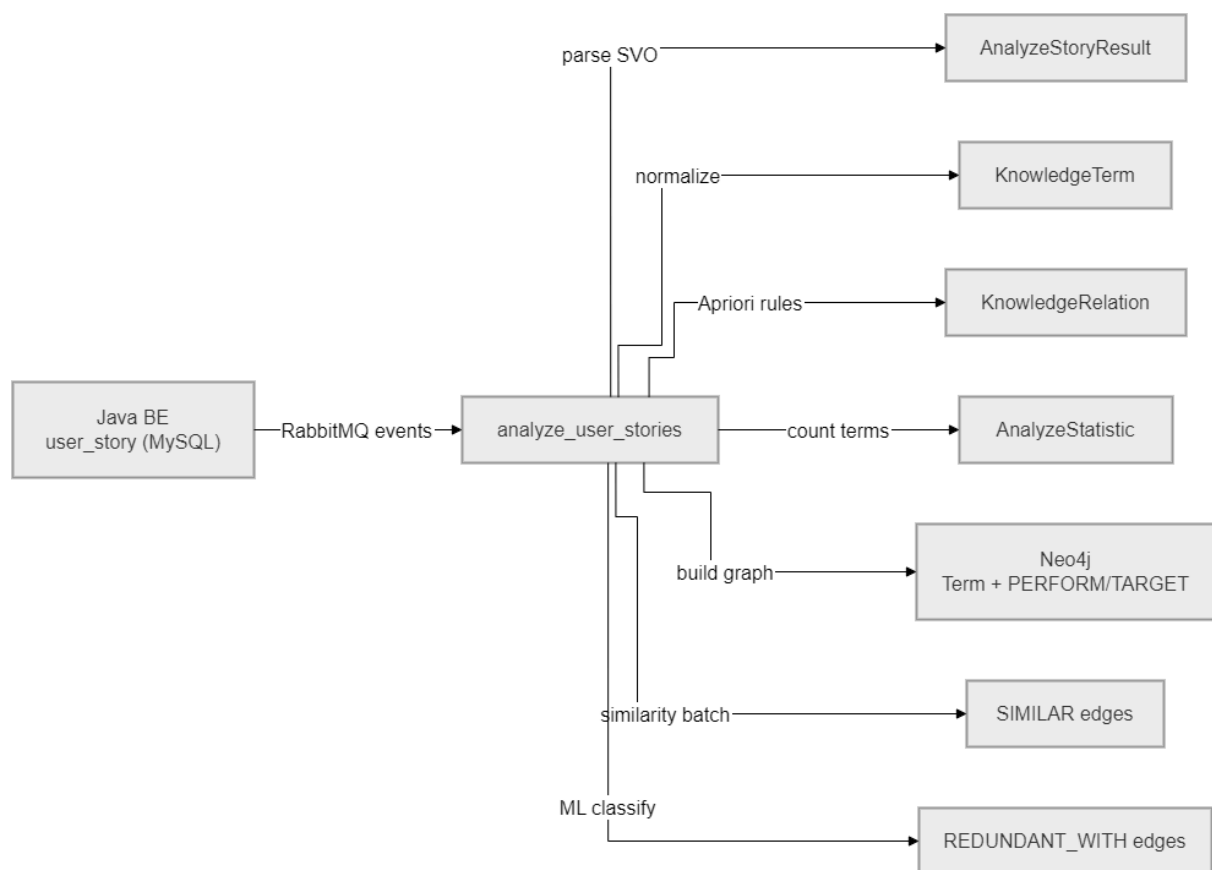


Hình 48: Đồ thị tri thức & ML

Bảng 51: Bảng mô tả Đồ thị tri thức & ML

Quan hệ	Bộ số	Ghi chú nghiệp vụ
Term → Term (PERFORM)	N–M	Subject thực hiện Action; cạnh mang story_id, sprint_id, backlog_id, source (REALTIME BATCH) để lọc theo ngữ cảnh
Term → Term (TARGET)	N–M	Action nhắm tới Object; cùng metadata ngữ cảnh như PERFORM
Term → Term (SIMILAR)	N–M	Tương đồng ngữ nghĩa giữa hai term (score); sinh từ batch/realtime merge
Term → Term (ASSOCIATED)	N–M	Luật kết hợp từ Apriori (confidence, lift) [5]; đẩy từ KnowledgeRelation (MySQL) lên đồ thị
UserStory (node) → Term (PERFORM)	1–N logic	Liên kết qua thuộc tính story_id trên cạnh PERFORM/TARGET, không phải FK

UserStory (node) ↔ UserStory (node)	N-M	Qua cạnh REDUNDANT_WITH (score, group_id, model, is_redundant); kết quả pipeline redundancy classification
UserStory (node) → UserStory (BE)	1-1 logic	id node = us_id trên MySQL BE; node chỉ phục vụ trực quan hóa và ML
Term → Workspace	N-1 logic	Mọi node Term gắn workspace_id; lọc graph theo phạm vi workspace/sprint/backlog
ClassificationMetrics → Workspace	1-1	Một bản ghi audit mô hình ML theo workspace (model_name, metrics_json)
MySQL AnalyzeStoryResult → Neo4j Term	N-N logic	Nhiều kết quả phân tích có thể map cùng term canonical; build graph qua pipeline parse → normalize → persist



Hình 49: Luồng logic

Bảng 52: Bảng mô tả Luồng logic

Quan hệ	Bộ số	Ghi chú nghiệp vụ
Java BE → UserStory (read)	1–1 mirror	Python đọc user_story từ MySQL slave; không sửa dữ liệu nghiệp vụ gốc
RabbitMQ event → AnalyzeStoryResult	1–N	USER_STORY_CREATED kích hoạt parse SVO realtime; USER_STORY_MOVED cập nhật asr_sprint_id
AnalyzeStoryResult → Neo4j (Term + cạnh)	1–N	Mỗi lần phân tích sinh/cập nhật node Term và cạnh PERFORM/TARGET
KnowledgeRelation (MySQL) →	1–1 logic	Luật kết hợp lưu MySQL trước, batch đẩy lên cạnh

ASSOCIATED (Neo4j)		ASSOCIATED
AnalyzeStatistic → ASSOCIATED	Hỗ trợ gián tiếp	Tần suất term theo sprint phục vụ Apriori và retrain
Neo4j REDUNDANT_WITH → UserStory (BE)	N–M logic	Chỉ phục vụ hiển thị ưu tiên/trùng lặp trên UI graph; không ghi ngược BE

4.3. Công thức và thuật toán tính toán trong hệ thống

4.3.1. Phân tích ngữ nghĩa và chuẩn hóa từ vựng (Semantic Normalization)

4.3.1.1. Mục đích

Trong quy trình Agile, cùng một nghiệp vụ có thể được diễn đạt bằng nhiều từ khác nhau (ví dụ create, add, register). Giai đoạn chuẩn hóa ngữ nghĩa nhằm phát hiện các hành động (action) hoặc đối tượng (object) có ý nghĩa tương đương, sau đó gộp chúng về một từ đại diện chung (Canonical Word). Nhờ đó, các bước khai phá luật kết hợp và phân tích đồ thị tri thức không bị phân mảnh do trùng từ khác chữ.

4.3.1.2. Độ tương đồng phi tuyến (Non-linear Fusion Similarity)

Hệ thống kết hợp thước đo tương đồng từ WordNet với vector từ nhúng Word2Vec, qua hàm phi tuyến với các tham số siêu cố định:

Tham số:

- $\beta_1 = 5,0$
- $\beta_2 = 1,3$
- $\text{bias} = -2,0$

Diễn giải: Các tham số điều khiển mức đóng góp Word2Vec và WordNet [7] trước khi ánh xạ điểm tương đồng về miền $[0, 1]$ dùng cho so khớp và gom nhóm.

4.3.1.3. Chỉ số Jaccard cho ngữ cảnh (Context Similarity)

Gọi Context_A và Context_B là hai tập Object đi kèm hai Action A và B.

Công thức (1): $\text{Context_Overlap} = |\text{Context_A} \cap \text{Context_B}| / |\text{Context_A} \cup \text{Context_B}|$

Diễn giải: Giá trị trong $[0, 1]$. Càng gần 1 thì hai action càng cùng tác động lên một nhóm object (ví dụ Create và Add cùng liên quan User, Account).

4.3.1.4. Thuật toán Union-Find — gom nhóm từ vựng

Hai từ được gom cùng cụm (một Canonical Word) khi đồng thời:

Similarity $\geq 0,75$ và Context $\geq 0,5$ [9]

- Similarity: điểm fusion phi tuyến [6]
- Context: giá trị Context_Overlap ở công thức (1)

Thuật toán Union-Find (Disjoint-Set) thực hiện union các cặp thỏa điều kiện.

4.3.2. Khai phá luật kết hợp (Association Rules)

4.3.2.1. Mục đích

Tìm mẫu nghiệp vụ: các từ (Subject, Action, Object) hay xuất hiện cùng nhau trong User Story.

4.3.2.2. Thuật toán Apriori

Tim Frequent Itemsets với ngưỡng: Min Support $\geq 0,1$ [5] (tức tập mục xuất hiện trong ít nhất 10% số User Story).

4.3.2.3. Các chỉ số sức mạnh của luật

- **Support (độ hỗ trợ):** Tỷ lệ số lần xuất hiện tập $\{A, B\}$ trên tổng số User Story.
- **Confidence (độ tin cậy):** Xác suất B xuất hiện khi A đã xuất hiện:
 - Confidence = $P(B | A)$ [5]
- **Lift (độ nâng):** Mức liên kết thực tế so với ngẫu nhiên. Lift $> 1 \rightarrow A$ thúc đẩy sự xuất hiện của B. [5]

Diễn giải: Ba chỉ số lưu vào đồ thị tri thức; dùng trong rule_confidence, rule_lift và Rule Signal ở công thức (6).

4.3.3. Chấm điểm mức độ ưu tiên (Priority Scoring)

4.3.3.1. Mục đích

Xác định User Story nào quan trọng hơn trong workspace, dựa trên ảnh hưởng S–V–O lên đồ thị tri thức và tín hiệu ngữ nghĩa–luật.

4.3.3.2. Độ trung tâm đồ thị (Graph Centrality)

- **Betweenness Centrality (Neo4j GDS) [4]:** số lần Term nằm trên đường đi ngắn nhất giữa hai Term khác.
- **Degree Centrality [11]:** tổng số kết nối (edges) của đỉnh; dùng fallback khi GDS lỗi.

4.3.3.3. Điểm tổng hợp cấu trúc (Struct Score)

Công thức (2): $x = 0,5 \times \text{Degree_Avg} + 0,5 \times \text{Betweenness_Avg}$ [4]

- **Degree_Avg**: trung bình degree các Object trong story
- **Betweenness_Avg**: trung bình betweenness các Object trong story

4.3.3.4. Chuẩn hóa (Scaling) và hàm Sigmoid

Tính trên toàn workspace:

- $IQR = Q_3 - Q_1$ [33]
- Median = trung vị của tập $\{x\}$ [33]

Công thức (3): $\alpha = 4 / IQR$ [33]

Công thức (4): $\beta = -\alpha \times \text{Median}$

Công thức (5) — Priority cơ sở: [33]

- $\text{Priority_base} = \frac{1}{1 + e^{-(\alpha x + \beta)}}$

Diễn giải: Dùng IQR và Median giảm ảnh hưởng outlier; sigmoid đưa điểm về khoảng (0, 1).

4.3.3.5. Công thức tính chỉnh cuối cùng

Công thức (6):

$\text{Priority_refined} = W_initial \times \text{Priority_base} + W_sim \times \text{Sim_Signal} + W_rule \times \text{Rule_Signal}$

Trọng số mặc định: $W_initial = 0,5$; $W_sim = 0,3$; $W_rule = 0,2$.

Công thức (7):

$\text{Priority_final} = \text{Priority_refined} \times (1 - \alpha_redundancy \times P_redundant)$

- $P_redundant$: xác suất dư thừa (mục 4.3.4)
- $\alpha_redundancy = 0,6$ (mặc định)
- Giá trị cuối giới hạn trong $[0, 1]$

4.3.4. Phân loại và phát hiện dư thừa (Redundancy Classification)

4.3.4.1. Mục đích

Tìm cặp/nhóm User Story trùng tính năng hoặc trùng mục đích để lược bỏ backlog.

4.3.4.2. Mô hình học máy

8 features [19]: same_subject, same_action, same_object, action_similarity, object_similarity, rule_confidence, rule_lift, chênh lệch priority.

Huấn luyện: Logistic Regression, Random Forest, Hist Gradient Boosting → chọn mô hình tốt nhất theo F1-score [19] → dự đoán $P_redundant$.

4.3.4.3. Công thức dự phòng (Rule-based Fallback)

Công thức (8):

$$P_{\text{redundant}} = 0,35 \times \text{Obj_Sim} + 0,2 \times \text{Action_Sim} + 0,15 \times \text{Same_Obj} + 0,15 \times \text{Rule_Conf} + 0,15 \times \text{Same_Action}$$

- **Obj_Sim, Action_Sim**: độ tương đồng object/action
- **Same_Obj, Same_Action**: biến 0/1 trùng khớp
- **Rule_Conf**: confidence luật kết hợp
- Kết quả giới hạn trong $[0, 1]$

4.3.4.4. Gom nhóm

$$P_{\text{redundant}} \geq \text{Threshold} \quad (\text{Threshold mặc định} = 0,6)$$

→ Union-Find → Redundancy Groups. [9]

4.3.5. Công thức trên đồ thị

4.3.5.1. Phân bố tọa độ S–V–O (Column Layout)

Hàm: `_calculateLayout`.

Trục X:

- $X(\text{Subject}) = 350$
- $X(\text{Verb}) = 650$
- $X(\text{Object}) = 950$

Trục Y:

- $Y_0 = 200$
- $Y_{\square} = Y_{\square-1} + \text{Spacing} \quad (\text{Spacing} = 120)$

4.3.5.2. Kéo thả (Drag Node và Matrix Transformation)

Hàm: `onPanStart`, `onPanUpdate`.

- $\text{ScenePoint} = \text{Ma}_{\text{trận_nghịch_đảo}} \times \text{Tọa_độ_chuột_Local}$
- $\text{Drag_Offset} = \text{NodePosition_old} - \text{ScenePoint_start}$
- $\text{NodePosition_new} = \text{ScenePoint_current} + \text{Drag_Offset}$

4.3.5.3. Max-Pooling Priority

Hàm: `_buildNodeKeyPriorityCache`.

$$\text{Priority}(k) = \text{MAX}(P(s_1), P(s_2), \dots, P(s_{\square}))$$

4.3.5.4. Lọc ngưỡng (Threshold Culling)

Hàm: `_updateDimmedSets`.

$$H_{\text{nodes}} = \{ \text{Node} \mid \text{Priority}(\text{Node}) < \text{Slider_Threshold} \}$$

Cạnh (A, B) bị ẩn khi và chỉ khi A thuộc H_{nodes} HOẶC B thuộc H_{nodes} .

4.3.5.5. *Active Neighbor Search*

IsActive(V) = (V trùng H) HOẶC (có cạnh $V \rightarrow H$) HOẶC (có cạnh $H \rightarrow V$)

- Active: Opacity = 100%
- Không active: Opacity $\approx$ 20%

4.3.5.6. *Quadratic Bezier Curve*

Hàm: `_drawCurvedLine`.

Mid_X = (X_from + X_to) / 2

Mid_Y = (Y_from + Y_to) / 2

Path: From $\rightarrow$ Mid Point $\rightarrow$ To.

4.3.5.7. *Glow Pulse Animation*

Hàm: `GlowCirclePainter`.

Pulse = $0,5 + 0,5 \times \sin(\text{AnimValue} \times 2\pi)$
(AnimValue từ 0,0 đến 1,0, chu kỳ 2 giây.)

Blur = $8 + 4 \times \text{Pulse}$

Opacity = $0,15 + 0,1 \times \text{Pulse}$

4.3.5.8. *Dashed Circle Zoning*

Hàm: `_drawDashedCircle`.

DashWidth = 8

DashSpace = 6

d_mới = d_cũ + DashWidth + DashSpace

(lặp d từ 0 đến hết chu vi.)

4.3.5.9. Lasso Polygon Tracing

Hàm: `LassoPainter` — thu List<Offset>, nối MoveTo + LineTo $\rightarrow$ đa giác khép kín.

4.3.6. *Tổng kết*

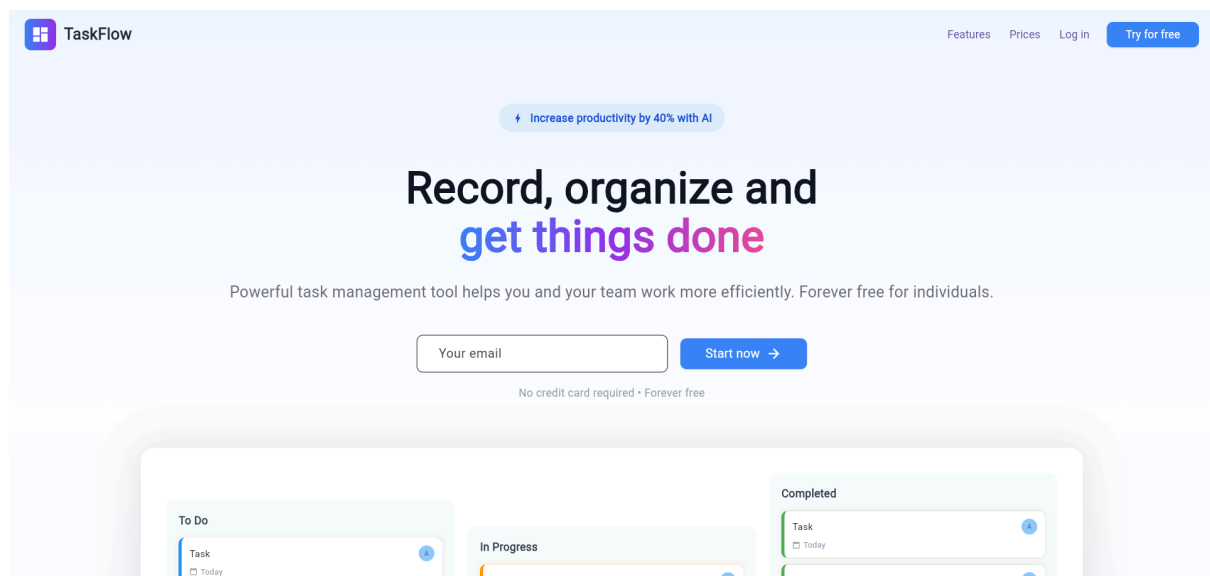
Công thức số	Công thức	Nội dung
--------------	-----------	----------

(1)	$\text{Context_Overlap} = \text{Context_A} \cap \text{Context_B} / \text{Context_A} \cup \text{Context_B} $	Jaccard ngữ cảnh
(2)	$x = 0,5 \times \text{Degree_Avg} + 0,5 \times \text{Betweenness_Avg}$	Điểm cấu trúc
(3)	$\alpha = 4 / \text{IQR}$	Hệ số co giãn
(4)	$\beta = -\alpha \times \text{Median}$	Hệ số dịch
(5)	$\text{Priority_base} = 1 / (1 + e^{-(\alpha \times x + \beta)})$	Priority cơ sở
(6)	$\text{Priority_refined} = W_{\text{initial}} \times \text{Priority_base} + W_{\text{sim}} \times \text{Sim_Signal} + W_{\text{rule}} \times \text{Rule_Signal}$	Tinh chỉnh
(7)	$\text{Priority_final} = \text{Priority_refined} \times (1 - \alpha_{\text{redundancy}} \times P_{\text{redundant}})$	Priority cuối sau khi xử lý dư thừa
(8)	$P_{\text{redundant}} = 0,35 \times \text{Obj_Sim} + 0,2 \times \text{Action_Sim} + 0,15 \times \text{Same_Obj} + 0,15 \times \text{Rule_Conf} + 0,15 \times \text{Same_Action}$	Fallback

4.4. Thiết kế giao diện người dùng (Giao diện Web)

4.4.1. Giao diện trang đích (Landing page)

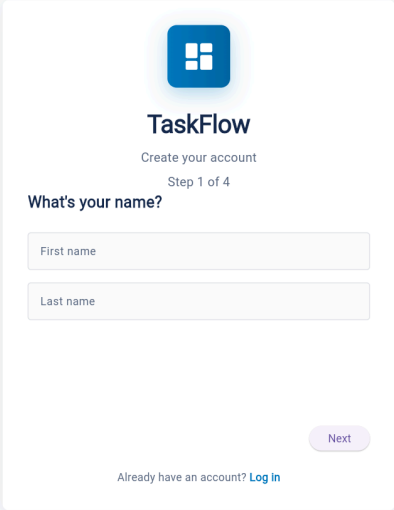
Giao diện trang đích là giao diện đầu tiên người dùng nhìn thấy khi truy cập vào hệ thống, được miêu tả trong Hình 50. Trang này được thiết kế với mục tiêu giới thiệu tổng quan về sản phẩm, truyền tải giá trị cốt lõi và khuyến khích người dùng đăng ký sử dụng.



Hình 50: Giao diện trang đích

4.4.2. Giao diện đăng ký

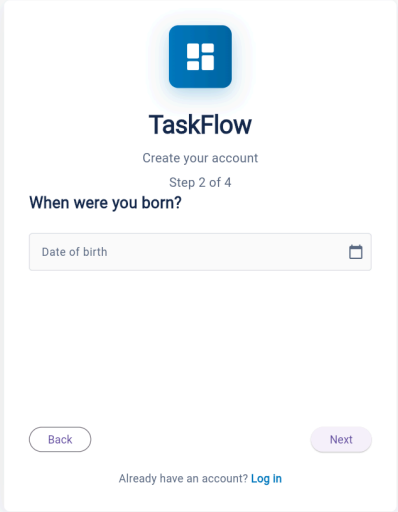
Giao diện đăng ký của hệ thống được thiết kế theo mô hình **Xác thực nhiều bước (Multi-Step Authentication)** nhằm tăng cường bảo mật và đảm bảo tính chính xác của thông tin người dùng. Hình 51 là giao diện khi vừa vào đăng ký với họ và tên cần điền ở bước 1:



The image shows a registration form for 'TaskFlow'. At the top is a blue square logo with a white grid pattern. Below the logo is the text 'TaskFlow' in bold, followed by 'Create your account' and 'Step 1 of 4'. The main heading is 'What's your name?'. There are two input fields: 'First name' and 'Last name'. At the bottom right is a 'Next' button. At the bottom left is a link: 'Already have an account? [Log in](#)'.

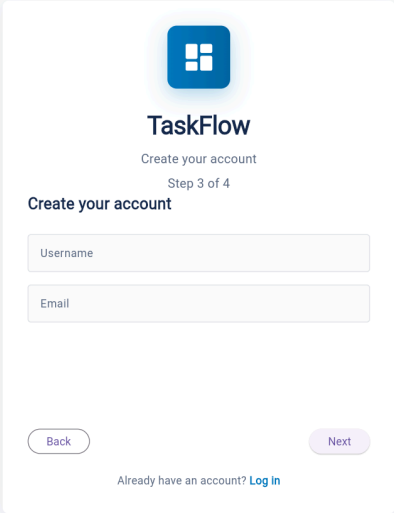
Hình 51: Giao diện đăng ký

Tiếp theo các giao diện sau cho phép người dùng thực hiện việc đăng ký bằng cách nhập ngày tháng năm sinh, tên người dùng, email và mật khẩu với các bước 2,3,4 để hình thành tài khoản cá nhân mới, miêu tả tuần tự từ Hình 52, Hình 53, và Hình 54



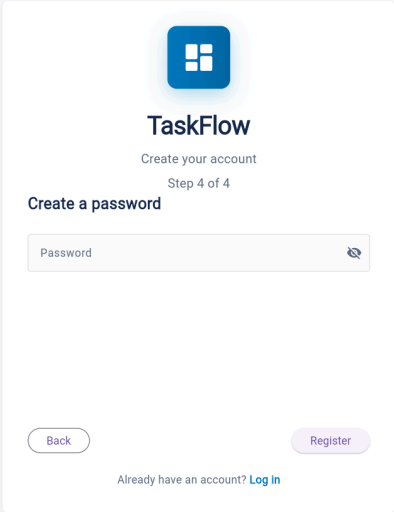
The image shows a mobile application registration screen for 'TaskFlow'. At the top is a blue square logo with a white grid pattern. Below the logo, the text 'TaskFlow' is displayed in a bold, dark font. Underneath, it says 'Create your account' and 'Step 2 of 4'. The main heading for this step is 'When were you born?'. Below this heading is a light gray input field with the placeholder text 'Date of birth' and a small calendar icon on the right. At the bottom of the screen, there are two rounded buttons: 'Back' on the left and 'Next' on the right. Below these buttons, there is a link that says 'Already have an account? Log in'.

Hình 52: Giao diện đăng ký



The image shows a registration form for TaskFlow, specifically Step 3 of 4. At the top is the TaskFlow logo, a blue square with a white grid pattern. Below the logo is the text "TaskFlow" in bold, followed by "Create your account" and "Step 3 of 4". The main heading is "Create your account". There are two input fields: "Username" and "Email". At the bottom, there are two buttons: "Back" and "Next". Below the buttons, there is a link: "Already have an account? [Log in](#)".

Hình 53: Giao diện đăng ký

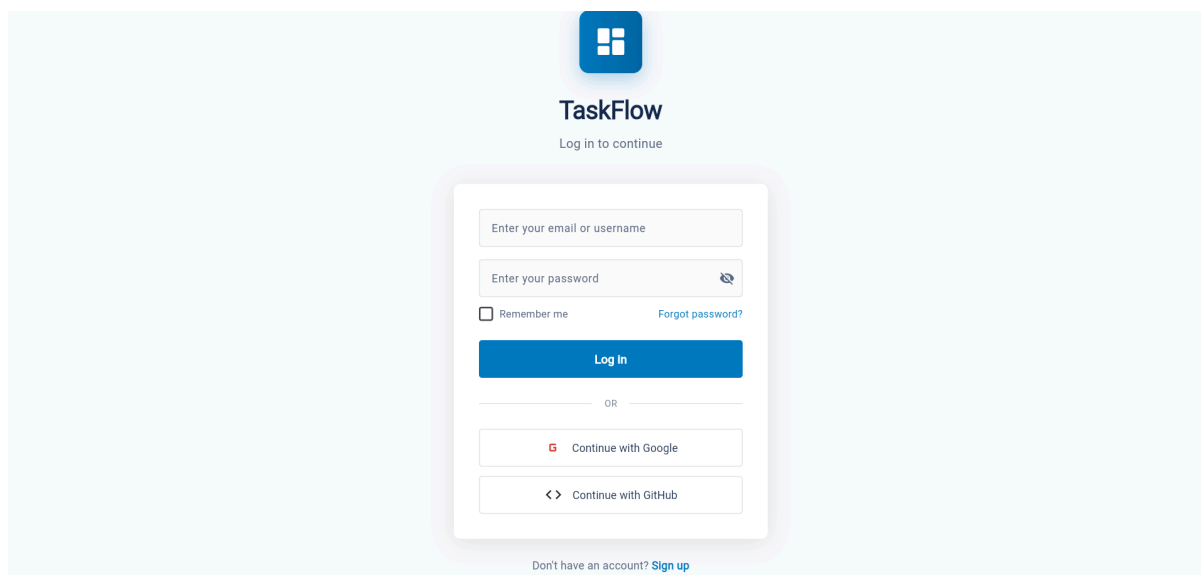


The image shows a registration form for TaskFlow, specifically Step 4 of 4. At the top is the TaskFlow logo, a blue square with a white grid pattern. Below the logo is the text "TaskFlow" in bold, followed by "Create your account" and "Step 4 of 4". The main heading is "Create a password". There is one input field: "Password". At the bottom, there are two buttons: "Back" and "Register". Below the buttons, there is a link: "Already have an account? [Log in](#)".

Hình 54: Giao diện đăng ký

4.4.3. Giao diện đăng nhập

Giao diện đăng nhập miêu tả trong Hình 55 cho phép người dùng truy cập vào tài khoản hiện có bằng thông tin xác thực của mình hoặc thông qua các nền tảng đăng nhập khác.

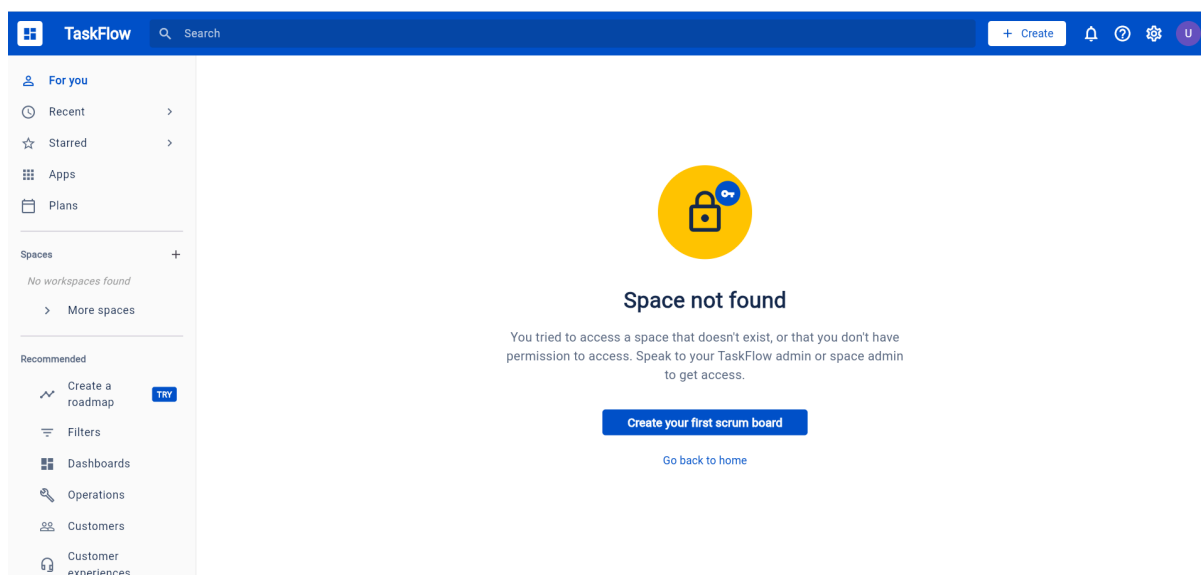


Hình 55: Giao diện đăng nhập

4.4.4. Giao diện trang chủ

Giao diện trang chủ, Hình 56 được hiển thị khi người dùng đăng nhập thành công nhưng chưa tạo bất kỳ không gian làm việc (Workspace/Space) nào trong hệ thống.

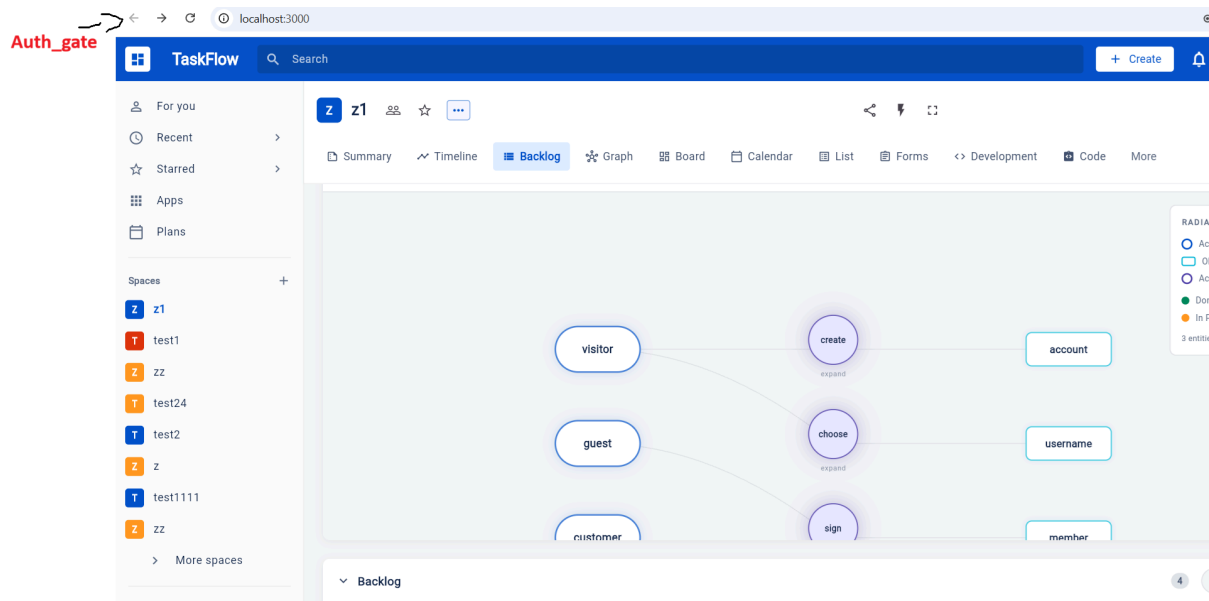
Hệ thống hiển thị trạng thái “Space not found” nhằm thông báo rằng tài khoản hiện chưa có dữ liệu làm việc.



Hình 56: Giao diện trang chủ

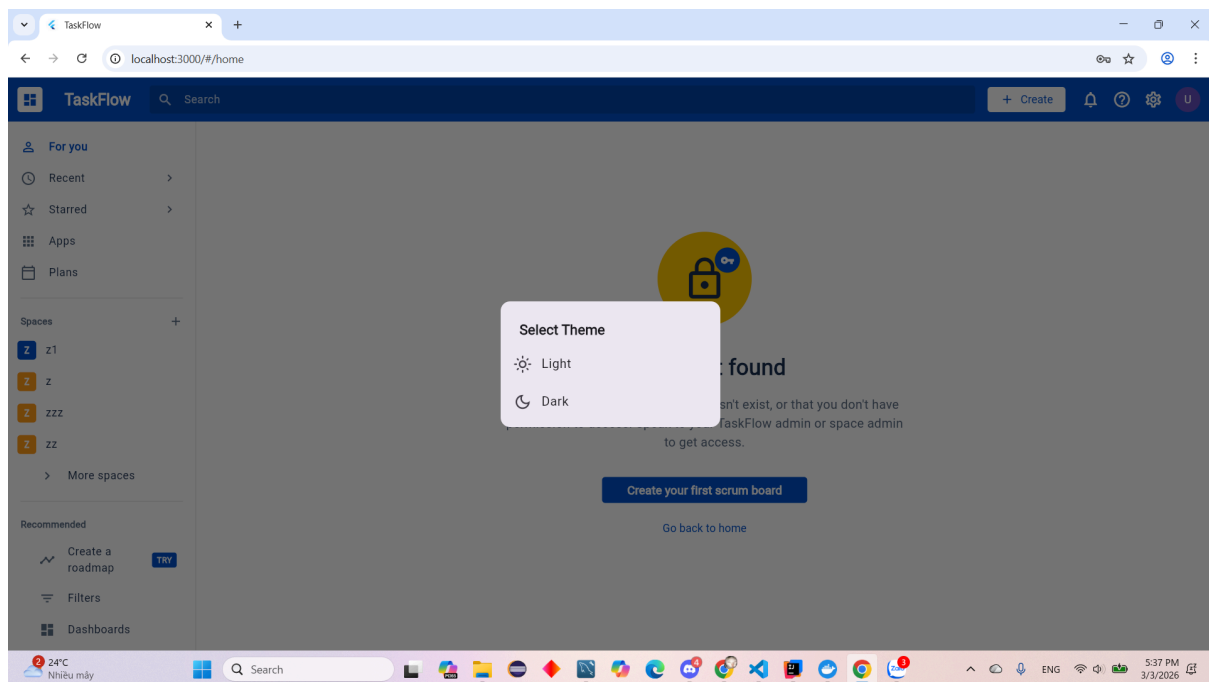
Sau khi truy cập vào giao diện trang chủ, hệ thống được thiết lập một cơ chế kiểm soát điều hướng (Auth Gate) nhằm đảm bảo tính nhất quán của phiên làm việc (session). Cụ thể, một khi người dùng đã đăng nhập thành công, hệ thống sẽ ngăn chặn việc quay ngược lại trang giới thiệu (Landing Page) hoặc trang đăng nhập. Nếu người dùng cố tình truy cập lại các

trang này thông qua URL hoặc nút 'Back' trên trình duyệt, Auth Gate sẽ tự động nhận diện trạng thái đã xác thực và điều hướng (redirect) họ trở lại trang chủ của ứng dụng. Điều này giúp tối ưu hóa trải nghiệm người dùng (UX) và tránh các luồng thao tác thừa thãi.

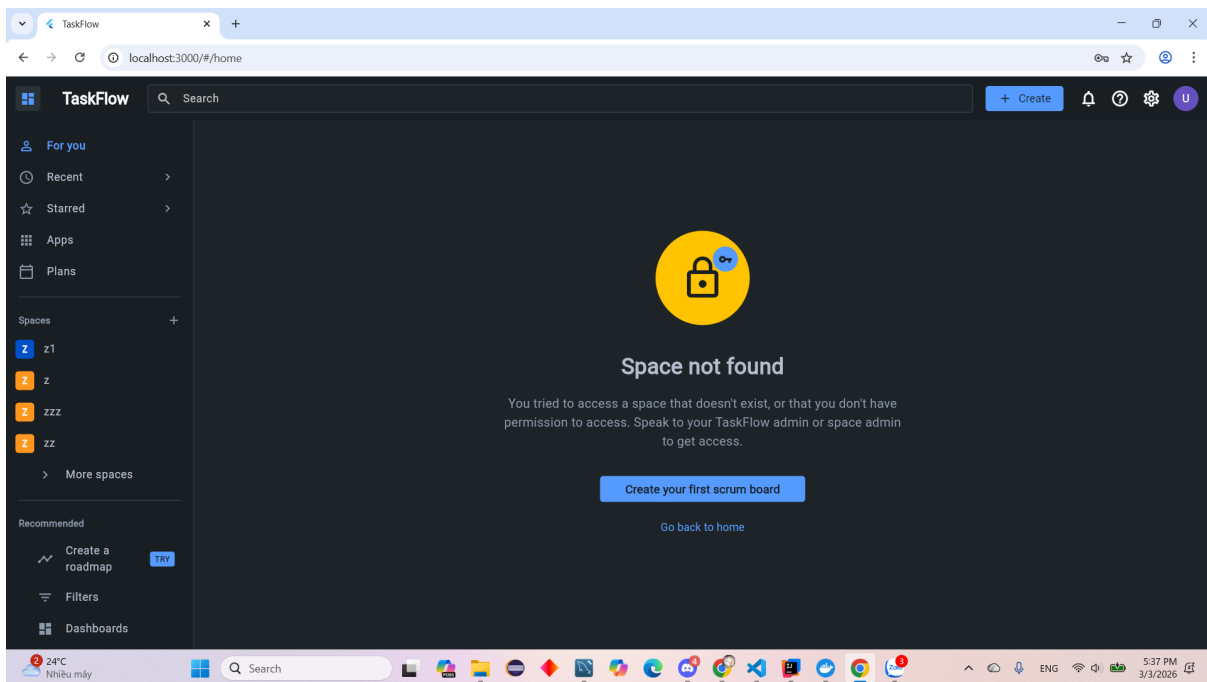


Hình 57: Giao diện trang chủ cơ chế điều hướng (Auth_gate)

Giao diện trang chủ có chức năng chuyển đổi chủ đề (Theme) từ chế độ sáng (Light Mode) sang chế độ tối (Dark Mode). Đây là một tùy chọn thiết kế cho phép người dùng chuyển đổi toàn bộ giao diện của ứng dụng TaskFlow từ nền sáng sang nền tối. Tính năng này cập nhật giao diện ngay lập tức mà không cần tải lại trang.



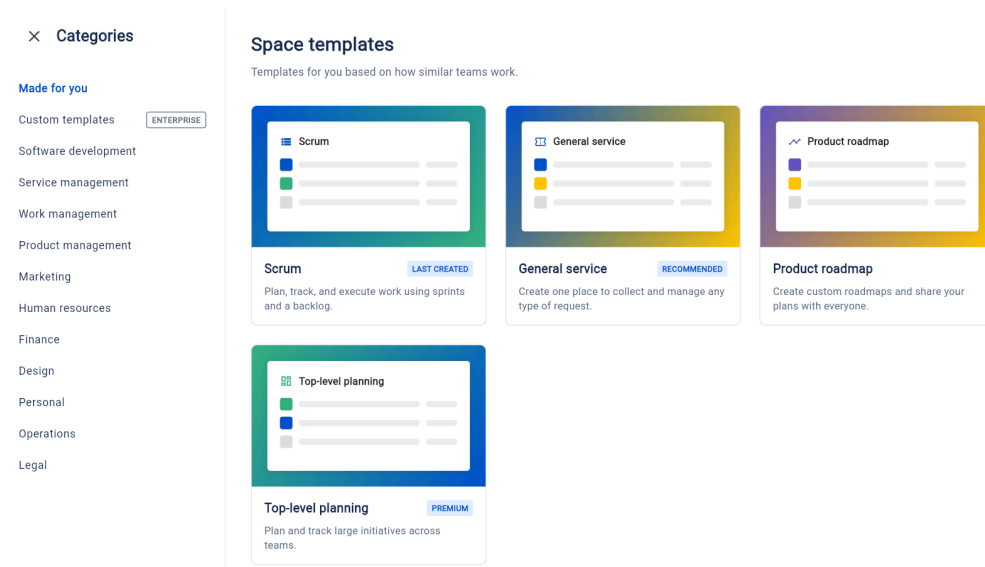
Hình 58: Giao diện trang chủ lựa chọn chế độ



Hình 59: Giao diện trang chủ chế độ đêm (Night mode)

4.4.5. Giao diện các mẫu không gian làm việc (*Space templates*)

Giao diện các mẫu không gian làm việc, được miêu tả trong Hình 60, cho phép người dùng tạo Space nhanh chóng dựa trên các mẫu (template) có sẵn thay vì phải cấu hình từ đầu.



Hình 60. Giao diện Space Templates

4.4.6. Giao diện khởi tạo *Workspace*

Giao diện khởi tạo, miêu tả trong Hình 61, dùng để **khởi tạo một dự án hoặc không gian làm việc**

(Workspace) mới. Bạn có thể nhập tên dự án, chọn cách thức quản lý dự án (Team-managed) và thiết lập quyền truy cập ban đầu (Open).

← [Back to project templates](#)

Create project

Name *

Try a team name, project goal, milestone...

How your space is managed

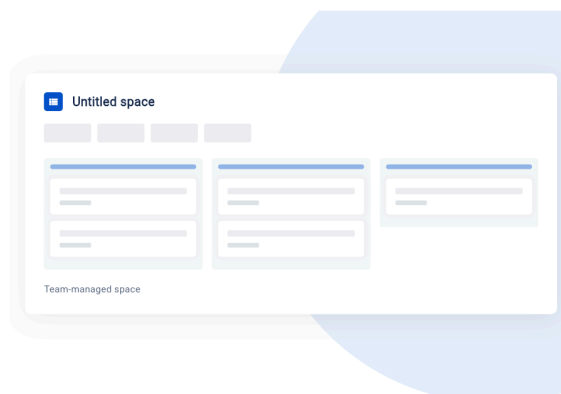
Team-managed

Access

Open

Cancel

Next



Hình 61: Giao diện tạo Workspace

4.4.7. Giao diện mời thành viên

Giao diện mời thành viên, Hình 62, có thể **mời đồng đội tham gia** vào không gian làm việc vừa tạo. Ở trạng thái này, bạn có thể nhập email và thiết lập vai trò mặc định là "Administrator" (Quản trị viên - người có quyền quản lý hệ thống) cho người được mời hoặc các lựa chọn khác như là "Member" và "Viewer".

← Back to project setup

Bring your team along

Collaborate instantly...

Email addresses

e.g., name@company.com

Role

Administrator

Admins can do most things...

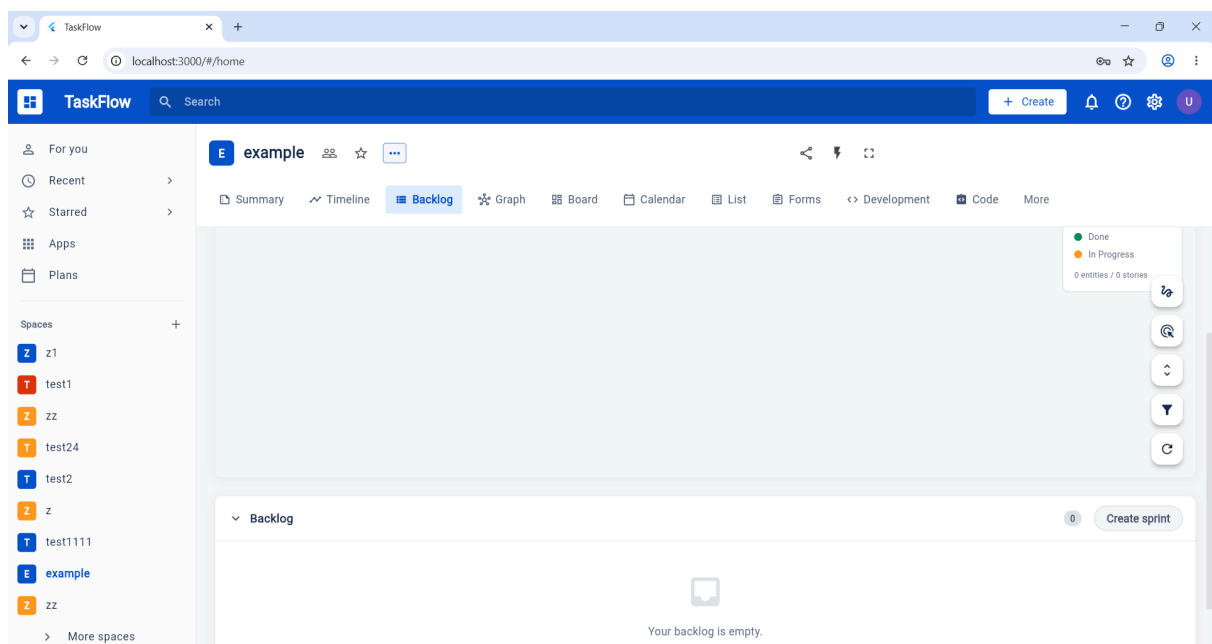
Skip for now

Invite and continue

Hình 62: Giao diện mời

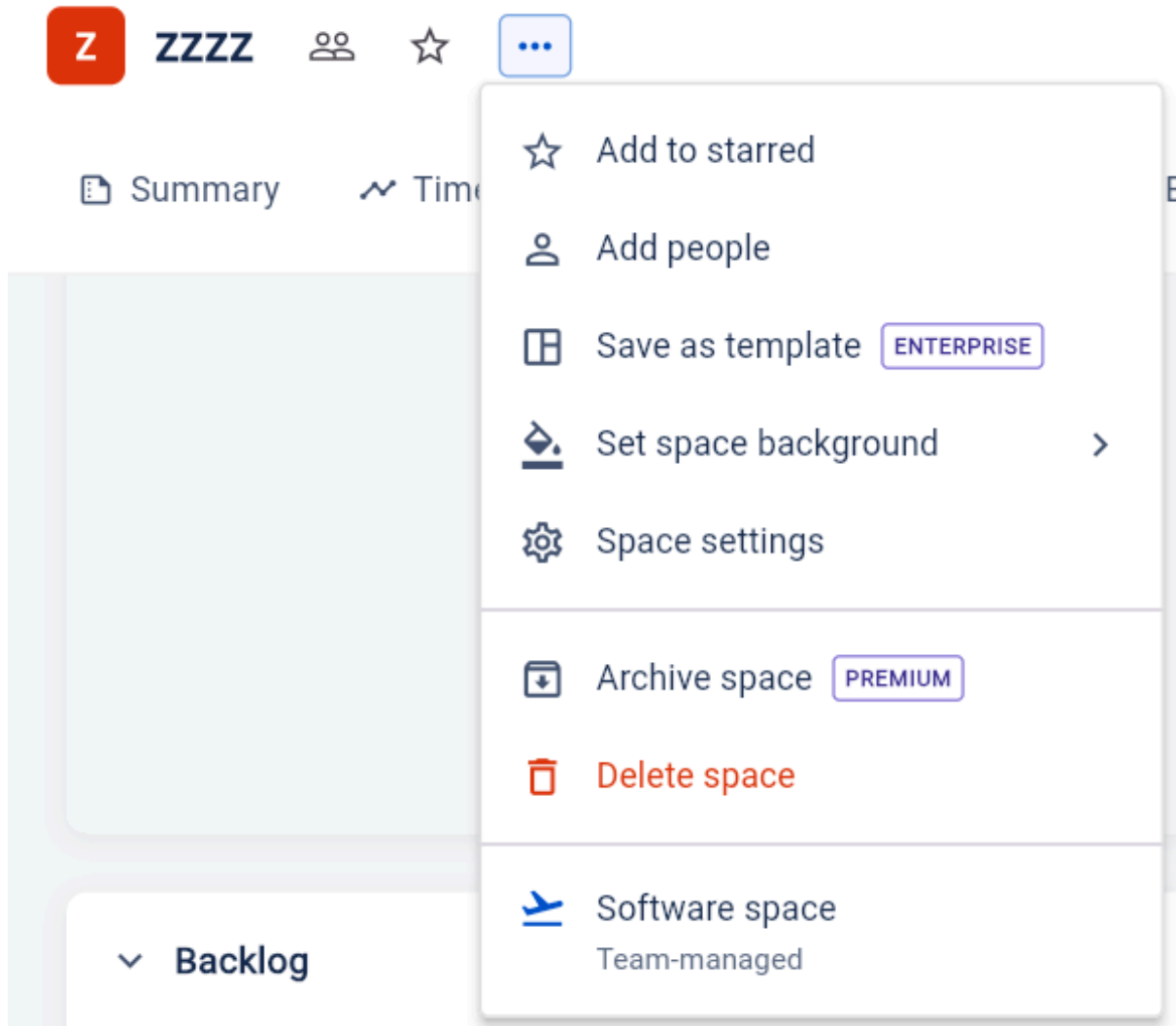
4.4.8. Giao diện Backlog

Giao diện Backlog được miêu tả trong Hình 63. Đây là nơi quản lý **danh sách các công việc tồn đọng**. Giao diện hiển thị trạng thái trống ("Your Backlog is empty") và cung cấp các nút thao tác để bạn bắt đầu tạo yêu cầu mới ("Create issue").



Hình 63: Giao diện Backlog

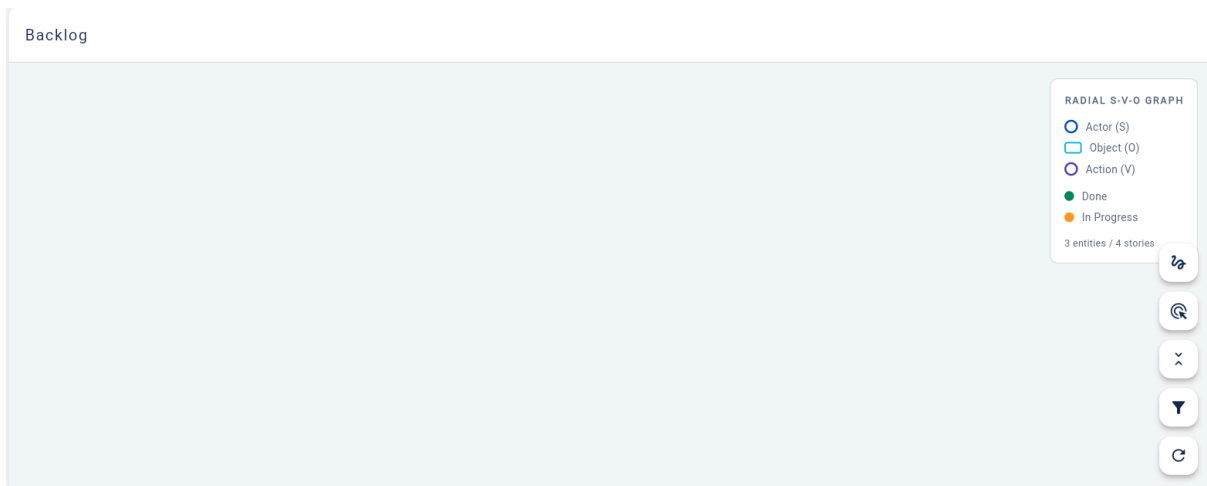
Hình 64 là menu tùy chọn (dropdown menu) của một không gian làm việc (Space). Nó cho phép quản trị viên thực hiện các thao tác quản lý cấp độ dự án như: đánh dấu sao yêu thích (Add to starred), mời thành viên (Add people), cài đặt không gian (Space settings), hoặc các thao tác nguy hiểm hơn như lưu trữ (Archive space) và xóa dự án (Delete space).



Hình 64: Giao diện Backlog

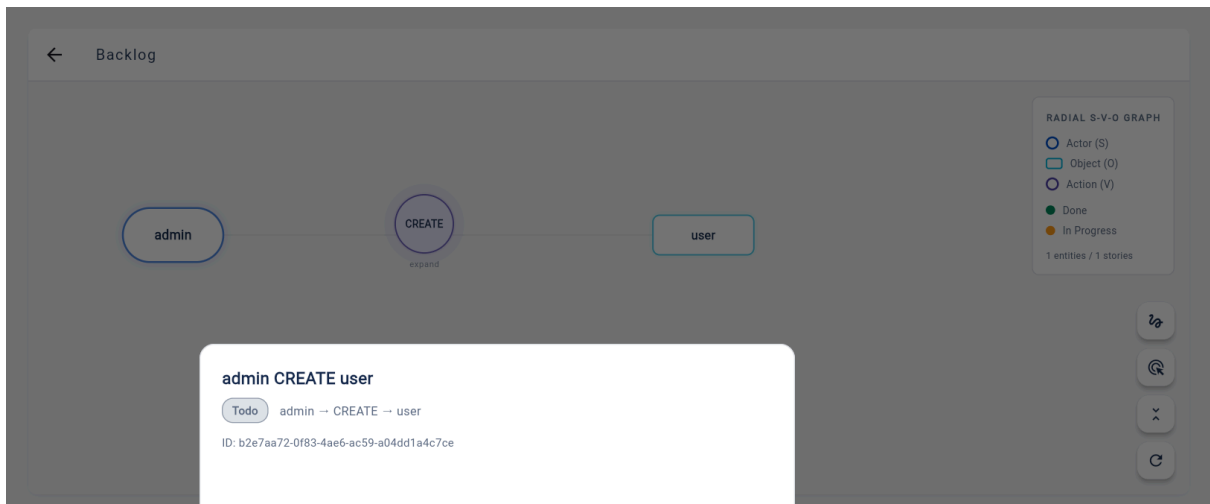
4.4.9. Giao diện Graph Backlog

Giao diện Graph Backlog, Hình 65, cung cấp một không gian trực quan hóa dữ liệu dạng "Radial S-V-O Graph" (Biểu đồ Chủ thể - Hành động - Đối tượng). Không gian màu đen hiện tại đang trống do dự án chưa có dữ liệu/công việc nào.



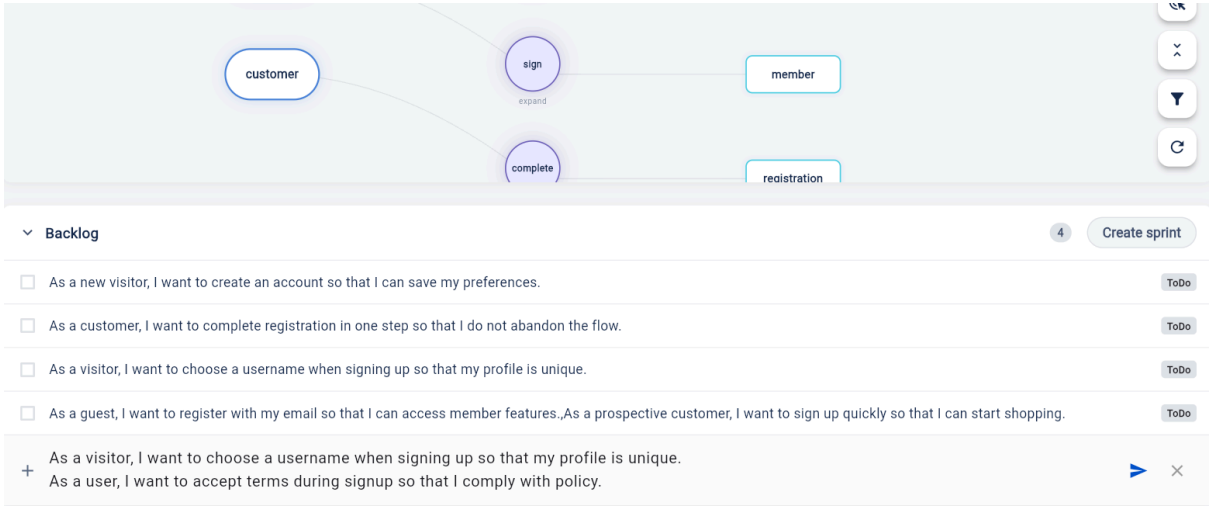
Hình 65: Giao diện Backlog

Sau khi thêm 1 yêu cầu tại Workspace, một công việc được mô phỏng qua các điểm nút. Khi tương tác với biểu đồ, một hộp thoại chi tiết sẽ hiện ra hiển thị ID và trạng thái (ToDo) của công việc đó, Hình 66.



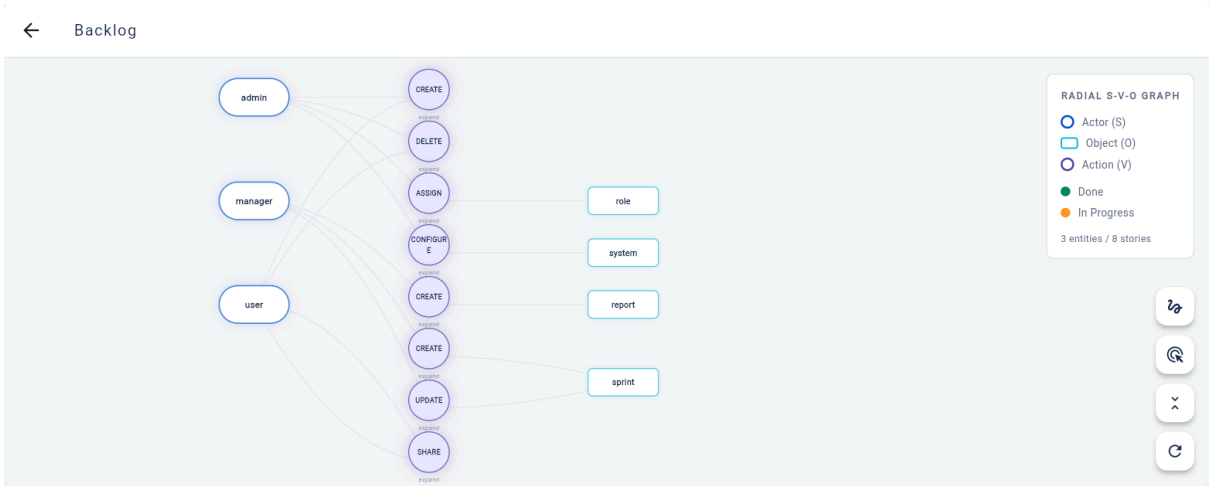
Hình 66: Giao diện Backlog

Có thể thêm nhiều công việc cùng 1 lúc bằng cách xuống dòng sau 1 câu chuyện người dùng.



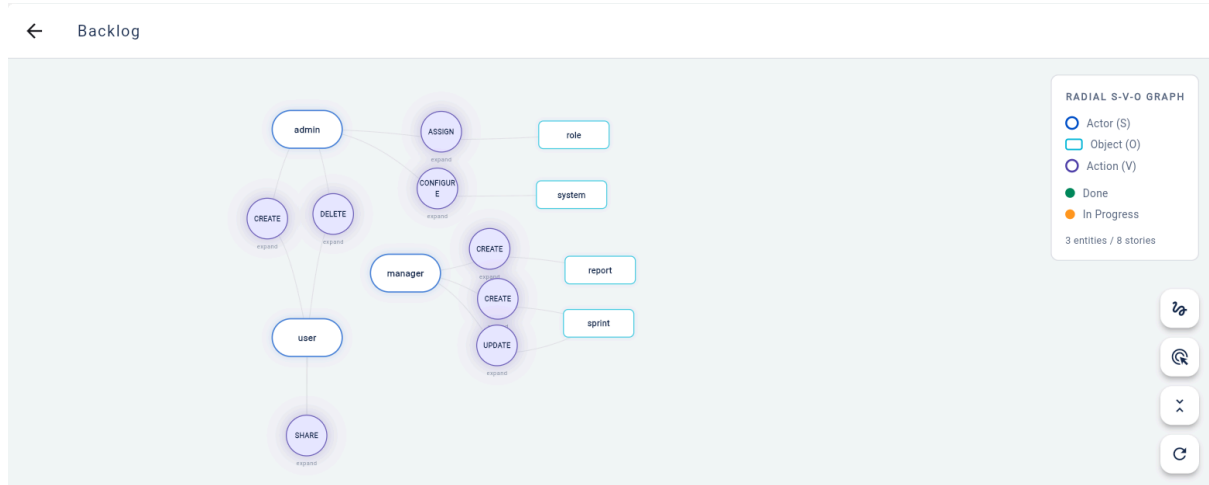
Hình 67: Giao diện Backlog

Giao diện, Hình 68, giúp team phát triển nhìn rõ tổng quan cấu trúc khi có nhiều khối lượng các công việc được chia theo các hàng Subject-Verb-Object (S-V-O).



Hình 68: Giao diện Backlog

Có thể di chuyển các node gần lại với nhau theo con trỏ chuột để giúp dễ nhìn và quản lý khối lượng công việc. Khi di chuyển 1 công việc sẽ in đậm các cạnh.



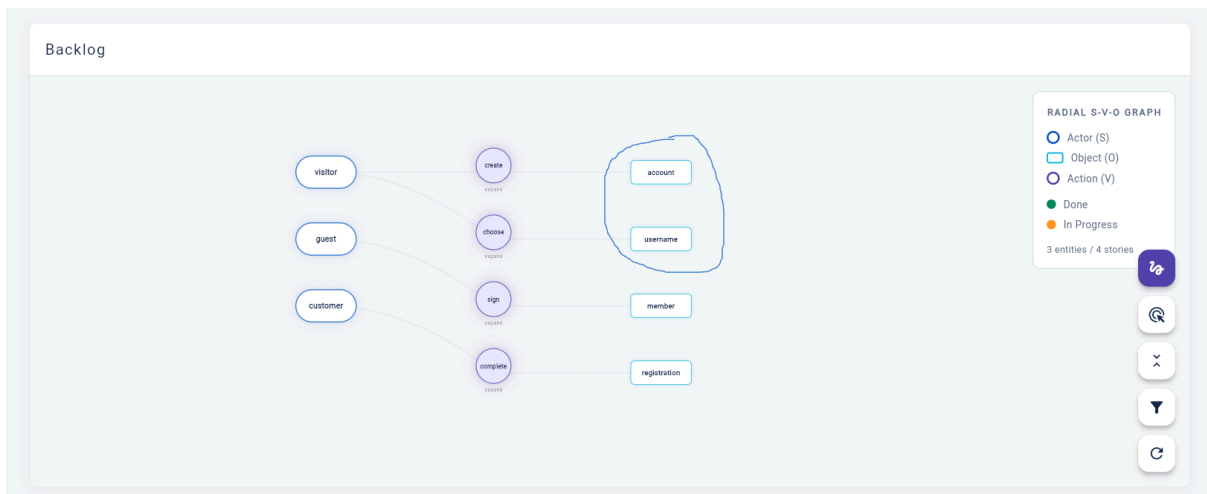
Hình 69: Giao diện Backlog

Trong đồ thị Backlog sẽ có các chức năng(Vẽ, Chọn, Thu/Mở, Làm mới) theo mô tả của Hình 70. Khi ấn vào các nút theo hình sẽ có thể thực hiện chức năng của nút đó.



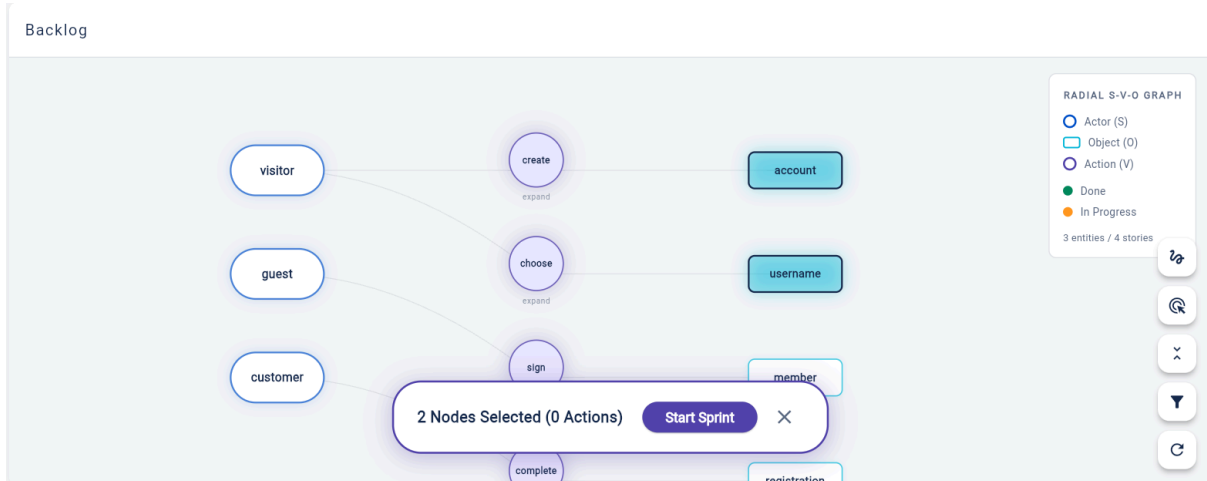
Hình 70: Giao diện Backlog

Dưới đây là chức năng vẽ, khi ấn vào nút có biểu tượng vẽ bạn sẽ có thể thực hiện chức năng khoanh tròn các câu chuyện người dùng Hình 71



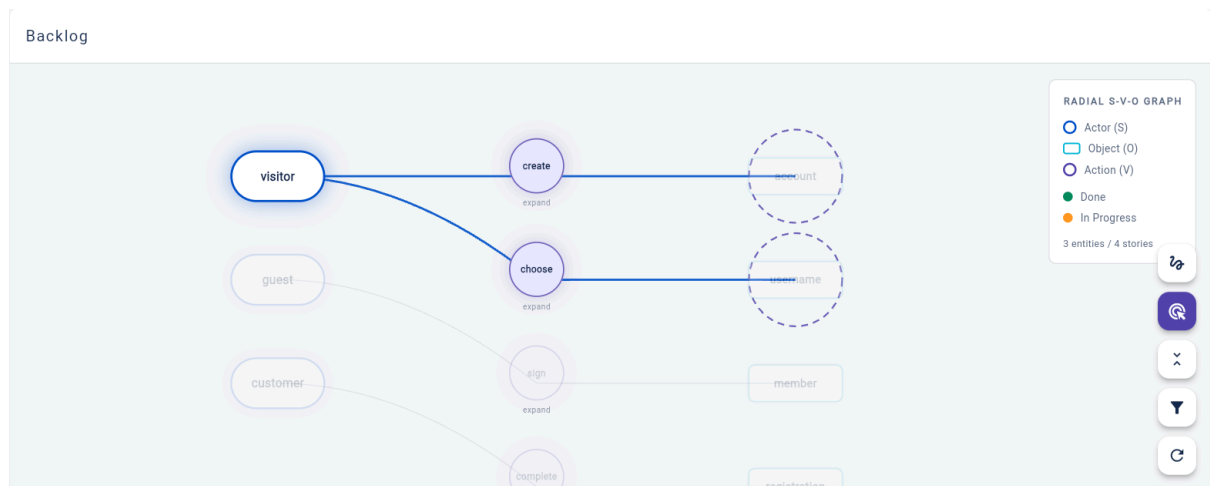
Hình 71: Giao diện Backlog chức năng vẽ

Sau khi khoanh tròn xong sẽ in đậm các câu chuyện người dùng như Hình 72



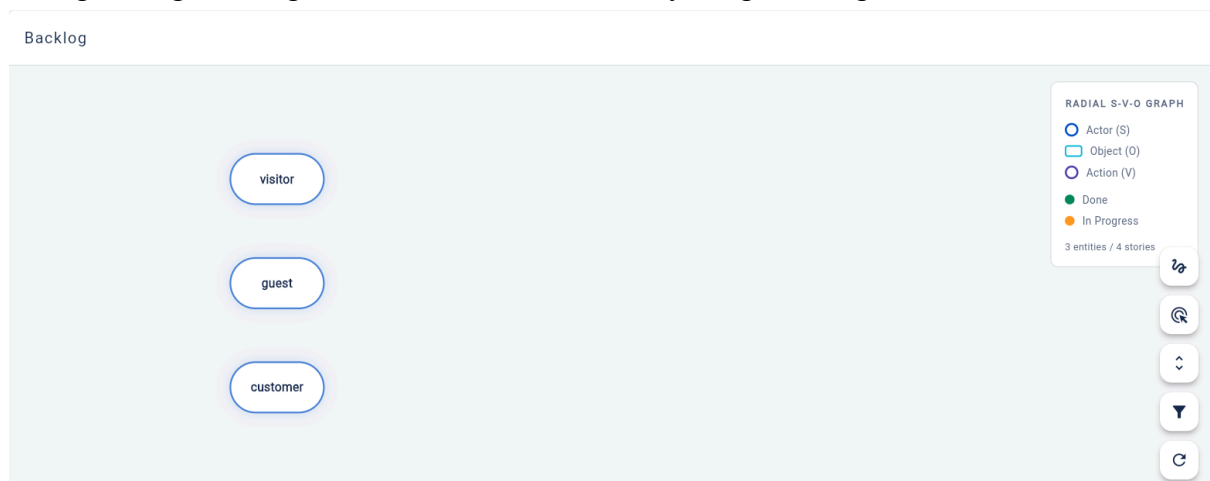
Hình 72: Giao diện Backlog chức năng vẽ

Tiếp theo là chức năng chọn, khi bạn nhấn vào chủ ngữ của một câu chuyện người dùng thì sẽ chọn toàn bộ các câu chuyện người dùng chứa chủ ngữ đó Hình 73



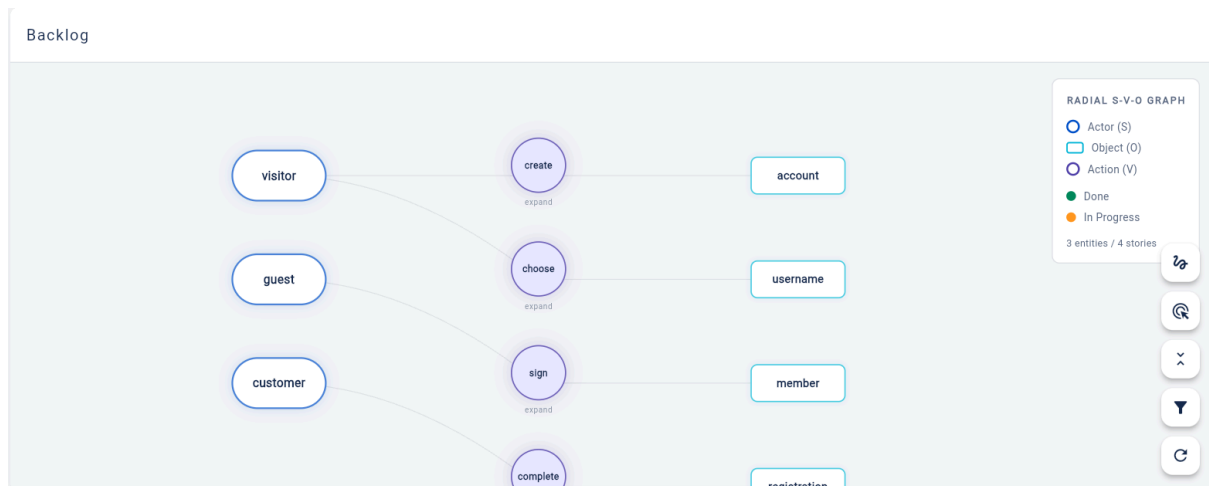
Hình 73: Giao diện Backlog chức năng chọn

Thêm nữa là chức năng thu/mở, khi thu sẽ thu gọn các câu chuyện người dùng và chỉ hiển thị chủ ngữ để người dùng dễ nhìn hơn với các câu chuyện người dùng tránh bị rối mắt Hình 74



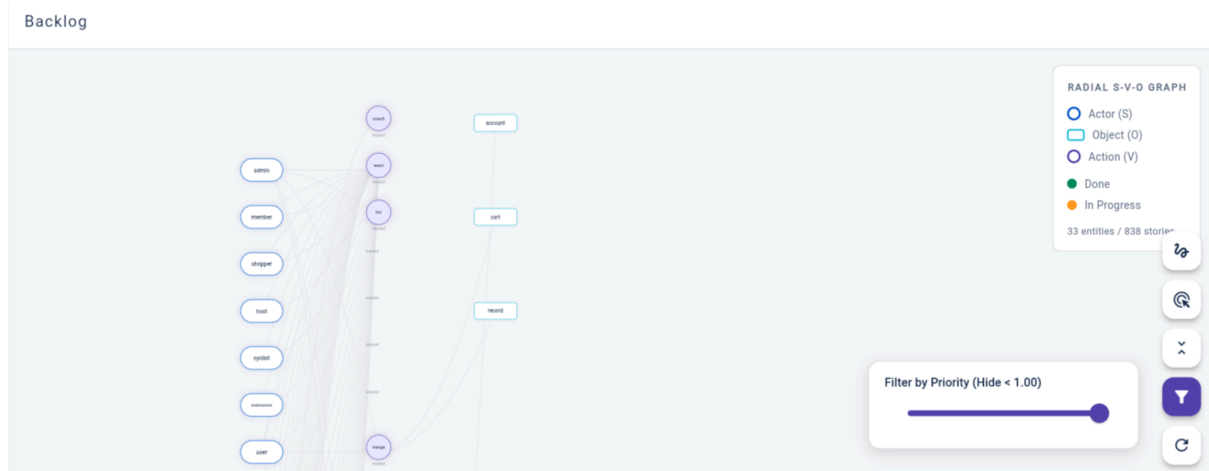
Hình 74: Giao diện Backlog chức năng thu

Khi thực hiện chức năng mở sẽ mở ra toàn bộ các câu chuyện người dùng để người dùng có cái nhìn tổng quan Hình 75



Hình 75: Giao diện Backlog chức năng mở

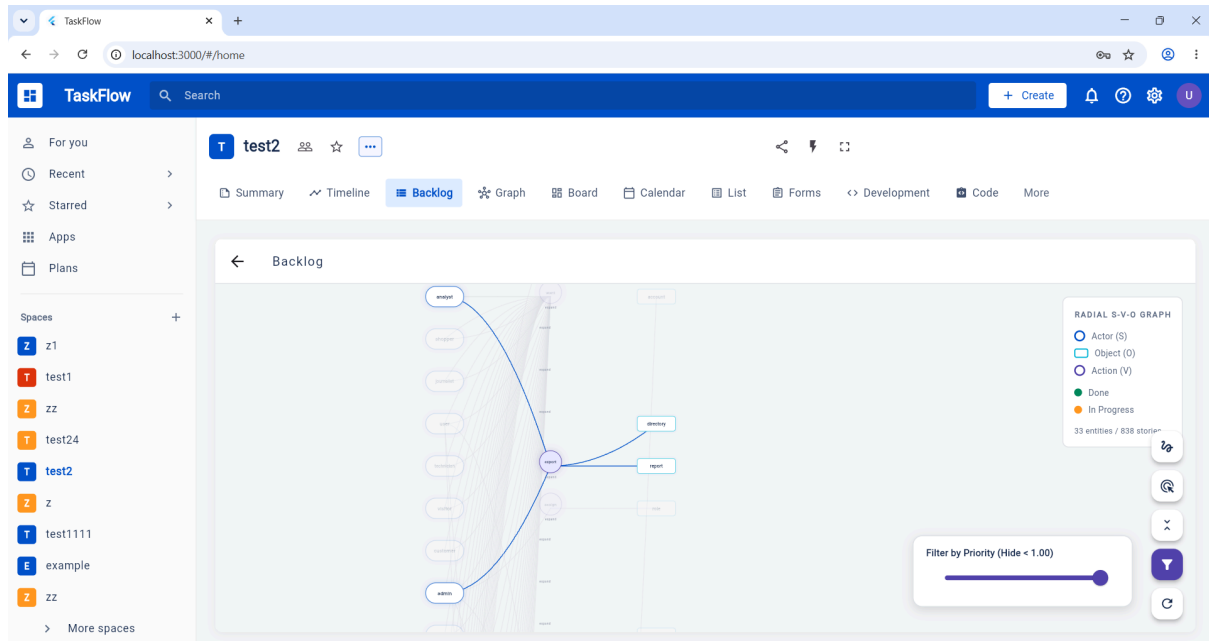
Bên cạnh khả năng mở rộng để quan sát tổng quan, hệ thống Đồ thị (Graph View) còn được tích hợp chức năng Lọc độ ưu tiên dựa trên mức độ quan trọng của các thực thể nghiệp vụ như Hình 76. Tính năng này được thiết kế nhằm giải quyết bài toán quá tải thông tin (Information Overload) khi số lượng User Story trong dự án tăng cao, khiến mạng lưới đồ thị trở nên chằng chịt và khó phân tích.



Hình 76: Giao diện Backlog chức năng lọc

Hình 77 minh họa kết quả trực quan trên đồ thị sau khi người dùng điều chỉnh thanh trượt Filter by Priority lên mức cao nhất (Hide < 1.00). Tại trạng thái này, thuật toán Lọc Ngưỡng (Threshold Culling) đã phát huy tác dụng: hệ thống tự động làm mờ và ẩn đi toàn bộ các đỉnh (Node) và cạnh (Edge) thuộc nhóm User Story có độ ưu tiên thấp hoặc không quan trọng. Đồ thị lúc này đã được khử nhiễu hoàn toàn, chỉ giữ lại độ sáng tối đa cho các luồng nghiệp vụ cốt lõi (Core Business Flows) mang tính sống còn của dự án. Thông qua tính năng này, người dùng (như Product Owner hoặc System Analyst) có thể dễ dàng truy vết đường đi của dữ liệu và tập trung phân tích các chức năng xương sống mà không bị phân tâm bởi mạng

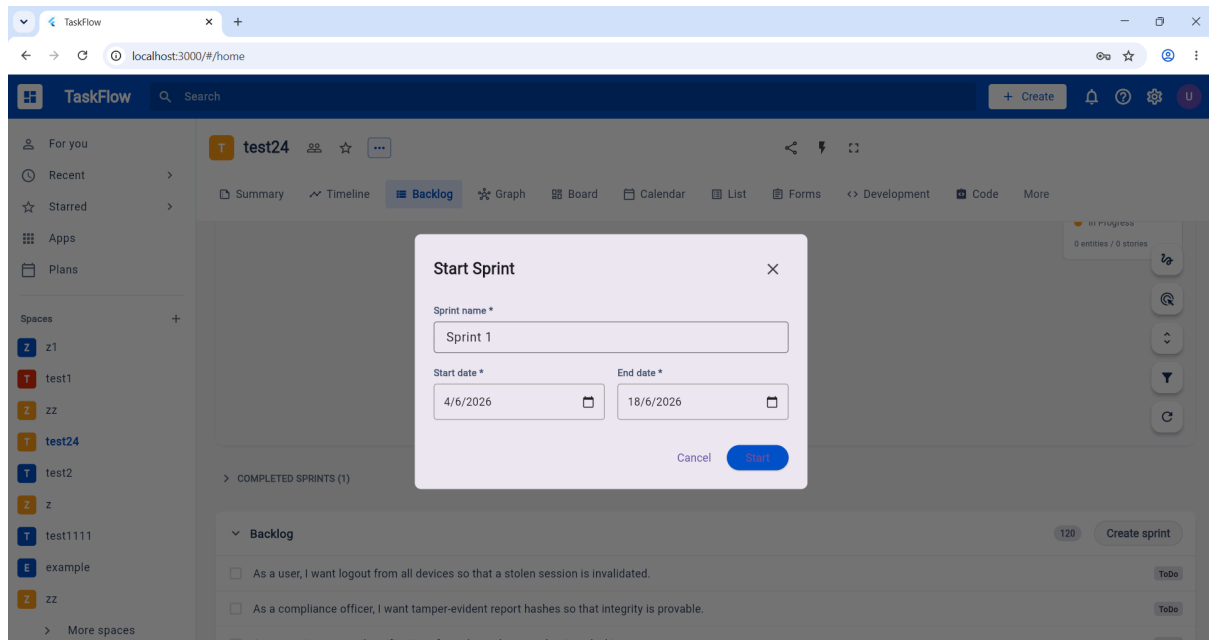
lưới thông tin chẳng chặt ban đầu. (Lọc theo con trỏ chuột được chỉ định đến nếu con trỏ ở S thì sẽ hiển thị kết nối đến V, $V \rightarrow S, O$ và $O \rightarrow V$).



Hình 77: Giao diện Backlog chức năng lọc

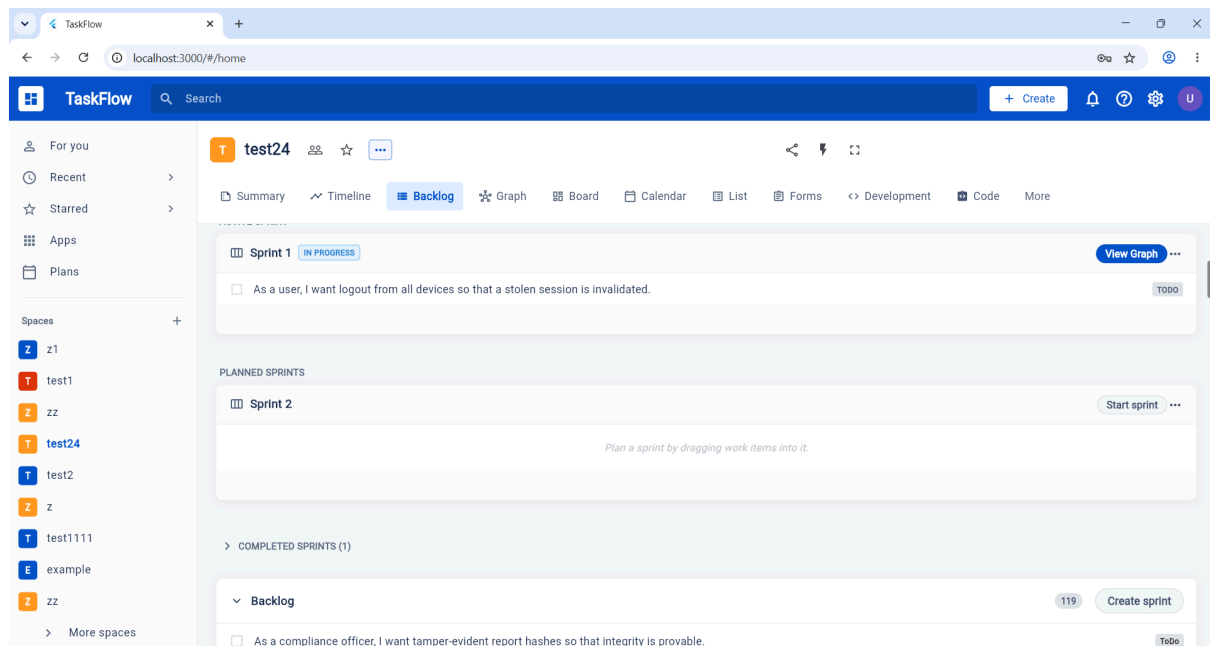
4.4.10. Giao diện Sprint

Giao diện Sprint, Hình 78, cho phép tạo 1 Sprint để đưa các hệ thống nhiệm vụ vào bao gồm các trường: Sprint name, ngày bắt đầu và ngày kết thúc:



Hình 78 Giao diện Sprint

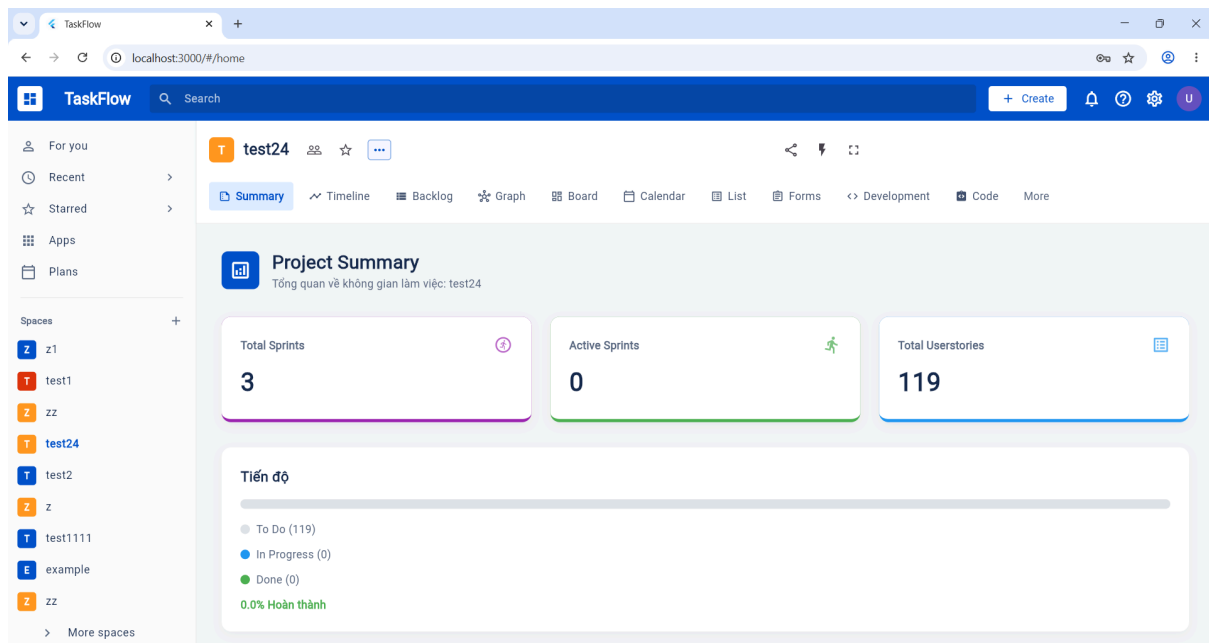
Chia các trạng thái của Sprint theo mô tả như Hình 79



Hình 79: Giao diện Sprint

4.4.11. Giao diện Summary

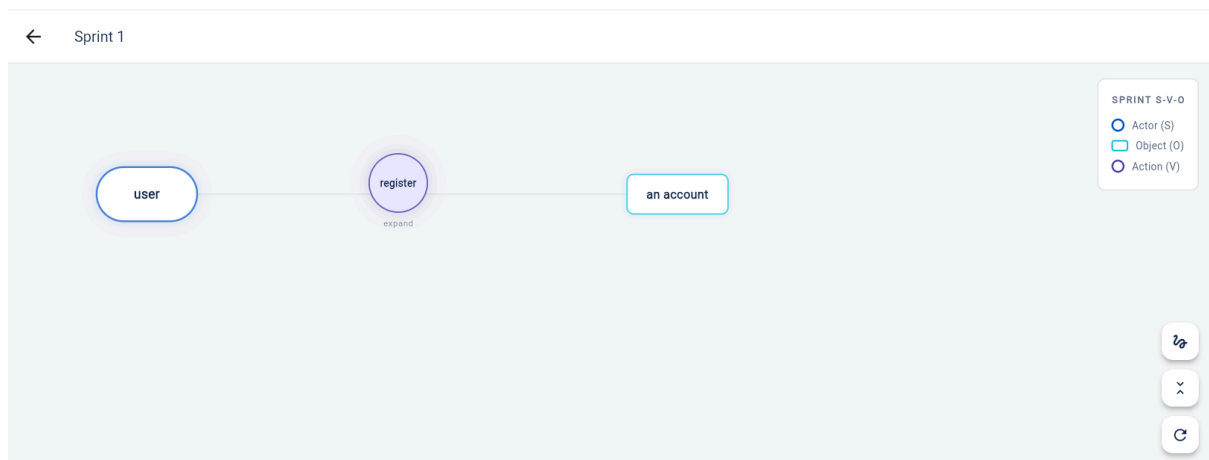
Giao diện Summary (Hình 80) đóng vai trò là màn hình báo cáo trung tâm của mỗi Không gian làm việc (Workspace). Giao diện này cung cấp cho người quản lý dự án (như Scrum Master hoặc Product Owner) một cái nhìn toàn cảnh, trực quan về khối lượng công việc và tiến độ thực thi thực tế mà không cần phải kiểm tra từng thẻ công việc thủ công.



Hình 80: Giao diện Summary

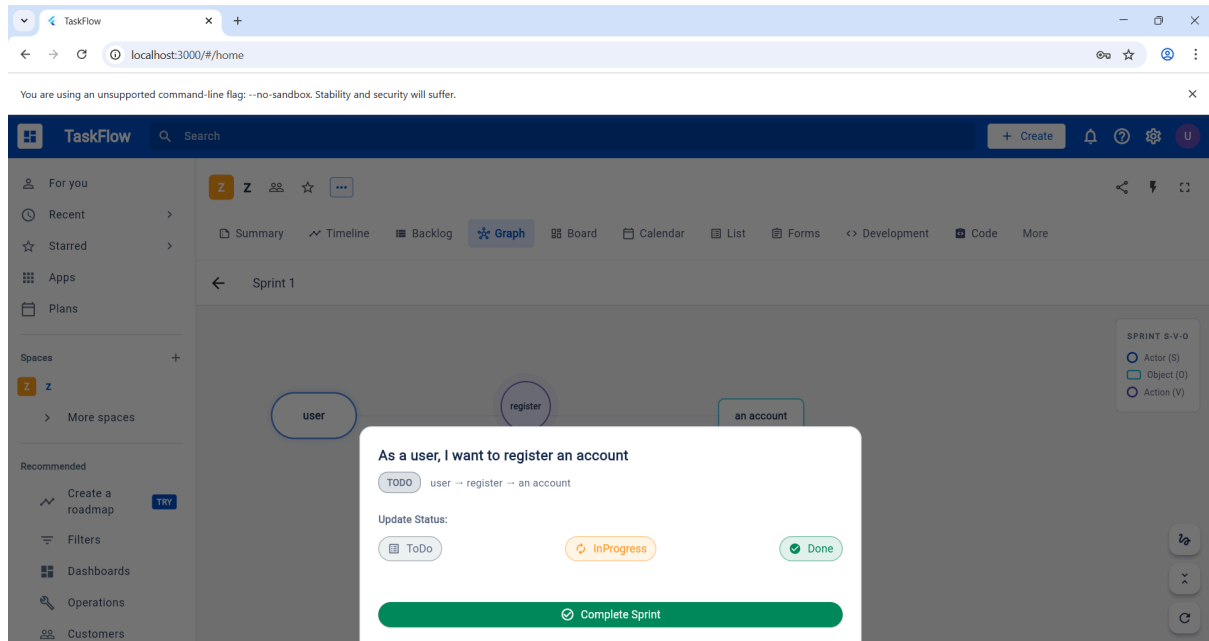
4.4.12. Giao diện Graph Sprint

Khi start Sprint sẽ hiển thị phần Sprint được start(hay các công việc được bắt đầu) ở tab Graph, được miêu tả Hình 81



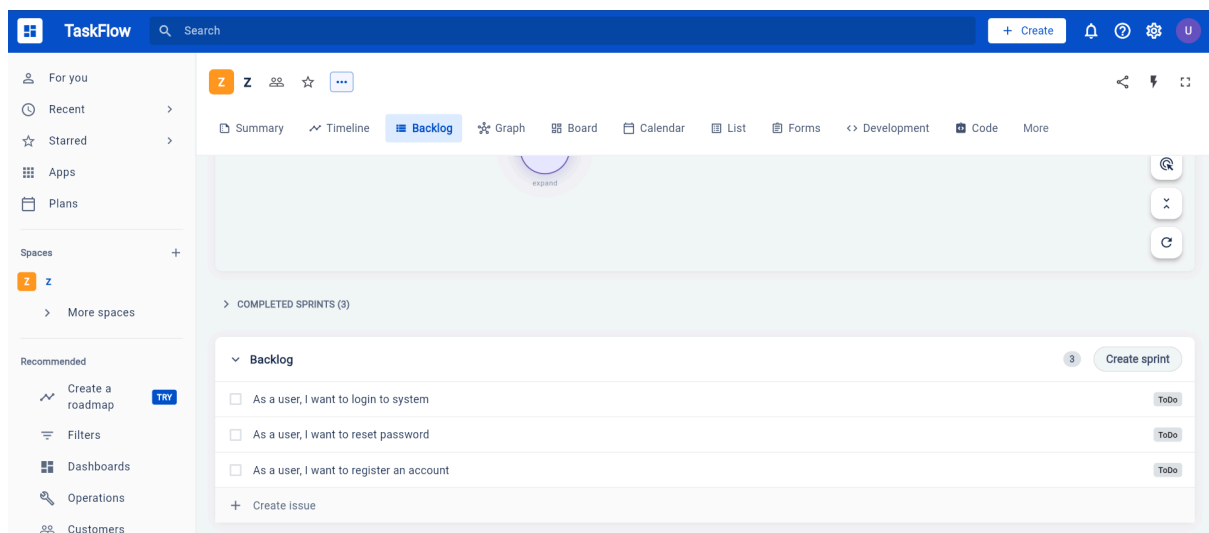
Hình 81: Giao diện Graph Sprint

Ấn vào node sẽ hiển thị lựa chọn “Complete Sprint” hoặc thay đổi trạng thái của user story, như Hình 82.



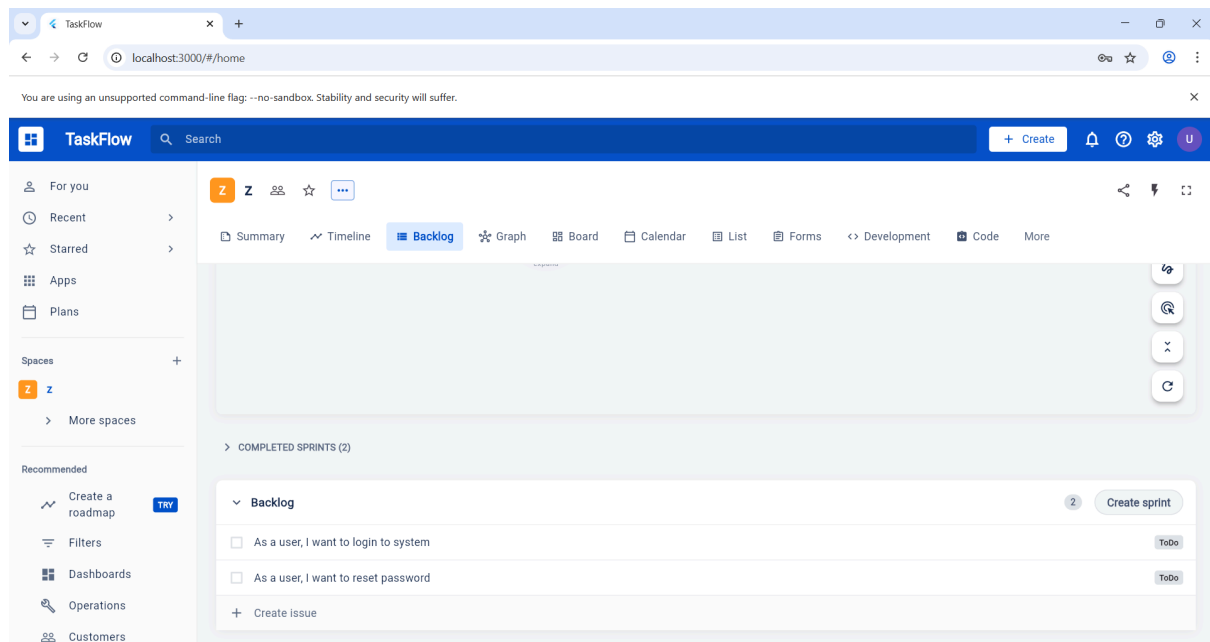
Hình 82: Giao diện Graph Sprint

Nếu user story ở trạng thái “Todo” mà khi “Complete” sẽ chuyển về tab Backlog, theo miêu tả Hình 83



Hình 83: Giao diện Graph Sprint

Nếu user story ở trạng thái “Done” mà khi “Complete” là công việc đã hoàn thành và sẽ biến mất khỏi backlog theo mô tả Hình 84

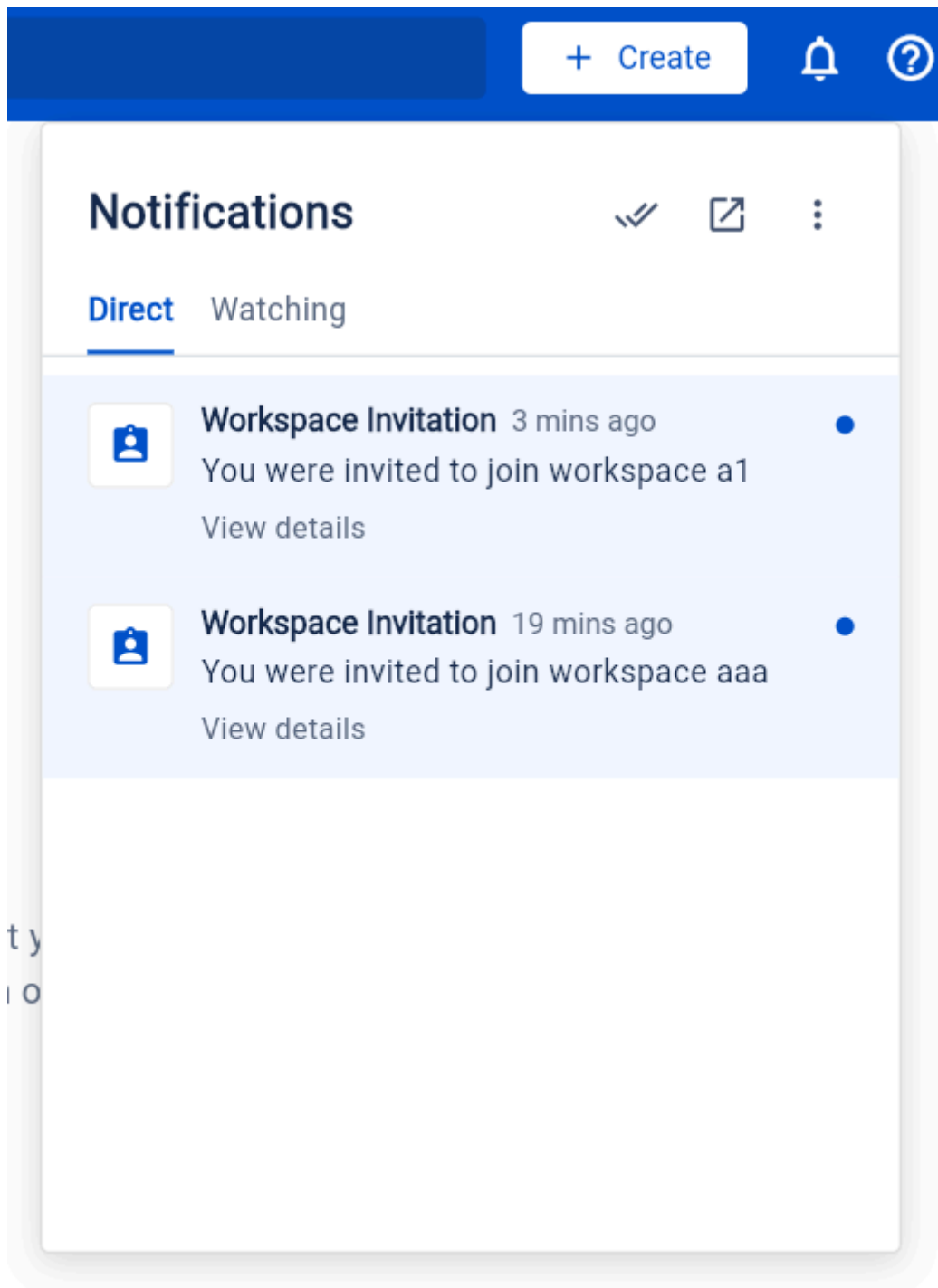


Hình 84 Giao diện Graph Sprint

4.4.13. Giao diện thông báo (Notifications)

Giao diện thông báo, hiển thị danh sách các lời mời ("Workspace Invitation") mà người dùng nhận được để tham gia vào các không gian làm việc. Quản lý trạng thái “Đã đọc”:

- **Đánh dấu từng cái:** Khi bạn bấm vào xem một thông báo cụ thể, hệ thống sẽ tự động hiểu là bạn đã xem và làm biến mất chấm xanh.
- **Đánh dấu tất cả:** Nếu bạn có quá nhiều thông báo và muốn dọn dẹp nhanh, hãy bấm vào **biểu tượng dấu tích kép (✓✓)** ở góc trên cùng bên phải. Thao tác này sẽ ngay lập tức chuyển toàn bộ danh sách thành trạng thái "đã đọc" mà không cần phải bấm vào từng cái một.

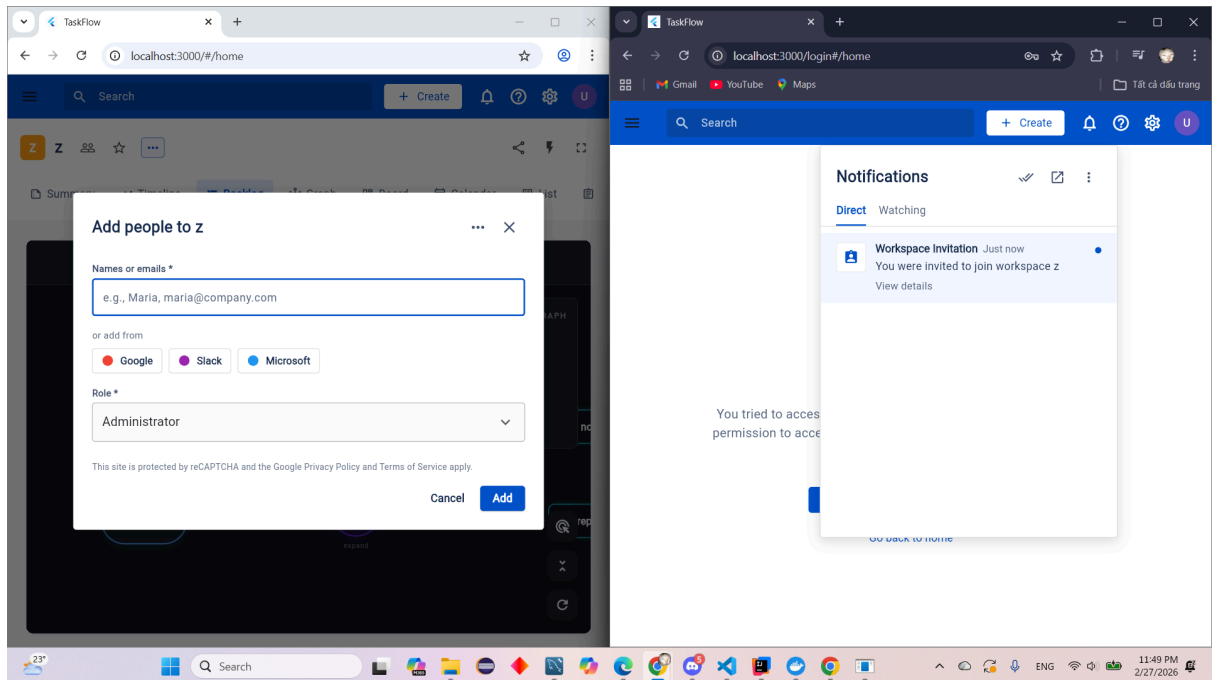


Hình 85: Giao diện thông báo

Giao diện thông báo, thể hiện toàn bộ quy trình thêm một người dùng mới vào dự án theo thời gian thực:

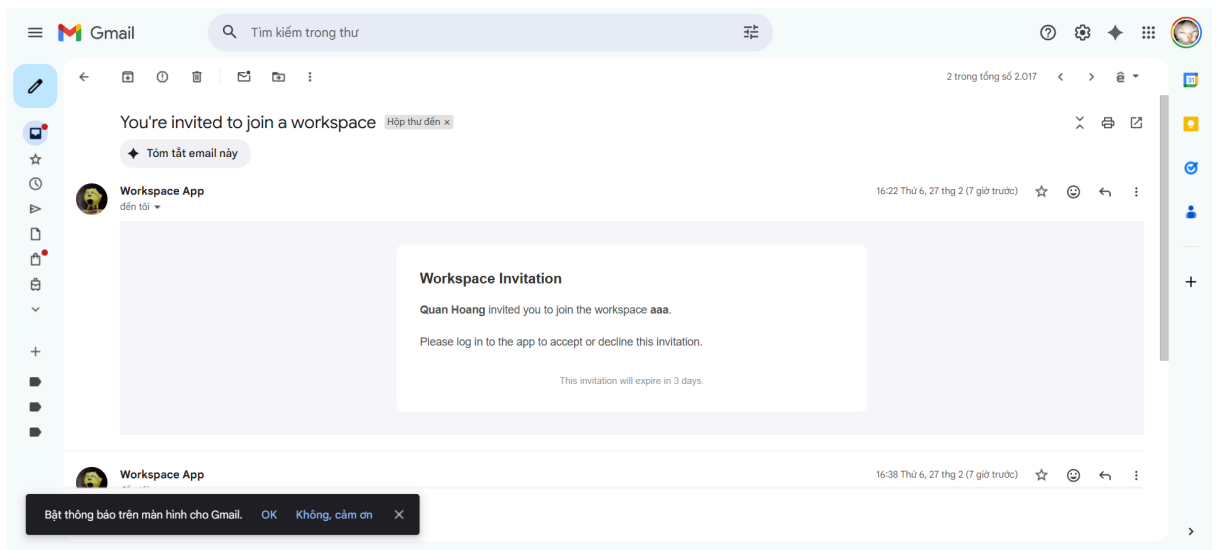
- **Cửa sổ bên trái:** Giao diện của người mời đang mở hộp thoại "Add people to z". Tại đây, họ có thể nhập email, phân quyền cho thành viên mới (ví dụ: cấp quyền Administrator) và nhấn gửi đi.
- **Cửa sổ bên phải:** Giao diện của người được mời. Nút chuông thông báo

(Notifications) hiển thị một lời mời tham gia Workspace vừa được gửi tới ("You were invited to join Workspace z").



Hình 86: Giao diện thông báo

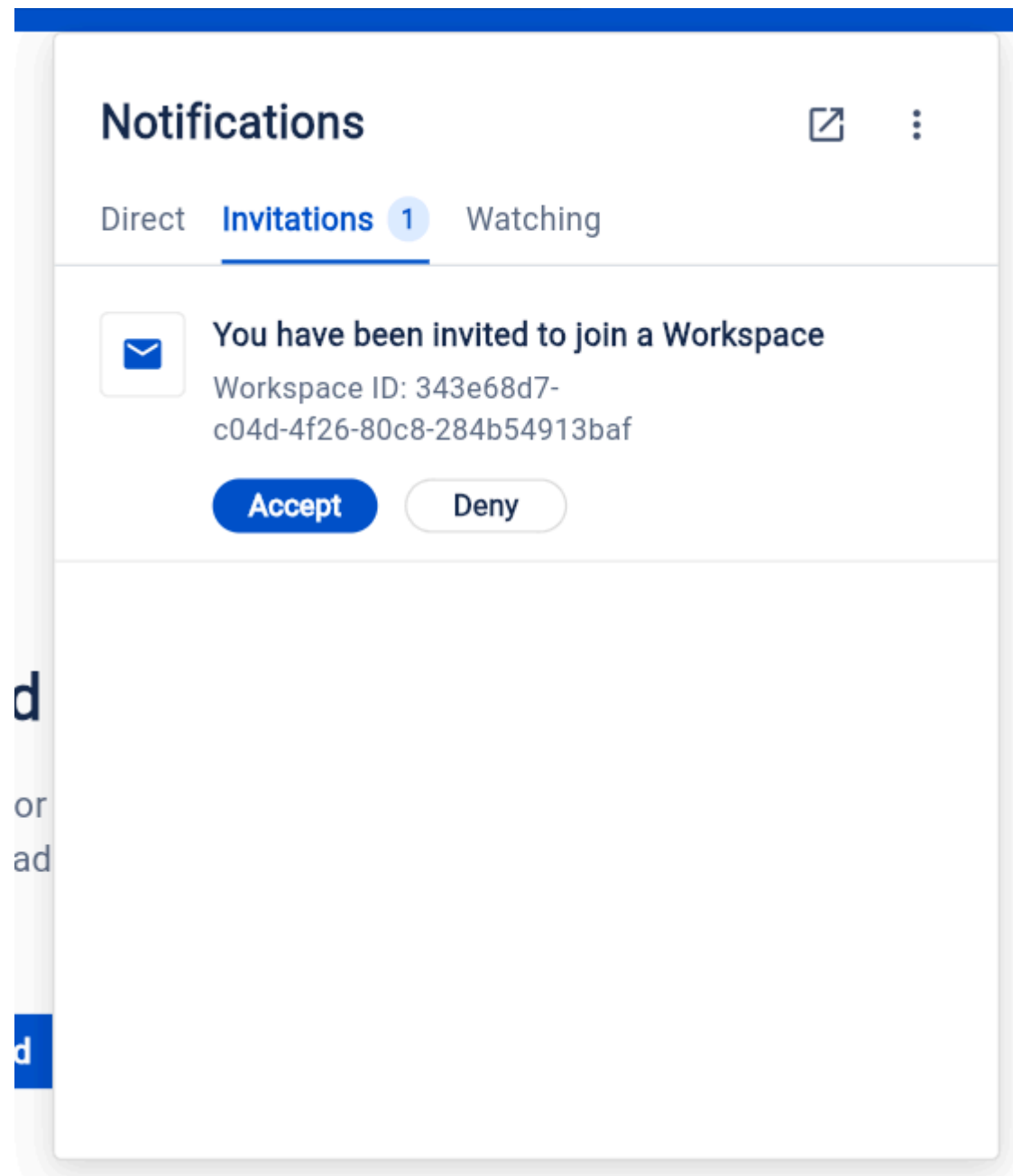
Khi được mời vào 1 Workspace sẽ được gửi thông báo vào email người dùng



Hình 87: Giao diện thông báo

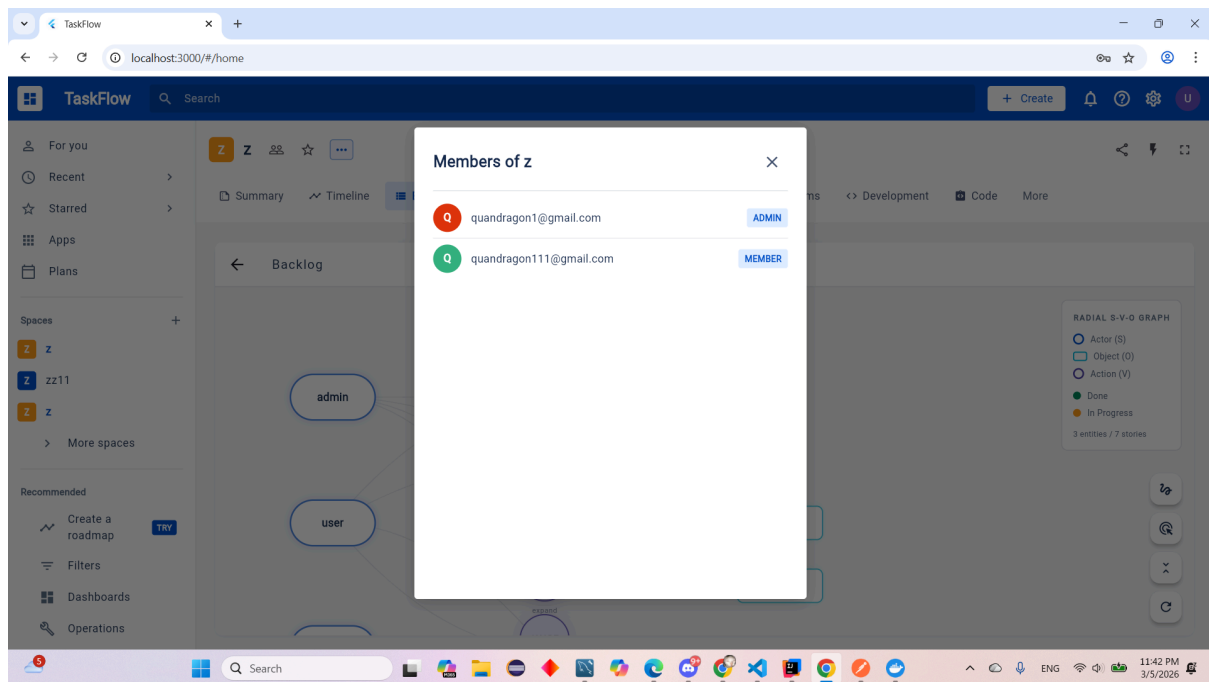
4.4.14. Giao diện mời (Invitations)

Sau khi xem thông báo lời mời ở Mục 4.5.13, có thể “Chấp nhận” hoặc “Từ chối” lời mời



Hình 88: Giao diện lời mời

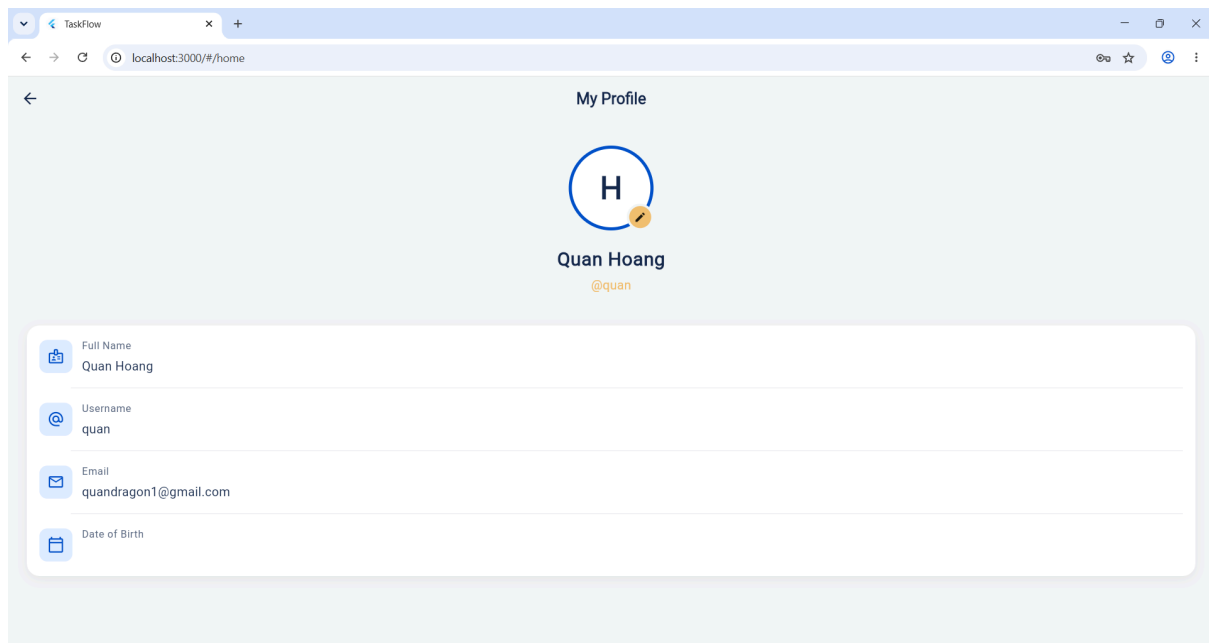
Khi “Chấp nhận” lời mời sẽ hiển thị thành viên ở Workspace đó với Email và Quyền



Hình 89: Giao diện lời mời

4.4.15. Giao diện hồ sơ (Profile)

Giao diện hồ sơ, được miêu tả trong Hình 90. Nó có bố cục gồm Avatar ở trên cùng, tên người dùng, và một khối card (thẻ) xám chứa danh sách các thông tin chi tiết (Full Name, Username, Email, Date of Birth) kèm theo icon cho từng dòng



Hình 90: Giao diện hồ sơ

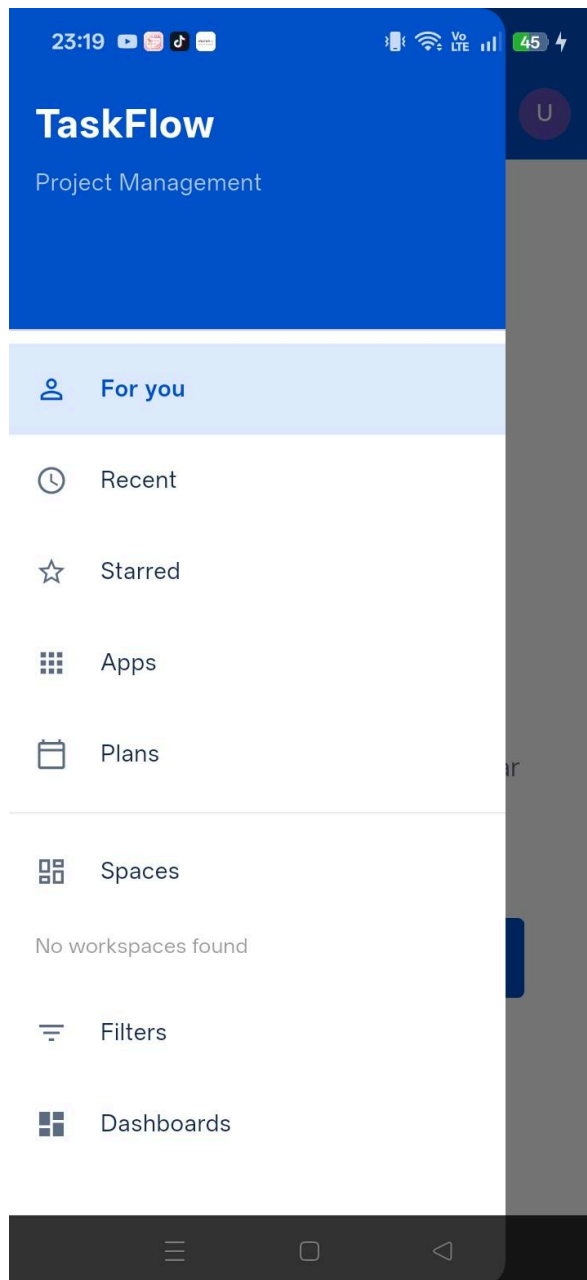
4.5. Thiết kế giao diện người dùng (Giao diện mobile)

Phiên bản ứng dụng di động được xây dựng với đầy đủ các tính năng lỗi hoàn toàn đồng bộ với phiên bản Web (đã được mô tả chi tiết tại mục 4.4). Tuy nhiên, để mang lại trải nghiệm tối ưu nhất trên thiết bị cầm tay, giao diện Mobile không sao chép y hệt Web mà được tái thiết kế (Redesign) để giải quyết bài toán về giới hạn không gian hiển thị và thói quen thao tác của người dùng.

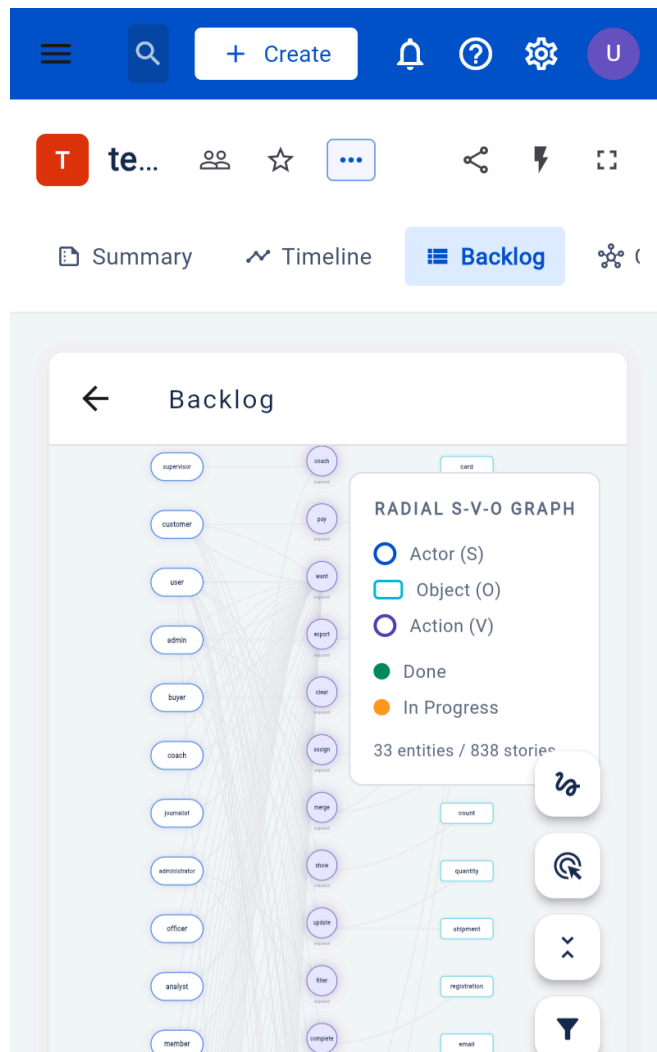
Cụ thể, giao diện Mobile tập trung vào 3 yếu tố thiết yếu sau:

4.5.1. Tối ưu hóa không gian và Điều hướng (Navigation)

Thay vì sử dụng thanh menu bên (Sidebar) chiếm diện tích như trên Web, phiên bản Mobile sử dụng thanh điều hướng dưới cùng (Bottom Navigation Bar) hoặc menu ẩn (Hamburger Menu). Cách bố trí này giúp giải phóng tối đa không gian màn hình cho nội dung chính, đồng thời giúp người dùng dễ dàng thao tác bằng một ngón tay cái.



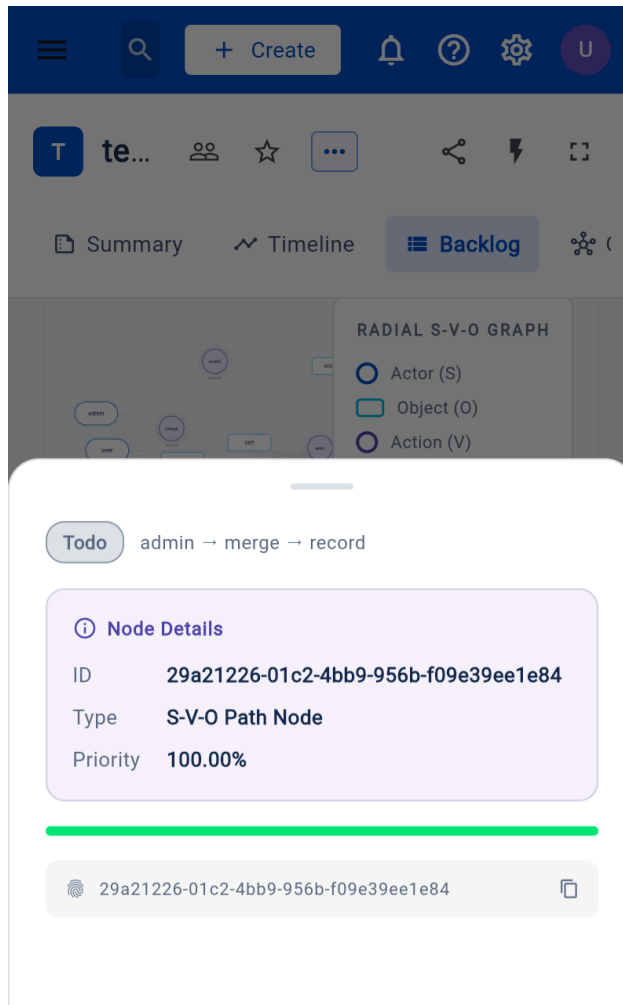
Hình 91: Giao diện bị ẩn



Hình 92: Giao diện backlog

4.5.2. Tái cấu trúc hiển thị dữ liệu (Data Layout)

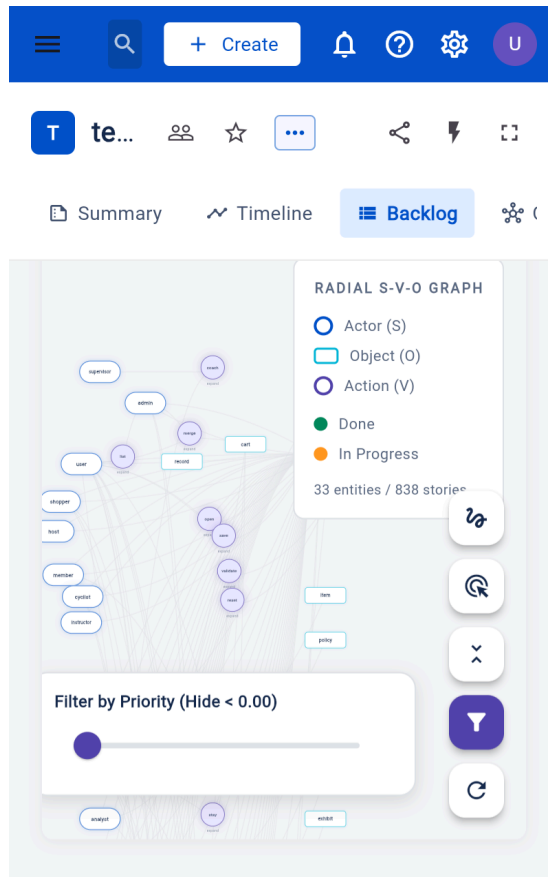
Các bảng dữ liệu lớn (như danh sách Backlog hay Sprint) trên Web được chuyển đổi thành cấu trúc thẻ (Card UI) hoặc danh sách cuộn (List View) trên Mobile. Mỗi thẻ chỉ hiển thị những thông tin quan trọng nhất (như Tên User Story, Mức độ ưu tiên), các thông tin phụ sẽ được giấu đi và chỉ hiện ra khi người dùng bấm vào xem chi tiết.



Hình 93: Giao diện backlog

4.5.3. Tương tác vuốt chạm (*Gesture-based Interaction*)

Tương tác vuốt chạm (*Gesture-based Interaction*) Phiên bản Mobile tận dụng tối đa các thao tác vật lý của thiết bị như kéo thả để cập nhật trạng thái (*Drag & Drop*), vuốt để xóa hoặc chỉnh sửa (*Swipe actions*), và hỗ trợ phóng to/thu nhỏ (*Pinch-to-zoom*) đặc biệt khi xem Đồ thị Backlog (*Graph*).



Hình 94: Giao diện backlog

4.6. Kết luận chương

Trong chương 4, nhóm đã trình bày chi tiết quá trình thiết kế hệ thống dựa trên các yêu cầu đã phân tích ở chương trước. Cụ thể, nhóm đã xây dựng kiến trúc tổng thể cho hệ thống, thiết kế cơ sở dữ liệu bảo đảm tính toàn vẹn và hiệu quả truy xuất, mô hình hóa các thành phần phần mềm thông qua các biểu đồ UML, và thiết kế giao diện người dùng trực quan, thân thiện.

Chương 5: Cài đặt và kết quả thử nghiệm

5.1. Môi trường triển khai

Để đảm bảo hệ thống mạng xã hội hoạt động ổn định, nhóm tiến hành triển khai và thử nghiệm trong môi trường có cấu hình như sau được miêu tả chi tiết trong **5.1.1.**, **5.1.2.**, và **5.1.3.**

5.1.1. Hệ điều hành và phần cứng

Hệ điều hành máy phát triển (Development): Windows 10 / 11, Ubuntu 22.04 (dành cho môi trường server).

CPU: Intel Core i5 / i7 hoặc tương đương.

RAM: Tối thiểu 8GB (khuyến nghị 16 GB để chạy đồng thời Docker, Java và Flutter).

Ổ cứng: SSD 256 GB trở lên.

Kết nối mạng: Internet ổn định để đồng bộ mã nguồn với GitHub.

5.1.2. Phần mềm và công cụ hỗ trợ

Visual Studio Code: IDE để viết mã frontend và gỡ lỗi.

IntelliJ IDEA 2024.3.3: IDE để viết mã backend.

Java 21: Môi trường chạy ứng dụng backend.

MySQL 8.0: Quản lý cơ sở dữ liệu quan hệ.

Neo4j Desktop: Quản lý cơ sở dữ liệu đồ thị.

Docker Desktop: Dùng để chạy Keycloak.

Git & GitHub: Quản lý mã nguồn.

Draw.io: Vẽ các biểu đồ UML (Use Case, Class, Sequence, Component, Package).

5.1.3. Kết quả mong đợi

1. Hệ thống có thể vận hành ổn định trong môi trường local thông qua Docker và cấu hình độc lập giữa các thành phần hạ tầng: **MySQL** (master–slave replication), **RabbitMQ** (message broker kèm giao diện quản trị), **Neo4j**, **Keycloak** (xác thực) và các application server (**Spring Boot** cho nghiệp vụ, **FastAPI** cho phân tích). Các tham số kết nối được tách qua biến môi trường (**NEO4J_URI**, **KEYCLOAK_***, **datasource master/slave**), giúp triển khai và thay đổi cấu hình không phụ thuộc cứng vào một máy chủ duy nhất.
2. Thời gian phản hồi trung bình đối với các thao tác đồng bộ chính trên luồng người dùng — **Tạo Workspace**, **Tạo Sprint**, **Tạo / Cập nhật User Story** — được duy trì ở mức thấp trong điều kiện tải thử nghiệm, vì các thao tác này chỉ ghi dữ liệu nghiệp vụ vào MySQL và trả kết quả ngay sau khi transaction commit; phần phân tích nặng được tách ra khỏi request–response.
3. Các tác vụ phân tích User Story và sinh/cập nhật đồ thị quan hệ được xử lý bất đồng bộ, hướng sự kiện thông qua **RabbitMQ** (**userstory.exchange**, routing key **userstory.created**, **userstory.moved**, **graph.rebuild**), kết hợp cơ chế **TransactionalEvents.afterCommit** trên tầng Application Spring — chỉ phát message sau khi giao dịch MySQL thành công. Cơ chế này giúp:

- a. Không làm chậm luồng CRUD chính của người dùng khi tạo hoặc di chuyển User Story.
 - b. Đảm bảo tách biệt rõ giữa xử lý nghiệp vụ (Java BE) và xử lý phân tích (Python analyze_user_stories).
 - c. Consumer Python xử lý idempotent (kiểm tra is_processed), cập nhật ngữ cảnh khi story chuyển sprint/backlog, và chạy batch rebuild toàn workspace khi được kích hoạt.
 - d. Duy trì tính ổn định khi khối lượng User Story tăng: phân tích realtime (parse SVO → Neo4j) tách khỏi batch (chuẩn hóa semantic, luật kết hợp, ưu tiên, phân loại trùng lặp).
4. Việc tách riêng **dữ liệu quan hệ (MySQL)** — lưu Workspace, Sprint, User Story, kết quả phân tích (AnalyzeStoryResult, KnowledgeTerm, KnowledgeRelation, AnalyzeStatistic) — và **dữ liệu đồ thị (Neo4j)** — node Term, UserStory, cạnh PERFORM, TARGET, SIMILAR, ASSOCIATED, REDUNDANT_WITH — giúp tối ưu truy vấn theo mục đích: CRUD và báo cáo trên SQL; trực quan hóa và suy luận quan hệ trên đồ thị (GraphQL đọc Neo4j). Kiến trúc master-slave MySQL hỗ trợ service phân tích đọc dữ liệu nghiệp vụ từ slave mà không ảnh hưởng tải ghi trên master.

5.2. Đánh giá hệ thống (Kết luận và hướng phát triển)

5.2.1. Kết quả đạt được

Sau quá trình nghiên cứu và phát triển, nhóm đã hoàn thành hệ thống quản lý và phân tích User Story với các chức năng cốt lõi sau.

- **Kiến trúc backend và tách tầng dịch vụ:**
 - Xây dựng backend nghiệp vụ bằng Java Spring Boot theo Domain-Driven Design (DDD) với các module nckh-domain, nckh-application, nckh-infrastructure, nckh-controller, nckh-start, tách bạch Domain, Application và Infrastructure.
 - Triển khai phân tích analyze_user_stories (FastAPI) độc lập, giao tiếp với BE qua RabbitMQ, không nhúng logic NLP/ML vào luồng đồng bộ của API nghiệp vụ.
- **Quản lý User Story theo mô hình Agile:**
 - CRUD và vòng đời cho Workspace, Sprint, User Story, kèm Backlog (quan hệ 1–1 với Workspace), Profile, WorkspaceMember, WorkspaceRole.
 - Luồng sprint: gán story vào sprint, cập nhật trạng thái trong sprint đang active, đưa story chưa hoàn thành về backlog khi complete sprint; mỗi sự kiện di chuyển phát USER_STORY_MOVED để đồng bộ ngữ cảnh phân tích.
 - Tích hợp Keycloak (OAuth2/JWT) cho xác thực; đồng bộ Profile cục bộ phục vụ membership và thông báo.
- **Cơ sở dữ liệu và đồ thị:**

- MySQL làm store quan hệ chính: nghiệp vụ Agile, lời mời workspace, thông báo in-app, API key; đồng thời lưu kho tri thức phân tích (SVO, term, relation, statistic).
- Neo4j lưu đồ thị tri thức: chuỗi Subject → Action → Object (PERFORM, TARGET), tương đồng term (SIMILAR), luật kết hợp (ASSOCIATED), trùng lặp giữa các story (REDUNDANT_WITH), metadata ưu tiên trên node UserStory.
- GraphQL (GraphServiceImpl) phục vụ truy vấn đồ thị theo workspaceId, sprintId, backlogId cho giao diện trực quan hóa.
- **Pipeline phân tích User Story (đã triển khai, không chỉ rule-based):**
 - Trích xuất SVO bằng spaCy (en_core_web_sm) [18]: phân tích dependency, tách mệnh đề phụ (advcl), xác định subject-verb-object [18].
 - Chuẩn hóa ngữ nghĩa bằng kết hợp WordNet và Word2Vec (Google News 300) qua chiến lược fusion phi tuyến [7], [6]; auto-merge term khi độ tương đồng $\geq$ ngưỡng cấu hình.
 - Luật kết hợp (Association Rules) với Apriori (mlxtend) [5] trên tập transaction SVO theo workspace/sprint.
 - Phân loại trùng lặp bằng pipeline ML (Logistic Regression, Random Forest, HistGradientBoosting) [19] trên đặc trưng tương đồng term, luật kết hợp và khoảng cách ưu tiên; xuất nhóm trùng lặp và cạnh REDUNDANT_WITH lên Neo4j.
 - Tính ưu tiên story (PriorityService) tích hợp vào batch rebuild.
- **Cơ chế xử lý bất đồng bộ:**
 - Realtime: USER_STORY_CREATED → consumer parse, lưu AnalyzeStoryResult, ghi Neo4j với source=REALTIME.
 - Batch: REBUILD_GRAPH → GraphBatchService rebuild toàn bộ đồ thị workspace (clear → SVO → similarity → rules → priority → redundancy).
 - JobRunr trên BE cho tác vụ nền nhẹ (gửi email mời tham gia workspace); Socket.IO (SocketService) đẩy thông báo realtime tới client đang online.
- **Kết quả tổng hợp:**
 - Hoàn thiện quy trình quản lý User Story theo mô hình Workspace – Sprint – Product Backlog.
 - Tự động hóa phân tích cấu trúc câu, chuẩn hóa thuật ngữ, sinh luật kết hợp, ưu tiên và phát hiện trùng lặp; biểu diễn kết quả dưới dạng đồ thị tương tác.
 - Thiết lập nền tảng kiến trúc event-driven, hybrid store, dễ mở rộng thêm mô hình NLP/ML mà không phá vỡ luồng nghiệp vụ đồng bộ.

5.2.2. Hạn chế

Mặc dù hệ thống đã đáp ứng các mục tiêu đề ra nhưng vẫn còn một số hạn chế cần cải thiện.

1. Hạn chế về hiệu năng, triển khai và khả năng mở rộng

- Hệ thống chủ yếu được triển khai và kiểm thử trong môi trường local / thử nghiệm; Docker compose hiện có cho MySQL replication và RabbitMQ, chưa

thể hiện bộ triển khai production đồng nhất (Neo4j, Keycloak, BE, Python, frontend trong một stack orchestrated).

- Chưa triển khai đầy đủ trên production các cơ chế Auto Scaling, Load Balancing, Kubernetes và tách cluster độc lập theo SLA.
- Chưa có benchmark chính thức (p95/p99 latency, throughput) được ghi nhận trong mã nguồn cho các API Tạo Workspace / Sprint / User Story dưới tải cao; đánh giá hiệu năng mới dừng ở quan sát thử nghiệm.
- Batch rebuild đòi hỏi tối thiểu 3 SVO hợp lệ trong workspace; workspace mới hoặc ít dữ liệu chưa có đồ thị phân tích đầy đủ.

2. Hạn chế trong phân tích User Story

- Parser phụ thuộc mô hình spaCy tiếng Anh (en_core_web_sm); user story tiếng Việt hoặc cấu trúc câu phức tạp ngoài mẫu dependency có thể trả về ERROR hoặc SVO không đầy đủ.
- Chuẩn hóa semantic dựa trên WordNet/Word2Vec tiếng Anh; thuật ngữ domain đặc thù hoặc từ viết tắt nội bộ có thể merge sai nếu không có từ điển riêng.
- Phân loại trùng lặp dùng weak labeling [10] và baseline ML trên tập cặp story tự sinh; chưa có bộ dữ liệu gán nhãn chuyên gia lớn để đánh giá F1/recall [19] ổn định trên production.
- Luật kết hợp Apriori nhạy ngưỡng support/confidence cố định [5]; workspace nhỏ có thể sinh ít hoặc nhiều rule nhiễu.

3. Hạn chế về giám sát và logging hệ thống

- Logging chủ yếu ở mức ứng dụng (log.info, print trong consumer Python); chưa có centralized logging, metrics (Prometheus/Grafana), distributed tracing hay dashboard Cloud Monitoring/Application Insights.
- RabbitMQ và Neo4j chưa có cảnh báo tự động khi queue tồn đọng, consumer lỗi liên tiếp hoặc rebuild batch thất bại.
- Khó truy vết end-to-end một User Story từ API tạo → message → parse → Neo4j chỉ bằng log rời rạc.

4. Hạn chế về trải nghiệm người dùng

Mặc dù giao diện đã đáp ứng được các chức năng chính, nhưng vẫn còn một số điểm cần cải thiện:

- Giao diện Flutter (frontend-mvvm) đã có responsive/mobile drawer và hỗ trợ dark mode theo theme hệ thống ở một số màn hình, nhưng chưa tối ưu hoàn toàn mobile-first trên toàn bộ luồng backlog/sprint/graph.
- Thông báo: có API Notification in-app và Socket.IO realtime khi client online; chưa có push notification native (FCM/APNs) khi người dùng offline.
- Đồ thị phụ thuộc dữ liệu Neo4j đã rebuild; người dùng có thể thấy đồ thị trống hoặc lệch nếu chưa kích hoạt rebuild hoặc batch chưa chạy xong.

- Mức độ cộng tác trong Workspace (mời thành viên, thông báo) đã có nền tảng nhưng chưa phong phú như các công cụ Agile thương mại (comment, mention, activity feed).

5.2.3. Hướng phát triển tiếp theo

Trong tương lai, nhóm dự kiến mở rộng và hoàn thiện hệ thống theo các hướng sau, bám sát kiến trúc đã có: Spring Boot (DDD), FastAPI analyze_user_stories, Flutter, MySQL, Neo4j, RabbitMQ và Keycloak.

5.2.3.1. Triển khai hệ thống trong môi trường thực tế (Production Environment)

Thành phần	Vai trò	Dịch vụ AWS
Frontend (Flutter)	Web/mobile client	S3 + CloudFront (static build) hoặc EC2/ECS
Backend nghiệp vụ (Spring Boot)	CRUD Agile, auth, GraphQL đọc graph	ECS/EKS hoặc EC2
NLP/ML service (Python analyze_user_stories)	Consumer RabbitMQ, batch rebuild	ECS (scale độc lập theo độ sâu queue)
MySQL	Nghiệp vụ + kho phân tích (AnalyzeStoryResult, KnowledgeTerm, ...)	RDS Multi-AZ, backup tự động
MySQL read replica	Python đọc user_story qua slave (đã có RoutingDataSource / get_slave_db)	RDS read replica
Neo4j	Đồ thị Term, PERFORM/TARGET, SIMILAR, ASSOCIATED, REDUNDANT_WITH	Neo4j Aura hoặc cluster self-managed trên EC2
Message broker	userstory.exchange (created / moved / rebuild)	Amazon MQ (RabbitMQ-compatible) hoặc RabbitMQ trên EC2
Identity	OAuth2/JWT	Keycloak trên EC2 hoặc managed IdP tương thích OIDC

File đính kèm	Avatar, tài liệu workspace	S3
---------------	----------------------------	----

Dịch vụ sử dụng:

- **Elastic Load Balancer (ELB):** phân phối tải tới các instance backend và (nếu cần) API gateway; health check tách riêng cho BE và Python service.
- **Auto Scaling:** scale theo CPU/latency cho BE; scale consumer Python theo độ dài queue userstory.created.queue / graph.rebuild.queue — phù hợp vì phân tích đã tách khỏi luồng đồng bộ.
- **IAM & CloudWatch:** giám sát RDS, queue depth, thời gian batch rebuild Neo4j, lỗi consumer; cảnh báo khi rebuild hoặc realtime ghi đồ thị thất bại liên tiếp.

5.2.3.2. Hoàn thiện quy trình CI/CD tự động

Để nâng cao chất lượng và tần suất release an toàn, hệ thống nên tích hợp GitHub Actions (hoặc tương đương) với pipeline đa artifact:

Khi merge vào nhánh main (hoặc release):

1. Java: kiểm tra biên dịch, unit/integration test, build JAR, đóng gói Docker image Spring Boot.
2. Python (analyze_user_stories): lint, test parser/normalization (nếu có), build image (kèm model spaCy + file Word2Vec .kv).
 - Backend: Spring Boot Docker Image
 - Frontend: Flutter Docker Image
3. Flutter (frontend-mvvm): analyze, test widget, build web/APK theo mục tiêu triển khai.

Registry: đẩy image lên Amazon ECR (tách repository).

Triển khai:

- BE và Python deploy độc lập (rolling update trên ECS/EKS); Python có thể release mà không restart BE khi chỉ đổi thuật toán phân tích.
- Cấu hình secret qua AWS Secrets Manager / Parameter Store: NEO4J_*, KEYCLOAK_*, datasource master/slave, RabbitMQ.
- Migration DB (Flyway/Liquibase nếu bổ sung) chạy trước khi bật traffic.

Mục tiêu: giảm downtime, đảm bảo mỗi release vẫn tương thích contract message RabbitMQ (USER_STORY_CREATED, USER_STORY_MOVED, REBUILD_GRAPH).

5.2.3.3. Nâng cấp pipeline phân tích ngôn ngữ và đồ thị tri thức

1. Những gì hệ thống đã đạt được:

- Python (analyze_user_stories) tách khỏi Spring Boot; giao tiếp bất đồng bộ qua RabbitMQ, không chặn API tạo/cập nhật User Story.
- Trích xuất SVO bằng spaCy (en_core_web_sm), xử lý mệnh đề phụ (advcl).
- Chuẩn hóa ngữ nghĩa: fusion WordNet + Word2Vec (Google News 300, Calculate_nonlinear_fusion); Association Rules (Apriori, mlxtend); phân loại trùng lặp (Logistic Regression, Random Forest, HistGradientBoosting).

- Lưu trữ: MySQL (analyze_story_results, knowledge_terms, ...) và Neo4j (Term, cạnh PERFORM/TARGET/SIMILAR/ASSOCIATED/REDUNDANT_WITH).
- Realtime vs batch: created → parse + Neo4j REALTIME; REBUILD_GRAPH → GraphBatchService rebuild toàn workspace.

2. Hướng phát triển: User Story ngoài ngữ cảnh thống nhất (domain / corpus):

Hiện tại, độ tương đồng lexical phụ thuộc Word2Vec pretrained tổng quát (word2vec-google-news-300). Trong Calc_w2v_similarity, nếu từ không có trong vocabulary thì similarity trả về 0.0 — các thuật ngữ domain (tên sản phẩm nội bộ, viết tắt, tiếng Việt latin) dễ bị bỏ qua, chỉ còn WordNet (cũng yếu với từ ngoài WordNet).

Đồng thời, SemanticNormalizationService xây action_context chỉ từ tập SVO trong cùng lần xử lý (batch workspace hoặc cụm story đang normalize): so khớp action dựa trên overlap object trong context đó. Các User Story không cùng domain (ví dụ nhóm J trong bộ synthetic — billing vs museum AR) vẫn có thể bị gộp term sai nếu similarity lexical cao tình cờ, hoặc không gộp được các cặp đồng nghĩa thật khi chúng nằm rải rác nhiều context.

Hướng cải thiện:

- Corpus Word2Vec theo workspace / domain: fine-tune hoặc train FastText/Word2Vec trên corpus User Story + KnowledgeTerm của từng workspace; lưu model riêng (S3) thay vì chỉ dùng Google News 300.
- Embedding cấp câu (sentence-level): Sentence-BERT / domain-adapted encoder cho cả user story, giảm phụ thuộc từng token OOV.
- Ngữ cảnh có cấu trúc: mở rộng action_context sang đồ thị domain (role, product line, sprint theme) hoặc tag metadata do người dùng gán trên Workspace.
- Chính sách merge có điều kiện: chỉ auto-merge khi cùng workspace + cùng nhóm domain (clustering trước normalize); tách pipeline cho story “độc lập” (ít overlap) như nhóm kiểm thử J trong tài liệu synthetic.
- Đa ngôn ngữ: mô hình spaCy/ViSpacy hoặc LLM-assisted parsing cho user story tiếng Việt, không chỉ en_core_web_sm.

3. Hướng phát triển: Đồng thời ghi Neo4j và “write lock”:

Kiến trúc hiện tại có hai luồng ghi Neo4j có thể chồng lên cùng workspace_id:

- Realtime: consumer handle_story_created → save_svo (MERGE Term, UserStory, PERFORM, TARGET).
- Batch: GraphBatchService.rebuild_workspace → clear_workspace (DETACH DELETE toàn bộ node có workspace_id) rồi ghi lại đồ thị.

Khi người dùng tạo story trong lúc rebuild, hoặc nhiều consumer/worker song song, Neo4j có thể phát sinh xung đột ghi (transaction retry, lock wait, dữ liệu tạm thời không nhất quán). Neo4jConnection.execute mở session ngắn từng query, chưa có cơ chế khóa logic theo workspace (distributed lock) hay hàng đợi serialize rebuild.

Hướng cải thiện:

- Khóa logic theo workspace: Redis lock / RabbitMQ “single consumer per workspace” — chỉ một rebuild hoặc một luồng ghi tại một thời điểm.
 - Chiến lược ghi incremental: hạn chế clear_workspace; dùng version/graph generation thay vì xóa toàn cục.
 - Neo4j Enterprise / Aura: tận dụng transaction isolation, causal cluster; theo dõi metric lock wait time trên CloudWatch.
 - Trạng thái rebuild trên UI: API báo REBUILDING để client không kỳ vọng đồ thị đầy đủ khi batch đang chạy.
 - Dead-letter queue cho message lỗi Neo4j, tránh mất sự kiện USER_STORY_CREATED im lặng.
4. Hướng phát triển: Học có giám sát và chất lượng mô hình
- Lưu phản hồi người dùng (xác nhận trùng/không trùng, sửa SVO) làm nhãn huấn luyện, thay weak-label heuristic hiện tại.
 - Đánh giá định kỳ trên bộ Wordsim353 / RG-65 / MC-30.
 - Tự động chọn hyperparameter fusion (beta1, beta2, bias_b) theo workspace thay vì hằng số cố định.

5.2.3.4. Tối ưu hiệu năng, bảo mật và trải nghiệm người dùng

- **Hiệu năng và hạ tầng**
 - CDN + caching: CloudFront cho static Flutter; cache GraphQL/read Neo4j theo workspaceId + version graph.
 - Tách read/write Neo4j: read replica hoặc routing read-only cho GraphServiceImpl; ghi tập trung qua queue.
 - Kubernetes (EKS): orchestration dài hạn thay chỉ Docker Compose local; HPA theo queue và CPU.
 - Giám sát: CloudWatch Dashboard, X-Ray/OpenTelemetry trace xuyên BE → RabbitMQ → Python → Neo4j.
- **Bảo mật:**
 - Đã có: thông báo in-app, Socket.IO realtime (SocketService); mời workspace + email nền (JobRunr).
 - Tiếp theo: push notification native (FCM/APNs); chat/comment trong workspace (mở rộng Socket.IO hoặc WebSocket room theo workspaceId).
 - Không ưu tiên ngay (trừ khi đổi sản phẩm): livestream/video — ngoài phạm vi quản lý User Story hiện tại.
- **UX/UI:**
 - Hoàn thiện mobile-first trên toàn bộ backlog, sprint board, màn graph.
 - Dark mode đồng bộ toàn app (hiện mới áp dụng một phần theo theme hệ thống).
 - Hiển thị trạng thái pipeline phân tích: đang parse, đang rebuild, đồ thị sẵn sàng.

Thuật ngữ chuyên môn

[illegible]

Tài Liệu Tham Khảo:

- [1] K. Beck et al., “Manifesto for Agile Software Development,” 2001. [Online]. Available: <https://agilemanifesto.org/>
- [2] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA, USA: Addison-Wesley, 2003.
- [3] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2015.
- [4] L. C. Freeman, “A Set of Measures of Centrality Based on Betweenness,” Sociometry, vol. 40, no. 1, pp. 35–41, Mar. 1977. doi: 10.2307/3033543
- [5] R. Agrawal and R. Srikant, “Fast Algorithms for Mining Association Rules in Large Databases,” in Proc. 20th Int. Conf. Very Large Data Bases (VLDB), Santiago, Chile, Sep. 1994, pp. 487–499. [Online]. Available: <https://www.vldb.org/conf/1994/P487.PDF>
- [6] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013. [Online]. Available: <https://arxiv.org/abs/1301.3781>
- [7] G. A. Miller, “WordNet: A Lexical Database for English,” Commun. ACM, vol. 38, no. 11, pp. 39–41, Nov. 1995. doi: 10.1145/219717.219748
- [8] Google, “Flutter Documentation,” Flutter.dev. [Online]. Available: <https://docs.flutter.dev/>
- [9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA, USA: MIT Press, 2009. (Union-Find / Disjoint-Set)
Từ khóa Scholar nếu cần: Cormen Introduction to Algorithms Union-Find
- [10] A. J. Ratner, C. De Sa, S. Wu, D. Selsam, and C. Ré, “Data Programming: Creating Large Training Sets, Quickly,” in Proc. 30th Conf. Neural Inf. Process. Syst. (NIPS), Barcelona, Spain, 2016. [Online]. Available: https://papers.nips.cc/paper_files/paper/2016/file/6709e8d64a5f47269ed5cea9f625f7ab-Paper.pdf
- [11] Neo4j, “Neo4j Graph Data Science Library — Centrality,” Neo4j Documentation. [Online]. Available: <https://neo4j.com/docs/graph-data-science/current/algorithms/centrality/>
- [12] VMware Spring Team, “Spring Boot Reference Documentation,” Spring.io. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

- [13] Google, “Flutter Documentation,” Flutter.dev. [Online]. Available: <https://docs.flutter.dev/>
- [14] Red Hat, “Keycloak Documentation,” Keycloak.org. [Online]. Available: <https://www.keycloak.org/documentation>
- [15] D. Hardt, Ed., “The OAuth 2.0 Authorization Framework,” RFC 6749, IETF, Oct. 2012. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6749>
- [16] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [17] GraphQL Foundation, “GraphQL Specification,” GraphQL.org. [Online]. Available: <https://spec.graphql.org/>
- [18] Explosion AI, “spaCy: Dependency Parsing,” spaCy Documentation. [Online]. Available: <https://spacy.io/usage/linguistic-features#dependency-parse>
- [19] scikit-learn developers, “scikit-learn User Guide,” scikit-learn.org. [Online]. Available: https://scikit-learn.org/stable/user_guide.html
- [20] VMware, “RabbitMQ Documentation,” RabbitMQ.com. [Online]. Available: <https://www.rabbitmq.com/documentation.html>
- [21] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” Commun. ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970. doi: 10.1145/362384.362685
- [22] W. W. Royce, “Managing the Development of Large Software Systems,” in Proc. IEEE WESCON, Aug. 1970, pp. 1–9.
Tủ khóa Scholar: Royce Managing the Development of Large Software Systems 1970
- [23] M. Cohn, User Stories Applied: For Agile Software Development. Boston, MA, USA: Addison-Wesley, 2004.
- [24] Microsoft, “Model–View–ViewModel (MVVM) Pattern,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm>
- [25] Object Management Group, “Unified Modeling Language (UML),” OMG Specification. [Online]. Available: <https://www.omg.org/spec/UML/>
- [26] Oracle, “MySQL 8.0 Reference Manual — Replication,” MySQL Documentation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/replication.html>

[27] JobRunr, “JobRunr Documentation,” JobRunr.io. [Online]. Available: <https://www.jobrunr.io/en/documentation/>

[28] Docker Inc., “Docker Documentation,” Docker Docs. [Online]. Available: <https://docs.docker.com/>

[29] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, Dec. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>

[30] Socket.IO, “Socket.IO Documentation,” Socket.io. [Online]. Available: <https://socket.io/docs/v4/>

[31] J. Klensin, “Simple Mail Transfer Protocol,” RFC 5321, IETF, Oct. 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5321>

[32] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. draft. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/>

[33] Bryant, D., Woodcock, J., Foster, S.: Formal analysis of the sigmoid function and formal proof of the universal approximation theorem (2025), <https://arxiv.org/abs/2512.03635>
Từ khóa: logistic sigmoid function machine learning calibration

Link Demo: <https://www.youtube.com/watch?v=jnOiHF75T6Q>

Link Github: <https://github.com/DucHuy74/daln>